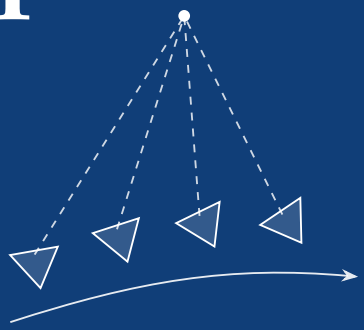
From Zero to MSCKF

Lecture Notes on Visual-Inertial Odometry

Lie theory · error-state Kalman filtering · multi-view geometry · the Multi-State Constraint Kalman Filter



These notes develop monocular visual-inertial odometry from first principles. They start with the linear algebra, probability and rotation geometry that state estimation is built on, develop calculus on the rotation group SO(3), derive the error-state Kalman filter for an inertial navigation system, build the geometric computer-vision front end, and assemble everything into the Multi-State Constraint Kalman Filter (MSCKF) evaluated on the EuRoC MAV benchmark.

Every convention matches the frozen specification of the `vio` project (`AGENTS.md`): Hamilton quaternions, body-to-world attitude q_{wb}, and *local (right) perturbations* in every Jacobian.

Edition. This edition *includes* the worked solutions of the project derivation exercises (Appendix E). If you intend to derive F_c, G_c, H and the cloning Jacobian J yourself (Guardrail 4 of `AGENTS.md`), attempt each *Derivation exercise* box before turning to the appendix, then verify your answer numerically.

Course notes for the `vio` project

October 7, 2026

Preface

What these notes are

Visual-inertial odometry (VIO) estimates the motion of a rigid body, such as a drone, a phone, a headset or a car, by fusing an inertial measurement unit (IMU) with a camera. It is the navigation core of most modern augmented-reality devices and of many autonomous aerial vehicles. VIO is also one of the best ways to learn state estimation properly, because it forces you to deal with almost every hard topic in the field at once: nonlinear dynamics, estimation on curved spaces, sensor noise modeling, multi-rate fusion, image processing, multi-view geometry, unobservable directions, outliers, numerical conditioning, and real-time computation.

These notes are meant to be the only text you need. They start from vectors and matrices and end with a complete, consistent, real-time multi-state constraint Kalman filter (MSCKF), and every tool used on the way is developed in the notes themselves: when a chapter needs the QR decomposition, the chi-square distribution, the matrix exponential, the inverse function theorem or image interpolation, an earlier chapter has derived it. In a university the same material is spread over a dozen courses; the table below shows which.

The notes are written in the style of a graduate course. Each chapter states its learning objectives, develops the theory with definitions, propositions and proofs, illustrates it with worked examples and figures, flags the pitfalls that cost practitioners weeks of debugging, and closes with key takeaways and exercises. All formulas and conventions match the `vio` project specification in `AGENTS.md`. Every identity, Jacobian and growth law stated here has been checked numerically. The figures and tables are generated by the Python scripts in `scripts/` (`make_figures.py`, `make_tables.py` and the planar simulations in `planar_sims.py`), with fixed random seeds, so you can reproduce them and change their parameters.

Who should read them

The assumed background is small:

- single-variable calculus (derivatives, integrals, Taylor series) and the idea of a partial derivative;
- matrix arithmetic (products, transposes, solving a small linear system by hand);
- for the implementation notes, the ability to read C++ and Python code.

Everything else, including linear algebra beyond matrix arithmetic, probability, differential equations, Lie groups, Kalman filtering, optimization, image processing and multi-view geometry, is developed from the beginning. Readers who already know some of it can skip ahead; the reading paths below say where.

The courses inside these notes

Typical course	Level	Chapters
Linear algebra	undergraduate	2
Numerical linear algebra	advanced undergraduate	3
Differential equations and numerical methods	undergraduate	4
Probability	undergraduate	5
Random processes and stochastic systems	graduate	6
Rigid-body kinematics	undergraduate	7
Lie groups for robotics	graduate	8
Estimation and detection theory	graduate	9, 12, 13
Numerical optimization	graduate	10
Linear systems and control	advanced undergraduate	11
Inertial navigation	graduate	15–17
Image processing	advanced undergraduate	21
Computer vision and multi-view geometry	graduate	20, 22, 23
State estimation for robotics, SLAM and VIO	graduate	14, 18, 19, 24–29
Real-time software engineering	professional	30

How the notes are organized

Part I, Mathematical Foundations. The vocabulary of the whole subject: linear algebra and the factorizations that make filters numerically sound; calculus, differential equations and their numerical solution; probability, Gaussian algebra and random processes; rotations, quaternions, and Lie theory, the calculus of the rotation group.

Part II, Estimation Theory. Least squares and Bayesian estimation, nonlinear least squares and optimization on manifolds, linear dynamical systems, the Kalman filter and its nonlinear relatives, and the error-state formulation that makes estimation on a manifold clean.

Part III, Inertial Navigation. IMU physics, noise and calibration, strapdown propagation, and the linearized error dynamics that drive covariance growth.

Part IV, The Error-State Filter in Practice. Measurement updates, gating, injection and reset, and the statistics that tell you whether the filter is lying to you.

Part V, Computer Vision. Camera models and calibration, image processing, corner detection and optical-flow tracking, epipolar geometry, robust estimation, triangulation and bundle adjustment.

Part VI, The MSCKF. From EKF-SLAM to the multi-state constraint, stochastic cloning, the visual update with left-nullspace projection, a complete planar MSCKF worked by hand and in simulation, and the full algorithm.

Part VII, Beyond the Baseline. Observability and consistency, and engineering a deterministic real-time implementation in C++20.

Table 1 lists the direct prerequisites of every chapter, for readers who want to jump in.

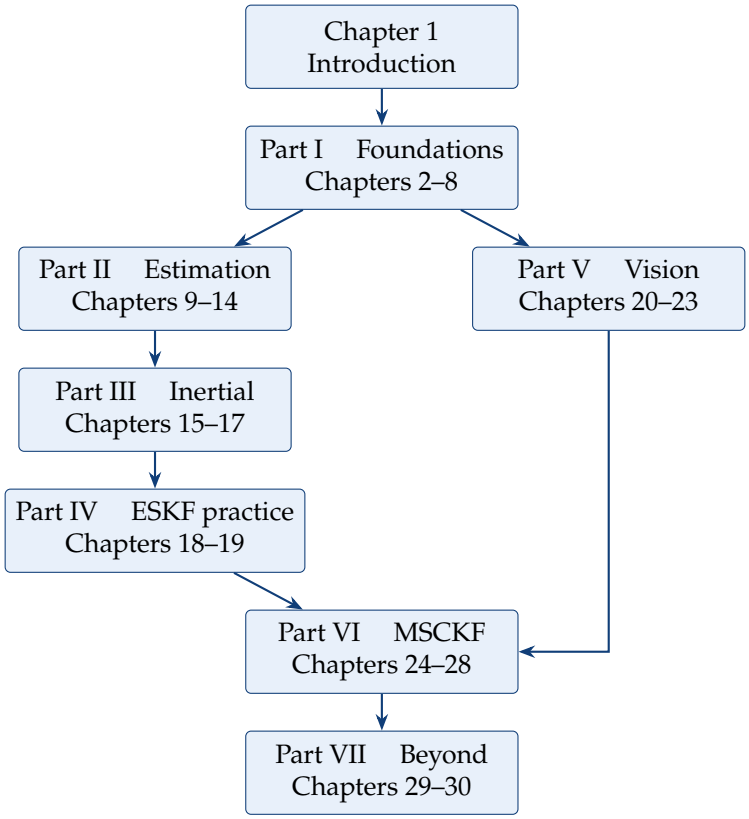


Figure 1: The parts of the notes and their dependencies. The inertial branch (Parts II–IV) and the vision branch (Part V) are independent of each other and meet in the MSCKF.

Reading paths

- **From scratch.** Read in order. Part I is long because it is the foundation of everything else; its chapters can be interleaved with the later ones as they are needed.
- **With an engineering mathematics background** (linear algebra, probability, ODEs). Skim Chapters 2 to 5 for notation and the results marked as theorems, read Chapters 6 to 8 carefully, then continue in order.
- **Knowing Kalman filtering.** Read Chapters 7, 8 and 14, then Parts III–IV, Part V, and Parts VI–VII.
- **Knowing computer vision.** Read Chapters 7 and 8, Parts II–IV, skim Part V, then Parts VI–VII.
- **Driven by the project.** Follow the milestone table below; each milestone lists the chapters it needs, and Table 1 tells you what to read first.

Reading the boxes

Several kinds of callout recur throughout:

Table 1: Direct prerequisites of each chapter (prerequisites of prerequisites are not repeated).

	Chapter	Needs		Chapter	Needs
1	Introduction	—	16	Strapdown	4, 8, 15
2	Linear algebra	—	17	Error dynamics	11, 14, 16
3	Numerical linear algebra	2	18	Updates	14, 17
4	Calculus and ODEs	2	19	Consistency	5, 18
5	Probability	2, 4	20	Image formation	2, 7
6	Random processes	4, 5	21	Image processing	4, 20
7	Rotations	2	22	Features and tracking	10, 21
8	Lie theory	4, 7	23	Multi-view geometry	3, 10, 20
9	Estimation	3, 5	24	EKF-SLAM to MSCKF	18, 23
10	Nonlinear least squares	8, 9	25	Cloning	17, 24
11	Linear systems	3, 4, 6	26	MSCKF update	23, 25
12	Kalman filter	9, 11	27	Planar MSCKF	26
13	Nonlinear filtering	12	28	Full algorithm	22, 26
14	Error-state KF	8, 13	29	Observability	11, 19, 27
15	IMU	6, 7	30	Engineering	17–28 (helpful)

Key idea: the one-sentence version

The idea to remember even if you forget the derivation.

Frozen convention: project specification

A rule from `AGENTS.md` that is frozen for the whole project. These are never changed silently. A change is a logged design decision.

Pitfall: what goes wrong in practice

A mistake that is easy to make, hard to see, and expensive to debug.

Implementation note: from math to code

How an equation becomes deterministic, allocation-free C++20 with Eigen.

Project link: Milestone N

Where the material meets a project milestone and its exit criteria.

Worked example: numbers you can check by hand

A complete calculation with concrete numbers, small enough to redo with pencil and paper.

Going deeper: optional depth

Background or extensions that can be skipped on a first reading.

One more box, the **Derivation exercise**, marks the Jacobians that the project asks you to derive yourself: F_c, G_c, the measurement Jacobians H, and the cloning Jacobian J. The surrounding

text teaches the method on closely related examples (the planar MSCKF of Chapter 27 derives every planar counterpart in full), and the box gives hints plus a finite-difference test that tells you when your answer is right. The worked solutions are collected in an appendix that exists only in the *solutions* edition of these notes (this one: Appendix E).

Milestones and chapters

Milestone	Topic	Chapters
M1	Build harness, sanitizers, allocation guard	30
M2	Lie group math on SO(3)	7, 8 (with 2, 4)
M3	EuRoC dataset streamer	15, Appendix D
M4	Nominal IMU kinematics	16 (with 4)
M5	Error covariance propagation	17 (with 6, 11)
M6	Pose update, injection and reset	14, 18 (with 12)
M7	Telemetry and consistency	19
M8	Feature detection and optical flow	20, 21, 22
M9	Epipolar geometry and outlier rejection	23, 22
M10a	Stochastic cloning and sliding window	24, 25
M10b	MSCKF visual update	26, 27, 28, 29

Sources and acknowledgments

These notes stand on a body of excellent literature. The treatment of quaternions and the error-state filter follows the conventions of Solà [47]; Lie theory follows Solà, Deray and Atchuthan [48] and Barfoot [4]; the MSCKF is due to Mourikis and Roumeliotis [40] and was refined by Li and Mourikis [34]; the consistency analysis follows Huang, Mourikis and Roumeliotis [28]; much practical wisdom comes from OpenVINS [18] and MSCKF-VIO [51]. The annotated bibliography at the end points to the best places to go deeper on each topic. You should never *need* it to follow these notes; you may well want it once you have.

Contents

Preface **i**

1 The Visual-Inertial Odometry Problem **1**

1.1 What odometry is . . . 1
1.2 An analogy: the blindfolded walk . . . 1
1.3 The sensors and their weaknesses . . . 2
1.4 Why fusion is hard . . . 4
1.5 A map of the field . . . 5
1.6 Why the MSCKF . . . 5
1.7 The estimator at a glance . . . 6
1.8 The EuRoC benchmark . . . 7
Exercises . . . 8

I Mathematical Foundations **9**

2 Vectors, Matrices and Linear Maps **10**

2.1 Vectors . . . 10
2.2 Inner products, lengths and angles . . . 12
2.3 The cross product . . . 13
2.4 Matrices as linear maps . . . 15
2.5 Linear systems, inverses and rank . . . 17
2.6 Determinants . . . 19
2.7 Orthogonal matrices and change of basis . . . 20
2.8 Eigenvalues and eigenvectors . . . 21
2.9 Symmetric matrices and quadratic forms . . . 22
2.10 Trace and norms . . . 24
Exercises . . . 25

3 Numerical Linear Algebra **26**

3.1 The cost of matrix operations . . . 26
3.2 Triangular systems and the LU factorization . . . 27
3.3 The Cholesky factorization . . . 27
3.4 Orthogonal factorizations: QR . . . 28
3.5 Least squares . . . 30
3.6 The singular value decomposition . . . 31
3.7 Conditioning and stability . . . 33
3.8 Block matrices and the Schur complement . . . 33
3.9 The matrix exponential . . . 34
3.10 Floating-point arithmetic . . . 35
Exercises . . . 37

4 Calculus, Differential Equations and Numerical Integration **39**

4.1 Derivatives of functions of several variables . . . 39
4.2 Ordinary differential equations . . . 41
4.3 Numerical integration of ODEs . . . 43
4.4 Numerical differentiation . . . 45
Exercises . . . 47

5 Probability and Random Vectors **49**

5.1 Probability . . . 49
5.2 Random variables . . . 50
5.3 Several random variables . . . 51
5.4 Random vectors . . . 52
5.5 The multivariate Gaussian . . . 53
5.6 Fusing information: Bayes' rule for Gaussians . . . 55
5.7 Limit theorems and Monte Carlo . . . 56
5.8 The chi-squared distribution . . . 57
5.9 Sample statistics and hypothesis tests . . . 58
5.10 Nonlinear transformations . . . 59
Exercises . . . 61

6 Random Processes, Spectra and Sensor Noise **62**

6.1 Random sequences . . . 62
6.2 Frequency-domain description . . . 64
6.3 Continuous-time white noise . . . 65
6.4 Sampling and aliasing . . . 65
6.5 Integrated noise . . . 66
6.6 Linear stochastic differential equations . . . 66
6.7 Colored noise and state augmentation . . . 67
6.8 The Allan variance . . . 68
Exercises . . . 69

7 Rotations, Quaternions and Rigid-Body Kinematics **71**

7.1 Frames and notation . . . 71
7.2 Rotations in the plane . . . 72
7.3 Rotation matrices and SO(3) . . . 73
7.4 Axis-angle and Rodrigues' formula . . . 73
7.5 Euler angles . . . 74
7.6 Quaternions . . . 75
7.7 Rigid-body transformations . . . 80
7.8 Rotational kinematics . . . 80
Exercises . . . 82

8 Lie Theory for Rotations **84**

8.1 Why estimation needs Lie theory . . . 84
8.2 Warm-up: rotations of the plane . . . 85
8.3 Manifolds, groups and Lie groups . . . 85
8.4 The Lie algebra and tangent spaces . . . 86
8.5 The exponential map . . . 87
8.6 The logarithm map . . . 90
8.7 The adjoint: moving a perturbation across a rotation . . . 91
8.8 Composition and the Baker–Campbell–Hausdorff formula . . . 91
8.9 Perturbations, $\boxplus$ and $\boxminus$ . . . 92
8.10 Jacobians on manifolds . . . 93
8.11 Uncertainty on SO(3) . . . 96
8.12 SE(3) and the choice of state manifold . . . 97

8.13 Verifying manifold Jacobians numerically . . . 98
Exercises . . . 99

II Estimation Theory **100**

9 Estimation Fundamentals **101**
9.1 Estimators and their errors . . . 101
9.2 Linear least squares . . . 102
9.3 Maximum likelihood . . . 103
9.4 Bayesian estimation . . . 104
9.5 Recursive least squares . . . 106
9.6 One answer, many derivations . . . 106
Exercises . . . 107

10 Nonlinear Least Squares and Optimization **108**
10.1 The problem . . . 108
10.2 Descent methods . . . 109
10.3 Gauss–Newton . . . 109
10.4 Levenberg–Marquardt . . . 110
10.5 Uncertainty of the solution . . . 110
10.6 Optimization on manifolds . . . 111
10.7 Robust estimation . . . 111
10.8 Structure: poses, landmarks and the Schur complement . . . 113
10.9 Filtering as optimization . . . 114
Exercises . . . 115

11 Linear Dynamical Systems **116**
11.1 State-space models . . . 116
11.2 Discretization . . . 117
11.3 Stability . . . 118
11.4 Observability . . . 118
Exercises . . . 120

12 The Kalman Filter **122**

12.1 The model . . . 122
12.2 The Bayes filter . . . 122
12.3 Derivation . . . 123
12.4 A worked example by hand . . . 124
12.5 Properties . . . 125
12.6 The Joseph form . . . 125
12.7 Implementing the update . . . 126
12.8 Extensions . . . 127
12.9 A simulated example . . . 127
Exercises . . . 128

13 Nonlinear Filtering **130**

13.1 The extended Kalman filter . . . 130
13.2 Sigma-point filters . . . 131
13.3 Particle filters . . . 133
13.4 Why VIO uses the EKF . . . 133
Exercises . . . 134

14 The Error-State Kalman Filter **135**

14.1 Filtering on a manifold . . . 135
14.2 The ESKF cycle . . . 136
14.3 Linearized error dynamics: the recipe . . . 137
14.4 Measurement updates on the manifold . . . 137
14.5 Injection and reset . . . 138
14.6 The complete error-state filter . . . 139
14.7 A complete example: attitude estimation . . . 139
Exercises . . . 141

III Inertial Navigation **143**

15 Inertial Sensors and Their Errors **144**

15.1 How a MEMS accelerometer works . . . 144
15.2 How a MEMS gyroscope works . . . 146
15.3 The measurement model . . . 146

15.4 Calibration . . . 147
15.5 Noise parameters and their units . . . 148
15.6 Tuning in practice . . . 149
15.7 The EuRoC IMU stream . . . 149
Exercises . . . 150

16 Strapdown Propagation of the Nominal State **151**
16.1 Continuous-time kinematics . . . 151
16.2 Exact solution for constant inputs . . . 152
16.3 Numerical integration schemes . . . 152
16.4 Coning, sculling and the rotation-vector equation . . . 154
16.5 Timing . . . 155
16.6 Initialization . . . 156
16.7 Sidebar: IMU preintegration . . . 157
Exercises . . . 157

17 Error-State Dynamics and Covariance Propagation **159**
17.1 True and nominal systems . . . 159
17.2 Deriving the error dynamics . . . 160
17.3 Discretization . . . 161
17.4 Propagating the full covariance . . . 162
17.5 How uncertainty grows . . . 163
17.6 Validating covariance propagation . . . 164
Exercises . . . 165

IV The Error-State Filter in Practice **166**

18 Measurement Updates, Gating and the Pose Sensor **167**
18.1 The generic update . . . 167
18.2 Worked example: position with a lever arm . . . 168
18.3 Worked example: a direction sensor . . . 168
18.4 The pose sensor of Milestone 6 . . . 168
18.5 Gating . . . 169
18.6 Pseudo-measurements and timing . . . 170
18.7 Testing the update . . . 171
Exercises . . . 171

19 Consistency and Evaluation **173**

19.1 What consistency means . . . 173
19.2 Normalized estimation error squared . . . 173
19.3 Normalized innovation squared . . . 174
19.4 Diagnosing inconsistency . . . 175
19.5 Trajectory accuracy . . . 176
Exercises . . . 177

V Computer Vision **179**

20 Image Formation and Camera Models **180**

20.1 How a camera forms an image . . . 180
20.2 The pinhole projection . . . 181
20.3 Homogeneous coordinates and projective geometry . . . 182
20.4 Lens distortion . . . 183
20.5 Camera calibration . . . 185
20.6 The measurement model . . . 185
20.7 The projection Jacobian . . . 186
20.8 Extrinsics and time offset . . . 186
Exercises . . . 187

21 Digital Image Processing **188**

21.1 Images as functions . . . 188
21.2 Linear filtering . . . 188
21.3 Image derivatives . . . 189
21.4 Sampling, aliasing and pyramids . . . 190
21.5 Interpolation . . . 191
21.6 Finding local maxima . . . 192
21.7 Photometric changes . . . 192
Exercises . . . 193

22 Feature Detection and Tracking **194**

22.1 What makes a good feature . . . 194
22.2 Lucas–Kanade optical flow . . . 196
22.3 Making tracking robust . . . 198

22.4 The feature life cycle . . . 199
22.5 Engineering the front end . . . 200
Exercises . . . 201

23 Multiple-View Geometry **202**
23.1 Epipolar geometry . . . 202
23.2 Estimating relative motion . . . 203
23.3 Homographies between views . . . 205
23.4 RANSAC . . . 205
23.5 Degeneracies . . . 206
23.6 Triangulation . . . 207
23.7 Bundle adjustment . . . 209
Exercises . . . 210

VI The Multi-State Constraint Kalman Filter **212**

24 From EKF-SLAM to the MSCKF **213**
24.1 EKF-SLAM . . . 213
24.2 The multi-state constraint . . . 214
24.3 Why past poses must be in the state . . . 214
24.4 The MSCKF state . . . 215
24.5 Relation to sliding-window optimization . . . 215
24.6 Variants . . . 216
Exercises . . . 216

25 Stochastic Cloning and the Sliding Window **217**
25.1 A scalar example . . . 217
25.2 State augmentation . . . 218
25.3 The project's clone . . . 218
25.4 Clones in propagation and updates . . . 219
25.5 Marginalizing the oldest clone . . . 219
25.6 Window policy and size . . . 220
Exercises . . . 220

26 The MSCKF Visual Update **222**

26.1 A feature track . . . 222
26.2 Residuals and Jacobians . . . 222
26.3 The problem with the feature . . . 223
26.4 Left-nullspace projection . . . 224
26.5 Gating, stacking and compression . . . 226
26.6 The update . . . 226
Exercises . . . 227

27 A Complete MSCKF in the Plane **229**

27.1 The planar world . . . 229
27.2 Sensors . . . 230
27.3 State, error state and propagation . . . 231
27.4 Cloning and marginalization . . . 233
27.5 The feature measurement . . . 233
27.6 Eliminating the feature . . . 235
27.7 The complete filter . . . 236
27.8 Simulation results . . . 237
27.9 The unobservable directions . . . 239
27.10 From the plane to three dimensions . . . 240
Exercises . . . 241

28 The Complete Algorithm **242**

28.1 The event loop . . . 242
28.2 Initialization and the first seconds . . . 242
28.3 Computational budget . . . 243
28.4 Software architecture . . . 244
28.5 Debugging guide . . . 244
28.6 What to expect on EuRoC . . . 245
Exercises . . . 246

VII Beyond the Baseline **247**

29 Observability and Consistency of VIO **248**

29.1 Observability . . . 248

29.2 The four unobservable directions of VIO . . . 250
29.3 Why the standard EKF becomes overconfident . . . 251
29.4 First-Estimates Jacobians . . . 253
29.5 Other remedies . . . 255
29.6 Degenerate motions . . . 257
Exercises . . . 258

30 Engineering a Real-Time Estimator in C++20 **259**
30.1 C++ essentials for estimator code . . . 259
30.2 Determinism and the memory tiers . . . 260
30.3 Eigen in brief . . . 261
30.4 Eigen pitfalls . . . 263
30.5 Time and events . . . 263
30.6 Numerical robustness checklist . . . 264
30.7 Sanitizers . . . 264
30.8 Modern CMake in brief . . . 265
30.9 Testing . . . 265
30.10 Debugging a diverging filter . . . 266
30.11 Going further . . . 266
Exercises . . . 267

Appendices **268**

A Notation and Conventions **269**

B Formula Sheet **271**

C Statistical and Numerical Tables **276**

D The EuRoC MAV Dataset **278**

E Solutions to the Project Derivation Exercises **280**
E.1 Error-state dynamics F_c and G_c . . . 280
E.2 Closed-form blocks of Φ . . . 281
E.3 Pose measurement Jacobian . . . 281
E.4 Cloning Jacobian . . . 282
E.5 Visual measurement Jacobians . . . 282

Further Reading **284**

References **285**

Index **289**

Chapter 1

The Visual-Inertial Odometry Problem

In this chapter

- State precisely what visual-inertial odometry estimates and what it cannot estimate.
- Understand why an IMU alone drifts, and quantify how fast.
- Understand why a monocular camera alone cannot recover metric scale.
- Place the MSCKF among the main families of VIO algorithms.
- See the complete estimator pipeline that the rest of the notes build piece by piece.

1.1 What odometry is

Odometry is the incremental estimation of a body's motion from its own sensors, without a prior map and without absolute positioning such as GNSS. The word comes from the wheel odometers of cars, which count wheel rotations. *Visual-inertial odometry* uses an IMU and one or more cameras rigidly attached to the body.

At every time t we want the *state*

$$x(t) = \big(p_{wb}(t),\ v_{wb}(t),\ q_{wb}(t),\ b_a(t),\ b_g(t) \big), \tag{1.1}$$

namely the position and velocity of the body in a world frame, its orientation, and the slowly varying biases of the accelerometer and gyroscope. We also want an honest measure of how uncertain that estimate is. Chapter 7 makes the frames and symbols precise. For now, w is a fixed, gravity-aligned world frame and b is the body frame of the IMU.

Odometry differs from *SLAM* (simultaneous localization and mapping) in one important way. A SLAM system builds a persistent map and recognizes places it has seen before (*loop closure*), which lets it cancel accumulated drift. An odometry system has no such memory: its error grows without bound, slowly if it is good and quickly if it is bad. The goal of VIO is to make that growth as slow as possible, typically well below 1% of the distance traveled, while running in real time on modest hardware.

1.2 An analogy: the blindfolded walk

Imagine standing blindfolded in an unfamiliar gymnasium and being asked to walk a 20-meter square and return to the start. You would rely on your inner ear to sense turning and on the feeling in your legs to judge distance. Integrating these internal sensations into a running estimate of your position is *dead reckoning*.

For the first few steps the estimate is good. But your sensors are imperfect: a 90-degree turn comes out as 87 degrees, and your strides are a little shorter than you think. Each new step is added on top of a slightly wrong estimate, so the small errors accumulate. After a few minutes you might believe you are back at the start while actually standing several meters away. This accumulation is called *drift*.

Now suppose you may lift the blindfold for a tenth of a second every few seconds. The room is dark except for a few glowing stickers on the walls. You do not know where the stickers are, but by watching how they shift between glimpses you can infer how you moved, and correct the walking estimate. That is visual-inertial odometry: the inner ear is the IMU, the glimpses are camera images, and the stickers are visual features.

Two details of the analogy carry over exactly. First, the glimpses are *relative*: they tell you how you moved between glimpses, not where you are in some absolute sense, so drift is slowed but never eliminated. Second, you need both senses. Without the glimpses, drift is fast. Without the inner ear, a single glimpse cannot tell whether a sticker is small and near or large and far.

1.3 The sensors and their weaknesses

1.3.1 The inertial measurement unit

An IMU contains a three-axis gyroscope and a three-axis accelerometer. Neither measures position.

- The **gyroscope** measures the angular velocity ω of the body, in rad/s, expressed in the body frame.
- The **accelerometer** measures *specific force*: the non-gravitational force per unit mass acting on the sensor. At rest on a table it reads +9.81 m/s^2 *upward*, because the table pushes up on it, and in free fall it reads zero. Chapter 15 explains this carefully.

IMUs are fast (typically 100–1000 Hz), cheap, self-contained, insensitive to lighting and texture, and their measurements are *metric*: an acceleration in m/s^2 carries an absolute scale. But to get position you must integrate angular rate once to get attitude, then rotate the specific force into the world frame, add gravity, and integrate twice. Any error in the sensor or in the attitude is integrated as well.

1.3.2 How fast inertial errors grow

Three error sources dominate short-term inertial drift. The derivations are one-liners, and the resulting growth laws (Figure 1.1) are worth memorizing.

1. **Accelerometer bias b_a.** A constant acceleration error integrated twice gives a position error $\frac{1}{2} b_a t^2$.
2. **Initial tilt error $\delta\theta_0$.** If the estimated attitude is tilted by a small angle $\delta\theta_0$, the 9.81 m/s^2 gravity compensation is applied along a slightly wrong axis. A component $g\,\delta\theta_0$ of gravity leaks into the horizontal acceleration, and the position error grows as $\frac{1}{2} g\,\delta\theta_0\,t^2$.

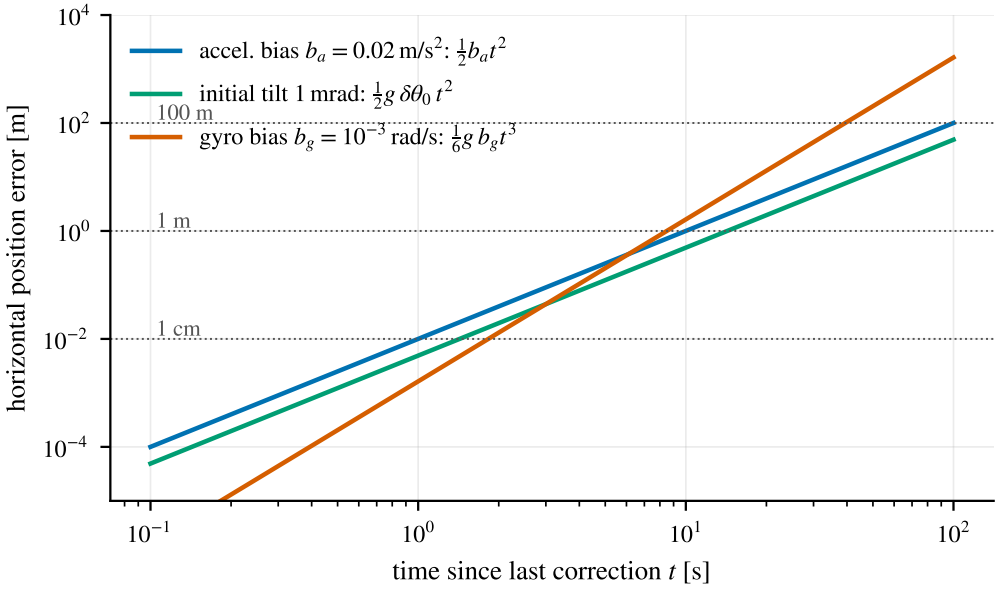


Figure 1.1: Deterministic dead-reckoning drift from three error sources of a consumer-grade IMU. A gyro bias of 10^{-3} rad/s ($\approx 0.06\,^\circ$/s) produces more than a meter of error after ten seconds and hundreds of meters after one minute. The cubic law is why attitude accuracy dominates inertial navigation.

3. **Gyroscope bias b_g.** Now the tilt itself grows linearly, $\delta\theta(t) = b_g t$, so the leaked acceleration is $g b_g t$, and integrating twice gives $\frac{1}{6} g\, b_g t^3$.

With $b_g = 10^{-3}$ rad/s, a modest value for a MEMS gyroscope, the gyro term alone gives 1.6 m after 10 s and 353 m after 60 s. Random sensor noise adds error that grows in a similar way. Chapter 17 derives that the position *standard deviation* due to accelerometer white noise grows as $t^{3/2}$, due to gyroscope white noise as $t^{5/2}$, and due to the gyroscope bias random walk as $t^{7/2}$.

Key idea: inertial navigation is an attitude problem

Because gravity is large (about 9.81 m/s^2) compared with typical accelerometer errors, a tilt of one milliradian costs as much as an accelerometer bias of about 10 mm/s^2. Position accuracy is therefore limited mostly by attitude accuracy. Any aiding sensor that constrains roll and pitch, as a camera does, pays off enormously.

1.3.3 The camera

A camera measures the *directions* (bearings) of light rays arriving from the scene. A point p_f in the world appears in the image at the projection of its camera-frame coordinates $p_c = (X_c, Y_c, Z_c)$:

$$z = \begin{bmatrix} X_c / Z_c \\ Y_c / Z_c \end{bmatrix}. \tag{1.2}$$

Because images are measurements of fixed scene geometry, they do not drift the way integrated inertial data does. But a single camera has a fundamental blind spot.

	IMU	Monocular camera
Rate	100–1000 Hz	10–60 Hz
Measures	angular rate, specific force	bearings to scene points
Scale	metric	unknown (Proposition 1.1)
Drift	fast (t^2 to t^3)	slow, relative to the scene
Gravity direction	observable (at rest or on average)	not observable
Fails when	always drifts; saturates on impacts	blur, darkness, no texture, pure rotation

Table 1.1: The IMU and the camera fail in complementary ways.

Proposition 1.1 (Monocular scale ambiguity). *Let a camera with poses (R_i, p_i), $i = 1, \dots, M$, observe points $p_{f,j}$. For any $s > 0$, the scaled configuration with poses $(R_i, s\,p_i)$ and points $s\,p_{f,j}$ produces exactly the same images.*

Proof. In camera i, the scaled point has coordinates $R_i^\mathsf{T}(s\,p_{f,j} - s\,p_i) = s\,R_i^\mathsf{T}(p_{f,j} - p_i)$. The projection divides the first two coordinates by the third, so the factor s cancels. ∎

A camera alone therefore recovers the trajectory and the scene only up to an unknown global scale. A photograph of a toy car from 10 cm can be identical to a photograph of a real car from 4 m. Cameras also fail under motion blur, in darkness, against texture-less surfaces such as white walls, and in scenes dominated by moving objects.

1.3.4 Why the combination works

Table 1.1 summarizes the complementarity. The camera bounds the inertial drift by constraining the motion between images. The IMU supplies metric scale, the direction of gravity (and hence absolute roll and pitch), high-rate motion between frames, and robustness during brief visual outages. Scale becomes observable because the accelerometer measures acceleration in $\mathrm{m/s^2}$: scaling a trajectory by s scales its acceleration by s, which the accelerometer would notice. The exception is unaccelerated motion. At constant velocity, the scale is unobservable again, which is one of several *degenerate motions* studied in Chapter 29.

Even fused, four directions remain unobservable: the three components of global position, and the rotation about the gravity vector (global yaw). Neither sensor senses where the origin is or which way north is. VIO estimates these relative to the starting pose, and they drift.

1.4 Why fusion is hard

The difficulty of VIO comes from several sources acting at once.

1. **Nonlinearity.** Rotations enter the equations through products and trigonometric functions, and the camera projection divides by depth.
2. **Curved state spaces.** Orientations do not live in a vector space. You cannot add two rotations, and every estimator must deal with this (Chapter 8).

3. **Multi-rate, asynchronous data.** The IMU runs at 200 Hz and the camera at 20 Hz, and every measurement must be processed in timestamp order.
4. **Unknown scene.** The 3D positions of the visual features are unknown and must be estimated, or eliminated, along with the motion.
5. **Outliers.** Feature tracks fail, latch onto moving objects, or slide along edges.
6. **Unobservability.** Some directions of the state cannot be determined. A careless estimator can become falsely confident about them, which is the consistency problem of Chapter 29.
7. **Real time.** All of this must run at camera rate, deterministically, on embedded processors.

1.5 A map of the field

VIO algorithms differ along three main axes.

Loosely vs. tightly coupled. A loosely coupled system runs a vision-only pipeline that produces up-to-scale poses and fuses those poses with the IMU. A tightly coupled system feeds the raw visual measurements (feature positions in the image) directly into a single estimator together with the inertial data. Tight coupling is more accurate and more robust, because the IMU helps the vision part and correlations are handled correctly. All modern high-accuracy systems, and this project, are tightly coupled.

Filtering vs. optimization. *Filtering* approaches (EKF-based) process each measurement once, keep a Gaussian belief over a bounded state, and discard the past. They are fast and have predictable cost. *Optimization* approaches solve a nonlinear least-squares problem over a sliding window of recent states, relinearizing as needed, and summarize discarded states as a prior (marginalization). They are typically more accurate per frame but more expensive and more complex. The two families are closer than they appear: the MSCKF is equivalent to a single Gauss–Newton iteration over a sliding window of poses with the landmarks marginalized (Chapter 26).

Feature-based vs. direct. Feature-based systems extract and track sparse keypoints and use their image coordinates as measurements. Direct systems use pixel intensities themselves (photometric error).

1.6 Why the MSCKF

In EKF-SLAM every tracked landmark adds three variables to the state. Covariance updates cost $O(n^2)$ to $O(n^3)$ in the state size n, so a few hundred landmarks quickly exhaust a real-time budget, and landmark initialization from a single camera is awkward because depth is unknown at first sight.

System	Family	Notes
EKF-SLAM / MonoSLAM [11]	filter, landmarks in state	cost grows quadratically with landmarks
MSCKF [40, 34]	filter, poses in state	landmarks eliminated by nullspace projection; this project
MSCKF-VIO [51]	filter (stereo MSCKF)	open-source, fast flight; good engineering reference
OpenVINS [18]	filter (MSCKF + SLAM features)	research platform with excellent documentation
ROVIO [6]	filter, direct	photometric patches in an iterated EKF
OKVIS [33]	optimization, keyframes	sliding-window bundle adjustment
VINS-Mono [42]	optimization	preintegration, robust initialization, loop closure
ORB-SLAM3 [9]	optimization + map	full visual-inertial SLAM with relocalization

Table 1.2: Representative visual-inertial estimators.

The **Multi-State Constraint Kalman Filter** of Mourikis and Roumeliotis [40] keeps instead a sliding window of past IMU poses, called *clones*, taken at image times. When a feature has been tracked across several clones, its observations constrain those poses relative to one another. The feature's 3D position is triangulated temporarily, then algebraically eliminated from the linearized equations by projecting onto the left nullspace of its Jacobian (Chapter 26). The landmark never enters the state. The result:

- cost linear in the number of features and bounded by the window size;
- no need to initialize landmark depth inside the filter;
- all the geometric information of each feature track is used exactly once.

These properties make the MSCKF the canonical filter-based VIO and an excellent vehicle for learning, because it requires every ingredient of the field to be understood and nothing more.

1.7 The estimator at a glance

Figure 1.2 shows the whole estimator. Between images, each IMU sample advances the nominal state with the nonlinear kinematics and grows the error covariance. When an image arrives, the filter propagates exactly to the image timestamp, appends a clone of the current pose, and updates the feature tracks. Tracks that have ended, or whose oldest observation is about to leave the window, are triangulated and converted into constraints between clones. The update corrects the IMU state and all clones together, the correction is injected into the nominal state, and the oldest clone is removed when the window is full. Chapter 27 builds a complete miniature of this pipeline for a robot moving in a plane, small enough to derive by hand and to run as a short Python script; it is the place to look whenever the general chapters feel abstract.

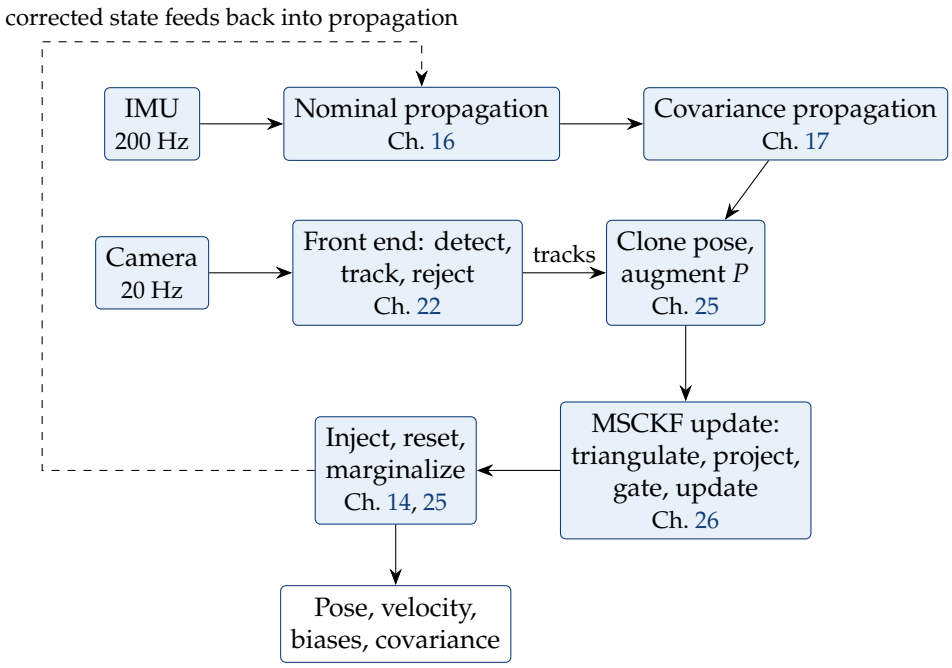


Figure 1.2: Data flow of the monocular MSCKF built in these notes.

Project link: the road map

The project builds this pipeline from the bottom up: the Lie group toolkit (M2), the data stream (M3), inertial propagation (M4–M5), a filter tested against synthetic pose measurements (M6–M7), the visual front end (M8–M9), and finally the MSCKF (M10a–M10b). Each layer is tested in isolation before the next is added, so that any later failure can be localized.

1.8 The EuRoC benchmark

The European Robotics Challenge (EuRoC) MAV dataset [8] is the standard VIO benchmark. A hexacopter carries a stereo pair of global-shutter cameras (752×480, 20 Hz) and an ADIS16448 MEMS IMU (200 Hz), hardware-synchronized and calibrated. Eleven sequences fly through an industrial machine hall (MH_01 to MH_05) and a motion-capture room (V1_01 to V2_03), graded easy, medium and difficult by speed, lighting and texture. Ground truth comes from a laser tracker or a motion-capture system. Appendix D documents the file formats and calibration values the project uses.

Key takeaways

- VIO estimates pose, velocity and IMU biases from an IMU and a camera, with no map and no absolute positioning, and its error grows without bound.
- Inertial drift is dominated by attitude error: a gyro bias produces position error growing like t^3.
- A monocular camera observes the scene only up to scale. The accelerometer supplies the scale, provided the body accelerates.

- Global position and yaw remain unobservable even with both sensors.
- The MSCKF keeps a sliding window of past poses rather than landmarks, which bounds its cost while using every feature track.

Exercises

1.1 A MEMS accelerometer has a bias of 5 mg ($1\,\text{mg} = 9.81 \times 10^{-3}\ \text{m/s}^2$). How long can you dead-reckon before the position error exceeds 1 m? What gyro bias produces the same error at the same time?

1.2 Prove that the gravity leak produced by a constant tilt error $\delta\theta_0$ is $g\,\delta\theta_0$ to first order. What is the exact expression, and at what tilt does the first-order formula err by 1%?

1.3 Extend Proposition 1.1: show that applying a common rigid transformation (R, t) to all camera poses and points also leaves the images unchanged. How many unobservable degrees of freedom does a monocular camera have? Which of them does an IMU remove, and why?

1.4 A drone hovers perfectly still for 30 s. Which states can a VIO system estimate during that time, and which cannot be distinguished from each other? (Revisit this after Chapter 29.)

1.5 Read the abstract and introduction of [40]. In two sentences, state the problem with EKF-SLAM that the paper claims to solve, and the key idea of the solution.

Part I

Mathematical Foundations

Chapter 2

Vectors, Matrices and Linear Maps

In this chapter

- Build linear algebra from scratch: vectors, vector spaces, bases, coordinates, inner products and the cross product.
- Understand a matrix as a linear map, matrix multiplication as composition, and the transpose, inverse, rank and determinant.
- Master the cross-product matrix $[a]_\times$ and the identities used in every rotation derivation.
- Understand orthogonal matrices, eigenvalues and eigenvectors, and prove that every 3D rotation has an axis.
- Understand symmetric and positive-definite matrices, quadratic forms and their ellipsoids, the language in which uncertainty is expressed.

Linear algebra is the grammar of state estimation. States are vectors, rotations and covariances are matrices, sensor models are linearized into matrices, and every filter step is a handful of matrix products and solutions of linear systems. This chapter develops the subject from first principles with the geometric meaning of each concept in view. Chapter 3 then turns to the numerical algorithms that make these operations reliable on a computer.

2.1 Vectors

2.1.1 Arrows and lists of numbers

A *vector* is first of all a geometric object: an arrow with a length and a direction, such as a displacement, a velocity or the direction of gravity. Two arrows with the same length and direction are the same vector, wherever they are drawn. Vectors can be *added* (place them head to tail; the sum goes from the first tail to the last head) and *scaled* (stretch by a real factor; a negative factor reverses the direction).

To compute, we pick three mutually perpendicular unit arrows e_1, e_2, e_3, the *axes*, and write every vector as a combination $v = v_1 e_1 + v_2 e_2 + v_3 e_3$. The three numbers (v_1, v_2, v_3) are the *coordinates* of v, and we stack them into a *column*

$$v = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} \in \mathbb{R}^3. \tag{2.1}$$

Addition and scaling act coordinate by coordinate: $(u + v)_i = u_i + v_i$ and $(\alpha v)_i = \alpha v_i$. More generally $\mathbb{R}^n$ is the set of columns of n real numbers. A state vector of a navigation filter,

such as the 15 numbers of an IMU error state, lives in $\mathbb{R}^{15}$, even though it cannot be drawn.

Key idea: a vector is not its coordinates

The same physical arrow has different coordinates in different sets of axes. The velocity of a drone is one arrow, but its coordinates in the world frame and in the body frame differ. Throughout these notes a subscript names the frame: v_w and v_b are the coordinates of the same vector in the world and body frames. Forgetting which frame a coordinate vector refers to is the most common bug in navigation software.

2.1.2 Vector spaces

Many objects besides arrows can be added and scaled: polynomials, functions, matrices. The abstraction that captures all of them is the vector space.

Definition 2.1 (Vector space)**.** A *vector space* over $\mathbb{R}$ is a set V with an addition $u + v$ and a scalar multiplication αv ($\alpha \in \mathbb{R}$) such that, for all $u, v, w \in V$ and $\alpha, \beta \in \mathbb{R}$: $u + v = v + u$; $(u + v) + w = u + (v + w)$; there is a zero vector with $v + 0 = v$; every v has a negative $-v$ with $v + (-v) = 0$; $\alpha(u + v) = \alpha u + \alpha v$; $(\alpha + \beta)v = \alpha v + \beta v$; $\alpha(\beta v) = (\alpha\beta)v$; and $1v = v$.

Examples: $\mathbb{R}^n$; the set of all $m \times n$ matrices; the set of polynomials of degree at most d; and, of great importance later, the set of 3×3 *skew-symmetric* matrices ($S^\mathsf{T} = -S$), which is closed under addition and scaling and will turn out to be the "tangent space" of the rotation group (Chapter 8). A non-example: the set of unit vectors in $\mathbb{R}^3$ (the sum of two unit vectors is usually not a unit vector). Likewise, the set of rotation matrices is *not* a vector space, which is the root of most of the difficulties of Chapters 7 and 8.

A *subspace* is a subset of a vector space that is itself a vector space under the same operations; it suffices to check that it contains 0 and is closed under addition and scaling. Lines and planes through the origin are the subspaces of $\mathbb{R}^3$; a line that misses the origin is not.

2.1.3 Linear combinations, span and independence

Definition 2.2. A *linear combination* of $v_1, \dots, v_k$ is a vector $\alpha_1 v_1 + \cdots + \alpha_k v_k$. The *span* of the v_i is the set of all their linear combinations; it is a subspace. The vectors are *linearly independent* if the only combination that gives zero is the trivial one:

$$\alpha_1 v_1 + \cdots + \alpha_k v_k = 0 \quad \Longrightarrow \quad \alpha_1 = \cdots = \alpha_k = 0. \tag{2.2}$$

Otherwise they are *linearly dependent*: one of them is a combination of the others.

In $\mathbb{R}^3$: two vectors are dependent exactly when they are parallel; three vectors are dependent exactly when they lie in a common plane through the origin. Testing independence means solving a homogeneous linear system, which Section 2.5 shows how to do.

2.1.4 Basis, dimension and coordinates

Definition 2.3 (Basis)**.** A *basis* of a vector space V is a linearly independent set of vectors that spans V.

Theorem 2.4 (Dimension)**.** *All bases of a vector space have the same number of elements, called its* dimension.

We omit the proof, which is a careful counting argument (a set of k independent vectors cannot lie in the span of fewer than k vectors). $\mathbb{R}^n$ has dimension n, the $m \times n$ matrices have dimension mn, and the 3×3 skew-symmetric matrices have dimension three (three free entries), the same as the number of degrees of freedom of a rotation.

Proposition 2.5 (Coordinates are unique)**.** *If $b_1, \dots, b_n$ is a basis, every $v \in V$ can be written as $v = \sum_i c_i b_i$ in exactly one way. The numbers c_i are the coordinates of v in that basis.*

Proof. Existence holds because the basis spans V. If also $v = \sum_i c'_i b_i$, subtracting gives $\sum_i (c_i - c'_i) b_i = 0$, and independence forces $c_i = c'_i$. ∎

Changing the basis changes the coordinates of every vector but not the vectors. Section 2.7 computes how coordinates change when one orthonormal basis is replaced by another, which is exactly what a rotation matrix between two frames does.

2.2 Inner products, lengths and angles

2.2.1 The dot product

Definition 2.6 (Dot product and norm)**.** The *dot product* (inner product) of $a, b \in \mathbb{R}^n$ is $a^{\mathsf{T}} b = a \cdot b = \sum_i a_i b_i$, and the (Euclidean) *norm* or length is $\|a\| = \sqrt{a^{\mathsf{T}} a}$. A *unit vector* has norm one; $a/\|a\|$ *normalizes* a non-zero a.

The dot product is symmetric ($a^{\mathsf{T}} b = b^{\mathsf{T}} a$), linear in each argument, and positive ($a^{\mathsf{T}} a > 0$ for $a \neq 0$). Its geometric meaning comes from the law of cosines.

Proposition 2.7 (Geometric dot product)**.** *If θ is the angle between non-zero a and b, then $a^{\mathsf{T}} b = \|a\| \|b\| \cos\theta$.*

Proof. The triangle with sides a, b and $a - b$ satisfies the law of cosines $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2\|a\|\|b\|\cos\theta$. Expanding the left side with bilinearity gives $\|a\|^2 - 2a^{\mathsf{T}} b + \|b\|^2$. Comparing the two expressions proves the claim. ∎

Vectors are *orthogonal* ($a \perp b$) when $a^{\mathsf{T}} b = 0$. For orthogonal vectors the Pythagorean theorem $\|a + b\|^2 = \|a\|^2 + \|b\|^2$ follows by expanding the left side.

Theorem 2.8 (Cauchy–Schwarz and triangle inequalities)**.** *For all $a, b \in \mathbb{R}^n$, $|a^{\mathsf{T}} b| \leq \|a\|\|b\|$ and $\|a + b\| \leq \|a\| + \|b\|$.*

Proof. For $b = 0$ both are trivial. Otherwise, for every real t, $0 \leq \|a - tb\|^2 = \|a\|^2 - 2t\, a^{\mathsf{T}} b + t^2 \|b\|^2$. A quadratic in t that is never negative has a non-positive discriminant: $4(a^{\mathsf{T}} b)^2 - 4\|a\|^2\|b\|^2 \leq 0$, which is the first inequality. Then $\|a + b\|^2 = \|a\|^2 + 2a^{\mathsf{T}} b + \|b\|^2 \leq \|a\|^2 + 2\|a\|\|b\| + \|b\|^2 = (\|a\| + \|b\|)^2$. ∎

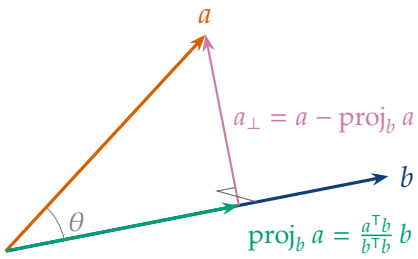


Figure 2.1: Orthogonal projection of a onto the direction of b. The remainder $a_\perp$ is orthogonal to b, and the projection is the closest point to a on the line spanned by b.

2.2.2 Orthonormal bases and projections

A basis $e_1, \dots, e_n$ is *orthonormal* if $e_i^\mathsf{T} e_j = 1$ for $i = j$ and 0 otherwise. In an orthonormal basis, coordinates are simply dot products: dotting $v = \sum_j c_j e_j$ with e_i leaves

$$c_i = e_i^\mathsf{T} v. \tag{2.3}$$

This is why navigation always uses orthonormal (Cartesian) axes: finding coordinates requires no equation solving.

Proposition 2.9 (Projection onto a direction)**.** *For $b \neq 0$, the point of the line $\{tb\}$ closest to a is $\mathrm{proj}_b a = \frac{a^\mathsf{T} b}{b^\mathsf{T} b} b$, and the remainder $a - \mathrm{proj}_b a$ is orthogonal to b (Figure 2.1).*

Proof. $f(t) = \|a - tb\|^2 = \|a\|^2 - 2t\, a^\mathsf{T} b + t^2 \|b\|^2$ is a parabola minimized where $f'(t) = 0$, i.e. $t = a^\mathsf{T} b / b^\mathsf{T} b$. Then $b^\mathsf{T}(a - tb) = a^\mathsf{T} b - t\, b^\mathsf{T} b = 0$. ∎

Every vector therefore splits into a component along b and a component perpendicular to it. This decomposition is the heart of Rodrigues' rotation formula (Chapter 7): a rotation leaves the component along its axis alone and turns the perpendicular component.

Gram–Schmidt. Any basis $a_1, \dots, a_n$ can be turned into an orthonormal basis $q_1, \dots, q_n$ spanning the same nested subspaces, by subtracting from each vector its projections onto the previous ones and normalizing:

$$\tilde{q}_k = a_k - \sum_{j<k} (q_j^\mathsf{T} a_k)\, q_j, \qquad q_k = \tilde{q}_k / \|\tilde{q}_k\|. \tag{2.4}$$

By construction $\tilde{q}_k$ is orthogonal to $q_1, \dots, q_{k-1}$, and it cannot be zero because a_k is not in the span of the previous vectors. Recording the coefficients $q_j^\mathsf{T} a_k$ gives the QR factorization of Chapter 3.

2.3 The cross product

2.3.1 Definition and geometry

Definition 2.10 (Cross product). For $a, b \in \mathbb{R}^3$,

$$a \times b = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}. \tag{2.5}$$

A convenient mnemonic is the formal determinant with rows (e_1, e_2, e_3), (a_1, a_2, a_3) and (b_1, b_2, b_3). The cross product is *bilinear*, *anti-commutative* ($a \times b = -b \times a$, so $a \times a = 0$), and orthogonal to both factors: $a^\mathsf{T}(a \times b) = 0$, which can be verified by expanding.

Proposition 2.11 (Length of the cross product). *$\|a \times b\|^2 = \|a\|^2\|b\|^2 - (a^\mathsf{T} b)^2$ (Lagrange's identity), hence $\|a \times b\| = \|a\|\|b\| \sin\theta$, the area of the parallelogram spanned by a and b.*

Proof. Expand both sides in coordinates; all terms match. Then substitute $a^\mathsf{T} b = \|a\|\|b\| \cos\theta$ and use $1 - \cos^2\theta = \sin^2\theta$. ∎

The direction of $a \times b$ follows the *right-hand rule*: with the fingers of the right hand curling from a to b, the thumb points along $a \times b$. In particular $e_1 \times e_2 = e_3$, $e_2 \times e_3 = e_1$ and $e_3 \times e_1 = e_2$. A coordinate system obeying these relations is *right-handed*; all frames in these notes are.

The *scalar triple product* $a^\mathsf{T}(b \times c)$ is the signed volume of the parallelepiped spanned by a, b and c; it equals the determinant of the matrix with columns a, b, c (Section 2.6), is unchanged by cyclic permutations, and changes sign when two vectors are swapped. The *vector triple product* obeys the "BAC–CAB" rule

$$a \times (b \times c) = b\,(a^\mathsf{T} c) - c\,(a^\mathsf{T} b), \tag{2.6}$$

which can be checked in coordinates (it suffices to check basis vectors, by linearity).

2.3.2 The cross-product matrix

Because $a \times b$ is linear in b, it is a matrix-vector product (Section 2.4).

Definition 2.12 (Skew-symmetric matrix). For $a = (a_1, a_2, a_3)^\mathsf{T}$, the *cross-product matrix* (or skew-symmetric matrix) of a is

$$[a]_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \qquad \text{so that} \qquad [a]_\times b = a \times b \quad \forall\, b \in \mathbb{R}^3. \tag{2.7}$$

It is skew-symmetric: $[a]_\times^\mathsf{T} = -[a]_\times$. Conversely, every 3×3 skew-symmetric matrix S equals $[s]_\times$ for the unique vector $s = (S_{32}, S_{13}, S_{21})^\mathsf{T}$; we write $s = S^\vee$ for this inverse operation, the *vee* map. In Lie theory $[\cdot]_\times$ is also written $(\cdot)^\wedge$ and called the *hat* map.

The following identities are used constantly in derivations; learn to recognize them on sight.

Proposition 2.13 (Cross-product identities). *For $a, b, c \in \mathbb{R}^3$ and a rotation matrix R (Sec-*

tion 2.7):

$$[a]_\times b = -[b]_\times a, \tag{2.8}$$
$$[a]_\times a = 0, \tag{2.9}$$
$$[a]_\times^2 = aa^\mathsf{T} - \|a\|^2 I, \tag{2.10}$$
$$[a]_\times^3 = -\|a\|^2 [a]_\times, \tag{2.11}$$
$$[a]_\times [b]_\times = ba^\mathsf{T} - (a^\mathsf{T} b)\, I, \tag{2.12}$$
$$[a \times b]_\times = [a]_\times [b]_\times - [b]_\times [a]_\times = ba^\mathsf{T} - ab^\mathsf{T}, \tag{2.13}$$
$$[Ra]_\times = R[a]_\times R^\mathsf{T}, \tag{2.14}$$
$$R\,(a \times b) = (Ra) \times (Rb). \tag{2.15}$$

Proof. (2.8) and (2.9) restate anti-commutativity. For (2.12), apply both sides to an arbitrary c and use (2.6): $[a]_\times [b]_\times c = a \times (b \times c) = b\, a^\mathsf{T} c - c\, (a^\mathsf{T} b)$. Setting $b = a$ gives (2.10), and multiplying (2.10) on the left by $[a]_\times$ and using (2.9) gives (2.11). Subtracting (2.12) from its version with a and b swapped gives the second equality of (2.13); applying $ba^\mathsf{T} - ab^\mathsf{T}$ to c and using (2.6) once more shows it equals $[a \times b]_\times c$. For (2.15), the cross product is defined by lengths, angles and orientation, all preserved by a rotation (Section 2.7). Finally (2.14) follows from (2.15): $[Ra]_\times c = (Ra) \times (RR^\mathsf{T} c) = R\,(a \times R^\mathsf{T} c) = R[a]_\times R^\mathsf{T} c$. ■

Key idea: moving a vector out of a cross product

Identity (2.8) is how unknowns are isolated in Jacobian derivations. A small rotation error $\delta\theta$ typically appears as $[\delta\theta]_\times v$; rewriting it as $-[v]_\times \delta\theta$ makes it a matrix times the unknown, and the matrix $-[v]_\times$ is the Jacobian.

Identity (2.11) is what makes closed-form rotation formulas possible: every power of $[a]_\times$ reduces to a multiple of $[a]_\times$ or $[a]_\times^2$, so infinite series in $[a]_\times$ collapse to two terms (Chapter 7).

2.4 Matrices as linear maps

2.4.1 Matrices and the matrix-vector product

An $m \times n$ *matrix* A is a rectangular array of numbers A_{ij} with m rows and n columns. Its product with $x \in \mathbb{R}^n$ is the vector $Ax \in \mathbb{R}^m$ with entries $(Ax)_i = \sum_j A_{ij} x_j$. There are two ways to read this product, and both are indispensable:

$$Ax = \begin{bmatrix} r_1^\mathsf{T} x \\ \vdots \\ r_m^\mathsf{T} x \end{bmatrix} \quad \text{(dot products with the rows } r_i^\mathsf{T}\text{)}, \tag{2.16}$$

$$Ax = x_1 a_1 + x_2 a_2 + \cdots + x_n a_n \quad \text{(combination of the columns } a_j\text{)}.$$

The column view says that the possible outputs Ax are exactly the linear combinations of the columns of A.

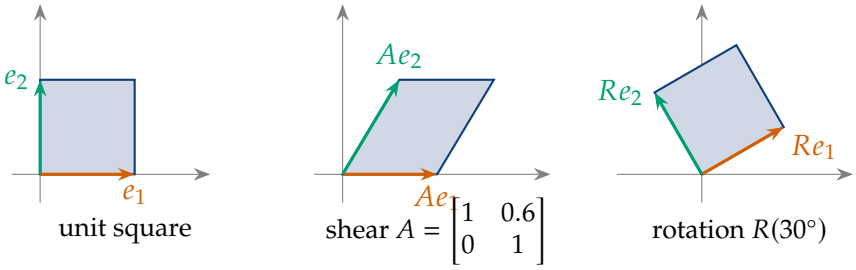


Figure 2.2: A linear map is determined by the images of the basis vectors, which are the columns of its matrix. The unit square becomes a parallelogram; its area is scaled by the determinant (1 for both maps shown).

2.4.2 Linear maps

Definition 2.14 (Linear map)**.** A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is *linear* if $f(u+v) = f(u) + f(v)$ and $f(\alpha u) = \alpha f(u)$ for all vectors u, v and scalars α.

Theorem 2.15 (Linear maps are matrices)**.** *Every linear map $f : \mathbb{R}^n \to \mathbb{R}^m$ is multiplication by a unique $m \times n$ matrix, whose j-th column is $f(e_j)$, the image of the j-th basis vector.*

Proof. Write $x = \sum_j x_j e_j$. By linearity $f(x) = \sum_j x_j f(e_j)$, which is the column view of Ax with columns $a_j = f(e_j)$. Uniqueness: two matrices that agree on all basis vectors have the same columns. ∎

This theorem turns geometric questions into matrix questions: to find the matrix of a geometric operation, ask where it sends each basis vector.

Example 2.16 (Rotation in the plane)**.** Rotating the plane counter-clockwise by θ sends $e_1 = (1, 0)$ to $(\cos\theta, \sin\theta)$ and $e_2 = (0, 1)$ to $(-\sin\theta, \cos\theta)$. Rotation is linear (it preserves parallelograms), so its matrix is

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}. \tag{2.17}$$

2.4.3 Matrix multiplication is composition

If B is $m \times n$ and A is $p \times m$, applying B and then A is again linear, and its matrix is the *matrix product* AB: $(AB)x = A(Bx)$. Its j-th column is A applied to the j-th column of B, and its entries are

$$(AB)_{ik} = \sum_j A_{ij} B_{jk}. \tag{2.18}$$

Because composition of functions is associative, so is matrix multiplication: $(AB)C = A(BC)$. It is *not* commutative: rotating and then shearing is not the same as shearing and then rotating. For rotations in three dimensions non-commutativity is unavoidable and is the defining difficulty of attitude estimation. The *identity matrix* I, with ones on the diagonal, leaves every vector unchanged: $AI = IA = A$.

Reading a product of transformations. In ABx, the transformation applied first is the one closest to x, here B. With frame-annotated rotations (Chapter 7), $R_{wb}R_{bc}x_c$ first expresses the camera vector x_c in the body frame and then in the world frame.

2.4.4 Transpose and special matrices

The *transpose* A^T swaps rows and columns: $(A^\mathsf{T})_{ij} = A_{ji}$. The dot product is a matrix product, $a^\mathsf{T}b$, and transposition reverses products:

$$(AB)^\mathsf{T} = B^\mathsf{T}A^\mathsf{T}, \tag{2.19}$$

because $((AB)^\mathsf{T})_{ki} = (AB)_{ik} = \sum_j A_{ij}B_{jk} = \sum_j (B^\mathsf{T})_{kj}(A^\mathsf{T})_{ji}$. A matrix is *symmetric* if $A^\mathsf{T} = A$ and *skew-symmetric* if $A^\mathsf{T} = -A$; every square matrix splits uniquely as $A = \frac{1}{2}(A + A^\mathsf{T}) + \frac{1}{2}(A - A^\mathsf{T})$, a symmetric plus a skew-symmetric part. Covariance matrices are symmetric; cross-product matrices are skew-symmetric.

Other matrices with names: *diagonal* matrices (non-zero entries only on the diagonal), *upper* and *lower triangular* matrices (zeros below or above the diagonal), and the *outer product* ab^T of two vectors, an $m \times n$ matrix with entries $a_i b_j$ whose columns are all multiples of a.

Block matrices. Matrices can be partitioned into blocks, and blocks multiply like entries as long as their sizes are compatible:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}. \tag{2.20}$$

Filters are written almost entirely in block form: the IMU state and the clones, position and attitude, poses and landmarks.

2.5 Linear systems, inverses and rank

2.5.1 Solving linear systems

Given A and b, find x with $Ax = b$. In the column view: find the combination of the columns of A that produces b. Three things can happen: exactly one solution, none (b is not a combination of the columns), or infinitely many (some combination of the columns gives zero, and it can be added to any solution).

Gaussian elimination solves the system by *row operations* that do not change the solution set: swapping two equations, scaling an equation by a non-zero number, and subtracting a multiple of one equation from another. They are used to create zeros below the diagonal (the *pivots*) column by column, after which the equations are solved from the bottom up by *back substitution*.

Example 2.17 (Elimination).

$$\begin{aligned} 2x_1 + x_2 + x_3 &= 5 \\ 4x_1 + 3x_2 + 3x_3 &= 13 \\ 2x_1 + 3x_2 + 4x_3 &= 13 \end{aligned} \xrightarrow{\text{(2)}-2\text{(1)},\ \text{(3)}-\text{(1)}} \begin{aligned} 2x_1 + x_2 + x_3 &= 5 \\ x_2 + x_3 &= 3 \\ 2x_2 + 3x_3 &= 8 \end{aligned} \xrightarrow{\text{(3)}-2\text{(2)}} \begin{aligned} 2x_1 + x_2 + x_3 &= 5 \\ x_2 + x_3 &= 3 \\ x_3 &= 2 \end{aligned}$$

Back substitution gives $x_3 = 2$, $x_2 = 1$ and $x_1 = 1$. The multipliers $2, 1, 2$ used along the way are the entries of the lower-triangular factor L of an LU factorization $A = LU$ (Chapter 3).

2.5.2 The inverse

A square matrix A is *invertible* if there is a matrix A^{-1} with $AA^{-1} = A^{-1}A = I$. Then $Ax = b$ has the unique solution $x = A^{-1}b$. Useful facts, each verified by multiplying out:

$$(AB)^{-1} = B^{-1}A^{-1}, \qquad (A^{\mathsf{T}})^{-1} = (A^{-1})^{\mathsf{T}}, \qquad \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}. \tag{2.21}$$

For a square matrix the following are equivalent: A is invertible; $Ax = 0$ only for $x = 0$; the columns are independent; elimination produces n non-zero pivots; $\det A \neq 0$ (Section 2.6). In numerical work one almost never computes A^{-1} explicitly; Chapter 3 explains why and what to do instead.

2.5.3 Range, nullspace and rank

Definition 2.18. For $A \in \mathbb{R}^{m \times n}$: the *range* (column space) $\operatorname{range}(A) = \{Ax\} \subseteq \mathbb{R}^m$; the *nullspace* $\operatorname{null}(A) = \{x : Ax = 0\} \subseteq \mathbb{R}^n$; and the *rank* $\operatorname{rank} A = \dim \operatorname{range}(A)$.

The rank is the number of independent columns, and also the number of pivots produced by elimination. A surprising and useful fact is that row rank equals column rank: $\operatorname{rank} A = \operatorname{rank} A^{\mathsf{T}}$.

Theorem 2.19 (Rank–nullity). *For $A \in \mathbb{R}^{m \times n}$, $\dim \operatorname{null}(A) + \operatorname{rank} A = n$.*

Proof sketch. Choose a basis $u_1, \dots, u_k$ of $\operatorname{null}(A)$ and extend it with $w_1, \dots, w_r$ to a basis of $\mathbb{R}^n$, so $k + r = n$. The vectors $Aw_1, \dots, Aw_r$ span $\operatorname{range}(A)$ (the Au_i vanish), and they are independent: if $\sum c_i Aw_i = 0$ then $\sum c_i w_i \in \operatorname{null}(A)$, which forces all $c_i = 0$ because the u's and w's together are independent. So $\operatorname{rank} A = r = n - k$. ∎

Each dimension of the nullspace is a direction of x that the map destroys. In estimation, the nullspace of a measurement Jacobian H is the set of state errors the measurement cannot see (Chapter 18), and the nullspace of an observability matrix is the set of state directions no measurement history can determine (Chapters 11 and 29).

2.5.4 The four fundamental subspaces

Each $A \in \mathbb{R}^{m \times n}$ of rank r has four associated subspaces:

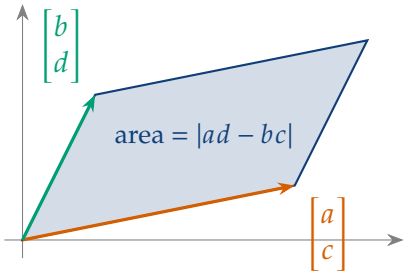


Figure 2.3: The determinant of a 2×2 matrix is the signed area of the parallelogram spanned by its columns; the sign is positive when the second column lies counter-clockwise from the first.

Subspace	Definition	Lives in	Dimension
column space (range)	$\text{range}(A) = \{Ax\}$	$\mathbb{R}^m$	r
nullspace	$\text{null}(A) = \{x : Ax = 0\}$	$\mathbb{R}^n$	$n - r$
row space	$\text{range}(A^\mathsf{T})$	$\mathbb{R}^n$	r
left nullspace	$\text{null}(A^\mathsf{T}) = \{y : y^\mathsf{T}A = 0\}$	$\mathbb{R}^m$	$m - r$

Theorem 2.20 (Fundamental theorem of linear algebra)**.** $\text{null}(A) = \text{range}(A^\mathsf{T})^\perp$ *and* $\text{null}(A^\mathsf{T}) = \text{range}(A)^\perp$*, where* $S^\perp$ *denotes the set of vectors orthogonal to every vector of* S*. Consequently every* $y \in \mathbb{R}^m$ *splits uniquely as* $y = y_r + y_n$ *with* $y_r \in \text{range}(A)$ *and* $y_n \in \text{null}(A^\mathsf{T})$*,* $y_r \perp y_n$*.*

Proof. $y \in \text{null}(A^\mathsf{T}) \iff A^\mathsf{T}y = 0 \iff x^\mathsf{T}A^\mathsf{T}y = 0 \;\forall x \iff (Ax)^\mathsf{T}y = 0 \;\forall x \iff y \perp \text{range}(A)$. The first statement is the same argument applied to A^T. The decomposition follows by projecting y onto $\text{range}(A)$ (Chapter 3). ∎

The *left nullspace* deserves special attention because the MSCKF is built on it. Suppose a stacked measurement depends on an unwanted variable f through a tall matrix $H_f \in \mathbb{R}^{m\times 3}$ of full column rank:

$$r = H_x\,\delta x + H_f\,\delta f + n. \tag{2.22}$$

Take any matrix A whose $m - 3$ columns form an orthonormal basis of $\text{null}(H_f^\mathsf{T})$. Then $A^\mathsf{T}H_f = 0$, and multiplying through by A^T gives $A^\mathsf{T}r = A^\mathsf{T}H_x\,\delta x + A^\mathsf{T}n$: the unwanted variable has disappeared. Chapter 26 proves that no information about δx is lost.

2.6 Determinants

2.6.1 Determinants as signed volumes

A linear map sends the unit square (cube) to a parallelogram (parallelepiped). The *determinant* is the factor by which it scales areas (volumes), with a sign that records whether it preserves or reverses orientation. For a 2×2 matrix the parallelogram spanned by the columns (a, c) and (b, d) has signed area $ad - bc$ (Figure 2.3); for a 3×3 matrix with columns a_1, a_2, a_3 the

signed volume is the triple product:

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc, \qquad \det \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} = a_1^{\mathsf{T}}(a_2 \times a_3). \tag{2.23}$$

2.6.2 Definition and properties

In general, the determinant is the unique function of the columns of a square matrix that is

1. linear in each column separately (*multilinear*),
2. zero whenever two columns are equal (*alternating*; equivalently, it changes sign when two columns are swapped), and
3. normalized: $\det I = 1$.

These three rules determine everything. Adding a multiple of one column to another does not change the determinant (by linearity, the change is a determinant with two equal columns, which is zero); hence the same holds for rows, and elimination reduces any determinant to that of a triangular matrix, which is the product of its diagonal entries. Expanding along the first row gives the *cofactor* (Laplace) formula

$$\det A = \sum_{j=1}^{n} (-1)^{1+j} A_{1j} \det M_{1j}, \tag{2.24}$$

where M_{1j} deletes row 1 and column j. It is convenient for 3×3 matrices by hand and hopeless for large ones, whose determinants are computed from an LU or Cholesky factorization.

Proposition 2.21 (Properties of the determinant)**.** *For square A and B of the same size:*

1. $\det(AB) = \det A \, \det B$ *(volume scaling factors multiply under composition);*
2. $\det A^{\mathsf{T}} = \det A$*;*
3. *A is invertible if and only if $\det A \neq 0$, and then $\det A^{-1} = 1/\det A$;*
4. *the determinant of a triangular or diagonal matrix is the product of its diagonal entries.*

Property 3 has a clear geometric meaning: $\det A = 0$ means the map flattens space onto a lower-dimensional subspace, destroying information, so it cannot be undone. The sign of the determinant distinguishes *rotations* (which preserve orientation, $\det = +1$) from *reflections* (which reverse it, $\det = -1$); the next section makes this precise.

2.7 Orthogonal matrices and change of basis

Definition 2.22 (Orthogonal matrix)**.** A square matrix Q is *orthogonal* if $Q^{\mathsf{T}}Q = I$, i.e. its columns are orthonormal. Then $Q^{-1} = Q^{\mathsf{T}}$.

Proposition 2.23 (Orthogonal matrices preserve geometry)**.** *If Q is orthogonal then $(Qa)^{\mathsf{T}}(Qb) = a^{\mathsf{T}}b$ for all a, b: lengths and angles are preserved. Moreover $\det Q = \pm 1$.*

Proof. $(Qa)^{\mathsf{T}}(Qb) = a^{\mathsf{T}}Q^{\mathsf{T}}Qb = a^{\mathsf{T}}b$. And $1 = \det I = \det(Q^{\mathsf{T}}Q) = (\det Q)^2$. ∎

Orthogonal matrices with $\det Q = +1$ are *rotations*; those with $\det Q = -1$ are rotations combined with a reflection, which turn right-handed frames into left-handed ones. The orthogonal $n \times n$ matrices form the *orthogonal group* $\mathrm{O}(n)$, and the rotations form the *special orthogonal group* $\mathrm{SO}(n)$. Products and inverses of rotations are rotations.

Change of orthonormal basis. Let e_1, e_2, e_3 be the axes of a frame W and b_1, b_2, b_3 those of a frame B, and let R be the matrix whose columns are the coordinates of b_1, b_2, b_3 in W. A vector with coordinates v_b in B is $v = \sum_j (v_b)_j b_j$; its coordinates in W are therefore

$$v_w = \sum_j (v_b)_j \,(\text{coordinates of } b_j \text{ in } W) = R\, v_b, \qquad v_b = R^{\mathsf{T}} v_w. \tag{2.25}$$

The matrix R is orthogonal because its columns are orthonormal, and it is a rotation when both frames are right-handed. This is exactly the rotation matrix R_{wb} of Chapter 7: its columns are the body axes in world coordinates, and it maps body coordinates to world coordinates.

2.8 Eigenvalues and eigenvectors

2.8.1 Definitions

Definition 2.24. A non-zero vector v is an *eigenvector* of a square matrix A, with *eigenvalue* λ, if $Av = \lambda v$: the map only stretches v, by the factor λ.

Rewriting $Av = \lambda v$ as $(A - \lambda I)v = 0$ shows that λ is an eigenvalue exactly when $A - \lambda I$ has a non-trivial nullspace, i.e. when the *characteristic polynomial* vanishes:

$$p(\lambda) = \det(A - \lambda I) = 0. \tag{2.26}$$

For a 2×2 matrix, $p(\lambda) = \lambda^2 - (\operatorname{tr} A)\lambda + \det A$, where $\operatorname{tr} A$ is the sum of the diagonal entries (the *trace*). An $n \times n$ matrix has n eigenvalues counted with multiplicity, which may be complex even when A is real. Comparing the coefficients of $p(\lambda) = \prod_i (\lambda_i - \lambda)$ shows that

$$\operatorname{tr} A = \sum_i \lambda_i, \qquad \det A = \prod_i \lambda_i. \tag{2.27}$$

2.8.2 Complex numbers in one paragraph

A complex number $z = x + iy$ has a real part x and an imaginary part y, with $i^2 = -1$. It can be pictured as the point (x, y) of the plane; its modulus is $|z| = \sqrt{x^2 + y^2}$ and its argument is the angle of the point. Euler's formula

$$e^{i\theta} = \cos\theta + i \sin\theta \tag{2.28}$$

follows from the power series of e^x, $\cos x$ and $\sin x$ by substituting $x = i\theta$. Multiplying a complex number by $e^{i\theta}$ rotates the corresponding point by θ: complex multiplication *is* planar rotation. Quaternions (Chapter 7) generalize this idea to three dimensions.

Example 2.25 (Eigenvalues of a planar rotation)**.** For $R(\theta)$ of Example 2.16, $\operatorname{tr} R = 2\cos\theta$ and $\det R = 1$, so $p(\lambda) = \lambda^2 - 2\cos\theta\,\lambda + 1$ with roots $\lambda = \cos\theta \pm i\sin\theta = e^{\pm i\theta}$. Unless θ is a multiple of π, no real direction is merely stretched, as expected of a rotation.

2.8.3 Diagonalization

If $A \in \mathbb{R}^{n\times n}$ has n independent eigenvectors $v_1, \dots, v_n$, collect them in $V = [v_1 \cdots v_n]$ and the eigenvalues in $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$. The relations $Av_i = \lambda_i v_i$ read $AV = V\Lambda$, so

$$A = V\Lambda V^{-1}. \tag{2.29}$$

In the basis of eigenvectors, A acts by independent scalings. Powers and polynomials become trivial, $A^k = V\Lambda^k V^{-1}$, and so do the power series used to define the matrix exponential (Chapter 3). Eigenvectors of distinct eigenvalues are always independent, so a matrix with n distinct eigenvalues is diagonalizable. Not every matrix is: the shear $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ has the double eigenvalue 1 but only one eigenvector direction.

2.8.4 Every 3D rotation has an axis

Theorem 2.26 (Euler's rotation theorem)**.** *Every $R \in \mathrm{SO}(3)$ has 1 as an eigenvalue. Its eigenvalues are $1, e^{i\theta}, e^{-i\theta}$ for some $\theta \in [0, \pi]$, and $\operatorname{tr} R = 1 + 2\cos\theta$. The real eigenvector u of eigenvalue 1 is fixed by R: it is the* axis *of the rotation, and R rotates the plane orthogonal to u by the angle θ.*

Proof. Using $R^\mathsf{T}R = I$ and $\det R = 1$: $\det(R - I) = \det\left(R(I - R^\mathsf{T})\right) = \det R \det\left((I - R)^\mathsf{T}\right) = \det(I - R) = (-1)^3 \det(R - I)$, so $\det(R - I) = 0$ and 1 is an eigenvalue. Every eigenvalue has modulus one, because R preserves lengths (this holds for complex eigenvectors as well). The characteristic polynomial is real and of degree three, so non-real eigenvalues come in conjugate pairs; with one eigenvalue equal to 1 and the product of all three equal to $\det R = 1$, the other two are $e^{\pm i\theta}$, or both $+1$ or both -1. The trace formula is (2.27). Finally, R maps the plane orthogonal to u to itself (it preserves orthogonality to the fixed vector u), and it acts there as a planar rotation with eigenvalues $e^{\pm i\theta}$, i.e. by the angle θ. ∎

This theorem is the reason a rotation can always be described by an axis and an angle, the representation from which all of rotation calculus is built (Chapters 7 and 8).

2.9 Symmetric matrices and quadratic forms

Covariance matrices, information matrices and the normal equations of least squares are all symmetric. Symmetric matrices have the most beautiful eigenstructure of all.

Theorem 2.27 (Spectral theorem)**.** *A real symmetric matrix A has real eigenvalues and an orthonormal basis of eigenvectors: $A = U\Lambda U^\mathsf{T}$ with U orthogonal and Λ real diagonal.*

Proof. Real eigenvalues. Let $Av = \lambda v$ with v possibly complex and $\bar{v}$ its complex conjugate. The number $s = \bar{v}^\mathsf{T}Av$ equals $\lambda\,\bar{v}^\mathsf{T}v$. Its complex conjugate is $v^\mathsf{T}A\bar{v}$ (because A is real),

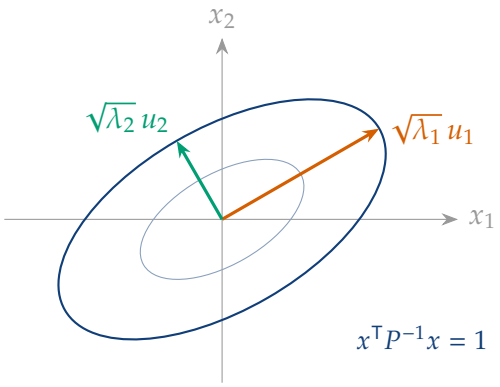


Figure 2.4: The level set $x^\mathsf{T}P^{-1}x = 1$ of a positive-definite $P = U\Lambda U^\mathsf{T}$ is an ellipse (ellipsoid) whose axes point along the eigenvectors u_i and have semi-lengths $\sqrt{\lambda_i}$. For a covariance matrix this is the one-sigma uncertainty ellipse.

and a 1×1 matrix equals its own transpose, so $\bar{s} = \bar{v}^\mathsf{T}A^\mathsf{T}v = \bar{v}^\mathsf{T}Av = s$: s is real. Since $\bar{v}^\mathsf{T}v = \sum_i |v_i|^2 > 0$, $\lambda = s/\bar{v}^\mathsf{T}v$ is real too. *Orthogonality.* If $Av = \lambda v$ and $Aw = \mu w$ with $\lambda \neq \mu$, then $\lambda\, w^\mathsf{T}v = w^\mathsf{T}Av = (Aw)^\mathsf{T}v = \mu\, w^\mathsf{T}v$, so $w^\mathsf{T}v = 0$. *Completeness* (sketch, by induction on n). Take a unit eigenvector u_1. The subspace $u_1^\perp$ is mapped into itself by A: if $x \perp u_1$ then $u_1^\mathsf{T}Ax = (Au_1)^\mathsf{T}x = \lambda_1 u_1^\mathsf{T}x = 0$. Restricted to this $(n-1)$-dimensional subspace, A is again symmetric and, by induction, has an orthonormal eigenbasis, which together with u_1 is an orthonormal eigenbasis of $\mathbb{R}^n$. ∎

2.9.1 Quadratic forms and definiteness

A *quadratic form* is a function $x \mapsto x^\mathsf{T}Ax$ with A symmetric. Substituting $x = Uy$, with U from the spectral theorem, decouples it into independent squares:

$$x^\mathsf{T}Ax = y^\mathsf{T}\Lambda y = \sum_i \lambda_i y_i^2. \tag{2.30}$$

Definition 2.28 (Definiteness). A symmetric matrix A is *positive definite* (PD, $A \succ 0$) if $x^\mathsf{T}Ax > 0$ for every $x \neq 0$, and *positive semi-definite* (PSD, $A \succeq 0$) if $x^\mathsf{T}Ax \geq 0$ for every x. We write $A \succeq B$ (the *Loewner order*) when $A - B \succeq 0$.

By the decoupled form, $A \succ 0$ if and only if all its eigenvalues are positive, and $A \succeq 0$ if and only if they are non-negative. Other tests: all pivots of elimination are positive; the Cholesky factorization exists (Chapter 3); or all leading principal minors are positive (Sylvester's criterion).

Ellipsoids. For $P \succ 0$ the set $\{x : x^\mathsf{T}P^{-1}x \leq 1\}$ is an ellipsoid with principal axes along the eigenvectors of P and semi-axis lengths $\sqrt{\lambda_i}$ (Figure 2.4): in the rotated coordinates $y = U^\mathsf{T}x$ it reads $\sum_i y_i^2/\lambda_i \leq 1$. When P is a covariance matrix this is the uncertainty ellipsoid of Chapter 5, and the eigen-decomposition of P tells in which directions the estimate is least certain.

Proposition 2.29 (Properties of PSD matrices)**.** *Let $P, Q \succeq 0$ and let A be any matrix of compatible size. Then*

1. *$APA^{\mathsf{T}} \succeq 0$ (a* congruence *preserves semi-definiteness);*
2. *$P + Q \succeq 0$, and $P + Q \succ 0$ if either is PD;*
3. *$P_{ii} \geq 0$ and $|P_{ij}| \leq \sqrt{P_{ii} P_{jj}}$;*
4. *every principal submatrix of P is PSD.*

Proof. (1) $x^{\mathsf{T}} APA^{\mathsf{T}} x = (A^{\mathsf{T}} x)^{\mathsf{T}} P (A^{\mathsf{T}} x) \geq 0$. (2) is immediate. (4) follows from (1) with A a selection matrix that picks some coordinates; (3) is (4) applied to 1×1 and 2×2 principal submatrices, whose determinants $P_{ii}P_{jj} - P_{ij}^2$ are non-negative. ∎

Property 1 explains why the covariance updates $\Phi P \Phi^{\mathsf{T}} + Q$ and the Joseph form (Chapter 12) preserve positive semi-definiteness exactly in exact arithmetic: they are sums of congruences of PSD matrices. The simpler update $(I - KH)P$ is not a congruence, which is why it can lose semi-definiteness under rounding.

Proposition 2.30 (Rayleigh quotient)**.** *For symmetric A with eigenvalues $\lambda_{\min} \leq \cdots \leq \lambda_{\max}$, $\lambda_{\min} \leq \dfrac{x^{\mathsf{T}} A x}{x^{\mathsf{T}} x} \leq \lambda_{\max}$ for every $x \neq 0$, with equality for the corresponding eigenvectors.*

Proof. With $y = U^{\mathsf{T}} x$, $x^{\mathsf{T}} A x / x^{\mathsf{T}} x = \sum_i \lambda_i y_i^2 / \sum_i y_i^2$, a weighted average of the eigenvalues. ∎

The Rayleigh quotient reappears in three places: minimizing $\|Ax\|^2$ over unit vectors (homogeneous least squares, Chapter 3), choosing good image features by the smallest eigenvalue of a structure tensor (Chapter 22), and bounding how much a matrix can amplify a vector (the matrix norm below).

2.10 Trace and norms

The trace is linear and invariant under cyclic permutations:

$$\operatorname{tr}(AB) = \operatorname{tr}(BA), \qquad \operatorname{tr}(ABC) = \operatorname{tr}(CAB) = \operatorname{tr}(BCA), \tag{2.31}$$

because $\operatorname{tr}(AB) = \sum_{i,j} A_{ij} B_{ji}$ is symmetric in the roles of A and B. Consequently the trace is unchanged by a change of basis, $\operatorname{tr}(V^{-1} A V) = \operatorname{tr} A$, and equals the sum of the eigenvalues. A quadratic form can be written as a trace, $x^{\mathsf{T}} A x = \operatorname{tr}(A x x^{\mathsf{T}})$, which is how expected values of quadratic forms are computed: $\mathbb{E}[x^{\mathsf{T}} A x] = \operatorname{tr}(A\, \mathbb{E}[x x^{\mathsf{T}}])$ (Chapter 5).

Vector norms other than the Euclidean one are occasionally useful: $\|x\|_1 = \sum |x_i|$ and $\|x\|_\infty = \max |x_i|$. For matrices, the *Frobenius norm* $\|A\|_F = \sqrt{\sum_{ij} A_{ij}^2} = \sqrt{\operatorname{tr}(A^{\mathsf{T}} A)}$ treats the matrix as a long vector, and the *induced 2-norm* measures the largest amplification of a vector:

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \sqrt{\lambda_{\max}(A^{\mathsf{T}} A)}, \tag{2.32}$$

the second equality being the Rayleigh quotient of $A^{\mathsf{T}} A$. It equals the largest singular value of A (Chapter 3). Norms are how tolerances are expressed in tests: "$\|R^{\mathsf{T}} R - I\|_F < 10^{-12}$" is a precise statement that R is numerically orthogonal.

Key takeaways

- A vector is a geometric object; its coordinates depend on the chosen basis (frame). In an orthonormal basis, coordinates are dot products.
- $a^\top b = \|a\|\|b\| \cos\theta$; projections split a vector into parallel and perpendicular parts.
- $[a]_\times b = a \times b = -[b]_\times a$, $[a]_\times^3 = -\|a\|^2 [a]_\times$ and $[Ra]_\times = R[a]_\times R^\top$ are the workhorses of rotation algebra.
- A matrix is a linear map; its columns are the images of the basis vectors; products compose maps right to left.
- Rank–nullity: $\dim \operatorname{null}(A) + \operatorname{rank} A = n$. The left nullspace is the orthogonal complement of the column space.
- Orthogonal matrices preserve geometry; those with determinant +1 are rotations. Every 3D rotation has an axis.
- Symmetric matrices have real eigenvalues and orthonormal eigenvectors; PD matrices define ellipsoids.

Exercises

2.1 Show that the set of 3×3 skew-symmetric matrices is a vector space of dimension three, and that the map $a \mapsto [a]_\times$ is linear and invertible.

2.2 Decide whether $(1,2,3)$, $(4,5,6)$ and $(7,8,9)$ are independent. Find a non-trivial combination giving zero.

2.3 Apply Gram–Schmidt to $(1,1,0)$, $(1,0,1)$, $(0,1,1)$.

2.4 Prove (2.6) by checking it for a, b, c chosen among e_1, e_2, e_3 and using linearity.

2.5 Prove (2.13) directly from (2.12), and verify the Jacobi identity $a \times (b \times c) + b \times (c \times a) + c \times (a \times b) = 0$ numerically.

2.6 Find the matrix of the reflection of $\mathbb{R}^3$ across the plane orthogonal to a unit vector n. Show that it is orthogonal with determinant -1.

2.7 Show that the eigenvalues of $[a]_\times$ are 0 and $\pm i\|a\|$. Which eigenvector belongs to 0?

2.8 For $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, find the eigen-decomposition and sketch the ellipse $x^\top A^{-1} x = 1$.

2.9 Let $H_f \in \mathbb{R}^{2M \times 3}$ have full column rank. What is the dimension of its left nullspace for $M = 2, 3, 10$?

2.10 Prove that the rank of $ab^\top$ is one for non-zero a, b, and that every rank-one matrix has this form.

2.11 Show that $\operatorname{tr}(A^\top B)$ is an inner product on matrices, and that $\|A\|_F^2 = \operatorname{tr}(A^\top A)$.

2.12 Prove that the product of two rotation matrices is a rotation matrix, and that $R(\alpha)R(\beta) = R(\alpha + \beta)$ in the plane.

Chapter 3

Numerical Linear Algebra

In this chapter

- Count the cost of matrix operations and understand why a filter must never form an explicit inverse.
- Derive the LU, Cholesky, QR and singular value decompositions, and know which one to use when.
- Solve least-squares problems stably, and compute nullspaces, projectors and pseudo-inverses.
- Use block matrices, Schur complements and the Woodbury identity fluently.
- Understand the matrix exponential, and why the exponential of a skew-symmetric matrix is a rotation.
- Understand floating-point arithmetic: rounding, cancellation, conditioning and stability.

Chapter 2 described what matrices *are*. This chapter is about computing with them reliably. A Kalman filter spends nearly all of its time in a few operations: multiplying matrices, solving linear systems with a covariance matrix, and factorizing Jacobians. Done naively, these operations are slow, lose accuracy, or silently destroy the symmetry and positive definiteness of covariances. Done well, they are fast and accurate to nearly machine precision.

3.1 The cost of matrix operations

Numerical analysts count *flops* (floating-point operations: one addition or multiplication each). Some basic costs for $n \times n$ matrices:

Operation	Approximate flops
dot product of two n-vectors	$2n$
matrix-vector product	$2n^2$
matrix-matrix product	$2n^3$
solving a triangular system	n^2
LU factorization	$\frac{2}{3}n^3$
Cholesky factorization	$\frac{1}{3}n^3$
explicit inverse	$2n^3$

For the MSCKF with $n = 75$, a matrix product costs about 8×10^5 flops, a fraction of a millisecond. Costs grow with the cube of the dimension, which is why filters keep their state small, exploit block structure, and update only the blocks that change. We write $O(n^3)$ ("order n^3") to mean "at most a constant times n^3 for large n".

3.2 Triangular systems and the LU factorization

3.2.1 Forward and back substitution

A triangular system is solved by direct substitution. For an upper-triangular U with non-zero diagonal, the last equation $U_{nn}x_n = b_n$ gives x_n, the previous one then gives x_{n-1}, and so on:

$$x_i = \frac{1}{U_{ii}}\Big(b_i - \sum_{j>i} U_{ij}x_j\Big), \qquad i = n, n-1, \dots, 1. \tag{3.1}$$

This is *back substitution*; *forward substitution* solves lower-triangular systems from the top. Each costs about n^2 flops. Every method in this chapter reduces a hard problem to one or two triangular solves.

3.2.2 LU factorization

Gaussian elimination (Section 2.5) subtracts multiples ℓ_{ik} of row k from the rows below it, column by column, until the matrix is upper triangular. Recording the multipliers in a unit lower-triangular matrix L gives the *LU factorization*

$$A = LU. \tag{3.2}$$

Then $Ax = b$ is solved by forward substitution $Ly = b$ and back substitution $Ux = y$. The factorization costs $\frac{2}{3}n^3$ flops once; every additional right-hand side costs only $2n^2$.

Pivoting. Elimination divides by the pivots. A tiny pivot creates huge multipliers that amplify rounding errors. The classic example is $\begin{bmatrix} 10^{-20} & 1 \\ 1 & 1 \end{bmatrix}$: eliminating with the pivot 10^{-20} gives a multiplier of 10^{20}, and in double precision the computed solution of $Ax = (1, 2)$ is completely wrong, although the matrix is perfectly well conditioned. Swapping the two rows first fixes it. *Partial pivoting* always swaps the row with the largest available pivot into place, giving $PA = LU$ with a permutation matrix P. Symmetric positive-definite matrices, which is what filters mostly factor, need no pivoting at all, as the next section shows.

3.3 The Cholesky factorization

Theorem 3.1 (Cholesky factorization). *A symmetric matrix S is positive definite if and only if $S = LL^{\mathsf{T}}$ for a lower-triangular L with positive diagonal. This L is unique.*

Proof. If $S = LL^{\mathsf{T}}$ with L invertible, then $x^{\mathsf{T}}Sx = \|L^{\mathsf{T}}x\|^2 > 0$ for $x \neq 0$. Conversely, partition

$$S = \begin{bmatrix} s_{11} & s^{\mathsf{T}} \\ s & S_{22} \end{bmatrix} = \begin{bmatrix} \ell_{11} & 0 \\ \ell & L_{22} \end{bmatrix} \begin{bmatrix} \ell_{11} & \ell^{\mathsf{T}} \\ 0 & L_{22}^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} \ell_{11}^2 & \ell_{11}\ell^{\mathsf{T}} \\ \ell_{11}\ell & \ell\ell^{\mathsf{T}} + L_{22}L_{22}^{\mathsf{T}} \end{bmatrix}.$$

Matching blocks: $\ell_{11} = \sqrt{s_{11}}$ (positive because $s_{11} = e_1^{\mathsf{T}}Se_1 > 0$), $\ell = s/\ell_{11}$, and $L_{22}L_{22}^{\mathsf{T}} = S_{22} - ss^{\mathsf{T}}/s_{11}$. The last matrix (a Schur complement, Section 3.8) is again positive definite,

so the construction continues recursively on a matrix one size smaller. Each step is forced, which proves uniqueness. ∎

Unrolling the recursion column by column gives the algorithm

$$L_{jj} = \Big(S_{jj} - \sum_{k<j} L_{jk}^2\Big)^{1/2}, \qquad L_{ij} = \frac{1}{L_{jj}}\Big(S_{ij} - \sum_{k<j} L_{ik}L_{jk}\Big), \quad i > j, \tag{3.3}$$

at a cost of $\frac{1}{3}n^3$ flops, half that of LU. The algorithm is backward stable without pivoting, and it fails (the argument of a square root is not positive) exactly when S is not numerically positive definite, which makes it a useful diagnostic too. The variant $S = LDL^\mathsf{T}$, with unit-diagonal L and diagonal D, avoids square roots; with symmetric pivoting (Eigen's `LDLT`) it also handles semi-definite and slightly indefinite matrices gracefully.

Uses of the Cholesky factor.

- *Solving* $Sx = b$: forward-substitute $Ly = b$, back-substitute $L^\mathsf{T}x = y$.
- *Whitening*: if a random vector n has covariance $S = LL^\mathsf{T}$, then $L^{-1}n$ has covariance I (Chapter 5). Mahalanobis distances are computed as $r^\mathsf{T}S^{-1}r = \|L^{-1}r\|^2$ with a single triangular solve.
- *Sampling*: if s has covariance I, then Ls has covariance S.
- *Determinants*: $\det S = \prod_i L_{ii}^2$, and $\log\det S = 2\sum_i \log L_{ii}$ without overflow.

3.4 Orthogonal factorizations: QR

Orthogonal transformations do not amplify errors (they preserve lengths), so factorizations built from them are the most stable available.

3.4.1 QR from Gram–Schmidt

Applying Gram–Schmidt to the columns $a_1, \dots, a_n$ of a full-column-rank $A \in \mathbb{R}^{m\times n}$ produces orthonormal $q_1, \dots, q_n$ with $a_k = \sum_{j\le k} r_{jk}q_j$, where $r_{jk} = q_j^\mathsf{T}a_k$ and $r_{kk} = \|\tilde{q}_k\|$. In matrix form

$$A = Q_1R_1, \qquad Q_1 = [q_1 \cdots q_n] \in \mathbb{R}^{m\times n}, \quad Q_1^\mathsf{T}Q_1 = I, \quad R_1 \text{ upper triangular.} \tag{3.4}$$

This is the *QR factorization*. Classical Gram–Schmidt, implemented literally, loses orthogonality in floating point when the columns are nearly dependent; the computed q's drift away from being orthogonal. Householder reflections avoid this problem.

3.4.2 Householder reflections

Definition 3.2 (Householder reflection). For a non-zero $v \in \mathbb{R}^m$, the matrix $H_v = I - 2\,\dfrac{vv^\mathsf{T}}{v^\mathsf{T}v}$ reflects vectors across the hyperplane orthogonal to v: it reverses the component along v and leaves the orthogonal components unchanged. It is symmetric and orthogonal, and

is its own inverse: $H_v = H_v^\top = H_v^{-1}$.

Proposition 3.3 (Zeroing a vector below its first entry). *For $x \in \mathbb{R}^m$ not a multiple of e_1, let $\alpha = -\operatorname{sign}(x_1)\|x\|$ and $v = x - \alpha e_1$. Then $H_v x = \alpha e_1$.*

Proof. Because $\|x\| = \|\alpha e_1\|$, the vectors x and αe_1 have the same length, so the reflection across the perpendicular bisector of the segment joining them swaps them. That bisecting hyperplane is orthogonal to $v = x - \alpha e_1$. Algebraically: $v^\top v = 2\|x\|^2 - 2\alpha x_1$ and $v^\top x = \|x\|^2 - \alpha x_1 = \tfrac{1}{2} v^\top v$, so $H_v x = x - 2v\,\frac{v^\top x}{v^\top v} = x - v = \alpha e_1$. ∎

The sign choice makes $v_1 = x_1 + \operatorname{sign}(x_1)\|x\|$ a sum of two numbers of the same sign, avoiding the cancellation that the other choice would suffer when x is nearly parallel to e_1.

Householder QR. Apply a reflection H_1 that zeroes the first column of A below the diagonal; then a reflection H_2, acting on rows $2, \dots, m$, that zeroes the second column below the diagonal; and so on for all n columns:

$$H_n \cdots H_2 H_1 A = \begin{bmatrix} R_1 \\ 0 \end{bmatrix} \quad \Longleftrightarrow \quad A = Q \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1 R_1, \tag{3.5}$$

with $Q = H_1 H_2 \cdots H_n$ orthogonal ($m \times m$), $R_1 \in \mathbb{R}^{n \times n}$ upper triangular, $Q_1 \in \mathbb{R}^{m \times n}$ and $Q_2 \in \mathbb{R}^{m \times (m-n)}$. The cost is about $2mn^2 - \frac{2}{3}n^3$ flops. The form $A = Q[R_1; 0]$ is the *full* QR factorization, and $A = Q_1 R_1$ the *thin* one.

Applying $Q^\top$ without forming Q. Each H_k is determined by its vector v_k, so $Q^\top b = H_n \cdots H_1 b$ is computed by applying the reflections one at a time:

$$b \;\leftarrow\; b - v_k \frac{2\, v_k^\top b}{v_k^\top v_k}, \qquad k = 1, \dots, n, \tag{3.6}$$

at a cost of $O(m)$ per reflection. Forming Q explicitly would cost $O(m^2 n)$ flops and m^2 storage.

Proposition 3.4 (Nullspaces from QR). *If A has full column rank n and $A = Q_1 R_1$ as in (3.5), then the columns of Q_1 are an orthonormal basis of* range(A) *and the columns of Q_2 are an orthonormal basis of the left nullspace* null$(A^\top)$. *In particular $Q_2^\top A = 0$.*

Proof. $Q^\top A = [R_1; 0]$, so $Q_2^\top A = 0$, i.e. the columns of Q_2 lie in null$(A^\top)$. They are orthonormal and there are $m - n$ of them, which is the dimension of null$(A^\top)$ by rank–nullity. Since R_1 is invertible, range(A) = range(Q_1). ∎

Implementation note: never form Q

Libraries store the Householder vectors compactly in the lower triangle of the factored matrix. In Eigen, `HouseholderQR::householderQ()` returns a lazy `HouseholderSequence` that can be applied to another matrix without materializing Q. This is exactly what the MSCKF needs: it applies the reflections that triangularize a feature Jacobian to the rest of the measurement equations (Chapter 26). Whether a particular call allocates must be

checked with the allocation guard, not assumed.

3.4.3 Givens rotations

A *Givens rotation* zeroes one entry at a time by rotating a pair of rows. To zero b in the pair (a, b), set $r = \sqrt{a^2 + b^2}$, $c = a/r$ and $s = b/r$:

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}. \tag{3.7}$$

Embedded in an identity matrix, it acts on two rows of a larger matrix and leaves the others untouched. Givens rotations are preferred when only a few entries need to be zeroed, when the matrix is sparse, or when rows arrive one at a time. The original MSCKF paper suggests them for its nullspace projection, because the feature Jacobian has a sparse block structure.

3.5 Least squares

3.5.1 The problem and the normal equations

An overdetermined system $Ax \approx b$, with $A \in \mathbb{R}^{m \times n}$, $m > n$, generally has no exact solution. The *least-squares* solution minimizes the squared residual

$$f(x) = \|Ax - b\|^2 = x^\mathsf{T} A^\mathsf{T} A x - 2x^\mathsf{T} A^\mathsf{T} b + b^\mathsf{T} b. \tag{3.8}$$

Setting the gradient $2A^\mathsf{T}Ax - 2A^\mathsf{T}b$ to zero gives the *normal equations*

$$A^\mathsf{T} A\, x = A^\mathsf{T} b. \tag{3.9}$$

If A has full column rank, $A^\mathsf{T}A$ is positive definite (since $x^\mathsf{T}A^\mathsf{T}Ax = \|Ax\|^2 > 0$ for $x \neq 0$), so the solution is unique and is a minimum.

Geometric meaning. The normal equations say $A^\mathsf{T}(b - Ax) = 0$: the residual is orthogonal to every column of A. The fitted vector Ax is the orthogonal projection of b onto range(A), the closest point to b that the model can produce.

> **Definition 3.5** (Orthogonal projector)**.** A matrix Π is an orthogonal projector if $\Pi^2 = \Pi$ and $\Pi^\mathsf{T} = \Pi$. If the columns of Q are an orthonormal basis of a subspace $\mathcal{S}$, the projector onto $\mathcal{S}$ is $\Pi_\mathcal{S} = QQ^\mathsf{T}$. For A of full column rank the projector onto range(A) is $A(A^\mathsf{T}A)^{-1}A^\mathsf{T}$, and $I - \Pi$ projects onto the orthogonal complement.

Combining the two descriptions of the projector onto range(A), and recalling that the columns of Q_2 in (3.5) span its complement null(A^T), gives an identity we will use to prove the optimality of the MSCKF: for any $A_\perp$ whose columns are an orthonormal basis of the left nullspace of a full-column-rank H_f,

$$A_\perp A_\perp^\mathsf{T} = I - H_f (H_f^\mathsf{T} H_f)^{-1} H_f^\mathsf{T}. \tag{3.10}$$

3.5.2 Three ways to solve

Cholesky on the normal equations. Form $A^\mathsf{T}A$ (mn^2 flops) and factor it. This is the fastest method, but forming $A^\mathsf{T}A$ squares the condition number of the problem (Section 3.7): $\kappa(A^\mathsf{T}A) = \kappa(A)^2$. When $\kappa(A) = 10^8$, the normal equations lose essentially all sixteen significant digits.

QR. With $A = Q[R_1; 0]$ and the invariance of lengths under Q^T,

$$\|Ax - b\|^2 = \|Q^\mathsf{T}(Ax - b)\|^2 = \|R_1 x - Q_1^\mathsf{T} b\|^2 + \|Q_2^\mathsf{T} b\|^2. \tag{3.11}$$

The second term does not depend on x, and the first vanishes for $x = R_1^{-1}Q_1^\mathsf{T}b$, computed by back substitution. The minimum residual is $\|Q_2^\mathsf{T}b\|$. This method works with the condition number of A itself and is the standard choice.

SVD. The most robust and most expensive method, required when A may be rank deficient (Section 3.6).

3.5.3 Weighted least squares

When the equations have different reliabilities, with errors of covariance W, the right objective weights each residual by its uncertainty (Chapter 9):

$$\min_x (Ax - b)^\mathsf{T} W^{-1} (Ax - b). \tag{3.12}$$

With $W = LL^\mathsf{T}$ this is ordinary least squares on the *whitened* system $L^{-1}Ax \approx L^{-1}b$, and the normal equations become $A^\mathsf{T}W^{-1}A\,x = A^\mathsf{T}W^{-1}b$.

3.6 The singular value decomposition

Theorem 3.6 (Singular value decomposition). *Every $A \in \mathbb{R}^{m\times n}$ can be written as $A = U\Sigma V^\mathsf{T}$ with $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ orthogonal and $\Sigma \in \mathbb{R}^{m\times n}$ diagonal with entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$, the* singular values.

Construction. $A^\mathsf{T}A$ is symmetric and PSD, so by the spectral theorem $A^\mathsf{T}A = V\Lambda V^\mathsf{T}$ with orthonormal v_i and eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$. Set $\sigma_i = \sqrt{\lambda_i}$. For each $\sigma_i > 0$ define $u_i = Av_i/\sigma_i$. These are orthonormal: $u_i^\mathsf{T}u_j = v_i^\mathsf{T}A^\mathsf{T}Av_j/(\sigma_i\sigma_j) = \lambda_j v_i^\mathsf{T}v_j/(\sigma_i\sigma_j) = \delta_{ij}$. Complete them to an orthonormal basis $u_1, \ldots, u_m$ of $\mathbb{R}^m$. Then $Av_i = \sigma_i u_i$ for all i (for $\sigma_i = 0$, $\|Av_i\|^2 = \lambda_i = 0$), which in matrix form is $AV = U\Sigma$. ∎

The SVD says that every linear map, in suitable orthonormal bases of its domain and codomain, is a pure scaling (Figure 3.1). It reveals everything about A at once:

- the rank r is the number of non-zero singular values;
- $u_1, \ldots, u_r$ span range(A), $u_{r+1}, \ldots, u_m$ span null(A^T), $v_1, \ldots, v_r$ span range(A^T) and $v_{r+1}, \ldots, v_n$ span null(A);

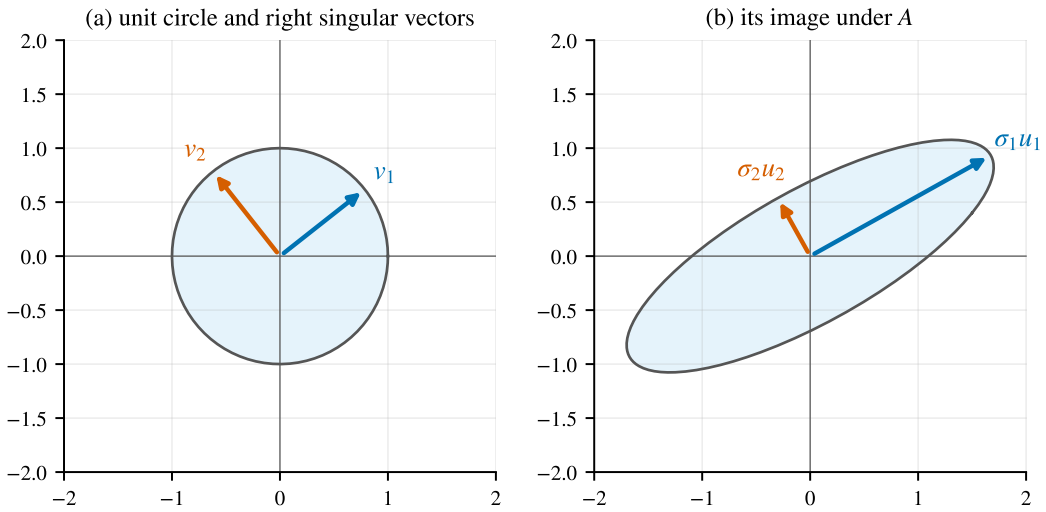


Figure 3.1: Geometry of the SVD of a 2×2 matrix. The right singular vectors v_1, v_2 (left) form an orthonormal basis that A maps to orthogonal vectors $\sigma_1 u_1, \sigma_2 u_2$ (right). The unit circle becomes an ellipse with semi-axes σ_1 and σ_2: every linear map is a rotation, an axis-aligned scaling, and another rotation.

- $\|A\|_2 = \sigma_1$ and $\|A\|_F^2 = \sum_i \sigma_i^2$.

Pseudo-inverse. The *pseudo-inverse* $A^+ = V\Sigma^+ U^\mathsf{T}$, where Σ^+ inverts the non-zero singular values and transposes the shape, gives the minimum-norm least-squares solution $x = A^+ b$ of any system, including rank-deficient ones. For full column rank, $A^+ = (A^\mathsf{T} A)^{-1} A^\mathsf{T}$.

> **Proposition 3.7** (Homogeneous least squares)**.** *The unit vector x minimizing $\|Ax\|$ is the right singular vector v_n belonging to the smallest singular value, and the minimum is σ_n.*

Proof. $\|Ax\|^2 = x^\mathsf{T} A^\mathsf{T} A x$, and by the Rayleigh quotient (Proposition 2.30) its minimum over unit vectors is the smallest eigenvalue of $A^\mathsf{T} A$, σ_n^2, attained at the corresponding eigenvector v_n. ∎

Many geometric estimation problems have the form $Ax = 0$ with x known only up to scale: estimating an essential matrix or a homography from correspondences, or triangulating a point in homogeneous coordinates (Chapter 23). With noisy data there is no exact solution, and Proposition 3.7 gives the best one.

> **Theorem 3.8** (Best low-rank approximation, Eckart–Young)**.** *Among all matrices of rank at most k, the one closest to A in the Frobenius or 2-norm is $A_k = \sum_{i \le k} \sigma_i u_i v_i^\mathsf{T}$, obtained by zeroing all but the k largest singular values.*

This is how an estimated matrix is projected onto a set defined by its singular values: for example, the nearest valid essential matrix to a noisy estimate is obtained by replacing its singular values by $(1, 1, 0)$ (Chapter 23).

3.7 Conditioning and stability

Two different things can make a computed answer inaccurate: the problem itself may be sensitive to its data, or the algorithm may add more error than necessary.

Definition 3.9 (Condition number)**.** For an invertible (or full-column-rank) A, $\kappa(A) = \sigma_{\max}/\sigma_{\min} = \|A\|_2 \, \|A^{+}\|_2$.

Proposition 3.10 (Sensitivity of linear systems)**.** *If $Ax = b$ and $A(x + \delta x) = b + \delta b$, then* $\dfrac{\|\delta x\|}{\|x\|} \le \kappa(A) \dfrac{\|\delta b\|}{\|b\|}$.

Proof. $\delta x = A^{-1}\delta b$ gives $\|\delta x\| \le \|A^{-1}\|\|\delta b\|$, and $b = Ax$ gives $\|b\| \le \|A\|\|x\|$. Multiply the two inequalities. ∎

A problem with condition number κ can lose about $\log_{10}\kappa$ significant digits whatever the algorithm; it is *ill-conditioned* when κ is large. An algorithm is *backward stable* if its computed answer is the exact answer to a problem whose data differ from the true data by a few units of rounding error; such an algorithm never loses more than the conditioning dictates. Householder QR, Cholesky on positive-definite matrices and LU with partial pivoting are backward stable in practice; classical Gram–Schmidt and the normal equations for ill-conditioned least squares are not. In geometric terms: triangulating a point seen with almost no parallax is an ill-conditioned problem that no algorithm can rescue (Chapter 23), while recovering the rotation angle from a rotation matrix with arccos is an unstable algorithm for a well-conditioned problem (Chapter 8).

Pitfall: explicit inverses

Formulas such as $K = PH^{\mathsf{T}}S^{-1}$ are notation, not algorithms. Computing S^{-1} and then multiplying costs about three times as much as solving, is less accurate, and destroys structure. Factor S once (Cholesky or LDL^{T}) and solve: $SK^{\mathsf{T}} = HP^{\mathsf{T}}$ gives K^{T} by two triangular substitutions per column. Writing `S.ldlt().solve(Identity)` is still an explicit inverse in disguise.

3.8 Block matrices and the Schur complement

Estimation constantly partitions vectors and matrices into blocks: IMU state versus clones, poses versus landmarks. Block elimination is the basic tool.

Proposition 3.11 (Block factorization and inverse)**.** *Let* $M = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$ *with A invertible, and define the* Schur complement *of A in M, $M/A := D - CA^{-1}B$. Then*

$$M = \begin{bmatrix} I & 0 \\ CA^{-1} & I \end{bmatrix} \begin{bmatrix} A & 0 \\ 0 & M/A \end{bmatrix} \begin{bmatrix} I & A^{-1}B \\ 0 & I \end{bmatrix}, \qquad \det M = \det A \cdot \det(M/A), \tag{3.13}$$

and if M/A is invertible,

$$M^{-1} = \begin{bmatrix} A^{-1} + A^{-1}B\,(M/A)^{-1}\,CA^{-1} & -A^{-1}B\,(M/A)^{-1} \\ -(M/A)^{-1}CA^{-1} & (M/A)^{-1} \end{bmatrix}. \tag{3.14}$$

Proof. Multiply out the three factors. The inverse is the product of the inverses of the three factors in reverse order, and the inverse of a unit block-triangular factor is obtained by negating its off-diagonal block. ∎

The Schur complement is what remains of the equations for the second block of unknowns after the first block has been eliminated. To see this, solve $\begin{bmatrix} A & B \\ C & D \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} f \\ g \end{bmatrix}$ by eliminating $x = A^{-1}(f - By)$ from the second equation: $(D - CA^{-1}B)\,y = g - CA^{-1}f$. For symmetric M with $A \succ 0$, $M \succ 0$ if and only if $M/A \succ 0$. Chapter 5 shows that the Schur complement is exactly Gaussian conditioning in covariance form and marginalization in information form, and Chapter 10 uses it to eliminate landmarks from bundle adjustment.

Theorem 3.12 (Woodbury matrix identity). *For invertible A and C and conformable U, V,*

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U\big(C^{-1} + VA^{-1}U\big)^{-1}VA^{-1}. \tag{3.15}$$

Proof. Write the block matrix $M = \begin{bmatrix} A & U \\ V & -C^{-1} \end{bmatrix}$ and compute the top-left block of M^{-1} in two ways with Proposition 3.11. Eliminating A first gives $A^{-1} + A^{-1}U(-C^{-1} - VA^{-1}U)^{-1}VA^{-1}$, the right-hand side. Eliminating the bottom-right block $-C^{-1}$ first (the same proposition with the roles of the blocks exchanged) gives the inverse of its Schur complement, $(A - U(-C^{-1})^{-1}V)^{-1} = (A + UCV)^{-1}$. ∎

Woodbury connects the two forms of the Kalman update (Chapter 12). It says that a low-rank update of a matrix gives a low-rank update of its inverse, which is why an update with m measurements needs only an $m \times m$ inverse no matter how large the state is. Its determinant counterpart is the *matrix determinant lemma* $\det(A + UCV) = \det(C^{-1} + VA^{-1}U)\det C \det A$.

3.9 The matrix exponential

Definition 3.13 (Matrix exponential). For $A \in \mathbb{R}^{n\times n}$, $\exp(A) = \sum_{k=0}^{\infty} \frac{A^k}{k!} = I + A + \frac{1}{2}A^2 + \frac{1}{6}A^3 + \cdots$.

The series converges for every A: the norm of the k-th term is at most $\|A\|^k/k!$, and $\sum_k \|A\|^k/k! = e^{\|A\|}$ is finite.

Proposition 3.14 (Properties of the matrix exponential).

1. $\frac{\mathrm{d}}{\mathrm{d}t}\exp(tA) = A\exp(tA) = \exp(tA)\,A$.
2. $x(t) = \exp(tA)\,x_0$ *is the unique solution of* $\dot{x} = Ax$, $x(0) = x_0$ *(Chapter 4).*
3. *If* $AB = BA$ *then* $\exp(A + B) = \exp(A)\exp(B)$. ***In general this fails.***
4. $\exp(A)^{-1} = \exp(-A)$, *and* $\exp(A^{\mathsf{T}}) = \exp(A)^{\mathsf{T}}$.
5. $\exp(TAT^{-1}) = T\exp(A)\,T^{-1}$ *for invertible* T.

6. $\det \exp(A) = e^{\operatorname{tr} A}$.

Proof. (1) Differentiate the series term by term: $\frac{\mathrm{d}}{\mathrm{d}t}\frac{t^k A^k}{k!} = A\frac{t^{k-1}A^{k-1}}{(k-1)!}$. (3) When A and B commute, the binomial theorem holds for $(A+B)^k$, and the Cauchy product of the two series reproduces the series of $\exp(A+B)$, exactly as for numbers. (4) A and $-A$ commute, so $\exp(A)\exp(-A) = \exp(0) = I$; transposing the series term by term gives the second statement. (5) $(TAT^{-1})^k = TA^kT^{-1}$. (6) For diagonalizable $A = V\Lambda V^{-1}$, (5) gives $\exp(A) = V\exp(\Lambda)V^{-1}$ with $\exp(\Lambda) = \operatorname{diag}(e^{\lambda_i})$, so $\det\exp(A) = \prod_i e^{\lambda_i} = e^{\sum \lambda_i} = e^{\operatorname{tr} A}$; the general case follows by continuity, since diagonalizable matrices are dense. Statement (2) is proved in Chapter 4. ∎

The failure of property 3 for non-commuting matrices is the algebraic root of everything subtle about rotations: two rotations generated by $[a]_\times$ and $[b]_\times$ do not compose into the rotation generated by $[a+b]_\times$. Chapter 8 measures the discrepancy precisely.

Proposition 3.15 (Exponentials of skew-symmetric matrices are rotations). *If $S^\mathsf{T} = -S$ then $\exp(S)$ is a rotation matrix.*

Proof. By property 4, $\exp(S)^\mathsf{T}\exp(S) = \exp(S^\mathsf{T})\exp(S) = \exp(-S)\exp(S) = I$, so $\exp(S)$ is orthogonal; and $\det\exp(S) = e^{\operatorname{tr} S} = e^0 = 1$ because a skew-symmetric matrix has a zero diagonal. ∎

This little proposition is the bridge between the flat world of vectors and the curved world of rotations: every vector $\phi \in \mathbb{R}^3$ defines the rotation $\exp([\phi]_\times)$, and Chapters 7 and 8 show that every rotation arises this way.

Computing the exponential.

- *Nilpotent matrices* ($A^k = 0$ for some k): the series is a finite sum.
- *Diagonalizable matrices*: $\exp(A) = V\operatorname{diag}(e^{\lambda_i})V^{-1}$.
- *Structured matrices*: when powers of A repeat, as for skew-symmetric 3×3 matrices, the series can be summed in closed form (Rodrigues' formula, Chapter 7).
- *General matrices*: *scaling and squaring*. Choose s such that $\|A\|/2^s$ is small, approximate $\exp(A/2^s)$ by a truncated series or a rational (Padé) approximation, and square the result s times, using $\exp(A) = \exp(A/2^s)^{2^s}$. This is what library routines such as `scipy.linalg.expm` do.

Example 3.16 (Nilpotent generator: the double integrator). For $F = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, $F^2 = 0$, so the series terminates: $\exp(F\Delta t) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$. This is the exact transition matrix of position and velocity under zero acceleration. The inertial error dynamics of Chapter 17 have a similar nilpotent structure, so their exponentials can be written in closed form.

3.10 Floating-point arithmetic

3.10.1 Representation

An IEEE 754 `double` represents a number as $\pm 1.f \times 2^e$, with a sign bit, an 11-bit exponent e and a 52-bit fraction f (53 significant bits counting the implicit leading one). Its range is about 10^{-308} to 10^{308}; results beyond the range *overflow* to $\pm\infty$, and results below it *underflow* gradually to zero. Undefined operations such as $0/0$ or $\infty - \infty$ produce `NaN` ("not a number"), which propagates through every subsequent operation and compares unequal to everything, including itself. A single `NaN` in a covariance matrix contaminates the whole filter within one update.

3.10.2 Rounding

Two constants describe the precision:

- the *machine epsilon* $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$, the gap between 1 and the next larger double (`std::numeric_limits<double>::epsilon()`);
- the *unit roundoff* $u = 2^{-53} \approx 1.11 \times 10^{-16}$: every basic operation satisfies $\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta)$ with $|\delta| \le u$.

The spacing between consecutive doubles is proportional to their magnitude: near x it is about $\varepsilon_{\text{mach}}|x|$. Relative accuracy is therefore excellent for a single operation. Errors accumulate in long computations (a sum of n terms can carry a relative error of order nu in the worst case, though typically $\sqrt{n}\,u$), but the dramatic losses come from *cancellation*.

3.10.3 Cancellation

Subtracting two nearly equal numbers, each carrying a relative rounding error of size u, produces a small result whose *absolute* error is still of size u times the operands, so its *relative* error can be enormous. The subtraction itself is exact; it merely exposes the errors already present.

> **Example 3.17** (Catastrophic cancellation in $1-\cos\theta$). For $\theta = 10^{-5}$, $\cos\theta = 1 - 5\times10^{-11} + \dots$. The stored value of $\cos\theta$ has an absolute error up to $u \approx 10^{-16}$, so $1 - \cos\theta \approx 5 \times 10^{-11}$ is computed with relative error up to $10^{-16}/(5 \times 10^{-11}) \approx 2 \times 10^{-6}$. Dividing by θ^2 to form $(1-\cos\theta)/\theta^2 \approx 1/2$ keeps that relative error. At $\theta = 10^{-8}$, $\cos\theta$ rounds to exactly 1, and the computed value is 0 instead of $1/2$. The Taylor expansion $\frac{1}{2} - \frac{\theta^2}{24} + \cdots$ has no cancellation at all. Figure 8.2 in Chapter 8 shows this effect for all the coefficient functions of the rotation group.

The remedies are always the same: rewrite the expression to avoid the subtraction (for example $1-\cos\theta = 2\sin^2(\theta/2)$), use a series expansion in the dangerous regime, or reorganize the algorithm (for example `atan2` instead of `acos` for angles, Chapter 8).

Pitfall: time stamps in floating point

EuRoC timestamps are nanoseconds since the Unix epoch, about 1.4×10^{18}, which lies between 2^{60} and 2^{61}. A `double` near that value can only represent multiples of $2^{60-52} = 256$ ns. Converting absolute timestamps to `double` therefore quantizes them to

256 ns, and accumulating time by repeated addition of Δt drifts further. Keep absolute time in 64-bit integers (`std::chrono::nanoseconds`) and convert only differences to `double` (AGENTS.md C5).

Key takeaways

- Costs grow as n^3; exploit structure and update only what changes.
- Cholesky ($S = LL^\mathsf{T}$) is the factorization of choice for covariances: half the cost of LU, stable without pivoting, and a built-in test of positive definiteness.
- Householder QR is the stable way to solve least squares and to obtain orthonormal bases of a column space and of its complement, the left nullspace. Apply Q^T through its reflections; never form Q.
- The SVD reveals rank, nullspaces and conditioning, and solves homogeneous least squares $\min \|Ax\|$, $\|x\| = 1$.
- Schur complements eliminate variables; Woodbury converts between large and small inverses.
- $\exp(A + B) \neq \exp(A)\exp(B)$ unless A and B commute; the exponential of a skew-symmetric matrix is a rotation.
- Cancellation, not single operations, destroys floating-point accuracy. Time stamps belong in 64-bit integers.

Exercises

3.1 Solve $\begin{bmatrix} 10^{-20} & 1 \\ 1 & 1 \end{bmatrix} x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ in double precision by elimination without pivoting and with partial pivoting. Explain the difference.

3.2 Compute the Cholesky factor of $\begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 3 \\ 2 & 3 & 6 \end{bmatrix}$ by hand and use it to solve $Sx = (8, 10, 11)$.

3.3 Show that if $P \succ 0$ then $P - KSK^\mathsf{T} \succeq 0$ for $S = HPH^\mathsf{T} + R$, $K = PH^\mathsf{T}S^{-1}$ and $R \succ 0$. (Hint: the Schur complement of $\begin{bmatrix} S & HP \\ PH^\mathsf{T} & P \end{bmatrix}$.)

3.4 Implement Householder QR in Python for a 10×3 matrix, check Proposition 3.4 numerically, and compare the accuracy of solving a least-squares problem with $\kappa(A) = 10^8$ by the normal equations and by QR.

3.5 Prove (3.10) using a full QR factorization of H_f.

3.6 Use Theorem 3.12 to show that $\left(P^{-1} + H^\mathsf{T}R^{-1}H\right)^{-1} = P - PH^\mathsf{T}(HPH^\mathsf{T} + R)^{-1}HP$. Which side is cheaper to evaluate when P is 75×75 and R is 2×2? When R is 500×500?

3.7 Compute $\exp\left(\begin{bmatrix} 0 & -\theta \\ \theta & 0 \end{bmatrix}\right)$ by summing the series, and recognize the result.

3.8 Show that for $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ and $B = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$, $\exp(A + B) \neq \exp(A)\exp(B)$.

3.9 What is the spacing of doubles near 2^{30} seconds expressed in nanoseconds? Would storing time as `double` seconds since the epoch be acceptable for a 200 Hz IMU?

3.10 Rewrite $\sqrt{x^2 + 1} - x$ for large x and $(1 - \cos x)/x^2$ for small x so that they are computed accurately.

Chapter 4

Calculus, Differential Equations and Numerical Integration

In this chapter

- Use partial derivatives, gradients, Jacobians and Hessians, and the chain rule in matrix form.
- Use Taylor's theorem with explicit error terms; this is what "linearization" and "first order" mean.
- Differentiate common matrix expressions: inverses, quadratic forms, determinants.
- Solve linear ordinary differential equations, including time-varying ones, with the state transition matrix.
- Integrate differential equations numerically, predict the order of the error, and verify it.
- Compute derivatives numerically with a step size chosen from first principles.

A navigation filter is a differential equation (the vehicle's motion) integrated numerically from noisy derivatives (the IMU readings), corrected with linearized measurement models. This chapter collects the calculus behind those three activities: derivatives and linearization, differential equations and their solutions, and the numerical methods that approximate both.

4.1 Derivatives of functions of several variables

4.1.1 Partial derivatives, gradients and Jacobians

For a scalar function $f(x_1, \dots, x_n)$, the *partial derivative* $\partial f / \partial x_j$ is the ordinary derivative with respect to x_j with the other variables held fixed. The *gradient* collects them: $\nabla f = (\partial f/\partial x_1, \dots, \partial f/\partial x_n)^\mathsf{T}$. For a vector function $f : \mathbb{R}^n \to \mathbb{R}^m$ the partial derivatives form the *Jacobian* matrix.

Definition 4.1 (Jacobian, numerator layout)**.** For $f : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian $\frac{\partial f}{\partial x} \in \mathbb{R}^{m \times n}$ has entries $\left[\frac{\partial f}{\partial x}\right]_{ij} = \frac{\partial f_i}{\partial x_j}$. Row i is the gradient of the i-th output (transposed); column j is the sensitivity of all outputs to the j-th input.

The Jacobian is the matrix of the best linear approximation of f near x:

$$f(x + \delta) = f(x) + \frac{\partial f}{\partial x}\,\delta + o(\|\delta\|), \tag{4.1}$$

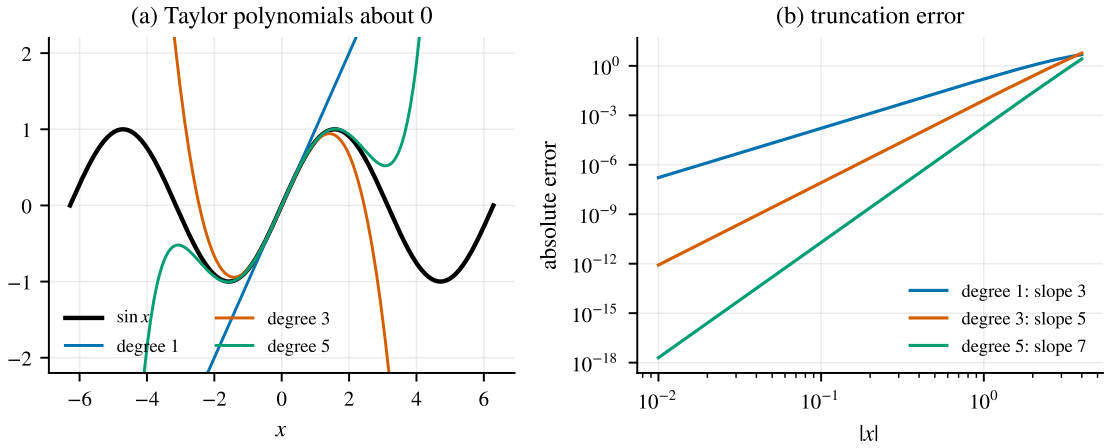


Figure 4.1: Taylor polynomials of $\sin x$ about 0 (left) and the absolute error of each (right, log-log scale). Near the expansion point the error of the degree-n polynomial falls like $|x|^{n+1}$ or faster. Because the even-order terms of $\sin x$ vanish, the polynomials of degree 1, 3 and 5 coincide with those of degree 2, 4 and 6, and their errors fall like $|x|^3$, $|x|^5$ and $|x|^7$ (slopes 3, 5 and 7). Far from the expansion point, every truncated series fails.

where $o(\|\delta\|)$ denotes a remainder that goes to zero faster than $\|\delta\|$. Column j is exactly what a finite-difference test computes by perturbing the j-th input (Section 4.4).

Proposition 4.2 (Chain rule). *If $y = f(x)$ and $z = g(y)$, then* $\dfrac{\partial z}{\partial x} = \dfrac{\partial g}{\partial y}\Big|_{y=f(x)} \dfrac{\partial f}{\partial x}$.

Proof. Substitute the linear approximation of f into that of g: $g(f(x+\delta)) = g\big(f(x) + J_f\delta + o(\|\delta\|)\big) = g(f(x)) + J_g J_f \delta + o(\|\delta\|)$. ∎

With the numerator layout the factors appear in the same order as the composition $g \circ f$, which makes long chains easy to write: the Jacobian of a camera measurement with respect to a pose is "(projection) times (frame change) times (pose perturbation)" (Chapter 26).

The *Hessian* of a scalar function is the matrix of second derivatives, $[\nabla^2 f]_{ij} = \partial^2 f / \partial x_i \partial x_j$. For smooth f it is symmetric. It describes curvature: the second-order behavior of f near a point.

4.1.2 Taylor's theorem

Theorem 4.3 (Taylor's theorem in one variable). *If f has $n+1$ continuous derivatives, then*

$$f(x+h) = \sum_{k=0}^{n} \frac{f^{(k)}(x)}{k!} h^k + \frac{f^{(n+1)}(\xi)}{(n+1)!} h^{n+1} \tag{4.2}$$

for some ξ between x and $x+h$ (Lagrange form of the remainder).

The theorem says more than "f is close to its Taylor polynomial": it bounds the error. If $|f^{(n+1)}| \le M$ on the interval, the error of the degree-n polynomial is at most $Mh^{n+1}/(n+1)!$. This is how we will judge every approximation in these notes.

Big-O and little-o. We write $g(h) = O(h^p)$ if $|g(h)| \le C h^p$ for some constant C as $h \to 0$, and $g(h) = o(h^p)$ if $g(h)/h^p \to 0$. An approximation is *first-order accurate* if its error is $O(h^2)$: the error of a linearization shrinks quadratically with the size of the perturbation. "Keeping terms to first order" in a derivation means writing the result as linear terms plus an $O(\|\delta\|^2)$ remainder, and then dropping the remainder.

For functions of several variables, Taylor's theorem to second order reads

$$f(x+\delta) = f(x) + \nabla f(x)^{\mathsf{T}}\delta + \tfrac{1}{2}\delta^{\mathsf{T}}\nabla^2 f(x)\,\delta + O(\|\delta\|^3), \tag{4.3}$$

and for vector functions each component expands this way. The size of the second-order term relative to the first measures how good a linearization is: when uncertainties are small compared with the scale over which the Jacobian changes, linearization is accurate. This is the quantitative content of the statement that an extended Kalman filter works well when errors are small (Chapter 13).

4.1.3 Matrix calculus

A few derivatives recur throughout estimation. We write them as differentials ($\mathrm{d}f$ = the linear change of f for a small change of the argument):

$$\mathrm{d}(Ax) = A\,\mathrm{d}x, \qquad \frac{\partial (Ax)}{\partial x} = A, \tag{4.4}$$

$$\mathrm{d}(x^{\mathsf{T}}Ax) = x^{\mathsf{T}}(A + A^{\mathsf{T}})\,\mathrm{d}x, \qquad \nabla_x (x^{\mathsf{T}}Ax) = (A + A^{\mathsf{T}})x \ \ (= 2Ax \text{ if } A \text{ symmetric}), \tag{4.5}$$

$$\mathrm{d}(a \times x) = [a]_\times\,\mathrm{d}x, \qquad \mathrm{d}(x \times a) = -[a]_\times\,\mathrm{d}x, \tag{4.6}$$

$$\mathrm{d}(A^{-1}) = -A^{-1}(\mathrm{d}A)A^{-1}, \qquad \mathrm{d}\log\det A = \operatorname{tr}(A^{-1}\mathrm{d}A). \tag{4.7}$$

Derivations. The first and third lines are linear maps, their own derivatives. For the quadratic form, $(x+\mathrm{d}x)^{\mathsf{T}}A(x+\mathrm{d}x) - x^{\mathsf{T}}Ax = \mathrm{d}x^{\mathsf{T}}Ax + x^{\mathsf{T}}A\,\mathrm{d}x + O(\|\mathrm{d}x\|^2) = x^{\mathsf{T}}(A^{\mathsf{T}} + A)\mathrm{d}x + O(\|\mathrm{d}x\|^2)$. For the inverse, differentiate $AA^{-1} = I$: $(\mathrm{d}A)A^{-1} + A\,\mathrm{d}(A^{-1}) = 0$. For the log-determinant of a positive-definite $A = U\Lambda U^{\mathsf{T}}$, a small change $\mathrm{d}A$ changes each eigenvalue by $u_i^{\mathsf{T}}(\mathrm{d}A)u_i$ to first order, so $\mathrm{d}\sum_i \log\lambda_i = \sum_i u_i^{\mathsf{T}}(\mathrm{d}A)u_i/\lambda_i = \operatorname{tr}(U\Lambda^{-1}U^{\mathsf{T}}\mathrm{d}A) = \operatorname{tr}(A^{-1}\mathrm{d}A)$. ∎

4.1.4 Stationary points

A smooth function has a local minimum at $x^\star$ only if $\nabla f(x^\star) = 0$, and it does have one if, in addition, $\nabla^2 f(x^\star) \succ 0$: by the second-order Taylor expansion, $f(x^\star+\delta) - f(x^\star) = \tfrac{1}{2}\delta^{\mathsf{T}}\nabla^2 f\,\delta + O(\|\delta\|^3) > 0$ for small $\delta \neq 0$. Every least-squares problem in these notes is solved by setting a gradient to zero; Chapter 10 develops the iterative methods needed when the resulting equations are nonlinear.

4.2 Ordinary differential equations

4.2.1 What a differential equation is

An *ordinary differential equation* (ODE) in state-space form is

$$\dot{x}(t) = f\big(x(t), t\big), \qquad x(t_0) = x_0, \tag{4.8}$$

where $x(t) \in \mathbb{R}^n$ is the *state* and $\dot{x} = \mathrm{d}x/\mathrm{d}t$. Any higher-order equation can be put in this form by adding derivatives to the state: Newton's law $\ddot{p} = a$ becomes the first-order system $\dot{p} = v$, $\dot{v} = a$ with state (p, v). The kinematics of a navigating vehicle are exactly such a system, with the IMU supplying the right-hand side.

Theorem 4.4 (Existence and uniqueness, Picard–Lindelöf)**.** *If f is continuous in t and Lipschitz continuous in x (there is an L with $\|f(x,t) - f(y,t)\| \le L\|x - y\|$), the initial value problem has a unique solution on some interval around t_0.*

Uniqueness is what makes prediction possible: the present state and the inputs determine the future. All the systems in these notes are smooth enough for the theorem to apply. (A non-example is $\dot{x} = \sqrt{|x|}$, $x(0) = 0$, solved both by $x \equiv 0$ and by $x = t^2/4$.)

4.2.2 Scalar linear equations

The scalar equation $\dot{x} = ax + b(t)$ is solved with an *integrating factor*. Multiplying by e^{-at} gives $\frac{\mathrm{d}}{\mathrm{d}t}\big(e^{-at}x\big) = e^{-at}b(t)$; integrating from t_0 to t,

$$x(t) = e^{a(t-t_0)}x_0 + \int_{t_0}^{t} e^{a(t-s)}\, b(s)\, \mathrm{d}s. \tag{4.9}$$

The solution is the free response of the initial condition plus a weighted sum of the inputs, each propagated from the time it was applied. Every linear system has this structure.

4.2.3 Linear time-invariant systems

Theorem 4.5 (Solution of linear time-invariant systems)**.** *The solution of $\dot{x} = Ax + Bu(t)$, $x(t_0) = x_0$, is*

$$x(t) = e^{A(t-t_0)}x_0 + \int_{t_0}^{t} e^{A(t-s)}Bu(s)\, \mathrm{d}s. \tag{4.10}$$

In particular $\dot{x} = Ax$ has the unique solution $x(t) = e^{A(t-t_0)}x_0$.

Proof. By Proposition 3.14, $\frac{\mathrm{d}}{\mathrm{d}t}\big(e^{-At}x\big) = e^{-At}(\dot{x} - Ax) = e^{-At}Bu$. Integrating from t_0 to t and multiplying by e^{At} gives (4.10). The same computation shows that *any* solution satisfies (4.10), which proves uniqueness. ∎

The matrix $e^{A\Delta t}$ propagates the state over an interval Δt. If $A = V\Lambda V^{-1}$, the solution is a combination of modes $e^{\lambda_i t}$: decaying for $\operatorname{Re}\lambda_i < 0$, growing for $\operatorname{Re}\lambda_i > 0$, oscillating for imaginary parts. Error dynamics of inertial navigation have eigenvalues at zero, so their errors grow polynomially rather than exponentially (Chapter 17).

4.2.4 Linear time-varying systems and the state transition matrix

When the system matrix changes with time, $\dot{x} = A(t)x$, the exponential no longer applies directly, but its key property survives.

Definition 4.6 (State transition matrix)**.** The *state transition matrix* $\Phi(t, s)$ of $\dot{x} = A(t)x$ is the solution of the matrix equation $\frac{\partial}{\partial t}\Phi(t, s) = A(t)\Phi(t, s)$ with $\Phi(s, s) = I$.

Proposition 4.7 (Properties of the state transition matrix)**.**

1. $x(t) = \Phi(t, s)\, x(s)$ *for every solution:* Φ *maps states at time* s *to states at time* t.
2. $\Phi(t, s)\,\Phi(s, r) = \Phi(t, r)$ *(the semigroup property) and* $\Phi(t, s)^{-1} = \Phi(s, t)$.
3. *For constant* A, $\Phi(t, s) = e^{A(t-s)}$.
4. *If* $A(t)$ *commutes with* $A(s)$ *for all* t, s*, then* $\Phi(t, s) = \exp\left(\int_s^t A(\tau)\,\mathrm{d}\tau\right)$*. In general this formula is* wrong.
5. *The solution of* $\dot{x} = A(t)x + B(t)u(t)$ *is* $x(t) = \Phi(t, t_0)x_0 + \int_{t_0}^{t} \Phi(t, s)B(s)u(s)\,\mathrm{d}s$.

Proof. (1) Both $x(t)$ and $\Phi(t, s)x(s)$ solve the ODE with the same value at $t = s$; by uniqueness they agree. (2) Both sides of the first identity, as functions of t, solve the matrix ODE with the value $\Phi(s, r)$ at $t = s$. Setting $r = t$ gives the second. (3) Check that $e^{A(t-s)}$ satisfies the definition. (4) If the $A(\tau)$ commute, so do $A(t)$ and $M(t) = \int_s^t A$, and then $\frac{\mathrm{d}}{\mathrm{d}t}e^{M(t)} = A(t)e^{M(t)}$ by the commuting case of the series; without commutativity this derivative formula fails. (5) Differentiate the right-hand side and use (1) and $\frac{\partial}{\partial t}\Phi = A\Phi$. ∎

Property 4 matters for attitude. The kinematics $\dot{R} = R[\omega(t)]_\times$ is a linear matrix ODE, and when the angular velocity changes direction over time the rotation is *not* $\exp\left(\int [\omega]_\times\right)$: rotations about different axes do not commute. Integrating the angular velocity vector and exponentiating the result produces the "coning" error of Chapter 16.

A general series for Φ, the *Peano–Baker series*

$$\Phi(t, s) = I + \int_s^t A(\tau_1)\,\mathrm{d}\tau_1 + \int_s^t \int_s^{\tau_1} A(\tau_1)A(\tau_2)\,\mathrm{d}\tau_2\,\mathrm{d}\tau_1 + \cdots,$$

obtained by repeatedly substituting the integral form of the ODE into itself, shows how products of non-commuting matrices enter. In practice Φ is approximated over short intervals, where A is nearly constant: $\Phi(t + \Delta t, t) \approx \exp(A(t)\Delta t) \approx I + A(t)\Delta t$, the discretization used by every filter in these notes.

4.3 Numerical integration of ODEs

4.3.1 Euler's method

Truncating the Taylor expansion $x(t + h) = x(t) + h\dot{x}(t) + O(h^2)$ gives *Euler's method*:

$$x_{k+1} = x_k + h\, f(x_k, t_k). \tag{4.11}$$

Each step makes a *local truncation error* of $O(h^2)$. Over a fixed interval T there are T/h steps, so one expects a *global error* of $O(h)$. The following proposition makes this precise; it also explains why local errors do not simply add up, but can be amplified by the dynamics.

Proposition 4.8 (Global error of Euler's method)**.** *If f is Lipschitz with constant L and the exact solution has a bounded second derivative $\|\ddot{x}\| \le M$, the error $e_k = x_k - x(t_k)$ of Euler's method satisfies $\|e_k\| \le \frac{Mh}{2L}\big(e^{L(t_k - t_0)} - 1\big)$.*

Proof. The exact solution satisfies $x(t_{k+1}) = x(t_k) + h f(x(t_k)) + \tau_k$ with $\|\tau_k\| \le Mh^2/2$ by Taylor's theorem. Subtracting from the numerical step, $e_{k+1} = e_k + h\big(f(x_k) - f(x(t_k))\big) - \tau_k$, so $\|e_{k+1}\| \le (1 + hL)\|e_k\| + Mh^2/2$. With $e_0 = 0$, induction gives $\|e_k\| \le \frac{Mh^2}{2}\sum_{j<k}(1+hL)^j = \frac{Mh}{2L}\big((1+hL)^k - 1\big) \le \frac{Mh}{2L}\big(e^{Lkh} - 1\big)$. ∎

Definition 4.9 (Order of a method)**.** A method has *order* p if its global error over a fixed interval is $O(h^p)$; equivalently its local error is $O(h^{p+1})$.

4.3.2 Higher-order methods

Runge–Kutta methods evaluate f at several points inside the step and combine the results so that the Taylor expansion of the step matches the true solution to higher order.

- **Midpoint method** (order 2): $k_1 = f(x_k, t_k)$, $x_{k+1} = x_k + h f(x_k + \tfrac{h}{2}k_1, t_k + \tfrac{h}{2})$.
- **Heun's method / trapezoidal rule** (order 2): $k_1 = f(x_k, t_k)$, $k_2 = f(x_k + hk_1, t_k + h)$, $x_{k+1} = x_k + \tfrac{h}{2}(k_1 + k_2)$.
- **Classical RK4** (order 4): $k_1 = f(x_k, t_k)$, $k_2 = f(x_k + \tfrac{h}{2}k_1, t_k + \tfrac{h}{2})$, $k_3 = f(x_k + \tfrac{h}{2}k_2, t_k + \tfrac{h}{2})$, $k_4 = f(x_k + hk_3, t_k + h)$, $x_{k+1} = x_k + \tfrac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$.

Verifying the order means expanding both the numerical step and the exact solution in Taylor series and comparing coefficients. For Heun's method on $\dot{x} = f(x)$: $k_2 = f(x + hf) = f + hf'f + O(h^2)$, so the step is $x + hf + \frac{h^2}{2}f'f + O(h^3)$, which matches $x(t+h) = x + h\dot{x} + \frac{h^2}{2}\ddot{x} + O(h^3)$ because $\ddot{x} = f'f$.

Verifying the order. If the error behaves as Ch^p, then halving the step divides the error by 2^p, and the order can be *measured*: $p \approx \log_2\big(e(h)/e(h/2)\big)$. Figure 4.2(a) shows this for three methods. A convergence-order test of this kind is one of the most powerful tests of an integrator: an implementation bug almost always destroys the expected order (Milestone 4).

Stability. Euler's method applied to the decaying equation $\dot{x} = -\lambda x$ gives $x_{k+1} = (1 - h\lambda)x_k$, which blows up when $h > 2/\lambda$, although the true solution decays. Stiff systems, with very fast decaying modes, need tiny steps or implicit methods. Applied to an oscillator, Euler's method multiplies the amplitude by $\sqrt{1 + h^2\omega^2} > 1$ per step and spirals outward (Figure 4.2b). Inertial navigation is not stiff, but this effect is the reason a naive attitude integrator slowly inflates the norm of its rotation matrix, as we now see.

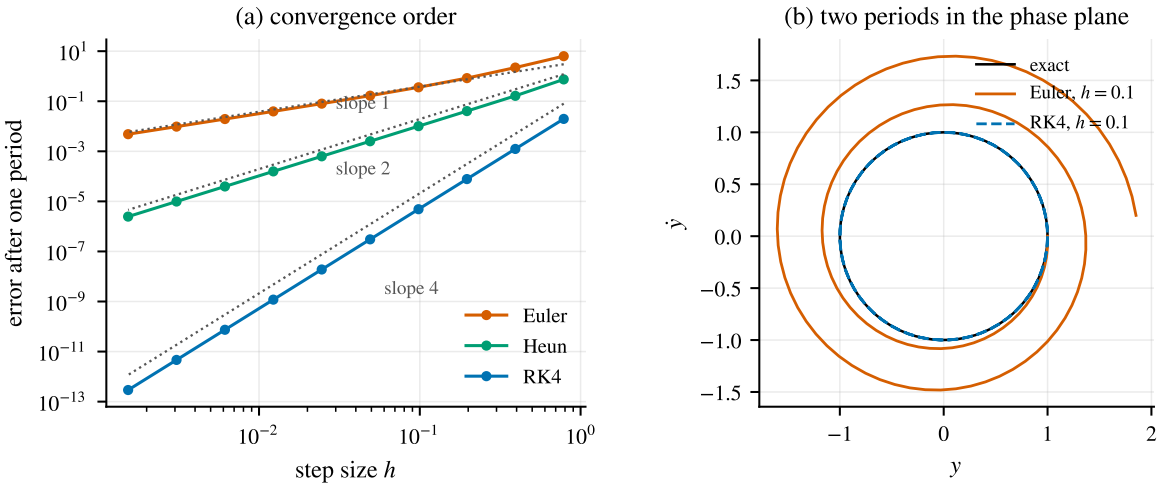


Figure 4.2: Integrating the harmonic oscillator $\ddot{y} = -y$. (a) Global error after one period versus step size: slopes 1, 2 and 4 confirm the orders of Euler, Heun and RK4. (b) With $h = 0.1$, Euler's method spirals outward, gaining energy every step, while RK4 stays on the circle to plotting accuracy.

4.3.3 Integrating on a curved space

Apply Euler's method to the attitude kinematics $\dot{R} = R[\omega]_\times$: $R_{k+1} = R_k + hR_k[\omega]_\times = R_k(I + h[\omega]_\times)$. The result is not a rotation matrix:

$$(I + h[\omega]_\times)^\mathsf{T}(I + h[\omega]_\times) = I - h^2[\omega]_\times^2 \neq I, \tag{4.12}$$

so every step stretches vectors perpendicular to ω by a factor of about $1 + \frac{1}{2}h^2\|\omega\|^2$, and the matrix drifts off the set of rotations. The remedy is to integrate in a way that respects the structure of the space. For constant ω over the step, the exact solution is $R_k \exp(h[\omega]_\times)$ (Theorem 4.5), so the *Lie–Euler* method

$$R_{k+1} = R_k \exp(h[\omega_k]_\times) \tag{4.13}$$

stays exactly on the set of rotations (Proposition 3.15) and is first order in general, exact when ω is constant. Higher-order generalizations (Runge–Kutta–Munthe-Kaas methods) apply Runge–Kutta to a vector-space parameterization of the increment. Every attitude update in these notes is multiplicative in this sense (AGENTS.md F1).

4.4 Numerical differentiation

Analytic Jacobians are derived by hand and verified numerically (AGENTS.md E6). Numerical differentiation is easy to write and easy to get subtly wrong, because it is a subtraction of nearly equal numbers.

Forward differences. By Taylor's theorem,

$$\frac{f(x + h) - f(x)}{h} = f'(x) + \frac{h}{2}f''(\xi), \tag{4.14}$$

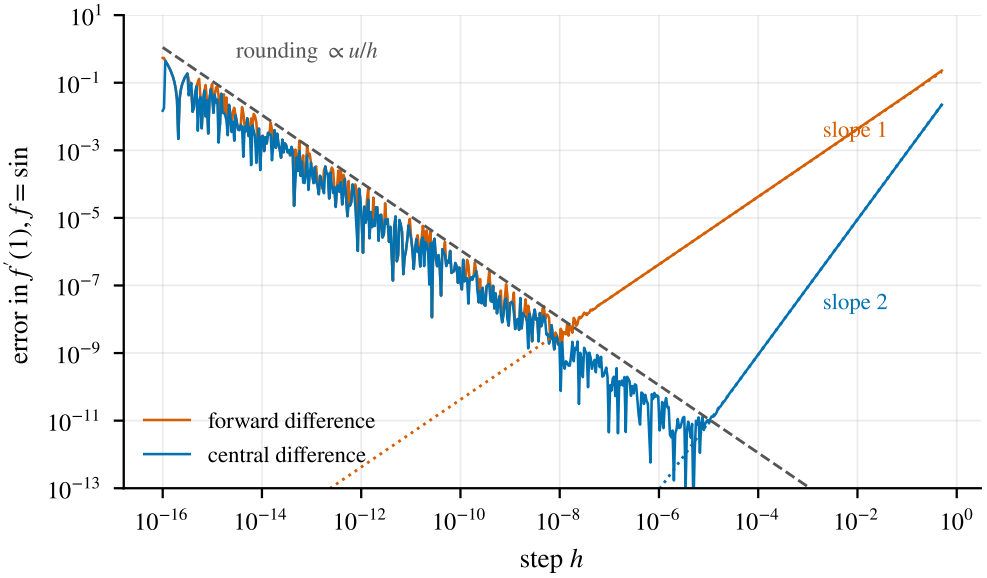


Figure 4.3: Error of finite-difference derivatives of $\sin x$ at $x = 1$ versus step size. To the right of the optimum, truncation error dominates (slopes 1 and 2); to the left, rounding error grows like u/h. Central differences reach about 10^{-11}; forward differences only about 10^{-8}.

so the *truncation error* is about $\frac{h}{2}|f''|$. But $f(x+h)$ and $f(x)$ are each computed with a rounding error of about $u|f|$, and their difference is divided by h, which adds a *rounding error* of about $2u|f|/h$. The total $\frac{h}{2}|f''| + \frac{2u|f|}{h}$ is minimized at $h^\star = 2\sqrt{u|f|/|f''|} \approx 10^{-8}$ for quantities of order one, with a total error of about 10^{-8}.

Central differences. Expanding both $f(x \pm h)$,

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6} f'''(\xi), \tag{4.15}$$

because the even-order terms cancel. The total error $\frac{h^2}{6}|f'''| + \frac{u|f|}{h}$ is minimized at

$$h^\star = \left(\frac{3\,u\,|f|}{|f'''|} \right)^{1/3} \approx 10^{-5} \text{ to } 10^{-6}, \tag{4.16}$$

with a total error of order $u^{2/3} \approx 10^{-11}$.

Figure 4.3 shows both error regimes. This is why the Jacobian tests of the project use central differences with $h \approx 10^{-6}$ and tolerances of about 10^{-6} to 10^{-8} relative to the size of the entries. A tolerance tighter than the achievable accuracy makes a correct implementation fail; a much looser one lets a wrong implementation pass.

Jacobians. For $f : \mathbb{R}^n \to \mathbb{R}^m$, column j of the numerical Jacobian is $\big(f(x + he_j) - f(x - he_j)\big)/2h$, costing $2n$ evaluations of f. On manifolds the perturbation and the difference are replaced by $\boxplus$ and $\boxminus$ (Chapter 8); nothing else changes.

Going deeper: the complex-step derivative

If f is analytic and can be evaluated with complex arguments, $f'(x) = \operatorname{Im} f(x + ih)/h + O(h^2)$, with no subtraction at all. With $h = 10^{-20}$ the result is exact to machine precision. The technique requires that every operation in f be implemented for complex numbers and avoid non-analytic functions (absolute values, comparisons), which rules it out for most estimator code. It is a superb way to generate reference Jacobians in Python, though.

Key takeaways

- The Jacobian is the matrix of the best linear approximation; the chain rule multiplies Jacobians in composition order.
- Taylor's theorem bounds approximation errors. "First order" means an error of $O(\|\delta\|^2)$.
- $\mathrm{d}(A^{-1}) = -A^{-1}\mathrm{d}A\,A^{-1}$ and $\mathrm{d}\log\det A = \operatorname{tr}(A^{-1}\mathrm{d}A)$.
- $\dot{x} = Ax + Bu$ is solved by e^{At} plus a convolution of the inputs; time-varying systems use the state transition matrix, which is *not* the exponential of the integrated system matrix unless the matrices commute.
- A method of order p has global error $O(h^p)$; halving the step should divide the error by 2^p. Measure it.
- Euler's method leaves the set of rotations; multiplicative updates $R\exp(h[\omega]_\times)$ do not.
- Central differences with $h \approx 10^{-6}$ reach about 10^{-11} accuracy; forward differences only about 10^{-8}.

Exercises

4.1 Compute the Jacobian of $f(x, y) = (x^2 y, \sin(xy), e^x)$ and verify it with central differences.

4.2 Use Taylor's theorem to bound the error of $\cos h \approx 1 - h^2/2$ for $|h| \le 0.1$.

4.3 Show that $\nabla_x \|Ax - b\|^2 = 2A^\mathsf{T}(Ax - b)$ and $\nabla_x^2 \|Ax - b\|^2 = 2A^\mathsf{T}A$.

4.4 Solve $\dot{x} = -2x + 1$, $x(0) = 3$, and check the solution by substitution.

4.5 For $A = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$, compute e^{At} and interpret the solution of $\dot{x} = Ax$.

4.6 Find the state transition matrix of $\dot{x} = \begin{bmatrix} 0 & t \\ 0 & 0 \end{bmatrix} x$. Do the matrices $A(t)$ commute?

4.7 Construct two angular velocity profiles with the same integral $\int \omega\,\mathrm{d}t$ that produce different final rotations.

4.8 Implement Euler, Heun and RK4 for the harmonic oscillator and reproduce Figure 4.2(a).

4.9 Show that one Euler step on $\dot{R} = R[\omega]_\times$ increases $\det R$ by a factor $1 + h^2\|\omega\|^2$, to leading order.

4.10 Reproduce Figure 4.3 and compare the observed optimal step sizes with the formulas of Section 4.4.

Chapter 5

Probability and Random Vectors

In this chapter

- Build probability from its axioms: conditional probability, independence and Bayes' rule.
- Work with random variables and random vectors: densities, expectations, covariances and changes of variables.
- Derive the Gaussian distribution and its key properties: affine maps, marginals, conditionals and the fusion of estimates.
- Understand the law of large numbers, the central limit theorem and Monte Carlo methods.
- Derive the χ^2 distribution and use it for hypothesis tests, the basis of gating and consistency checks.
- Understand how nonlinear functions distort distributions, and what linearization keeps and loses.

A filter does not merely estimate the state; it maintains a probability distribution over it. The estimate is the center of that distribution, and the covariance describes its spread. This chapter develops the probability theory that gives these words a precise meaning.

5.1 Probability

5.1.1 Events and the axioms

A random experiment has a set Ω of possible outcomes, the *sample space*. An *event* is a subset of outcomes, and a *probability* assigns to each event A a number $P(A)$ obeying three axioms (Kolmogorov):

1. $P(A) \geq 0$ for every event;
2. $P(\Omega) = 1$;
3. if $A_1, A_2, \ldots$ are mutually exclusive (pairwise disjoint), $P(A_1 \cup A_2 \cup \cdots) = \sum_i P(A_i)$.

Everything else follows: $P(\text{not } A) = 1 - P(A)$, $P(A \cup B) = P(A) + P(B) - P(A \cap B)$, and so on. Probability can be read as a long-run frequency (the fraction of repetitions in which A occurs) or as a degree of belief; the mathematics is the same, and filters use both readings.

5.1.2 Conditional probability and independence

Definition 5.1 (Conditional probability). For $P(B) > 0$, the probability of A given B is $P(A \mid B) = P(A \cap B)/P(B)$.

Conditioning restricts attention to the outcomes in which B happened and renormalizes. Two events are *independent* if $P(A \cap B) = P(A)P(B)$, equivalently $P(A \mid B) = P(A)$: knowing that B occurred tells nothing about A.

Proposition 5.2 (Law of total probability and Bayes' rule). *If $B_1, \dots, B_n$ are mutually exclusive and cover Ω, then $P(A) = \sum_i P(A \mid B_i)P(B_i)$, and*

$$P(B_i \mid A) = \frac{P(A \mid B_i)\,P(B_i)}{\sum_j P(A \mid B_j)\,P(B_j)}. \tag{5.1}$$

Proof. A is the disjoint union of the $A \cap B_i$, and $P(A \cap B_i) = P(A \mid B_i)P(B_i)$. Bayes' rule is the definition of conditional probability applied twice: $P(B_i \mid A)P(A) = P(A \cap B_i) = P(A \mid B_i)P(B_i)$. ∎

Bayes' rule turns the probability of an observation given a hypothesis (which a sensor model provides) into the probability of the hypothesis given the observation (which we want).

Worked example: how informative is a passed test?

Suppose 10% of the feature tracks reaching an outlier test are wrong. The test rejects 90% of wrong tracks, and (being a 95% gate) also rejects 5% of correct ones. A track passes. What is the probability that it is wrong? With W = "wrong" and A = "passes": $P(A \mid W) = 0.1$, $P(W) = 0.1$, $P(A \mid \text{correct}) = 0.95$, $P(\text{correct}) = 0.9$. Bayes' rule gives

$$P(W \mid A) = \frac{0.1 \times 0.1}{0.1 \times 0.1 + 0.95 \times 0.9} = \frac{0.01}{0.865} \approx 1.2\%.$$

The test reduces the outlier fraction from 10% to about 1%: very useful, but not perfect, which is why the estimator adds further checks downstream.

5.2 Random variables

5.2.1 Distributions, densities and expectations

A *random variable* X is a numerical quantity determined by the outcome of an experiment. Its *cumulative distribution function* is $F(x) = P(X \le x)$. A discrete random variable takes countably many values with probabilities $p(x) = P(X = x)$; a continuous one has a *probability density function* (pdf) $p(x) \ge 0$ with

$$P(a \le X \le b) = \int_a^b p(x)\,\mathrm{d}x, \qquad \int_{-\infty}^{\infty} p(x)\,\mathrm{d}x = 1. \tag{5.2}$$

A density is not a probability: $p(x)\,\mathrm{d}x$ is the probability of falling in a small interval of width $\mathrm{d}x$ around x, and a density can exceed one.

Definition 5.3 (Expectation and variance). The *expectation* (mean) of X is $\mathbb{E}[X] = \int x\, p(x)\,\mathrm{d}x$ (a sum for discrete variables). The expectation of a function of X is $\mathbb{E}[g(X)] = \int g(x)p(x)\,\mathrm{d}x$. The *variance* is $\operatorname{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$, and its square root is the *standard deviation* σ.

Expectation is linear: $\mathbb{E}[aX + bY + c] = a\mathbb{E}[X] + b\mathbb{E}[Y] + c$ for any random variables, dependent or not. Variance scales quadratically and ignores shifts: $\operatorname{Var}(aX + c) = a^2 \operatorname{Var}(X)$. The standard deviation has the units of X and measures its typical spread.

Example 5.4 (Uniform distribution). X uniform on $[a, b]$ has $p(x) = 1/(b - a)$ there, mean $(a + b)/2$ and variance $(b - a)^2/12$. Rounding a reading to a resolution Δ produces an error that is approximately uniform on $[-\Delta/2, \Delta/2]$, with standard deviation $\Delta/\sqrt{12}$: the quantization noise of a sensor.

5.2.2 The Gaussian distribution

Definition 5.5 (Gaussian distribution). $X \sim \mathcal{N}(\mu, \sigma^2)$ has density

$$p(x) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\!\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big). \tag{5.3}$$

The *standard normal* distribution is $\mathcal{N}(0, 1)$.

Proposition 5.6 (Normalization). $\int_{-\infty}^{\infty} e^{-x^2/2}\,\mathrm{d}x = \sqrt{2\pi}$.

Proof. Let I be the integral. Then $I^2 = \iint e^{-(x^2+y^2)/2}\,\mathrm{d}x\,\mathrm{d}y$. In polar coordinates, with area element $r\,\mathrm{d}r\,\mathrm{d}\theta$, $I^2 = \int_0^{2\pi}\int_0^{\infty} e^{-r^2/2} r\,\mathrm{d}r\,\mathrm{d}\theta = 2\pi\big[-e^{-r^2/2}\big]_0^{\infty} = 2\pi$. ∎

Substituting $x \to (x - \mu)/\sigma$ shows that the general density integrates to one, and similar integrals (using symmetry and integration by parts) give $\mathbb{E}[X] = \mu$ and $\operatorname{Var}(X) = \sigma^2$. If $X \sim \mathcal{N}(\mu, \sigma^2)$ then $(X - \mu)/\sigma \sim \mathcal{N}(0, 1)$, and the probabilities of falling within one, two and three standard deviations of the mean are 68.27%, 95.45% and 99.73%. A Gaussian is completely determined by its mean and variance, and is closed under the operations a linear filter performs, which is why Kalman filters can work with means and covariances alone.

5.3 Several random variables

5.3.1 Joint, marginal and conditional densities

Two random variables X, Y have a *joint density* $p(x, y)$, with $P((X, Y) \in A) = \iint_A p(x, y)\,\mathrm{d}x\,\mathrm{d}y$. From it:

$$\text{marginal:} \quad p(x) = \int p(x, y)\,\mathrm{d}y, \tag{5.4}$$

$$\text{conditional:} \quad p(x \mid y) = \frac{p(x, y)}{p(y)}, \tag{5.5}$$

$$\text{Bayes' rule:} \quad p(x \mid y) = \frac{p(y \mid x)\,p(x)}{p(y)} = \frac{p(y \mid x)\,p(x)}{\int p(y \mid x')p(x')\,\mathrm{d}x'}. \tag{5.6}$$

X and Y are independent if $p(x, y) = p(x)p(y)$. In estimation, x is the unknown state and y the measurement: $p(y \mid x)$ is the sensor model (the *likelihood*), $p(x)$ is the *prior*, and $p(x \mid y)$ the *posterior*.

5.3.2 Covariance and correlation

The *covariance* $\operatorname{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)]$ measures how two variables vary together, and the *correlation coefficient* $\rho = \operatorname{Cov}(X, Y)/(\sigma_X \sigma_Y) \in [-1, 1]$ normalizes it (the bound is the Cauchy–Schwarz inequality). Independent variables have zero covariance, and then $\operatorname{Var}(X + Y) = \operatorname{Var}(X) + \operatorname{Var}(Y)$: variances of independent errors add.

The converse fails: zero correlation does not imply independence. If $X \sim \mathcal{N}(0, 1)$ and $Y = X^2$, then $\operatorname{Cov}(X, Y) = \mathbb{E}[X^3] = 0$, yet Y is a function of X. Correlation measures only *linear* dependence. (For jointly Gaussian variables the converse does hold, as shown below.)

5.4 Random vectors

5.4.1 Mean and covariance

For a random vector $x \in \mathbb{R}^n$ with density $p(x)$, the mean and *covariance matrix* are

$$\mu = \mathbb{E}[x], \qquad \Sigma = \operatorname{Cov}(x) = \mathbb{E}\big[(x - \mu)(x - \mu)^{\mathsf{T}}\big] = \mathbb{E}[xx^{\mathsf{T}}] - \mu\mu^{\mathsf{T}}. \tag{5.7}$$

The diagonal of Σ holds the variances σ_i^2 and its off-diagonal entries the covariances. Σ is symmetric and positive semi-definite: for any fixed a, $a^{\mathsf{T}}\Sigma a = \mathbb{E}\big[(a^{\mathsf{T}}(x - \mu))^2\big] \geq 0$. The *cross-covariance* of two random vectors is $\Sigma_{xy} = \mathbb{E}[(x - \mu_x)(y - \mu_y)^{\mathsf{T}}]$, and $\Sigma_{yx} = \Sigma_{xy}^{\mathsf{T}}$.

Proposition 5.7 (Affine transformations)**.** *If $y = Ax + b$ then $\mathbb{E}[y] = A\mu + b$ and $\operatorname{Cov}(y) = A\Sigma A^{\mathsf{T}}$. If x and w are uncorrelated, $\operatorname{Cov}(Ax + Bw) = A\Sigma_x A^{\mathsf{T}} + B\Sigma_w B^{\mathsf{T}}$.*

Proof. Linearity of expectation gives the mean. Then $\operatorname{Cov}(y) = \mathbb{E}[A(x-\mu)(x-\mu)^{\mathsf{T}}A^{\mathsf{T}}] = A\Sigma A^{\mathsf{T}}$. With uncorrelated x and w, the cross terms $A\Sigma_{xw}B^{\mathsf{T}}$ vanish. ∎

This one proposition is the covariance propagation of the Kalman filter ($P \leftarrow \Phi P \Phi^{\mathsf{T}} + Q$), the covariance of a stacked measurement, and the augmentation formula of stochastic cloning ($P \leftarrow JPJ^{\mathsf{T}}$).

Proposition 5.8 (Expected value of a quadratic form)**.** *For any matrix A, $\mathbb{E}[x^\mathsf{T} A x] = \operatorname{tr}(A\Sigma) + \mu^\mathsf{T} A \mu$. In particular, if $\mu = 0$ then $\mathbb{E}[x^\mathsf{T}\Sigma^{-1}x] = \operatorname{tr}(I_n) = n$.*

Proof. $x^\mathsf{T} A x = \operatorname{tr}(A x x^\mathsf{T})$, and expectation commutes with the trace: $\mathbb{E}[x^\mathsf{T} A x] = \operatorname{tr}(A\,\mathbb{E}[x x^\mathsf{T}]) = \operatorname{tr}(A(\Sigma + \mu\mu^\mathsf{T}))$. ∎

The special case is the reason a consistent filter's normalized estimation error squared averages to the dimension of the state (Chapter 19).

5.4.2 Changing variables

Proposition 5.9 (Change of variables)**.** *If $y = g(x)$ with g invertible and differentiable, the densities are related by $p_y(y) = p_x\big(g^{-1}(y)\big)\left|\det \frac{\partial g^{-1}}{\partial y}\right|$.*

In one dimension this follows from equating probabilities of corresponding small intervals, $p_y(y)\,|\mathrm{d}y| = p_x(x)\,|\mathrm{d}x|$; in several dimensions the determinant measures how g^{-1} stretches volumes (Section 2.6). For an invertible linear map $y = Ax$ the factor is the constant $1/|\det A|$.

5.5 The multivariate Gaussian

5.5.1 Definition

Definition 5.10 (Multivariate Gaussian)**.** A random vector $x \in \mathbb{R}^n$ is Gaussian, $x \sim \mathcal{N}(\mu, \Sigma)$, if it can be written as $x = \mu + Ls$ where s has independent standard normal components and $LL^\mathsf{T} = \Sigma$. If $\Sigma > 0$ its density is

$$p(x) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \exp\!\Big(-\tfrac{1}{2}(x-\mu)^\mathsf{T}\Sigma^{-1}(x-\mu)\Big). \tag{5.8}$$

The quantity $d_M^2(x) = (x-\mu)^\mathsf{T}\Sigma^{-1}(x-\mu)$ is the squared *Mahalanobis distance* of x from the mean.

Derivation of the density. The density of s is the product of n standard normal densities, $(2\pi)^{-n/2}\exp(-\tfrac{1}{2}s^\mathsf{T}s)$. By Proposition 5.9 with $s = L^{-1}(x-\mu)$, $p(x) = (2\pi)^{-n/2}\exp\big(-\tfrac{1}{2}(x-\mu)^\mathsf{T}L^{-\mathsf{T}}L^{-1}(x-\mu)\big)/|\det L|$. Since $L^{-\mathsf{T}}L^{-1} = (LL^\mathsf{T})^{-1} = \Sigma^{-1}$ and $|\det L| = \sqrt{\det\Sigma}$, this is (5.8). ∎

By Proposition 5.7, the mean of x is μ and its covariance is $L\,\mathbb{E}[ss^\mathsf{T}]L^\mathsf{T} = \Sigma$. The definition also handles singular covariances (with L not invertible), which arise naturally in stochastic cloning (Chapter 25).

Geometry. The density is constant on the ellipsoids $d_M^2(x) = c$, whose axes point along the eigenvectors of Σ with lengths proportional to the square roots of its eigenvalues (Figures 2.4 and 5.1). The Mahalanobis distance measures distance in units of standard deviations along each principal axis, which makes it the natural way to ask whether a vector is "far" from a distribution.

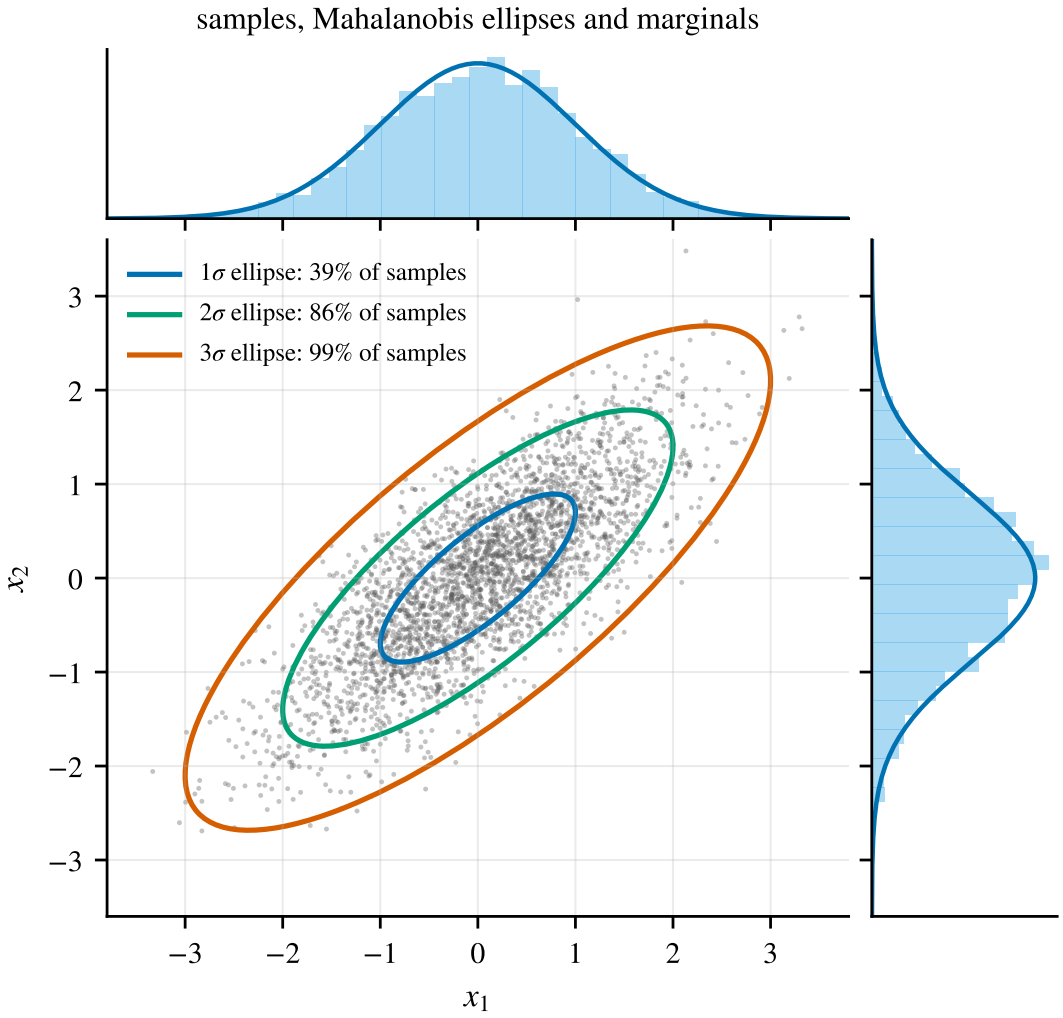


Figure 5.1: Samples of a correlated 2D Gaussian with the 1σ, 2σ and 3σ Mahalanobis ellipses, and the two marginal densities. The ellipses are aligned with the eigenvectors of Σ. In two dimensions they contain 39%, 86% and 99% of the probability, not the familiar 68–95–99.7% of one dimension.

Sampling. To draw from $\mathcal{N}(\mu, \Sigma)$, factor $\Sigma = LL^\mathsf{T}$ (Cholesky), draw $s \sim \mathcal{N}(0, I)$, and return $\mu + Ls$. Monte Carlo tests of covariance propagation (Milestone 5) use exactly this.

Closure. Affine maps of Gaussian vectors are Gaussian (directly from the definition). Marginals, conditionals and products of Gaussian densities are Gaussian (next section). Hence a Gaussian belief propagated through a linear system with Gaussian noise remains exactly Gaussian, and the Kalman filter is exact in that case. For jointly Gaussian vectors, uncorrelated implies independent: if the covariance is block diagonal, the density factors.

Information form. A Gaussian can equally be parameterized by the *information matrix* $\Lambda = \Sigma^{-1}$ and the *information vector* $\eta = \Sigma^{-1}\mu$. Then $-\log p(x) = \frac{1}{2}x^\mathsf{T}\Lambda x - \eta^\mathsf{T}x + \text{const}$. The information form is natural for combining independent pieces of evidence, since their information adds, and it is the form in which optimization-based estimators work.

5.5.2 Marginals and conditionals

Partition a jointly Gaussian vector as

$$\begin{bmatrix} a \\ b \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{bmatrix} \right), \qquad \Lambda = \Sigma^{-1} = \begin{bmatrix} \Lambda_{aa} & \Lambda_{ab} \\ \Lambda_{ba} & \Lambda_{bb} \end{bmatrix}. \tag{5.9}$$

Theorem 5.11 (Gaussian marginalization and conditioning). *1. The marginal of a is $\mathcal{N}(\mu_a, \Sigma_{aa})$.*

2. The conditional of a given b is $\mathcal{N}(\mu_{a|b}, \Sigma_{a|b})$ with

$$\mu_{a|b} = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(b - \mu_b), \qquad \Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}. \tag{5.10}$$

Proof. The marginal follows from Proposition 5.7 with the selection matrix $A = [I \;\; 0]$. For the conditional, define $\tilde{a} = a - \Sigma_{ab}\Sigma_{bb}^{-1}b$. Then $\mathrm{Cov}(\tilde{a}, b) = \Sigma_{ab} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{bb} = 0$, so $\tilde{a}$ and b are uncorrelated, and being jointly Gaussian they are independent. By Proposition 5.7, $\tilde{a} \sim \mathcal{N}(\mu_a - \Sigma_{ab}\Sigma_{bb}^{-1}\mu_b,\ \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba})$. Conditioning on b leaves the distribution of $\tilde{a}$ unchanged (independence), and $a = \tilde{a} + \Sigma_{ab}\Sigma_{bb}^{-1}b$, which gives (5.10). ∎

The conditional mean moves away from μ_a in proportion to how far b is from its mean, by the "regression" matrix $\Sigma_{ab}\Sigma_{bb}^{-1}$, and the conditional covariance shrinks by a positive semi-definite amount. It does not depend on the observed value of b, only on the fact that it was observed. The conditional covariance is the Schur complement Σ/Σ_{bb}; applying Proposition 3.11 to Σ shows that $\Lambda_{aa} = \Sigma_{a|b}^{-1}$ and that the marginal information matrix is a Schur complement of Λ. The two parameterizations therefore swap the roles of the two operations:

	covariance form (μ, Σ)	information form (η, Λ)
marginalize out b	Σ_{aa} (**delete rows/columns**)	$\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ (Schur compl.)
condition on b	$\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$ (Schur compl.)	Λ_{aa} (**delete rows/columns**)

Key idea: marginalization is free in covariance form

A Kalman filter stores covariances, so discarding a variable (the oldest clone of the MSCKF window) is literally deleting its rows and columns from P. Optimization-based estimators store information matrices, so the same operation requires a Schur complement, which creates fill-in and the "marginalization prior" of sliding-window smoothers (Chapter 10).

5.6 Fusing information: Bayes' rule for Gaussians

5.6.1 Combining two estimates

Suppose two independent sensors measure the same scalar x, with readings $z_1 = x + n_1$ and $z_2 = x + n_2$ and noise variances σ_1^2 and σ_2^2. With no other knowledge of x, the posterior

density is proportional to the product of the two likelihoods,

$$p(x \mid z_1, z_2) \propto \exp\Big(-\frac{(x-z_1)^2}{2\sigma_1^2}\Big)\exp\Big(-\frac{(x-z_2)^2}{2\sigma_2^2}\Big). \tag{5.11}$$

The exponent is a quadratic in x; completing the square shows that the posterior is Gaussian with

$$\frac{1}{\sigma^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}, \qquad \hat{x} = \sigma^2\Big(\frac{z_1}{\sigma_1^2} + \frac{z_2}{\sigma_2^2}\Big) = z_1 + \frac{\sigma_1^2}{\sigma_1^2+\sigma_2^2}(z_2 - z_1). \tag{5.12}$$

Inverse variances (information) add, and the fused estimate is an information-weighted average. The second form already has the shape of a Kalman update: start from z_1 and correct it by a gain times the discrepancy $z_2 - z_1$, with a gain between 0 and 1 that is large when the first estimate is poor. The fused variance is smaller than both, so combining independent information always helps.

5.6.2 The linear-Gaussian model

Proposition 5.12 (Linear-Gaussian posterior). *If $x \sim \mathcal{N}(\mu, P)$ and $z = Hx + n$ with $n \sim \mathcal{N}(0, R)$ independent of x, the posterior $p(x \mid z)$ is Gaussian, and its mean (which is also its mode) solves the weighted least-squares problem*

$$\hat{x} = \arg\min_x \ \|x-\mu\|_{P^{-1}}^2 + \|z - Hx\|_{R^{-1}}^2, \qquad \|e\|_W^2 := e^{\mathsf{T}} W e. \tag{5.13}$$

Its covariance is $(P^{-1} + H^{\mathsf{T}}R^{-1}H)^{-1}$.

Proof. By Bayes' rule $p(x \mid z) \propto p(z \mid x)p(x)$, and $-\log p(x \mid z) = \tfrac{1}{2}\|x-\mu\|_{P^{-1}}^2 + \tfrac{1}{2}\|z-Hx\|_{R^{-1}}^2 +$ const. This is a convex quadratic in x whose quadratic term is $\tfrac{1}{2}x^{\mathsf{T}}(P^{-1} + H^{\mathsf{T}}R^{-1}H)x$, so the posterior is Gaussian with that information matrix, and its mode, which for a Gaussian is the mean, minimizes the quadratic. ∎

This is the scalar fusion rule in matrix form: the prior contributes information P^{-1}, the measurement contributes $H^{\mathsf{T}}R^{-1}H$, and they add. Chapter 9 develops this into the general theory of estimators, and Chapter 12 turns it into the Kalman filter.

5.7 Limit theorems and Monte Carlo

5.7.1 The law of large numbers

Let $X_1, X_2, \dots$ be independent draws from the same distribution, with mean μ and variance σ^2, and let $\bar{X}_N = \frac{1}{N}\sum_{i=1}^{N} X_i$ be their *sample mean*. Then $\mathbb{E}[\bar{X}_N] = \mu$ and, because variances of independent terms add, $\operatorname{Var}(\bar{X}_N) = \sigma^2/N$.

Theorem 5.13 (Weak law of large numbers). *For every $\epsilon > 0$, $P(|\bar{X}_N - \mu| > \epsilon) \to 0$ as $N \to \infty$.*

Proof. Chebyshev's inequality $P(|Y - \mathbb{E}Y| \ge \epsilon) \le \operatorname{Var}(Y)/\epsilon^2$ (which follows from $\operatorname{Var}(Y) \ge \mathbb{E}[(Y-\mathbb{E}Y)^2\,\mathbf{1}_{|Y-\mathbb{E}Y|\ge\epsilon}] \ge \epsilon^2 P(|Y-\mathbb{E}Y|\ge\epsilon)$) applied to $\bar{X}_N$ gives the bound $\sigma^2/(N\epsilon^2) \to 0$. ∎

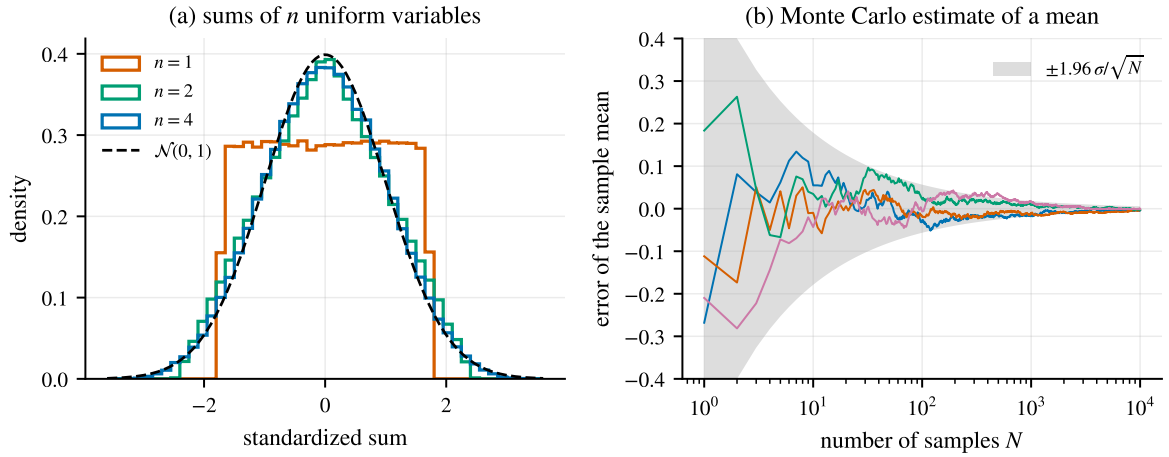


Figure 5.2: (a) The central limit theorem: histograms of the standardized sum of n independent uniform variables approach the standard normal density already for $n = 4$. (b) Monte Carlo estimation of a mean: the error of the sample mean shrinks like $1/\sqrt{N}$ (gray band: $\pm 1.96\,\sigma/\sqrt{N}$, the 95% interval).

The law of large numbers justifies *Monte Carlo* methods: to estimate an expectation, average many independent samples. The error shrinks only like $\sigma/\sqrt{N}$, so each additional digit of accuracy costs a hundred times more samples.

5.7.2 The central limit theorem

Theorem 5.14 (Central limit theorem)**.** *Under the same assumptions, the distribution of* $\sqrt{N}(\bar{X}_N - \mu)/\sigma$ *converges to* $\mathcal{N}(0, 1)$.

Sums of many small independent effects are approximately Gaussian, whatever the distribution of each effect (Figure 5.2). Electronic sensor noise is the sum of many microscopic fluctuations, and integrating a noisy signal sums many samples; both are why Gaussian noise models work well in navigation. They work less well for rare large errors (outliers), which is why estimators also need gating and robust methods (Chapters 10 and 18).

5.8 The chi-squared distribution

Definition 5.15. If $s_1, \dots, s_k$ are independent standard normal variables, $\gamma = \sum_{i=1}^{k} s_i^2$ has the χ^2 distribution with k degrees of freedom, written $\gamma \sim \chi^2_k$.

Proposition 5.16 (Properties of χ^2_k)**.** $\mathbb{E}[\gamma] = k$ *and* $\mathrm{Var}(\gamma) = 2k$. *Its density is*

$$p(\gamma) = \frac{\gamma^{k/2-1} e^{-\gamma/2}}{2^{k/2}\,\Gamma(k/2)}, \qquad \gamma > 0, \tag{5.14}$$

where Γ *is the gamma function* ($\Gamma(m) = (m-1)!$ *for integers,* $\Gamma(\tfrac{1}{2}) = \sqrt{\pi}$).

Proof of the moments and of the case $k = 1$. For $s \sim \mathcal{N}(0, 1)$, $\mathbb{E}[s^2] = 1$ and $\mathbb{E}[s^4] = 3$ (integration by parts), so $\mathrm{Var}(s^2) = 2$; the moments of the sum follow by additivity over independent terms.

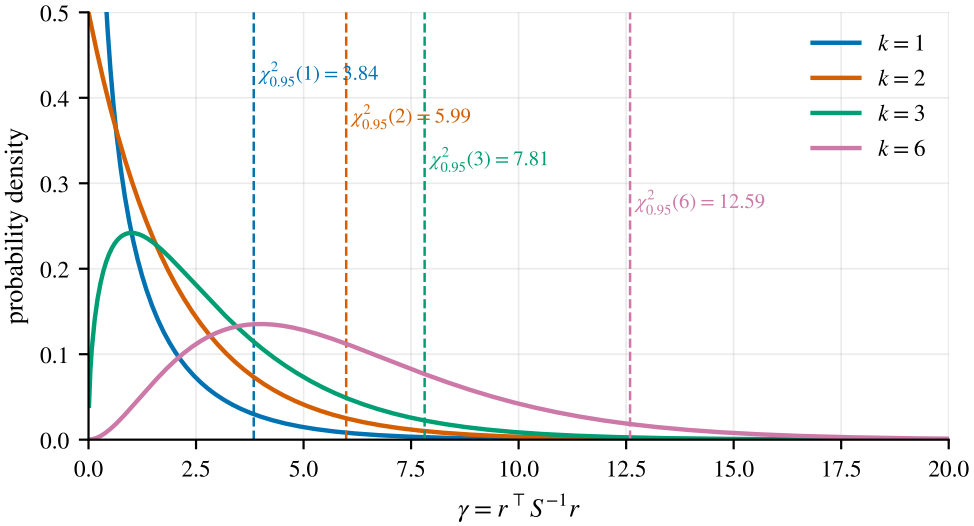


Figure 5.3: χ^2 densities and their 95% quantiles. A 2D image measurement whose normalized innovation squared exceeds 5.99 would occur by chance only 5% of the time if the filter's covariance were correct.

For $k = 1$, $\gamma = s^2$ takes the value γ when $s = \pm\sqrt{\gamma}$, so $p(\gamma) = 2 \cdot \frac{1}{\sqrt{2\pi}} e^{-\gamma/2} \cdot \frac{\mathrm{d}\sqrt{\gamma}}{\mathrm{d}\gamma} = \frac{1}{\sqrt{2\pi\gamma}} e^{-\gamma/2}$, which is the formula with $k = 1$. The general case follows by convolving densities (or by integrating the Gaussian density over spherical shells). ∎

Theorem 5.17 (Mahalanobis distance of a Gaussian). *If $x \sim \mathcal{N}(\mu, \Sigma)$ in $\mathbb{R}^k$ with $\Sigma \succ 0$, then $(x-\mu)^\top \Sigma^{-1}(x-\mu) \sim \chi^2_k$.*

Proof. With $\Sigma = LL^\top$, the whitened vector $s = L^{-1}(x-\mu)$ is $\mathcal{N}(0, I)$ by Proposition 5.7, and $(x-\mu)^\top\Sigma^{-1}(x-\mu) = s^\top s = \sum_{i} s_i^2$. ∎

The *quantile* $\chi^2_p(k)$ is the value below which a χ^2_k variable falls with probability p (Figure 5.3; tables in Appendix C). Because independent χ^2 variables add, with their degrees of freedom adding, the sum of N independent squared Mahalanobis distances of n-dimensional Gaussians is χ^2_{Nn}. As k grows, χ^2_k is approximately $\mathcal{N}(k, 2k)$, so its quantiles approach $k \pm 1.96\sqrt{2k}$.

5.9 Sample statistics and hypothesis tests

5.9.1 Estimating a mean and a variance

Given independent samples $X_1, \dots, X_N$ of a distribution with unknown mean and variance, the natural estimates are the sample mean $\bar{X}$ and the *sample variance*

$$s^2 = \frac{1}{N-1} \sum_{i=1}^{N} (X_i - \bar{X})^2. \tag{5.15}$$

The divisor $N-1$ makes s^2 unbiased. Expanding, $\sum_i (X_i - \bar{X})^2 = \sum_i (X_i - \mu)^2 - N(\bar{X} - \mu)^2$, whose expectation is $N\sigma^2 - N \cdot \sigma^2/N = (N-1)\sigma^2$: one degree of freedom is used up by estimating

the mean. For Gaussian samples one can show moreover that $(N-1)s^2/\sigma^2 \sim \chi^2_{N-1}$. This gives exact acceptance intervals for comparing a Monte Carlo sample variance with a predicted variance, as Milestone 5 requires: the ratio s^2/σ^2 should lie between $\chi^2_{0.025}(N-1)/(N-1)$ and $\chi^2_{0.975}(N-1)/(N-1)$ 95% of the time (for $N = 500$, between 0.880 and 1.128; Appendix C).

5.9.2 Hypothesis testing

A *hypothesis test* decides whether data are compatible with a model. The recipe:

1. State a *null hypothesis* H_0, for example "this measurement is consistent with the filter's prediction".
2. Choose a *test statistic* whose distribution under H_0 is known, for example the squared Mahalanobis distance of the innovation, which is χ^2_m under H_0.
3. Choose a *significance level* α (often 5%) and an *acceptance region* that contains the statistic with probability $1-\alpha$ under H_0.
4. Reject H_0 if the statistic falls outside the region.

Two errors are possible. A *type I error* rejects a true H_0; by construction it happens with probability α. A *type II error* accepts a false H_0; its probability depends on how different the truth is from H_0, and one minus it is the *power* of the test. Lowering α reduces false rejections at the price of more missed detections.

One-sided tests reject only large values. Measurement gating is one-sided: a measurement is rejected when its Mahalanobis distance exceeds $\chi^2_{1-\alpha}(m)$, because only large innovations indicate outliers (Chapter 18). *Two-sided* tests reject both unusually large and unusually small values. Filter consistency tests are two-sided: an average normalized error that is too small reveals an underconfident filter just as one that is too large reveals an overconfident one (Chapter 19).

Pitfall: a 5% test rejects 5% of good data

At a 95% gate, one in twenty perfectly good measurements is rejected. Over thousands of measurements this is expected and harmless, provided rejections are counted. A rejection rate far from the design value is itself a symptom worth investigating.

5.10 Nonlinear transformations

If $y = f(x)$ with f nonlinear and $x \sim \mathcal{N}(\mu, \Sigma)$, then y is generally *not* Gaussian. The extended Kalman filter approximates it by a Gaussian, linearizing f about the mean:

$$y \approx f(\mu) + J(x-\mu), \quad J = \left.\frac{\partial f}{\partial x}\right|_{\mu} \qquad \Longrightarrow \qquad y \mathrel{\dot\sim} \mathcal{N}\big(f(\mu),\, J\Sigma J^{\mathsf{T}}\big). \tag{5.16}$$

How much does this miss? In one dimension, Taylor expansion to second order gives $f(x) \approx f(\mu) + f'(\mu)(x-\mu) + \tfrac{1}{2}f''(\mu)(x-\mu)^2$, and taking expectations,

$$\mathbb{E}[f(x)] \approx f(\mu) + \tfrac{1}{2} f''(\mu)\,\sigma^2. \tag{5.17}$$

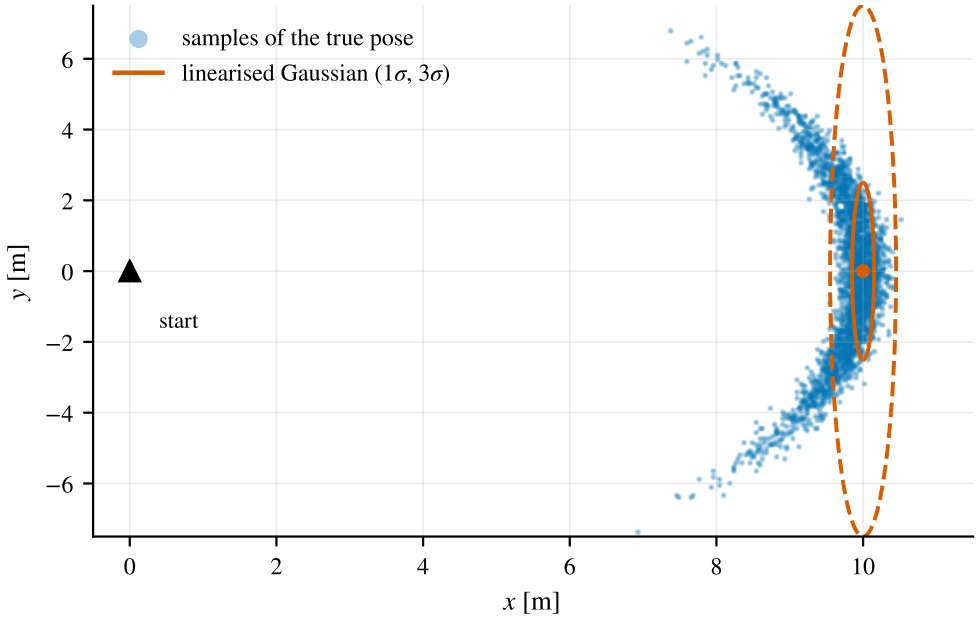


Figure 5.4: A robot drives 10 m with a heading uncertainty of $\sigma_\psi = 0.25$ rad. The true distribution of its final position (samples) is a curved "banana". The linearized Gaussian (ellipses) matches its first two moments only locally, puts probability where the robot cannot be, and misses the curvature. Errors of this kind are why an estimator must keep its errors small, the central design principle of the error-state filter.

Linearization therefore ignores a *bias* proportional to the curvature times the variance. The approximation is good when f is nearly linear over the region where x has most of its probability mass, a few standard deviations around μ.

Figure 5.4 shows a classic failure. When heading uncertainty is large, positions are distributed along an arc, and the linearized Gaussian is a poor description: its mean lies outside the arc, and it assigns probability to positions the robot cannot reach. Two remedies exist. One is to reduce the uncertainty that must be linearized: the error-state filter of Chapter 14 keeps the estimation error, not the state, as the quantity it linearizes, and continually resets it to zero. The other is to choose coordinates in which the distribution is close to Gaussian: a Gaussian on the heading angle and travelled distance, pushed through the exact kinematics, reproduces the banana. Chapter 8 develops that idea for rotations, and Chapter 13 presents a third remedy, sigma-point methods that propagate a few carefully chosen samples instead of a linearization.

Key takeaways

- Bayes' rule converts a sensor model $p(z \mid x)$ and a prior $p(x)$ into the posterior $p(x \mid z)$.
- Affine maps propagate covariances as $A\Sigma A^\mathsf{T}$. This single fact drives prediction, cloning and stacking.
- $\mathbb{E}[x^\mathsf{T} A x] = \operatorname{tr}(A\Sigma) + \mu^\mathsf{T} A \mu$; the normalized squared error of an n-dimensional Gaussian averages to n.

- In covariance form, marginalization deletes rows and columns and conditioning takes a Schur complement. Information form is the reverse.
- Fusing independent Gaussian information adds inverse covariances; the result is a gain-weighted correction.
- Monte Carlo errors shrink like $1/\sqrt{N}$; sums of many small effects are nearly Gaussian.
- The squared Mahalanobis distance of an n-dimensional Gaussian is χ^2_n: the basis of gating and consistency tests.
- Linearization ignores curvature-induced bias and fails when uncertainty is large relative to curvature.

Exercises

5.1 A gyro is either healthy (99%) or faulty (1%). A self-test flags 98% of faulty units and 3% of healthy ones. A unit is flagged; what is the probability that it is faulty?

5.2 Let $x \sim \mathcal{N}(0, \mathrm{diag}(1, 4))$ and $y = Ax$ with A a rotation by 30°. Compute $\mathrm{Cov}(y)$ and sketch the 1σ ellipses of x and y.

5.3 Prove that $\Sigma_{a|b} \preceq \Sigma_{aa}$: conditioning never increases uncertainty in the Loewner order. When is the inequality an equality?

5.4 Derive the information-form marginal $\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ from Proposition 3.11 and show that it equals Σ_{aa}^{-1}.

5.5 Complete the square to derive (5.12).

5.6 Solve (5.13) by setting the gradient to zero, and show that the solution can be written $\hat{x} = \mu + K(z - H\mu)$ for $K = (P^{-1} + H^\mathsf{T}R^{-1}H)^{-1}H^\mathsf{T}R^{-1}$.

5.7 For $x \sim \mathcal{N}(0, \sigma^2)$ and $y = x^2$, compute the exact mean of y and its first-order linearized prediction. Explain the discrepancy.

5.8 What fraction of a 2D Gaussian's probability lies inside its 1σ Mahalanobis ellipse? (Hint: $P(\chi^2_2 \le 1)$.) Inside its 3σ ellipse?

5.9 Show that $\mathbb{E}[s^4] = 3$ for $s \sim \mathcal{N}(0, 1)$ by integrating by parts.

5.10 A Monte Carlo test with $N = 100$ runs finds a sample variance 25% larger than predicted. Is this evidence of an error at the 5% level?

Chapter 6

Random Processes, Spectra and Sensor Noise

In this chapter

- Describe noise that evolves in time: random sequences, autocorrelation, stationarity, white noise and random walks.
- Understand power spectral densities and why sensor noise is specified in units per $\sqrt{\text{Hz}}$.
- Understand continuous-time white noise, its sampling, and aliasing.
- Derive how integrated noise grows, and how covariance evolves in a linear stochastic system.
- Derive the Allan variance signatures used to identify IMU noise parameters.
- Model colored noise with shaping filters, the reason sensor biases become filter states.

Chapter 5 dealt with random vectors: a fixed number of uncertain quantities. Sensor noise is a different kind of object: a random *signal*, a function of time, whose values at different times may or may not be related. This chapter develops the theory of such random processes, as far as it is needed to understand IMU datasheets, to model sensor errors in a filter, and to discretize those models correctly.

6.1 Random sequences

6.1.1 Definitions

A *random sequence* (discrete-time random process) is a sequence of random variables $x_0, x_1, x_2, \ldots$, for example the noise added to successive IMU samples. One run of the experiment produces one *sample path*; repeating the experiment produces another. Statistics are taken across this *ensemble* of paths. The *mean function* is $\mu_k = \mathbb{E}[x_k]$ and the *autocovariance*

$$C(k, j) = \mathbb{E}\big[(x_k - \mu_k)(x_j - \mu_j)\big] \tag{6.1}$$

says how strongly the values at two times are related. A sequence is *wide-sense stationary* if its mean is constant and its autocovariance depends only on the lag, $C(k, j) = C(k - j)$: its statistical character does not change with time.

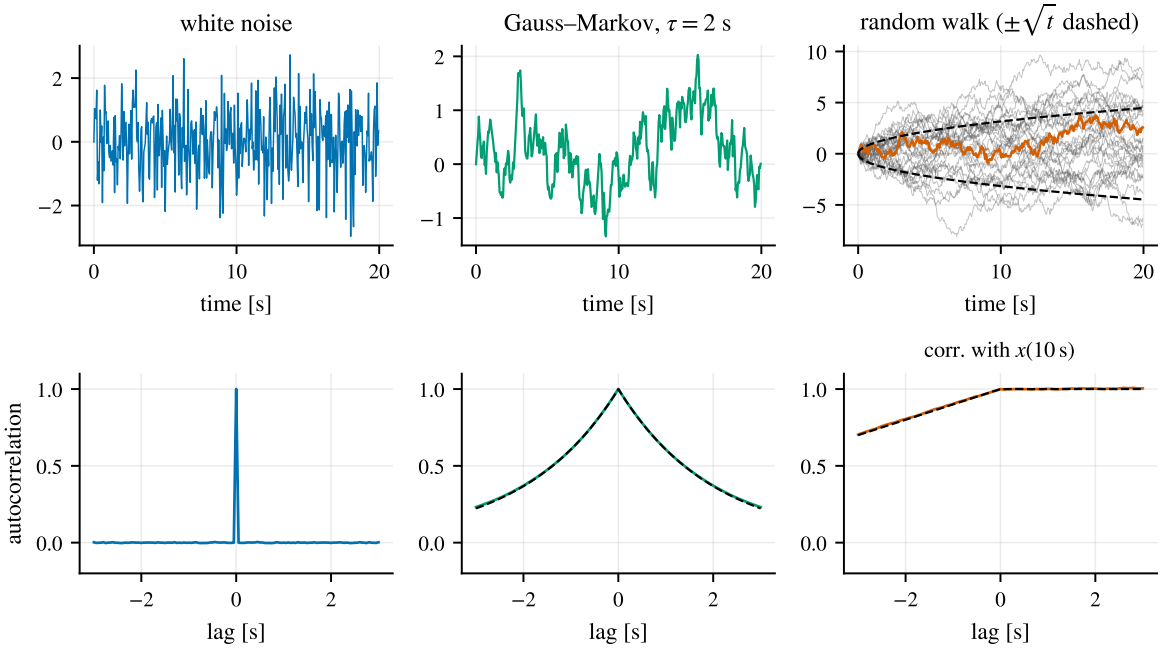


Figure 6.1: Sample paths (top) and autocorrelation functions (bottom) of white noise, a first-order Gauss–Markov sequence with correlation time $\tau = 2$ s, and a random walk, all driven by the same white sequence. White noise is uncorrelated with itself at any non-zero lag; the Gauss–Markov sequence forgets its past exponentially; the random walk never forgets and its spread grows like $\sqrt{t}$.

6.1.2 Three fundamental sequences

White noise. A *white sequence* has zero mean and uncorrelated values: $C(k) = \sigma^2$ for $k = 0$ and 0 otherwise. Each sample carries no information about the others. The measurement noise of a well-behaved sensor is approximately white.

Random walk. Summing white noise gives a *random walk*: $x_{k+1} = x_k + w_k$, $x_0 = 0$, with w_k white of variance σ^2. Then $x_k = \sum_{j<k} w_j$ and, because variances of uncorrelated terms add,

$$\mathrm{Var}(x_k) = k\sigma^2. \tag{6.2}$$

The spread grows like $\sqrt{k}$ without bound, and the sequence is not stationary. Its autocovariance $C(k, j) = \min(k, j)\sigma^2$ shows that it remembers its whole past. Slowly drifting sensor biases are modeled as random walks.

First-order Gauss–Markov sequence. Between the two lies $x_{k+1} = a x_k + w_k$ with $|a| < 1$. If the sequence is stationary, taking the variance of both sides gives $V = a^2 V + \sigma^2$, so

$$\mathrm{Var}(x_k) = \frac{\sigma^2}{1 - a^2}, \qquad C(j) = \frac{\sigma^2}{1 - a^2}\, a^{|j|}, \tag{6.3}$$

the second because $x_{k+j} = a^j x_k + (\text{noise uncorrelated with } x_k)$. The sequence forgets its past exponentially, with a *correlation time* of about $1/(1 - a)$ steps. It is a bounded, slowly varying

error: a useful model for biases that wander but do not diverge, and for many environmental disturbances.

6.2 Frequency-domain description

6.2.1 Fourier analysis in brief

Every reasonable signal is a superposition of sinusoids. The *Fourier transform*

$$X(f) = \int_{-\infty}^{\infty} x(t)\, e^{-i2\pi f t}\, \mathrm{d}t, \qquad x(t) = \int_{-\infty}^{\infty} X(f)\, e^{i2\pi f t}\, \mathrm{d}f \tag{6.4}$$

gives the complex amplitude $X(f)$ of the component at frequency f (in Hz). Two facts are all we need. First, Parseval's theorem $\int |x(t)|^2 \mathrm{d}t = \int |X(f)|^2 \mathrm{d}f$: energy can be counted in time or in frequency. Second, linear time-invariant systems (filters) act on each frequency independently, multiplying $X(f)$ by a *frequency response* $H(f)$. A low-pass filter has $|H(f)| \approx 1$ at low frequencies and $|H(f)| \approx 0$ at high ones.

6.2.2 Power spectral density

A random signal has infinite energy (it goes on forever), but finite *power*. Its *power spectral density* (PSD) describes how that power is distributed over frequency.

Definition 6.1 (Power spectral density)**.** For a wide-sense stationary process with autocovariance $C(\tau) = \mathbb{E}[x(t+\tau)x(t)]$ (zero mean), the (two-sided) PSD is

$$S(f) = \int_{-\infty}^{\infty} C(\tau)\, e^{-i2\pi f \tau}\, \mathrm{d}\tau. \tag{6.5}$$

This relation between the autocovariance and the PSD is the *Wiener–Khinchin theorem*. Evaluating the inverse transform at $\tau = 0$ shows that the variance is the total area under the PSD:

$$\mathrm{Var}(x) = C(0) = \int_{-\infty}^{\infty} S(f)\, \mathrm{d}f. \tag{6.6}$$

So $S(f)\,\mathrm{d}f$ is the contribution of the frequency band $[f, f + \mathrm{d}f]$ to the variance, and the PSD has units of $[x]^2/\mathrm{Hz}$. Passing the process through a filter with frequency response $H(f)$ multiplies its PSD by $|H(f)|^2$.

Pitfall: one-sided and two-sided PSDs

Engineers often use a *one-sided* PSD defined only for $f \geq 0$ and equal to twice the two-sided one, so that the variance is $\int_0^\infty S_1(f)\,\mathrm{d}f$. Datasheets rarely say which convention they use, and the two differ by a factor of $\sqrt{2}$ in the noise density. These notes use the convention of the EuRoC calibration files and of most of the VIO literature: a continuous white noise w with $\mathbb{E}[w(t)w(s)] = \sigma^2\delta(t-s)$, i.e. two-sided PSD σ^2.

6.3 Continuous-time white noise

Definition 6.2 (Continuous-time white noise)**.** A zero-mean process $w(t) \in \mathbb{R}^n$ is white with *noise density* (intensity) Q_c if $\mathbb{E}[w(t)\,w(s)^\mathsf{T}] = Q_c\,\delta(t-s)$, where δ is the Dirac delta. For a scalar process we write $Q_c = \sigma^2$ and call σ the noise density.

Continuous white noise is an idealization: its PSD is flat at all frequencies, so its variance $\int S\,\mathrm{d}f$ is infinite. Physical noise is white only up to some bandwidth. What makes the idealization useful is that its *integrals* are perfectly well behaved: the integral of white noise over an interval of length T is a zero-mean random variable with variance

$$\operatorname{Var}\left(\int_0^T w(t)\,\mathrm{d}t\right) = \int_0^T \int_0^T \sigma^2 \delta(t-s)\,\mathrm{d}s\,\mathrm{d}t = \sigma^2 T. \tag{6.7}$$

Since δ has units of $1/\text{time}$, σ has units of $[w]\cdot\sqrt{\text{s}} = [w]/\sqrt{\text{Hz}}$. For a gyroscope, $[w] = \text{rad/s}$, so its noise density is in $\text{rad/s}/\sqrt{\text{Hz}}$, the units of `gyroscope_noise_density` in EuRoC's `sensor.yaml`.

From continuous to discrete noise. A digital sensor reports samples every Δt. If each sample is the average of the continuous white input over its interval, which an anti-aliasing filter approximates, the sample noise has variance

$$\operatorname{Var}\left(\frac{1}{\Delta t}\int_0^{\Delta t} w(s)\,\mathrm{d}s\right) = \frac{\sigma^2}{\Delta t}, \qquad \sigma_{\text{discrete}} = \frac{\sigma}{\sqrt{\Delta t}}. \tag{6.8}$$

The frequency-domain view agrees: an ideal low-pass filter that keeps the band $|f| < f_s/2$ (with $f_s = 1/\Delta t$) passes a variance of $\int_{-f_s/2}^{f_s/2} \sigma^2\,\mathrm{d}f = \sigma^2 f_s = \sigma^2/\Delta t$. Sampling faster does not reduce the noise per unit time; it merely spreads it over more samples, each noisier. For EuRoC's gyroscope, $\sigma_g = 1.6968 \times 10^{-4}$ $\text{rad/s}/\sqrt{\text{Hz}}$ at 200 Hz gives a per-sample standard deviation of 2.4×10^{-3} rad/s.

Conversely, a random walk driven by white noise, $\dot{b} = w$, gains variance $\sigma^2\Delta t$ over a step Δt, so its exact discretization is $b_{k+1} = b_k + \sqrt{\Delta t}\,\sigma\,s_k$ with $s_k \sim \mathcal{N}(0,1)$. The two conversions go in opposite directions, $\sigma/\sqrt{\Delta t}$ for the sample noise and $\sigma\sqrt{\Delta t}$ for the walk; confusing them is a classic error in noise simulators.

6.4 Sampling and aliasing

Theorem 6.3 (Sampling theorem, Nyquist–Shannon)**.** *A signal containing no frequencies at or above $f_s/2$ is completely determined by its samples at rate f_s. The frequency $f_s/2$ is the* Nyquist frequency.

A component at a frequency f above the Nyquist frequency does not disappear when sampled: it masquerades as a lower frequency $|f - k f_s|$ for the integer k that brings it into $[0, f_s/2]$ (Figure 6.2). This *aliasing* cannot be undone after sampling, so it must be prevented before, by an analog or digital *anti-aliasing* low-pass filter. IMUs contain such filters, but strong vibration near or above the sampling rate (propellers at hundreds of Hz) can still alias into low frequencies and appear as bias-like errors. The same principle governs images: a

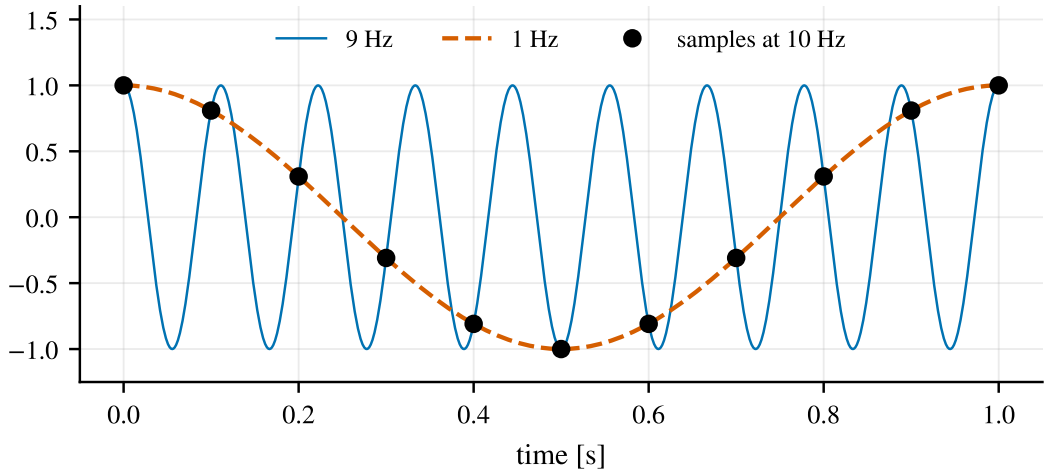


Figure 6.2: Aliasing. A 9 Hz sinusoid sampled at 10 Hz (dots) produces exactly the same samples as a 1 Hz sinusoid (dashed). After sampling, the two are indistinguishable.

camera samples a scene spatially, and an image must be blurred before it is down-sampled, or fine textures turn into false coarse patterns (Chapter 21).

6.5 Integrated noise

Proposition 6.4 (Integrated white noise). *Let w be scalar white noise with density σ. The m-fold integral $I_m(t) = \int_0^t \int_0^{t_1} \cdots \int_0^{t_{m-1}} w(t_m)\,\mathrm{d}t_m \cdots \mathrm{d}t_1$ has zero mean and*

$$\operatorname{Var}\big(I_m(t)\big) = \sigma^2 \,\frac{t^{2m-1}}{(2m-1)\,\big((m-1)!\big)^2}. \tag{6.9}$$

For $m = 1, 2, 3, 4$ this gives $\sigma^2 t$, $\sigma^2 t^3/3$, $\sigma^2 t^5/20$ and $\sigma^2 t^7/252$.

Proof. By Cauchy's formula for repeated integration (proved by induction, exchanging the order of integration), $I_m(t) = \int_0^t \frac{(t-s)^{m-1}}{(m-1)!}\, w(s)\,\mathrm{d}s$. For a weighted integral $\int g(s) w(s)\,\mathrm{d}s$ of white noise, $\operatorname{Var} = \iint g(s)g(s')\,\sigma^2\delta(s-s')\,\mathrm{d}s\,\mathrm{d}s' = \sigma^2 \int g(s)^2\,\mathrm{d}s$. Here $\int_0^t (t-s)^{2m-2}\,\mathrm{d}s/((m-1)!)^2 = t^{2m-1}/((2m-1)((m-1)!)^2)$. ∎

The single integral I_1 is the continuous random walk, or *Wiener process*, whose standard deviation grows like $\sqrt{t}$. Integrating accelerometer white noise twice into position gives $\sigma_a^2 t^3/3$: position uncertainty grows like $t^{3/2}$. Each further integration adds two powers of t to the variance. Chapter 17 uses this proposition to explain how fast an inertial navigator degrades.

6.6 Linear stochastic differential equations

Most noise models in navigation are linear systems driven by white noise:

$$\dot{x}(t) = F(t)\,x(t) + G(t)\,w(t), \qquad \mathbb{E}[w(t)w(s)^{\mathsf{T}}] = Q_c\,\delta(t-s). \tag{6.10}$$

Because white noise is so irregular, such equations are made rigorous by stochastic calculus. For linear systems, however, the results we need follow from a simple limiting argument.

Proposition 6.5 (Mean and covariance propagation). *For* (6.10)*, the mean* $m(t) = \mathbb{E}[x(t)]$ *and covariance* $P(t)$ *evolve as*

$$\dot{m} = Fm, \qquad \dot{P} = FP + PF^{\mathsf{T}} + GQ_cG^{\mathsf{T}}, \tag{6.11}$$

with solution $P(t) = \Phi(t,t_0)P(t_0)\Phi(t,t_0)^{\mathsf{T}} + \int_{t_0}^{t} \Phi(t,s)G(s)Q_cG(s)^{\mathsf{T}}\Phi(t,s)^{\mathsf{T}}\,\mathrm{d}s$.

Proof. Over a short interval h, $x(t+h) = (I + Fh)x(t) + \int_t^{t+h} Gw\,\mathrm{d}s + O(h^2)$, and the noise integral is zero-mean with covariance $GQ_cG^{\mathsf{T}}h$ and uncorrelated with $x(t)$. By Proposition 5.7, $P(t+h) = (I+Fh)P(I+Fh)^{\mathsf{T}} + GQ_cG^{\mathsf{T}}h + O(h^2) = P + (FP + PF^{\mathsf{T}} + GQ_cG^{\mathsf{T}})h + O(h^2)$. Subtract $P(t)$, divide by h and let $h \to 0$. The integral form is verified by differentiating it (Proposition 4.7). ∎

Equation (6.11) is the *Lyapunov differential equation*. The first two terms move and shape the existing uncertainty with the dynamics; the last term pumps in new uncertainty from the noise. Its integral form over one time step is exactly the discrete process noise covariance Q_d of the Kalman filter (Chapters 11 and 12). For a stable time-invariant system, P approaches the solution of the algebraic Lyapunov equation $0 = FP + PF^{\mathsf{T}} + GQ_cG^{\mathsf{T}}$.

6.6.1 The continuous Gauss–Markov process

The scalar system $\dot{x} = -x/\tau + w$ with noise density σ is the continuous counterpart of the Gauss–Markov sequence. Its stationary variance solves $0 = -2P/\tau + \sigma^2$:

$$P_\infty = \frac{\sigma^2\tau}{2}, \qquad C(\Delta t) = P_\infty\, e^{-|\Delta t|/\tau}, \qquad S(f) = \frac{\sigma^2\tau^2}{1 + (2\pi f\tau)^2}. \tag{6.12}$$

(The PSD is the white input PSD σ^2 times $|H(f)|^2$ for the system's frequency response $H(f) = 1/(i2\pi f + 1/\tau)$; integrating it over all frequencies returns P_∞.) The autocovariance decays with time constant τ, and the PSD is flat up to the corner frequency $1/(2\pi\tau)$ and falls off beyond it. Its exact discretization over Δt is $x_{k+1} = e^{-\Delta t/\tau}x_k + w_k$ with $\mathrm{Var}(w_k) = P_\infty(1 - e^{-2\Delta t/\tau})$. As $\tau \to \infty$ it becomes a random walk; as $\tau \to 0$ with the low-frequency level $\sigma^2\tau^2$ of its PSD held fixed, it becomes white noise.

6.7 Colored noise and state augmentation

A Kalman filter assumes that its process and measurement noises are white. When a disturbance is *colored* (correlated in time), the standard remedy is to model it as the output of a linear system driven by white noise, a *shaping filter*, and to add the state of that system to the filter state. A sensor bias that drifts as a random walk is exactly this: b is colored, but $\dot{b} = w_b$ is white-driven, so b becomes part of the state. This is why IMU biases appear in the state vector of every inertial filter (Chapter 15). Ignoring the correlation, and treating a slowly varying error as white, makes the filter believe that errors average out when they do not, and leads to overconfidence.

6.8 The Allan variance

How are the noise densities of a sensor measured? Record it at rest for hours and compute its *Allan variance*, a tool developed for characterizing atomic clocks that separates noise processes by their time scale.

Definition 6.6 (Allan variance)**.** Split a record of rate samples $y(t)$ into consecutive clusters of duration τ, and let $\bar{y}_k(\tau)$ be the average over cluster k. The Allan variance is half the mean squared difference of consecutive cluster averages:

$$\sigma_A^2(\tau) = \tfrac{1}{2}\,\mathbb{E}\big[(\bar{y}_{k+1}(\tau) - \bar{y}_k(\tau))^2\big]. \tag{6.13}$$

In practice it is computed from the integrated signal $\theta(t) = \int_0^t y$ (for a gyroscope, the angle), since $\bar{y}_{k+1} - \bar{y}_k = (\theta_{k+2} - 2\theta_{k+1} + \theta_k)/\tau$ with $\theta_j = \theta(j\tau)$. The *overlapping* estimator averages over all starting times rather than only disjoint clusters, which uses the data more efficiently.

Proposition 6.7 (Allan variance signatures)**.** *For white noise of density σ, $\sigma_A^2(\tau) = \sigma^2/\tau$. For a random walk $\dot{b} = w_b$ with density σ_b, $\sigma_A^2(\tau) = \sigma_b^2\tau/3$.*

Proof. White noise. Each cluster average is an integral of white noise divided by τ, with variance $\sigma^2\tau/\tau^2 = \sigma^2/\tau$; consecutive averages involve disjoint intervals and are uncorrelated, so their difference has variance $2\sigma^2/\tau$, and half of it is σ^2/τ. *Random walk.* Write $b(t) = \int_0^t w_b$. The difference of two consecutive cluster averages is $D = \frac{1}{\tau}\int_\tau^{2\tau} b\,\mathrm{d}t - \frac{1}{\tau}\int_0^\tau b\,\mathrm{d}t$. Substituting b and exchanging the order of integration expresses D as $\frac{1}{\tau}\int_0^{2\tau} h(s)w_b(s)\,\mathrm{d}s$ with $h(s) = s$ for $s \le \tau$ and $h(s) = 2\tau - s$ for $s > \tau$ (a triangular weight). Its variance is $\frac{\sigma_b^2}{\tau^2}\big(\int_0^\tau s^2\mathrm{d}s + \int_\tau^{2\tau}(2\tau - s)^2\mathrm{d}s\big) = \frac{2}{3}\sigma_b^2\tau$, and half of it is $\sigma_b^2\tau/3$. ∎

On a log-log plot of the *Allan deviation* $\sigma_A(\tau)$, independent noise processes therefore appear as straight lines with characteristic slopes:

Process	$\sigma_A^2(\tau)$	slope of σ_A
quantization noise	$3Q^2/\tau^2$	-1
white noise (density σ)	σ^2/τ	$-\frac{1}{2}$
bias instability (B)	$\approx (0.664B)^2$	0
bias random walk (density σ_b)	$\sigma_b^2\tau/3$	$+\frac{1}{2}$
rate ramp (R)	$R^2\tau^2/2$	$+1$

The white-noise density is read off the $-\frac{1}{2}$ line at $\tau = 1$ s, and the random-walk density off the $+\frac{1}{2}$ line at $\tau = 3$ s. The minimum of the curve marks the averaging time beyond which a stationary sensor's bias starts to wander; its height is related to the *bias instability*, a flicker-noise floor that neither white noise nor a random walk describes exactly.

Figure 6.3 shows the procedure on simulated data with the EuRoC parameters of Chapter 15. On real sensors, a flat bias-instability region appears near the minimum, which the random-walk model only approximates. The IEEE standard procedure [30] and El-Sheimy et al. [14] give practical details.

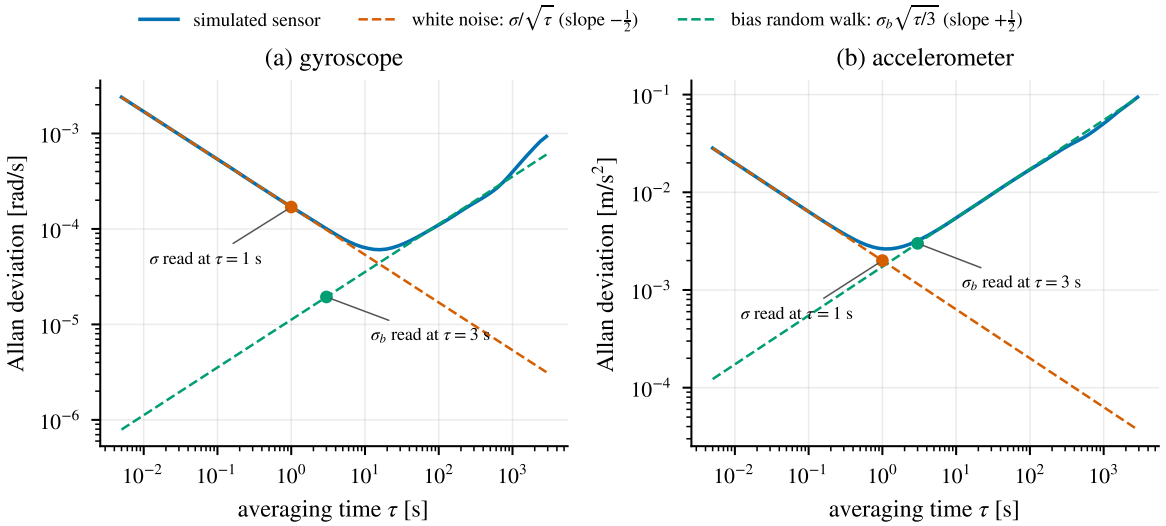


Figure 6.3: Allan deviation of a simulated 4-hour static recording with EuRoC's IMU noise parameters. The $-\frac{1}{2}$ slope of white noise dominates short averaging times and the $+\frac{1}{2}$ slope of the bias random walk dominates long ones. Reading the asymptotes at $\tau = 1$ s and $\tau = 3$ s recovers the densities.

Key takeaways

- White noise is uncorrelated in time; a random walk is its running sum, with variance growing linearly; a Gauss–Markov process forgets its past exponentially.
- The PSD distributes a process's variance over frequency; noise densities are square roots of PSDs, in units per $\sqrt{\text{Hz}}$.
- Sampling white noise of density σ at period Δt gives per-sample noise $\sigma/\sqrt{\Delta t}$; a random walk gains $\sigma_b\sqrt{\Delta t}$ per step.
- Frequencies above Nyquist alias; filter before you sample or down-sample.
- The m-fold integral of white noise has variance growing like t^{2m-1}.
- Covariance in a linear stochastic system obeys $\dot{P} = FP + PF^{\mathsf{T}} + GQ_cG^{\mathsf{T}}$.
- Colored noise is modeled with a shaping filter whose state joins the filter state: this is why biases are states.
- The Allan deviation separates white noise (slope $-\frac{1}{2}$, read at 1 s) from bias random walk (slope $+\frac{1}{2}$, read at 3 s).

Exercises

6.1 Show that the autocovariance of a random walk is $C(k, j) = \min(k, j)\,\sigma^2$.

6.2 For the Gauss–Markov sequence with $a = 0.99$ and $\sigma = 1$, compute the stationary standard deviation and the lag at which the correlation drops below $1/e$.

6.3 A 1 kHz accelerometer has white noise of 2×10^{-3} m/s$^2/\sqrt{\text{Hz}}$. What is the standard deviation of a single sample? Of the average of 1000 consecutive samples? Of a single

sample of the same sensor read at 100 Hz?

6.4 A 105 Hz vibration is sampled at 100 Hz. At what frequency does it appear?

6.5 Verify Proposition 6.4 by Monte Carlo for $m = 2$ and $m = 3$, simulating at 200 Hz for 10 s with 1000 runs.

6.6 Solve the Lyapunov equation for the double integrator $\dot{p} = v$, $\dot{v} = w$ with $P(0) = 0$ and verify that $P_{pp}(t) = \sigma^2 t^3/3$.

6.7 Derive the stationary variance and the exact discretization of the continuous Gauss–Markov process.

6.8 Simulate 2 hours of a gyro with EuRoC parameters, compute the overlapping Allan deviation, and recover σ_g and σ_{b_g} from the asymptotes. How long a recording is needed to estimate σ_{b_g} to 10%?

Chapter 7

Rotations, Quaternions and Rigid-Body Kinematics

In this chapter

- Read and write frame-annotated transformations such as R_{wb}, p_{wb} and T_{bc} without ambiguity, and catch frame bugs with the adjacency rule.
- Understand rotations in the plane, then in space: rotation matrices, axis-angle and Rodrigues' formula.
- Understand Euler angles, intrinsic versus extrinsic sequences, their conversions, and why they are banned from the estimator.
- Derive Hamilton quaternion algebra and its rotation formula; convert between quaternions and matrices; interpolate; and translate formulas from the JPL convention.
- Derive the kinematics of rotating bodies: $\dot{R} = R[\omega]_\times$, $\dot{q} = \frac{1}{2} q \otimes [0, \omega]$, the transport theorem and the Coriolis acceleration, and quantify the effects of the Earth's rotation.

7.1 Frames and notation

A *frame* is an origin together with three orthonormal, right-handed axes. Three frames appear in this project (Figure 7.1).

Frozen convention: frames, AGENTS.md C1

- W, the **world** frame: fixed, gravity-aligned with z pointing up, the frame of the ground truth (EuRoC calls it `R`). Gravity is $g_w = [0, 0, -9.81]^\mathsf{T}$ m/s^2.
- B, the **body** frame: the IMU frame `imu0` (EuRoC's `S` in `imu0`).
- C, the left **camera** frame `cam0`: origin at the optical center, z along the optical axis, x to the right and y down in the image.

R_{wb} (or q_{wb}) maps body-frame coordinates to world-frame coordinates, $v_w = R_{wb} v_b$. p_{wb} is the position of the origin of B expressed in W. v_{wb} is the velocity of B relative to W, expressed in W. $T_{bc} = (R_{bc}, p_{bc})$ maps camera points into the body frame, $p_b = R_{bc} p_c + p_{bc}$; it is `T_BS` from `cam0/sensor.yaml`. A world-to-body rotation is never used as a state.

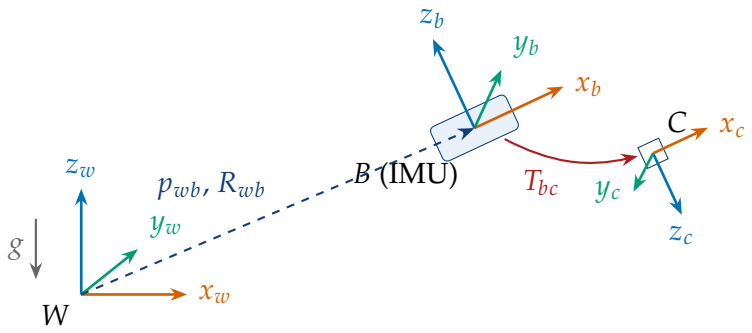


Figure 7.1: The world frame W (gravity-aligned, z up), the body/IMU frame B moving with the vehicle, and the camera frame C rigidly attached to the body through the constant extrinsic calibration T_{bc}.

7.1.1 Reading subscripts

The naming rule `<quantity>_<to><from>` makes products self-checking. R_{wb} takes a vector *from B to W*. Composition chains frames like dominoes:

$$R_{wc} = R_{wb}\, R_{bc}, \qquad v_w = R_{wb}\, R_{bc}\, v_c. \tag{7.1}$$

Key idea: the adjacency rule

In a valid product, adjacent subscripts match: $R_{w\underline{b}}\, R_{\underline{b}c}$. A product such as $R_{wb}R_{cb}$ has mismatched neighbors and is a bug. The fix is a transpose, $R_{wb}R_{cb}^{\mathsf{T}} = R_{wb}R_{bc}$, since $R_{bc} = R_{cb}^{-1} = R_{cb}^{\mathsf{T}}$. Apply the rule to every line of code that multiplies rotations or transforms points.

7.1.2 Active and passive views

A rotation matrix can be read in two ways. In the *passive* view, R_{wb} re-expresses the same physical vector in another frame: v_w and v_b are coordinates of one arrow (Section 2.7). In the *active* view, R_{wb} rotates the world axes onto the body axes: its columns are the body axes x_b, y_b, z_b expressed in world coordinates. Both readings describe the same matrix:

$$R_{wb} = \begin{bmatrix} x_b^{(w)} & y_b^{(w)} & z_b^{(w)} \end{bmatrix}. \tag{7.2}$$

This gives a quick sanity check when debugging: the third column of R_{wb} is the body z-axis in world coordinates. For a level vehicle whose z-axis points up, it is $[0, 0, 1]^{\mathsf{T}}$; its third row is the world vertical expressed in the body frame.

7.2 Rotations in the plane

Planar rotations contain most of the ideas of this chapter in their simplest form. The rotation by θ is (Example 2.16)

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}. \tag{7.3}$$

Rotations compose by adding angles, $R(\alpha)R(\beta) = R(\alpha+\beta)$ (multiply out and use the angle-addition formulas), so planar rotations commute. The angle is a perfect coordinate: $R(\theta)$ is determined by θ, and every planar rotation is $R(\theta)$ for a θ that is unique up to multiples of 2π. The only subtlety is wrap-around: angles must be compared modulo 2π, which is why the difference between two headings is computed as $\operatorname{atan2}(\sin(\theta_2-\theta_1), \cos(\theta_2-\theta_1))$ rather than $\theta_2 - \theta_1$.

Identifying the plane with the complex numbers, rotation by θ is multiplication by $e^{i\theta} = \cos\theta + i\sin\theta$. Unit complex numbers compose by multiplication, and their angles add. Quaternions (Section 7.6) are the three-dimensional generalization of this idea, with one crucial difference: in three dimensions, rotations do not commute.

7.3 Rotation matrices and SO(3)

Definition 7.1 (Special orthogonal group)**.** $\mathrm{SO}(3) = \{R \in \mathbb{R}^{3\times 3} : R^{\mathsf{T}}R = I,\ \det R = +1\}$.

Orthogonality ($R^{\mathsf{T}}R = I$) means R preserves lengths and angles; $\det R = +1$ excludes reflections, which would turn a right-handed frame into a left-handed one (Section 2.7). The set is closed under multiplication and inversion, $R^{-1} = R^{\mathsf{T}}$, so it is a *group*. A rotation matrix has nine entries but only three degrees of freedom: orthonormality imposes six constraints (three unit lengths, three orthogonalities). This redundancy is why SO(3) is a curved three-dimensional surface inside the nine-dimensional space of matrices rather than a vector space; the sum of two rotation matrices is not a rotation matrix.

The elementary rotations about the coordinate axes follow from the planar case:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\alpha & -s_\alpha \\ 0 & s_\alpha & c_\alpha \end{bmatrix}, \quad R_y(\alpha) = \begin{bmatrix} c_\alpha & 0 & s_\alpha \\ 0 & 1 & 0 \\ -s_\alpha & 0 & c_\alpha \end{bmatrix}, \quad R_z(\alpha) = \begin{bmatrix} c_\alpha & -s_\alpha & 0 \\ s_\alpha & c_\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{7.4}$$

with $c_\alpha = \cos\alpha$, $s_\alpha = \sin\alpha$. (The signs of R_y look reversed but are not: in the z-x plane, ordered cyclically after y, it is the usual planar rotation.) Rotations in space do not commute: $R_x(90^\circ)R_y(90^\circ) \neq R_y(90^\circ)R_x(90^\circ)$. Try it with a book.

7.4 Axis-angle and Rodrigues' formula

By Euler's theorem (Theorem 2.26), every rotation is a rotation by some angle $\theta \in [0,\pi]$ about some unit axis u. The pair (u,θ) is the *axis-angle* representation, and $\phi = \theta u \in \mathbb{R}^3$ is the *rotation vector*: its direction is the axis and its length the angle. The rotation vector is the central object of Chapter 8.

Theorem 7.2 (Rodrigues' rotation formula)**.** *The rotation by angle θ about the unit axis u is*

$$R = I + \sin\theta\,[u]_\times + (1-\cos\theta)\,[u]_\times^2 = \cos\theta\, I + (1-\cos\theta)\,uu^{\mathsf{T}} + \sin\theta\,[u]_\times. \tag{7.5}$$

Geometric proof. Split any vector into components parallel and perpendicular to the axis (Proposition 2.9), $v = v_\parallel + v_\perp$ with $v_\parallel = uu^{\mathsf{T}}v$. Rotation leaves $v_\parallel$ unchanged and rotates $v_\perp$

by θ within the plane spanned by $v_\perp$ and $u \times v_\perp = u \times v$ (a vector of the same length, rotated 90° about u):

$$Rv = v_\parallel + \cos\theta\, v_\perp + \sin\theta\, u \times v.$$

Substituting $v_\perp = v - uu^\mathsf{T}v = -[u]_\times^2 v$ (from (2.10) with $\|u\| = 1$) and $u \times v = [u]_\times v$ gives $Rv = (I + \sin\theta[u]_\times + (1 - \cos\theta)[u]_\times^2)v$. ∎

Algebraic proof. The rotation about u at unit angular rate satisfies $\dot{R} = [u]_\times R$ (Section 7.8), so $R(\theta) = \exp(\theta[u]_\times)$ by Theorem 4.5. With $[u]_\times^3 = -[u]_\times$ from (2.11), all odd powers of $[u]_\times$ are $\pm[u]_\times$ and all even powers (except the zeroth) are $\pm[u]_\times^2$, so the exponential series regroups as

$$\exp(\theta[u]_\times) = I + \left(\theta - \tfrac{\theta^3}{3!} + \tfrac{\theta^5}{5!} - \cdots\right)[u]_\times + \left(\tfrac{\theta^2}{2!} - \tfrac{\theta^4}{4!} + \cdots\right)[u]_\times^2,$$

and the two series are $\sin\theta$ and $1 - \cos\theta$. ∎

Two consequences of (7.5) recover (u, θ) from R. Taking the trace (using $\operatorname{tr}[u]_\times = 0$ and $\operatorname{tr}[u]_\times^2 = -2$) and the skew-symmetric part,

$$\operatorname{tr} R = 1 + 2\cos\theta, \qquad R - R^\mathsf{T} = 2\sin\theta\,[u]_\times. \tag{7.6}$$

Chapter 8 shows that *how* these are combined numerically matters a great deal.

Worked example: a quarter turn

Rotate by $\theta = 90°$ about $u = e_z$: $[e_z]_\times = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ and $[e_z]_\times^2 = \operatorname{diag}(-1, -1, 0)$, so (7.5) gives $R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = R_z(90°)$, which sends e_x to e_y, as a counter-clockwise quarter turn about z should. Its trace is $1 = 1 + 2\cos 90°$.

7.5 Euler angles

7.5.1 Definition and the two readings

Euler angles describe a rotation as three successive rotations about coordinate axes. The aerospace yaw-pitch-roll (ZYX) convention writes

$$R_{wb} = R_z(\psi)\, R_y(\vartheta)\, R_x(\varphi), \tag{7.7}$$

with yaw ψ, pitch ϑ and roll φ. The same product can be read in two ways, which are a constant source of confusion:

- *Intrinsic* (rotating axes): start aligned with the world, yaw by ψ about z, then pitch by ϑ about the *new* y-axis, then roll by φ about the *newest* x-axis. This is how a pilot thinks.
- *Extrinsic* (fixed axes): roll by φ about the world x-axis, then pitch by ϑ about the world y-axis, then yaw by ψ about the world z-axis.

Both readings produce the same matrix (7.7); the intrinsic sequence z, y', x'' equals the extrinsic sequence x, y, z in reverse order. Twelve different axis sequences are in use, and each library documents its own. Always check the convention against a known example before trusting a conversion.

7.5.2 Conversions

Multiplying out (7.7) gives the third row of R as $(-\sin\vartheta,\ \cos\vartheta\sin\varphi,\ \cos\vartheta\cos\varphi)$ and the first column as $\cos\vartheta(\cos\psi, \sin\psi, \cdot)$. Hence

$$\vartheta = \operatorname{atan2}\big(-R_{31},\ \sqrt{R_{11}^2 + R_{21}^2}\big), \qquad \varphi = \operatorname{atan2}(R_{32},\ R_{33}), \qquad \psi = \operatorname{atan2}(R_{21},\ R_{11}). \tag{7.8}$$

When $\cos\vartheta = 0$ ($\vartheta = \pm 90°$), the formulas for φ and ψ become $\operatorname{atan2}(0, 0)$: only their sum or difference is determined.

7.5.3 Gimbal lock

The relation between the body angular velocity measured by a gyroscope and the rates of the Euler angles is

$$\begin{bmatrix} \dot\varphi \\ \dot\vartheta \\ \dot\psi \end{bmatrix} = \begin{bmatrix} 1 & \sin\varphi\tan\vartheta & \cos\varphi\tan\vartheta \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \sin\varphi/\cos\vartheta & \cos\varphi/\cos\vartheta \end{bmatrix} \omega_b, \tag{7.9}$$

obtained by differentiating (7.7) and comparing with $[\omega_b]_\times = R^\mathsf{T}\dot R$ (Section 7.8). At $\vartheta = \pm 90°$ the matrix blows up. Geometrically, when the nose points straight up, yaw and roll rotate about the same physical axis, so the three angles no longer describe three independent motions. This is *gimbal lock*, named after mechanical gimbal platforms that physically jam in this configuration. Near the singularity, small angular velocities produce enormous Euler-angle rates, and a filter that linearizes in Euler angles becomes numerically meaningless.

Frozen convention: AGENTS.md C2

Euler angles never appear in the estimator. They may appear only in telemetry and plots.

No three-parameter representation of SO(3) is free of singularities everywhere: a topological fact (the space SO(3) cannot be covered by a single chart). The solution adopted by modern estimators is to store attitude in a singularity-free *redundant* representation (a unit quaternion or a rotation matrix) and to use three parameters only for *small* deviations around the current estimate, where no singularity is near. That is the content of Chapter 8.

7.6 Quaternions

7.6.1 Quaternion algebra

A *quaternion* is a four-component number $q = w + x\,i + y\,j + z\,k$, written as a scalar part and a vector part, $q = [w,\ v]$ with $v = (x, y, z)^\mathsf{T}$. Hamilton's multiplication rules for the three

imaginary units are

$$i^2 = j^2 = k^2 = ijk = -1, \tag{7.10}$$

from which $ij = k = -ji$, $jk = i = -kj$ and $ki = j = -ik$. (For example, multiplying $ijk = -1$ on the right by k gives $ij(-1) = -k$.) Expanding the product of two quaternions term by term with these rules and collecting the scalar and vector parts gives

$$p \otimes q = \begin{bmatrix} p_w q_w - p_v^{\mathsf{T}} q_v \\ p_w q_v + q_w p_v + p_v \times q_v \end{bmatrix}. \tag{7.11}$$

The products of imaginary units produce exactly the components of the cross product, which makes the product non-commutative: $p \otimes q - q \otimes p = [0,\ 2p_v \times q_v]$. It is associative. Because it is linear in each factor, it can be written with matrices,

$$p \otimes q = [p]_L\, q = [q]_R\, p, \qquad [p]_L = \begin{bmatrix} p_w & -p_v^{\mathsf{T}} \\ p_v & p_w I + [p_v]_\times \end{bmatrix}, \quad [q]_R = \begin{bmatrix} q_w & -q_v^{\mathsf{T}} \\ q_v & q_w I - [q_v]_\times \end{bmatrix}, \tag{7.12}$$

where quaternions are treated as 4-vectors $[q_w;\ q_v]$.

The *conjugate* is $q^* = [w,\ -v]$ and the *norm* is $\|q\| = \sqrt{w^2 + \|v\|^2}$. By (7.11), $q \otimes q^* = [\|q\|^2,\ 0]$, so the inverse is $q^{-1} = q^*/\|q\|^2$. Conjugation reverses products, $(p \otimes q)^* = q^* \otimes p^*$, and norms multiply, $\|p \otimes q\| = \|p\|\|q\|$ (because $\|p \otimes q\|^2 = p \otimes q \otimes q^* \otimes p^* = \|q\|^2\, p \otimes p^* = \|p\|^2\|q\|^2$). Consequently the product of two *unit* quaternions is a unit quaternion: unit quaternions form a group, the three-dimensional sphere $S^3 \subset \mathbb{R}^4$.

7.6.2 Unit quaternions represent rotations

Identify a vector $a \in \mathbb{R}^3$ with the *pure* quaternion $[0, a]$.

Theorem 7.3 (Rotation by a unit quaternion). *For a unit quaternion $q = [w, v]$, the map $a \mapsto q \otimes [0, a] \otimes q^*$ sends pure quaternions to pure quaternions and acts on a as the rotation matrix*

$$R(q) = (w^2 - v^{\mathsf{T}}v)\, I + 2\, vv^{\mathsf{T}} + 2w\, [v]_\times = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}. \tag{7.13}$$

The unit quaternion of a rotation by θ about the unit axis u is

$$q = \left[\cos(\theta/2),\ \sin(\theta/2)\, u\right]. \tag{7.14}$$

Proof. By (7.11), $[0, a] \otimes q^* = [a^{\mathsf{T}}v,\ wa - a \times v]$. Multiplying on the left by $q = [w, v]$: the scalar part is $w\, a^{\mathsf{T}}v - v^{\mathsf{T}}(wa - a \times v) = 0$ (since $v^{\mathsf{T}}(a \times v) = 0$), and the vector part is $w(wa - a \times v) + (a^{\mathsf{T}}v)v + v \times (wa - a \times v) = w^2 a + 2w\, v \times a + (v^{\mathsf{T}}a)v - v \times (a \times v)$. With the BAC–CAB rule, $v \times (a \times v) = a\,(v^{\mathsf{T}}v) - v\,(v^{\mathsf{T}}a)$, so the vector part is $(w^2 - v^{\mathsf{T}}v)a + 2(v^{\mathsf{T}}a)v + 2w\, v \times a$, which is (7.13). This matrix is orthogonal with determinant one (it preserves norms, because quaternion norms multiply, and depends continuously on q, equal to I at $q = [1, 0]$). For (7.14), substitute $w = \cos\frac{\theta}{2}$, $v = \sin\frac{\theta}{2} u$ and use the double-angle identities $w^2 - v^{\mathsf{T}}v = \cos\theta$, $2\sin^2\frac{\theta}{2} = 1 - \cos\theta$ and $2\sin\frac{\theta}{2}\cos\frac{\theta}{2} = \sin\theta$; the result is the second form of Rodrigues' formula (7.5). ∎

The *half* angle in (7.14) is the most common source of factor-of-two bugs. It arises because q appears twice in the sandwich product $q \otimes [0, a] \otimes q^*$, each occurrence contributing half of the rotation.

Key idea: double cover

q and $-q$ give the same $R(q)$, since (7.13) is quadratic in q. Every rotation has exactly two unit quaternions, at opposite points of the sphere S^3. They correspond to rotating by θ about u, or by $2\pi - \theta$ about $-u$. Code that compares, averages, interpolates or takes logarithms of quaternions must account for this. The project canonicalizes to $w \geq 0$ before taking Log_q (AGENTS.md C2).

7.6.3 Composition

Because $(p \otimes q) \otimes [0, a] \otimes (p \otimes q)^* = p \otimes \big(q \otimes [0, a] \otimes q^*\big) \otimes p^*$, the Hamilton product composes rotations in the same order as matrix multiplication:

$$R(p \otimes q) = R(p)\, R(q), \qquad q_{wc} = q_{wb} \otimes q_{bc} \;\leftrightarrow\; R_{wc} = R_{wb} R_{bc}. \tag{7.15}$$

The adjacency rule therefore applies to quaternions unchanged. Composing quaternions costs 16 multiplications and 12 additions, against 27 and 18 for matrices, and renormalizing a quaternion is trivial compared with re-orthogonalizing a matrix. These are the practical reasons to store attitude as a quaternion.

Worked example: composing two quarter turns

The quarter turn about z is $q_z = [\cos 45°,\ \sin 45° e_z] = \frac{1}{\sqrt{2}}[1, (0, 0, 1)]$, and about x it is $q_x = \frac{1}{\sqrt{2}}[1, (1, 0, 0)]$. By (7.11), $q_z \otimes q_x = \frac{1}{2}\big[1,\ (1, 0, 1) + e_z \times e_x\big] = \frac{1}{2}[1, (1, 1, 1)]$: a rotation by $2 \arccos \frac{1}{2} = 120°$ about $(1, 1, 1)/\sqrt{3}$. (Check: it cycles the axes, $x \to y \to z \to x$.) In the other order, $q_x \otimes q_z = \frac{1}{2}[1, (1, -1, 1)]$, a different rotation.

7.6.4 Converting a rotation matrix to a quaternion

Inverting (7.13) naively via $w = \frac{1}{2}\sqrt{1 + \operatorname{tr} R}$ and $v = (R - R^{\mathsf{T}})^{\vee}/(4w)$ divides by a number that vanishes for 180° rotations. *Shepperd's method* avoids this. From (7.13), the four quantities $4w^2 = 1 + \operatorname{tr} R$, $4x^2 = 1 + R_{11} - R_{22} - R_{33}$, $4y^2 = 1 - R_{11} + R_{22} - R_{33}$ and $4z^2 = 1 - R_{11} - R_{22} + R_{33}$ can all be computed from the diagonal. Choose the largest (at least $1/4$, since $w^2 + x^2 + y^2 + z^2 = 1$), take its square root, and compute the other three components from the off-diagonal sums and differences, for example $4wx = R_{32} - R_{23}$, $4xy = R_{21} + R_{12}$, $4xz = R_{13} + R_{31}$, dividing only by the large component. Eigen implements this method.

7.6.5 The Hamilton and JPL conventions

Unfortunately, two incompatible quaternion conventions are in wide use. Much of the classic VIO literature, including the original MSCKF paper [40] and the indirect Kalman filter report of Trawny and Roumeliotis [52], uses the *JPL* convention. Formulas cannot be copied between the two without translation [49].

	Hamilton (this project)	**JPL**
Multiplication rule	$ij = k$ ($ijk = -1$)	$ij = -k$ ($ijk = +1$)
Component order	usually $[w, x, y, z]$	usually $[x, y, z, w]$
Typical meaning	body-to-world (active), q_{wb}	world-to-body (passive), ${}^{B}_{W}\bar{q}$
Composition	$q_{wc} = q_{wb} \otimes q_{bc}$	${}^{C}_{W}\bar{q} = {}^{C}_{B}\bar{q} \otimes {}^{B}_{W}\bar{q}$
Rotation matrix	$R(q) = (w^2 - v^\mathsf{T}v)I + 2vv^\mathsf{T} + 2w[v]_\times$	$C(\bar{q}) = (2q_4^2 - 1)I - 2q_4[q]_\times + 2qq^\mathsf{T}$
Used by	Eigen, ROS, Solà [47], Ceres, Sophus	MSCKF papers, Trawny [52], NASA

Table 7.1: Hamilton versus JPL quaternions. The JPL product of two quaternions equals the Hamilton product of the same numbers in reverse order.

A useful fact for reading the JPL literature: the JPL quaternion ${}^{B}_{W}\bar{q}$ of the world-to-body rotation has *the same four numbers* as the Hamilton quaternion q_{wb} of the body-to-world rotation, merely stored in the order $[x, y, z, w]$. (Substitute into the two rotation formulas of Table 7.1: $C(\bar{q}) = R(q)^\mathsf{T}$.) What changes is the meaning of products and of perturbations: a JPL error quaternion $\delta\bar{q}$ in ${}^{B}_{W}\bar{q} = \delta\bar{q} \otimes {}^{B}_{W}\hat{\bar{q}}$ is a perturbation in the body frame, but written on the left, and its Jacobians differ in sign and order from ours. Translate a JPL derivation by re-deriving it, not by pattern-matching symbols.

Frozen convention: quaternions, AGENTS.md C2

Hamilton quaternions, `Eigen::Quaterniond`, always of unit norm, normalized after every multiplicative update. Never substitute JPL formulas, even when a paper you are reading uses them. Translate them first.

7.6.6 Quaternions in code and data

Pitfall: coefficient order

Three orderings meet at the project's input boundary:

- `Eigen::Quaterniond(w, x, y, z)`: the constructor takes w *first*;
- `q.coeffs()`: Eigen's storage is $[x, y, z, w]$, with w *last*;
- EuRoC CSV files store `qw, qx, qy, qz`, with w first.

Mapping a raw EuRoC row with `Eigen::Map<Quaterniond>` interprets it as $[x, y, z, w]$ and silently produces a different, valid-looking rotation. Always construct explicitly, `Eigen::Quaterniond(row[0], row[1], row[2], row[3])`, and cover the parser with a fixture test.

Other Eigen facts worth knowing: `q * v` for a `Vector3d` applies $R(q)$ to v; `q1 * q2` is the Hamilton product; `q.conjugate()` is the inverse of a unit quaternion; `q.normalize()` works in place while `q.normalized()` returns a copy; and `q.toRotationMatrix()` evaluates (7.13).

Normalization. Each multiplicative update of a quaternion rounds its four components, so $\|q\|$ performs a slow random walk away from 1. A non-unit quaternion inserted into (7.13) produces a matrix that scales vectors by $\|q\|^2$ as well as rotating them. Dividing by the

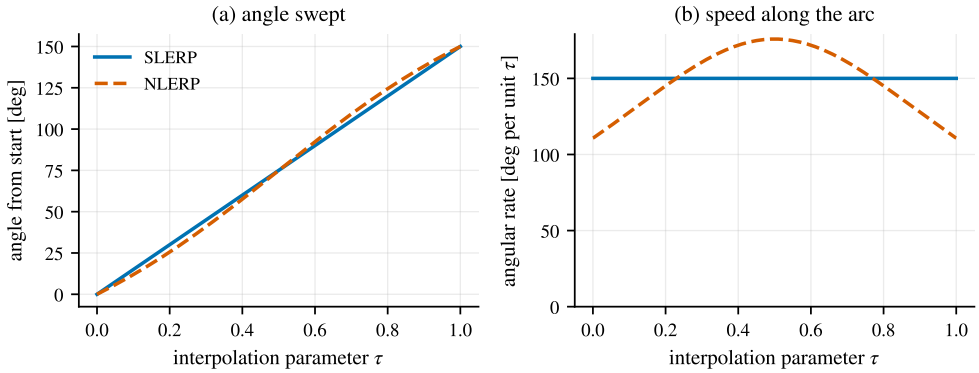


Figure 7.2: Interpolating between two attitudes 150° apart. SLERP sweeps the angle uniformly in the interpolation parameter; normalized linear interpolation (NLERP) follows the same path but at a non-uniform speed, fastest in the middle.

norm after every update costs a few flops and removes the problem. Milestone 4 checks that $|\|q\| - 1| < 10^{-12}$ after 10^6 updates.

7.6.7 Distance and interpolation

The angle of the relative rotation between two attitudes q_1 and q_2 is

$$\theta_{12} = 2 \arccos \left| q_1^{\mathsf{T}} q_2 \right| = \left\| \operatorname{Log}_q (q_1^{-1} \otimes q_2) \right\|, \tag{7.16}$$

where $q_1^{\mathsf{T}} q_2$ is the four-dimensional dot product (the scalar part of $q_1^* \otimes q_2$) and the absolute value accounts for the double cover. (For small angles prefer the Log_q form, which avoids arccos near 1; Chapter 8.)

Ground truth and IMU data rarely share timestamps with images, so attitudes must be interpolated. The correct interpolation follows the shortest great arc at constant angular velocity: *spherical linear interpolation* (SLERP),

$$q(\tau) = q_0 \otimes \operatorname{Exp}_q\big(\tau \, \operatorname{Log}_q(q_0^{-1} \otimes q_1)\big) = \frac{\sin((1-\tau)\Omega)}{\sin \Omega} \, q_0 + \frac{\sin(\tau\Omega)}{\sin \Omega} \, q_1, \qquad \tau \in [0,1], \tag{7.17}$$

with $\cos \Omega = q_0^{\mathsf{T}} q_1$ after flipping the sign of q_1 if needed so that $q_0^{\mathsf{T}} q_1 \geq 0$. The first form (using the exponential and logarithm of Chapter 8) shows that the relative rotation is applied with a uniformly growing angle; the second is the formula for interpolating along a great circle of the sphere. Component-wise linear interpolation followed by normalization (NLERP) follows the same path at a non-uniform speed (Figure 7.2); it is acceptable only for small angles.

7.7 Rigid-body transformations

A rigid transformation combines a rotation and a translation. Point coordinates transform as

$$p_w = R_{wb}\, p_b + p_{wb} \quad \Longleftrightarrow \quad \begin{bmatrix} p_w \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} R_{wb} & p_{wb} \\ 0 & 1 \end{bmatrix}}_{T_{wb}} \begin{bmatrix} p_b \\ 1 \end{bmatrix}. \tag{7.18}$$

Appending a 1 to the point (*homogeneous coordinates*, developed in Chapter 20) turns the affine map into a single matrix product. The 4×4 matrices of this form make up the group SE(3) of rigid motions. Composition and inversion follow from block multiplication:

$$T_{wc} = T_{wb} T_{bc} = \begin{bmatrix} R_{wb} R_{bc} & R_{wb} p_{bc} + p_{wb} \\ 0 & 1 \end{bmatrix}, \qquad T_{wb}^{-1} = T_{bw} = \begin{bmatrix} R_{wb}^{\mathsf{T}} & -R_{wb}^{\mathsf{T}} p_{wb} \\ 0 & 1 \end{bmatrix}. \tag{7.19}$$

The camera pose in the world therefore is $R_{wc} = R_{wb} R_{bc}$ and $p_{wc} = p_{wb} + R_{wb} p_{bc}$, and a world point p_f has camera coordinates

$$p_c = R_{bc}^{\mathsf{T}} \big(R_{wb}^{\mathsf{T}} (p_f - p_{wb}) - p_{bc} \big), \tag{7.20}$$

which is the first half of the camera measurement model (AGENTS.md C6).

Pitfall: points versus directions

Translations apply to points, not to free vectors. A velocity, a gravity vector or a bearing direction transforms with the rotation only, $v_w = R_{wb} v_b$. In homogeneous coordinates directions carry a 0 in the fourth component, which drops the translation automatically.

7.8 Rotational kinematics

7.8.1 Angular velocity

Let $R(t) = R_{wb}(t)$ describe a rotating body. Differentiating $R^{\mathsf{T}} R = I$ gives $\dot{R}^{\mathsf{T}} R + R^{\mathsf{T}} \dot{R} = 0$, so $R^{\mathsf{T}} \dot{R}$ is skew-symmetric and equals $[\omega_b]_\times$ for a unique vector ω_b:

$$\dot{R}_{wb} = R_{wb}\, [\omega_b]_\times = [\omega_w]_\times\, R_{wb}, \qquad \omega_w = R_{wb}\, \omega_b, \tag{7.21}$$

where the second form uses (2.14). Why is ω an angular velocity? A point fixed in the body, with constant body coordinates r_b, has world coordinates $r_w = R_{wb} r_b$ and velocity $\dot{r}_w = \dot{R}_{wb} r_b = [\omega_w]_\times R_{wb} r_b = \omega_w \times r_w$: every point moves as if the body were spinning about the axis ω_w through the origin with angular speed $\|\omega\|$. ω_b is this angular velocity of B relative to W, expressed in B, and it is exactly what a strapdown gyroscope measures (up to bias and noise). The two forms of (7.21) are the kinematic origin of the two perturbation conventions of Chapter 8: a body-frame quantity multiplies on the right, a world-frame quantity on the left.

Proposition 7.4 (Constant body rate). *If ω_b is constant on $[t_0, t_0 + \Delta t]$, then $R(t_0 + \Delta t) = R(t_0) \exp(\Delta t\, [\omega_b]_\times)$.*

Proof. $R(t) = R(t_0)\exp((t-t_0)[\omega_b]_\times)$ satisfies (7.21) by Proposition 3.14, and the solution is unique (Theorem 4.5). ∎

This exact solution is the basis of the multiplicative attitude update used in every integration scheme of Chapter 16. When the rate varies, the update is still applied step by step with the rate held constant over each short step (Section 4.3.3). Note that the solution is *not* $R(t_0) + \Delta t\, R(t_0)[\omega_b]_\times$, which would leave SO(3).

7.8.2 Quaternion kinematics

The quaternion counterpart of (7.21) is

$$\dot{q}_{wb} = \tfrac{1}{2}\, q_{wb} \otimes [0,\ \omega_b] = \tfrac{1}{2}\, \Omega(\omega_b)\, q_{wb}, \qquad \Omega(\omega) = [\,[0,\omega]\,]_R = \begin{bmatrix} 0 & -\omega^\mathsf{T} \\ \omega & -[\omega]_\times \end{bmatrix}, \tag{7.22}$$

with constant-rate solution $q(t_0+\Delta t) = q(t_0) \otimes \left[\cos\frac{\|\omega\|\Delta t}{2},\ \sin\frac{\|\omega\|\Delta t}{2}\,\frac{\omega}{\|\omega\|}\right]$. To derive (7.22), compose $q(t+\mathrm{d}t) = q(t)\otimes \mathrm{d}q$, where $\mathrm{d}q$ is the small body-frame rotation by $\omega_b\,\mathrm{d}t$. By (7.14), $\mathrm{d}q = [1,\ \frac{1}{2}\omega_b\,\mathrm{d}t] + O(\mathrm{d}t^2)$, so $q(t+\mathrm{d}t) - q(t) = q \otimes [0, \frac{1}{2}\omega_b]\,\mathrm{d}t$. The factor $\frac{1}{2}$ is the half angle again.

7.8.3 Derivatives in a rotating frame

Differentiating $v_w = R_{wb}v_b$ with (7.21) gives the *transport theorem*

$$\dot{v}_w = R_{wb}\left(\dot{v}_b + \omega_b \times v_b\right): \tag{7.23}$$

the rate of change seen from the fixed frame equals the rate seen in the rotating frame plus the change due to the rotation of the frame itself. Applying it twice to the position r of a particle relative to a frame rotating with angular velocity Ω (constant) gives the acceleration in the fixed (inertial) frame, expressed in rotating coordinates:

$$a_{\text{inertial}} = \ddot{r} + 2\,\Omega \times \dot{r} + \Omega \times (\Omega \times r), \tag{7.24}$$

where $\ddot{r}$ and $\dot{r}$ are the acceleration and velocity seen in the rotating frame. The extra terms are the *Coriolis acceleration* $2\Omega \times \dot{r}$ and the *centripetal acceleration* $\Omega \times (\Omega \times r)$. This formula is the working principle of the vibratory MEMS gyroscope (Chapter 15), and it tells us what we neglect by treating the world frame as inertial.

7.8.4 The Earth is not an inertial frame

The world frame W is fixed to the Earth, which rotates at $\Omega_e = 7.292 \times 10^{-5}$ rad/s. Three effects follow; their sizes justify the project's simplifications.

- *Gyroscopes sense the Earth rate.* 7.3×10^{-5} rad/s is comparable to the bias instability of the EuRoC ADIS16448 and far below its noise over short horizons. The project neglects it; its constant body-frame component (while the attitude changes little) is absorbed by the gyro bias estimate. High-grade systems model it explicitly and can even find north from it (gyrocompassing).

- *Centripetal acceleration.* The term $\Omega_e \times (\Omega_e \times r)$ reaches 0.034 m/s^2 at the equator. It is absorbed into the local gravity vector: *gravity*, as measured by a plumb line, is gravitation plus this centrifugal term. Its magnitude varies from 9.780 m/s^2 at the equator to 9.832 m/s^2 at the poles; the project uses a constant 9.81 (AGENTS.md C5), accurate to about 10^{-3} relative in Zurich, where EuRoC was recorded.
- *Coriolis acceleration.* $2\Omega_e \times v$ is about 1.5×10^{-4} m/s^2 at 1 m/s: below accelerometer noise and bias for a small drone, and neglected.

Over the minutes and hundreds of meters of a EuRoC sequence, the world frame may therefore be treated as inertial and flat, with constant gravity.

Key takeaways

- R_{wb} maps body coordinates to world coordinates; its columns are the body axes in the world. Adjacent subscripts must match.
- Every rotation is an axis-angle rotation, $R = I + \sin\theta[u]_\times + (1 - \cos\theta)[u]_\times^2$; $\operatorname{tr} R = 1 + 2\cos\theta$.
- Euler angles have intrinsic and extrinsic readings and a singularity at ±90° pitch; they stay out of the estimator.
- Hamilton unit quaternions compose like matrices; q and $-q$ are the same rotation; the quaternion of a rotation by θ uses $\theta/2$.
- Eigen's constructor is w-first, its storage w-last, EuRoC is w-first. JPL formulas must be re-derived, not copied.
- $\dot{R}_{wb} = R_{wb}[\omega_b]_\times$; with constant ω_b the update $R \leftarrow R\exp(\Delta t[\omega_b]_\times)$ is exact.
- Derivatives in rotating frames pick up $\omega\times$ terms; for a small drone over minutes, the Earth's rotation is negligible.

Exercises

7.1 Show that the columns of any $R \in \mathrm{SO}(3)$ are orthonormal and that $\det R = +1$ implies the third column is the cross product of the first two.

7.2 Verify $R_x(90°)R_y(90°) \neq R_y(90°)R_x(90°)$ by computing both products, and describe each rotation physically.

7.3 Derive (7.9) from (7.7) and $[\omega_b]_\times = R^\mathsf{T}\dot{R}$.

7.4 Convert the rotation of the worked example (120° about $(1,1,1)/\sqrt{3}$) to ZYX Euler angles.

7.5 Prove that $[p]_L[q]_R = [q]_R[p]_L$. What does this say about associativity of the quaternion product?

7.6 Show that $R(-q) = R(q)$ and compute the quaternion of a 270° rotation about z in its canonical form ($w \geq 0$). What rotation angle and axis does the canonical form describe?

7.7 Implement Shepperd's method and test it on rotations by 0°, 90°, 179.9999° and 180°.

7.8 Write a function that converts a EuRoC ground-truth row into p_{wb}, q_{wb}, v_{wb}, b_g and b_a. Design a fixture row whose values would expose a swapped w/z and swapped biases.

7.9 Given T_{bc} from `cam0/sensor.yaml` (Appendix D), compute the camera's optical axis in body coordinates.

7.10 Show that if ω_b varies in time, $R(t) = R(0)\exp\left(\int_0^t [\omega_b]_\times \, \mathrm{d}s\right)$ is in general *wrong*. Construct a counterexample with piecewise-constant rates.

7.11 Derive (7.24) by applying (7.23) twice.

Chapter 8

Lie Theory for Rotations

In this chapter

- Understand SO(3) as a curved manifold with a flat tangent space, and why an estimator needs both.
- Use the exponential and logarithm maps, in matrix and quaternion form, and implement them robustly.
- Distinguish local (right) and global (left) perturbations, and move perturbations across a rotation with the adjoint.
- Define Jacobians of functions of rotations, derive the right Jacobian J_r, and build a table of elementary Jacobians from which every Jacobian in the project can be assembled.
- Represent and propagate uncertainty on SO(3), and test manifold Jacobians numerically.

This chapter is the mathematical core of the notes. Its conclusions fit in a few lines (the $\boxplus/\boxminus$ operators of Section 8.9 and the Jacobian table of Table 8.1), but the reasoning behind them is what lets you derive any new Jacobian correctly. The treatment follows the "micro Lie theory" of Solà et al. [48], specialized to SO(3) with the conventions of this project. Barfoot [4] covers the same material in greater depth.

8.1 Why estimation needs Lie theory

A Kalman filter represents uncertainty as a mean and a covariance and corrects the mean by adding a vector. Both operations assume the state lives in a vector space. Attitude does not.

- **Addition leaves the space.** If $\hat{q}$ is a unit quaternion and δq a small 4-vector, $\hat{q} + \delta q$ is generally not unit length. The same holds for rotation matrices.
- **Redundant parameters give singular covariances.** A unit quaternion has four components but three degrees of freedom. A 4×4 covariance of a unit quaternion must be singular along the radial direction, and keeping it exactly singular under rounding is hopeless.
- **Minimal parameters have singularities.** Euler angles (Chapter 7) are minimal but singular at gimbal lock.

Key idea: the Earth and its tangent planes

The surface of the Earth is curved, yet a town map drawn on flat paper is perfectly accurate for a walk across town. The map is the *tangent plane* at your location. It works

as long as you do not walk too far, and when you move to a new town you draw a new map. An estimator does the same: it stores the attitude on the curved space SO(3) (the globe), and expresses small uncertainties and corrections in the flat 3D tangent space at the current estimate (the town map). The exponential map turns a step on the map into a point on the globe, and the logarithm map does the reverse.

8.2 Warm-up: rotations of the plane

Every idea of this chapter appears, in its simplest form, for planar rotations $R(\theta)$. Work through them here; the rest of the chapter asks only what changes in three dimensions.

- *The set.* Planar rotations form a circle: $R(\theta)$ and $R(\theta + 2\pi)$ are the same. The set is one-dimensional but curved, and it is not a vector space ($R(\alpha) + R(\beta)$ is not a rotation).
- *The tangent space.* Differentiating $R(\theta)$ at $\theta = 0$ gives $\frac{\mathrm{d}R}{\mathrm{d}\theta}(0) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$; every velocity at the identity is a multiple of this skew-symmetric matrix. The tangent space is a line, coordinatized by a single number.
- *Exponential and logarithm.* $\exp\left(\theta \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}\right) = R(\theta)$ (sum the series, or solve $\dot{R} = R \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$), so "Exp" maps an angle to a rotation. Its inverse "Log" recovers the angle, $\theta = \operatorname{atan2}(R_{21}, R_{11}) \in (-\pi, \pi]$, unique only up to the choice of branch.
- *Perturbations.* An estimate $R(\hat{\theta})$ with a small error is $R(\hat{\theta})R(\delta\theta) = R(\hat{\theta} + \delta\theta)$. Because planar rotations commute, perturbing on the left or on the right is the same, and the error of the composition is just the sum of the angles.
- *Jacobians.* The derivative of Exp is the identity (1), and nothing like the right Jacobian appears.

In the plane, therefore, one can simply estimate the angle as a scalar, as long as differences are wrapped to $(-\pi, \pi]$. In three dimensions rotations do not commute, the identity $R(\alpha)R(\beta) = R(\alpha + \beta)$ fails for rotation vectors with different axes, and left and right perturbations differ. Each of these failures produces a new object in this chapter: the adjoint, the Baker–Campbell–Hausdorff formula, and the right and left Jacobians.

8.3 Manifolds, groups and Lie groups

8.3.1 Manifolds and tangent spaces

A smooth n-dimensional *manifold* is a space that looks like $\mathbb{R}^n$ near every point. Precisely, each point has a neighborhood that can be mapped smoothly and invertibly onto an open set of $\mathbb{R}^n$ (a *chart*), and where two charts overlap, the change of coordinates between them is smooth. The surface of a sphere is a 2D manifold: latitude and longitude form a chart away from the poles, and a second chart covers the poles. A manifold usually needs several charts;

a single chart for the whole sphere does not exist, and neither does one for SO(3), which is why every three-number parameterization of attitude has singularities.

A *tangent vector* at a point x of a manifold is the velocity $\dot{\gamma}(0)$ of a smooth curve $\gamma(t)$ through $\gamma(0) = x$. The tangent vectors at x form a vector space, the *tangent space* $T_x\mathcal{M}$, of the same dimension as the manifold. For a manifold that sits inside a larger vector space (as SO(3) sits inside $\mathbb{R}^{3\times3}$), tangent vectors are ordinary vectors tangent to the surface: the tangent plane of a sphere in the Earth analogy.

8.3.2 Groups

Definition 8.1 (Group). A *group* $(\mathcal{G}, \circ)$ is a set with an operation $\circ$ that is associative, has an identity element $\mathcal{E}$ ($\mathcal{E} \circ \mathcal{X} = \mathcal{X} \circ \mathcal{E} = \mathcal{X}$), and an inverse $\mathcal{X}^{-1}$ for every element ($\mathcal{X} \circ \mathcal{X}^{-1} = \mathcal{E}$).

Commutativity is *not* required. Examples: $(\mathbb{R}^n, +)$ (identity 0, inverse $-x$); the invertible $n \times n$ matrices under multiplication; SO(3); the unit quaternions under the Hamilton product; the rigid motions SE(3). A group can also *act* on another set: a rotation acts on vectors by $v \mapsto Rv$, and a rigid motion acts on points by $p \mapsto Rp + t$. These actions are how groups enter measurement models.

Definition 8.2 (Lie group). A *Lie group* is a group that is also a smooth manifold, such that composition and inversion are smooth maps. A *matrix Lie group* is a Lie group whose elements are invertible matrices and whose operation is matrix multiplication.

Being a group as well gives a Lie group a remarkable property. Multiplication by a fixed element maps a neighborhood of the identity smoothly onto a neighborhood of that element: the space looks the same everywhere. All local calculus can therefore be done once, at the identity, and carried to any other element by composition. This is what makes estimation on SO(3) tractable: one tangent space (at the identity), one exponential map, and one set of Jacobian formulas serve every attitude.

Group	Elements	Dimension	Role in VIO
$(\mathbb{R}^n, +)$	vectors	n	position, velocity, biases
SO(2)	planar rotations	1	toy examples
SO(3)	3×3 rotation matrices	3	attitude
S^3	unit quaternions	3	attitude (double cover of SO(3))
SE(3)	rigid transformations	6	poses, extrinsics

8.4 The Lie algebra and tangent spaces

Consider a smooth curve $R(t) \in \mathrm{SO}(3)$ with $R(0) = I$. Differentiating $R^\mathsf{T}R = I$ at $t = 0$ gives $\dot{R}(0)^\mathsf{T} + \dot{R}(0) = 0$: the velocity at the identity is skew-symmetric. The set of all such velocities is the *Lie algebra*

$$\mathfrak{so}(3) = \{\, \Omega \in \mathbb{R}^{3\times3} : \Omega^\mathsf{T} = -\Omega \,\} = \{\, [\phi]_\times : \phi \in \mathbb{R}^3 \,\}, \tag{8.1}$$

a three-dimensional vector space identified with $\mathbb{R}^3$ by the hat and vee maps, $\phi^\wedge = [\phi]_\times$ and $[\phi]_\times^\vee = \phi$. Unlike SO(3) itself, the Lie algebra is flat: we can add its elements, scale them, and

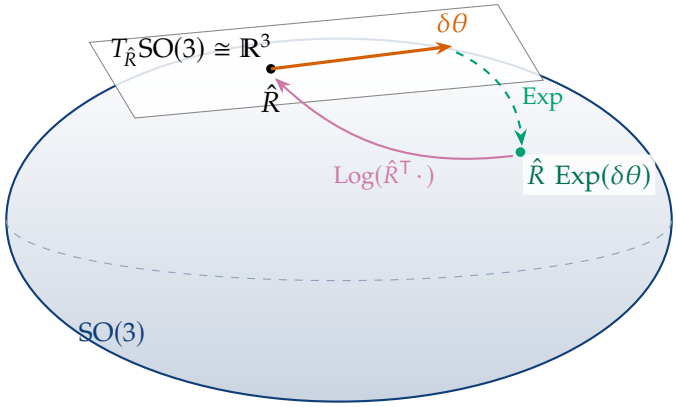


Figure 8.1: The manifold SO(3) and its tangent space at the estimate $\hat{R}$. A small error or correction $\delta\theta \in \mathbb{R}^3$ lives in the flat tangent space. The exponential map wraps it onto the manifold as $\hat{R}\,\mathrm{Exp}(\delta\theta)$, and the logarithm recovers it.

put Gaussian distributions on them.

The Lie algebra carries one more operation, the *Lie bracket* (commutator) $[X, Y] = XY - YX$, which measures how much two infinitesimal motions fail to commute. The commutator of two skew-symmetric matrices is skew-symmetric, and by (2.13)

$$\big[[a]_\times, [b]_\times\big] = [a \times b]_\times. \tag{8.2}$$

So $\mathfrak{so}(3)$ with its bracket is $\mathbb{R}^3$ with the cross product. The cross product, which is zero only for parallel vectors, is the infinitesimal version of the non-commutativity of rotations: rotations about parallel axes commute, all others do not.

At a general point R, the tangent space consists of the velocities of curves through R. From (7.21), every such velocity can be written in two ways,

$$\dot{R} = R\,[\omega_b]_\times = [\omega_w]_\times\,R, \qquad \omega_w = R\,\omega_b, \tag{8.3}$$

so the tangent space at R can be identified with $\mathbb{R}^3$ in two ways: by a vector expressed in the *local* (body) frame, multiplying on the right, or by a vector expressed in the *global* (world) frame, multiplying on the left. This is the origin of the two perturbation conventions of Section 8.9. The project uses the local one.

8.5 The exponential map

Definition 8.3 (Exponential map of SO(3))**.** $\exp : \mathfrak{so}(3) \to \mathrm{SO}(3)$ is the matrix exponential. Composed with the hat map it gives the *capitalized* exponential

$$\mathrm{Exp} : \mathbb{R}^3 \to \mathrm{SO}(3), \qquad \mathrm{Exp}(\phi) = \exp([\phi]_\times). \tag{8.4}$$

By Rodrigues' formula (Theorem 7.2), with $\theta = \|\phi\|$,

$$\mathrm{Exp}(\phi) = I + \frac{\sin\theta}{\theta}\,[\phi]_\times + \frac{1-\cos\theta}{\theta^2}\,[\phi]_\times^2. \tag{8.5}$$

$\mathrm{Exp}(\phi)$ is the rotation by angle $\|\phi\|$ about the axis $\phi/\|\phi\|$. Its main properties are $\mathrm{Exp}(0) = I$, $\mathrm{Exp}(-\phi) = \mathrm{Exp}(\phi)^{\mathsf{T}}$, and $\mathrm{Exp}(s\phi)\,\mathrm{Exp}(t\phi) = \mathrm{Exp}((s+t)\phi)$: rotations about a *common* axis compose by adding angles. Exp is onto SO(3), and one-to-one on the open ball $\|\phi\| < \pi$.

Why the exponential? The curve $R(t) = \mathrm{Exp}(t\phi)$ is the motion of a body spinning at the constant body rate ϕ: it solves $\dot{R} = R[\phi]_\times$ with $R(0) = I$ (Proposition 7.4), and it is a *one-parameter subgroup*, $R(s)R(t) = R(s+t)$. The exponential map thus sends each tangent vector at the identity to the point reached by moving "straight" in that direction for unit time. In every Lie group, the exponential is defined the same way, which is why the same machinery works for SO(3), SE(3) and the extended poses used by invariant filters.

Quaternion exponential. The unit quaternion corresponding to $\mathrm{Exp}(\phi)$ follows from (7.14):

$$\mathrm{Exp}_q(\phi) = \begin{bmatrix} \cos(\theta/2) \\ \dfrac{\sin(\theta/2)}{\theta}\,\phi \end{bmatrix}, \qquad R\big(\mathrm{Exp}_q(\phi)\big) = \mathrm{Exp}(\phi). \tag{8.6}$$

This is the quaternion exponential of the pure quaternion $[0, \phi/2]$; the half angle is the double cover again. Since $\mathrm{Exp}_q(\phi)$ has $w = \cos(\theta/2) \geq 0$ for $\theta \leq \pi$, its output is already in canonical form on that range.

8.5.1 Numerical evaluation and the Taylor branches

Formulas (8.5) and (8.6), and the Jacobian formulas later in this chapter, are built from a handful of scalar *coefficient functions* of θ:

$$A(\theta) = \frac{\sin\theta}{\theta}, \quad B(\theta) = \frac{1-\cos\theta}{\theta^2}, \quad C(\theta) = \frac{\theta - \sin\theta}{\theta^3}, \quad D(\theta) = \frac{1}{\theta^2} - \frac{\cot(\theta/2)}{2\theta}. \tag{8.7}$$

All four are smooth, even functions with finite limits at $\theta = 0$, but their formulas read $0/0$ there. Their Taylor series are

$$\begin{aligned} A &= 1 - \tfrac{\theta^2}{6} + \tfrac{\theta^4}{120} - \cdots, & B &= \tfrac{1}{2} - \tfrac{\theta^2}{24} + \tfrac{\theta^4}{720} - \cdots, \\ C &= \tfrac{1}{6} - \tfrac{\theta^2}{120} + \tfrac{\theta^4}{5040} - \cdots, & D &= \tfrac{1}{12} + \tfrac{\theta^2}{720} + \tfrac{\theta^4}{30240} + \cdots, \end{aligned} \tag{8.8}$$

and for the quaternion map $\cos\frac{\theta}{2} = 1 - \frac{\theta^2}{8} + \frac{\theta^4}{384} - \cdots$ and $\frac{\sin(\theta/2)}{\theta} = \frac{1}{2} - \frac{\theta^2}{48} + \frac{\theta^4}{3840} - \cdots$.

There are two distinct numerical issues.

1. **Division by zero.** At $\theta = 0$ exactly (a perfectly still IMU, or a zero correction), every formula evaluates to `NaN`, and a single `NaN` propagates through P and destroys the filter. For tiny θ, θ^2 can also underflow to zero.
2. **Cancellation.** B, C and D subtract nearly equal quantities. As Figure 8.2(a) shows, their naive evaluation has relative error of order u/θ^2: about 10^{-8} at $\theta = 10^{-4}$, and complete loss below $\theta \approx 10^{-8}$. A and the quaternion coefficients involve no subtraction and are accurate everywhere except at $0/0$.

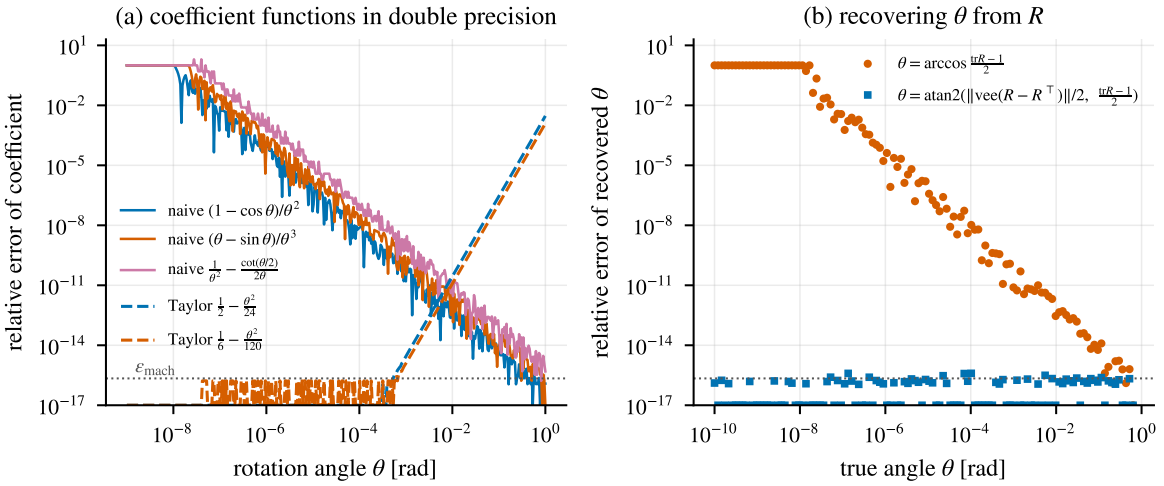


Figure 8.2: (a) Relative error of the coefficient functions evaluated naively in double precision (solid) and with a two-term Taylor series (dashed), against a 60-digit reference. The naive error grows like u/θ^2 as $\theta \to 0$, and the truncation error of the series grows like θ^4. (b) Recovering the angle from a rotation matrix: arccos loses accuracy like u/θ^2 and returns exactly zero below 10^{-8}, while the atan2 form is accurate to machine precision at every angle.

How much does the cancellation matter? It depends on what the coefficient multiplies. A coefficient with absolute error ϵ_c that multiplies $[\phi]_\times^k$ contributes an error of about $\epsilon_c \theta^k$ to the matrix. For B, C and D, $\epsilon_c \approx u/\theta^2$, so:

Function	cancelling coefficient	multiplies	naive absolute error
$\mathrm{Exp}(\phi)$	B	$[\phi]_\times^2$	$\approx u$ (harmless)
$J_r(\phi)$	B	$[\phi]_\times$	$\approx u/\theta$ (**harmful**: 3×10^{-9} at $\theta = 10^{-8}$)
$J_r(\phi)$	C	$[\phi]_\times^2$	$\approx u$
$J_r^{-1}(\phi)$	D	$[\phi]_\times^2$	$\approx u$

Key idea: why the Taylor branch exists, and how to choose its threshold

Every function needs a guard against $0/0$. Beyond that, the branch matters for accuracy wherever a cancelling coefficient multiplies a low power of $[\phi]_\times$, as in J_r. To choose the threshold θ_0 and the number of terms, balance the two error curves of Figure 8.2(a): keeping terms through θ^{2m} leaves a truncation error of order θ^{2m+2}, and the exact formula has a rounding error of order u/θ^2. Below the crossing point use the series, above it the formula. For two-term series ($m = 1$) the curves cross near $\theta \approx 3\times10^{-3}$, where both relative errors are about 10^{-12}; adding the θ^4 term moves the crossing to larger angles and lowers the error there. Remember to multiply by θ^k when translating a coefficient error into a matrix error. Record the choice and this justification next to the code (Milestone 2).

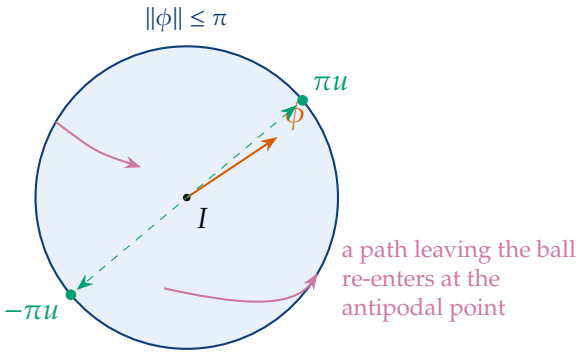


Figure 8.3: SO(3) pictured as the ball of rotation vectors of radius π, with antipodal points on the boundary identified (πu and $-\pi u$ are the same rotation). Log is smooth inside the ball but must jump when a rotation crosses $\theta = \pi$. Near the boundary a tiny change in R can flip the returned vector to the opposite side.

8.6 The logarithm map

Definition 8.4 (Logarithm map). $\mathrm{Log} : \mathrm{SO}(3) \to \mathbb{R}^3$ returns the rotation vector ϕ with $\|\phi\| \le \pi$ such that $\mathrm{Exp}(\phi) = R$.

For angles strictly between 0 and π the answer is unique. Combining the two equations of (7.6),

$$\theta = \operatorname{atan2}\Big(\tfrac{1}{2}\big\|(R - R^{\mathsf{T}})^{\vee}\big\|,\ \tfrac{1}{2}(\operatorname{tr} R - 1)\Big), \qquad \mathrm{Log}(R) = \frac{\theta}{2\sin\theta}\,(R - R^{\mathsf{T}})^{\vee}. \tag{8.9}$$

The atan2 form is essential. The common textbook formula $\theta = \arccos\big((\operatorname{tr} R - 1)/2\big)$ is correct mathematically but numerically unstable near $\theta = 0$, where arccos has infinite slope, and loses half the significant digits (Figure 8.2b). Three regimes need care in code.

1. **Small angles.** $\theta/(2\sin\theta) \to \tfrac{1}{2}$, so use $\mathrm{Log}(R) \approx \tfrac{1}{2}(1 + \tfrac{\theta^2}{6})(R - R^{\mathsf{T}})^{\vee}$.
2. **Generic angles.** Use (8.9) directly.
3. **Angles near π.** $\sin\theta \to 0$ and $(R - R^{\mathsf{T}})^{\vee} = 2\sin\theta\, u$ vanishes, so the axis cannot be recovered from the skew-symmetric part. Use the symmetric part instead: from (7.5), $\tfrac{1}{2}(R + R^{\mathsf{T}}) = \cos\theta\, I + (1 - \cos\theta)\, uu^{\mathsf{T}}$, so $uu^{\mathsf{T}} = \big(\tfrac{1}{2}(R + R^{\mathsf{T}}) - \cos\theta\, I\big)/(1 - \cos\theta)$. Take the column with the largest diagonal entry, normalize it to get $\pm u$, and fix the sign so that $u^{\mathsf{T}}(R - R^{\mathsf{T}})^{\vee} \ge 0$.

Remark **8.5** (why Log is ill-conditioned near π). Near $\theta = \pi$ two different effects combine. Algebraically, the axis must be extracted from a nearly vanishing skew-symmetric part, so the naïve formula amplifies rounding errors by $1/\sin\theta$. Topologically, Log is discontinuous at $\theta = \pi$ (Figure 8.3): a rotation by $\pi - \epsilon$ about u and one by $\pi - \epsilon$ about $-u$ are close in SO(3) but have Log values 2π apart. No implementation can remove the second effect. In an estimator it rarely matters, because Log is applied to *differences* between nearby rotations, whose angles are small. Milestone 2 nevertheless tests the near-π regime, because a broken branch there produces `NaN`.

Quaternion logarithm. For a unit quaternion $q = [w, v]$, first canonicalize to $w \geq 0$ (replace q by $-q$ if needed), then

$$\operatorname{Log}_q(q) = 2\,\operatorname{atan2}(\|v\|, w)\,\frac{v}{\|v\|} \;\approx\; \frac{2}{w}\Big(1 - \frac{\|v\|^2}{3w^2}\Big)\,v \quad \text{for small } \|v\|. \tag{8.10}$$

Canonicalization guarantees $\|\operatorname{Log}_q(q)\| \leq \pi$ and makes $\operatorname{Log}_q(q) = \operatorname{Log}_q(-q)$, so the double cover is invisible to the estimator. The atan2 is well-conditioned at all angles, including near π where $w \approx 0$: the quaternion logarithm does not suffer the algebraic ill-conditioning of the matrix version. The topological discontinuity at π remains.

8.7 The adjoint: moving a perturbation across a rotation

Proposition 8.6 (Adjoint of SO(3)). *For all $R \in \mathrm{SO}(3)$ and $\phi \in \mathbb{R}^3$,*

$$R\,\operatorname{Exp}(\phi)\,R^{\mathsf{T}} = \operatorname{Exp}(R\phi), \tag{8.11}$$

or equivalently $\operatorname{Exp}(\phi)\,R = R\,\operatorname{Exp}(R^{\mathsf{T}}\phi)$ and $R\,\operatorname{Exp}(\phi) = \operatorname{Exp}(R\phi)\,R$.

Proof. By (2.14), $R[\phi]_\times R^{\mathsf{T}} = [R\phi]_\times$. By property 4 of Proposition 3.14, $R\exp([\phi]_\times)R^{\mathsf{T}} = \exp(R[\phi]_\times R^{\mathsf{T}}) = \exp([R\phi]_\times)$. The equivalent forms follow by multiplying on the right or left by R or R^{T}. ∎

The map $\phi \mapsto R\phi$ is the *adjoint action* Ad_R, which for SO(3) is simply the rotation matrix itself. Geometrically, a rotation about an axis ϕ expressed in one frame equals the rotation about the same physical axis expressed in another frame. Equation (8.11) is the single most used identity in Jacobian derivations: it lets you slide a perturbation $\operatorname{Exp}(\delta)$ through other rotations until it reaches the position the convention requires.

8.8 Composition and the Baker–Campbell–Hausdorff formula

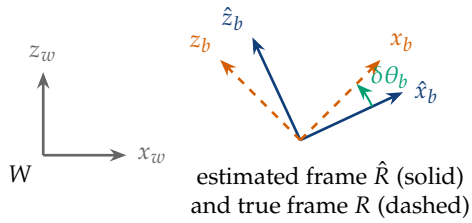
Rotations about different axes do not commute, so $\operatorname{Exp}(a)\operatorname{Exp}(b) \neq \operatorname{Exp}(a+b)$ in general. The Baker–Campbell–Hausdorff (BCH) formula quantifies the discrepancy. For matrices X, Y near zero,

$$\log\big(\exp X \exp Y\big) = X + Y + \tfrac{1}{2}[X,Y] + \tfrac{1}{12}\big([X,[X,Y]] + [Y,[Y,X]]\big) + \cdots, \tag{8.12}$$

where $[X,Y] = XY - YX$. For $\mathfrak{so}(3)$, (2.13) gives $[[a]_\times, [b]_\times] = [a \times b]_\times$, so

$$\operatorname{Log}\big(\operatorname{Exp}(a)\operatorname{Exp}(b)\big) = a + b + \tfrac{1}{2}\,a \times b + \tfrac{1}{12}\big(a \times (a \times b) + b \times (b \times a)\big) + \cdots. \tag{8.13}$$

Derivation of the second-order term. Expand both exponentials to second order: $\exp X \exp Y = (I + X + \frac{1}{2}X^2)(I + Y + \frac{1}{2}Y^2) + O(3) = I + X + Y + \frac{1}{2}X^2 + XY + \frac{1}{2}Y^2 + O(3)$, where $O(3)$ collects terms of third order in X and Y. Write the result as $\exp Z$ with $Z = X + Y + Z_2 + O(3)$ and Z_2 of second order. Then $\exp Z = I + Z + \frac{1}{2}Z^2 + O(3) = I + X + Y + Z_2 + \frac{1}{2}(X+Y)^2 + O(3)$. Comparing the two expansions, $Z_2 = \frac{1}{2}X^2 + XY + \frac{1}{2}Y^2 - \frac{1}{2}(X^2 + XY + YX + Y^2) = \frac{1}{2}(XY - YX)$.



Local: $R = \hat{R}\,\mathrm{Exp}(\delta\theta_b)$ **(this project)**
Global: $R = \mathrm{Exp}(\delta\theta_w)\hat{R}$
Same error: $\delta\theta_w = \hat{R}\,\delta\theta_b$
Covariances: $P_w = \hat{R}P_b\hat{R}^\mathsf{T}$

Figure 8.4: The local (right) perturbation measures the attitude error as a small rotation about the axes of the *estimated* body frame. The global (left) perturbation measures the same error about the world axes.

The higher-order terms are obtained the same way, with more patience; they involve only nested brackets, a theorem in itself. ■

The first correction is second order. Two small rotations therefore compose approximately additively, $\mathrm{Exp}(a)\,\mathrm{Exp}(b) \approx \mathrm{Exp}(a+b)$, with an error of order $\|a\|\|b\|$. When only one of them is small, summing the BCH series in the large argument gives the two most important first-order formulas of this chapter:

$$\mathrm{Log}\left(\mathrm{Exp}(\phi)\,\mathrm{Exp}(\delta)\right) = \phi + J_r^{-1}(\phi)\,\delta + O(\|\delta\|^2), \tag{8.14}$$

$$\mathrm{Log}\left(\mathrm{Exp}(\delta)\,\mathrm{Exp}(\phi)\right) = \phi + J_l^{-1}(\phi)\,\delta + O(\|\delta\|^2), \tag{8.15}$$

with the Jacobians J_r and J_l defined in Section 8.10.

8.9 Perturbations, ⊞ and ⊟

Let $\hat{R}$ be an estimate and R the truth. There are two natural ways to write the small rotation separating them:

$$\underbrace{R = \hat{R}\,\mathrm{Exp}(\delta\theta_b)}_{\text{local (right) perturbation}} \qquad \underbrace{R = \mathrm{Exp}(\delta\theta_w)\,\hat{R}}_{\text{global (left) perturbation}}. \tag{8.16}$$

By Proposition 8.6 they describe the same error, related by $\delta\theta_w = \hat{R}\,\delta\theta_b$. $\delta\theta_b$ is the error expressed in the estimated body frame, $\delta\theta_w$ the same error expressed in the world frame. If $\delta\theta_b \sim \mathcal{N}(0, P_b)$, then $\delta\theta_w \sim \mathcal{N}(0, \hat{R}P_b\hat{R}^\mathsf{T})$.

Both conventions are mathematically valid and widely used, but they are not interchangeable. Every Jacobian, covariance and residual depends on the choice. The project uses the local one for three reasons.

1. **It matches the sensor.** The gyroscope measures ω_b in the body frame and (7.21) puts it on the right, so body-frame errors couple to body-frame quantities.
2. **It is decoupled from global yaw.** The local attitude error dynamics (Chapter 17) depend on $\hat{\omega}$ and the biases but not on the absolute heading.
3. **It is the convention of the standard ESKF reference [47],** which makes the literature easy to cross-check.

Frozen convention: retraction and local difference, AGENTS.md C4

For the full state $x = (p, v, q, b_a, b_g, \{p_i, q_i\})$ and $\delta x \in \mathbb{R}^{15+6N}$:

$$x = \hat{x} \boxplus \delta x: \quad p = \hat{p} + \delta p, \quad v = \hat{v} + \delta v, \quad q = \hat{q} \otimes \mathrm{Exp}_q(\delta\theta), \quad b = \hat{b} + \delta b,$$
$$\delta x = x \boxminus \hat{x}: \quad \delta\theta = \mathrm{Log}_q(\hat{q}^{-1} \otimes q), \quad \text{all other blocks subtract.}$$

Clone blocks follow the same rule. Every residual, Jacobian, injection and numerical-differentiation test uses these two operators and nothing else.

The operators satisfy the axioms that make them a consistent local parameterization:

$$x \boxplus 0 = x, \qquad (x \boxplus \delta) \boxminus x = \delta, \qquad x \boxplus (y \boxminus x) = y, \tag{8.17}$$

the second for $\|\delta\theta\| < \pi$ and the third for any y. The state is a *product manifold* $\mathbb{R}^3 \times \mathbb{R}^3 \times \mathrm{SO}(3) \times \mathbb{R}^3 \times \mathbb{R}^3 \times (\mathbb{R}^3 \times \mathrm{SO}(3))^N$. Each factor has its own $\boxplus$, and the composite operator acts block-wise.

8.10 Jacobians on manifolds

Definition 8.7 (Jacobian with respect to local perturbations)**.** Let $f : \mathcal{M} \to \mathcal{N}$ map between manifolds with $\boxplus/\boxminus$ operators. Its Jacobian at x is the matrix

$$J_x^f = \frac{\partial}{\partial \delta}\Big[f(x \boxplus \delta) \boxminus f(x)\Big]_{\delta=0}, \qquad \text{so that} \qquad f(x \boxplus \delta) = f(x) \boxplus \big(J_x^f\, \delta + O(\|\delta\|^2)\big). \tag{8.18}$$

When both spaces are vector spaces this is the ordinary Jacobian. The chain rule holds unchanged, $J_x^{g\circ f} = J_{f(x)}^g J_x^f$, so complicated Jacobians are products of simple ones. The column j of J_x^f is what a finite-difference test computes by perturbing direction j, which is how every such Jacobian is verified.

8.10.1 The right and left Jacobians of SO(3)

Definition 8.8 (Right and left Jacobians)**.** The *right Jacobian* $J_r(\phi)$ is the Jacobian of $\mathrm{Exp} : \mathbb{R}^3 \to \mathrm{SO}(3)$ with the local $\boxminus$ on SO(3); the *left Jacobian* $J_l(\phi)$ uses the global one:

$$\mathrm{Exp}(\phi + \delta) = \mathrm{Exp}(\phi)\ \mathrm{Exp}\left(J_r(\phi)\,\delta\right) + O(\|\delta\|^2) = \mathrm{Exp}\left(J_l(\phi)\,\delta\right)\ \mathrm{Exp}(\phi) + O(\|\delta\|^2). \tag{8.19}$$

$J_r(\phi)$ answers the question: if I change the rotation *vector* ϕ by δ, what small *body-frame* rotation does the rotation *matrix* undergo? Only for $\phi = 0$ is the answer δ itself.

Theorem 8.9 (Closed forms)**.** *With* $\theta = \|\phi\|$,

$$J_r(\phi) = \int_0^1 \mathrm{Exp}(-s\phi)\,\mathrm{d}s = I - \frac{1-\cos\theta}{\theta^2}[\phi]_\times + \frac{\theta - \sin\theta}{\theta^3}[\phi]_\times^2, \tag{8.20}$$

$$J_r^{-1}(\phi) = I + \tfrac{1}{2}[\phi]_\times + \left(\frac{1}{\theta^2} - \frac{1+\cos\theta}{2\theta\sin\theta}\right)[\phi]_\times^2, \tag{8.21}$$

$$J_l(\phi) = J_r(-\phi) = J_r(\phi)^{\mathsf{T}} = \mathrm{Exp}(\phi)\,J_r(\phi), \qquad J_l^{-1}(\phi) = J_r^{-1}(-\phi). \tag{8.22}$$

The coefficient in (8.21) equals $D(\theta) = \frac{1}{\theta^2} - \frac{\cot(\theta/2)}{2\theta}$, which is finite at $\theta = \pi$ (where $D = 1/\pi^2$) and singular only at $\theta = 2\pi$.

Proof. Step 1: the derivative of the matrix exponential. Let $K(t) = \exp\big(t(X + \epsilon Y)\big)$, which solves $\dot{K} = (X + \epsilon Y)K$, $K(0) = I$. Its derivative with respect to ϵ at $\epsilon = 0$, $W(t) = \partial K/\partial\epsilon$, solves the equation obtained by differentiating the ODE: $\dot{W} = XW + Y\exp(tX)$, $W(0) = 0$. This is a linear ODE with an input, so by variation of constants (Proposition 4.7) $W(t) = \int_0^t \exp((t-s)X)\, Y \exp(sX)\, \mathrm{d}s$. At $t = 1$, after substituting $s \to 1 - s$,

$$\frac{\mathrm{d}}{\mathrm{d}\epsilon}\exp(X + \epsilon Y)\Big|_{\epsilon=0} = \int_0^1 \exp(sX)\, Y \exp((1-s)X)\, \mathrm{d}s.$$

Step 2: the integral form. With $X = [\phi]_\times$ and $Y = [\delta]_\times$,

$$\begin{aligned}\operatorname{Exp}(\phi)^{-1}\frac{\mathrm{d}}{\mathrm{d}\epsilon}\operatorname{Exp}(\phi + \epsilon\delta)\Big|_0 &= \int_0^1 \operatorname{Exp}(-(1-s)\phi)\,[\delta]_\times \operatorname{Exp}((1-s)\phi)\,\mathrm{d}s \\ &= \Big[\int_0^1 \operatorname{Exp}(-s\phi)\,\mathrm{d}s\;\delta\Big]_\times,\end{aligned}$$

using $R[\delta]_\times R^\mathsf{T} = [R\delta]_\times$ with $R = \operatorname{Exp}(-(1-s)\phi)$ and substituting $s \to 1-s$ again. Comparing with $\operatorname{Exp}(\phi)\operatorname{Exp}(\epsilon J_r\delta) = \operatorname{Exp}(\phi)(I + \epsilon[J_r\delta]_\times) + O(\epsilon^2)$ gives $J_r(\phi) = \int_0^1 \operatorname{Exp}(-s\phi)\,\mathrm{d}s$.

Step 3: the closed form. With $u = \phi/\theta$, Rodrigues' formula gives $\operatorname{Exp}(-s\phi) = I - \sin(s\theta)[u]_\times + (1 - \cos(s\theta))[u]_\times^2$. Integrating term by term, $\int_0^1 \sin(s\theta)\,\mathrm{d}s = (1 - \cos\theta)/\theta$ and $\int_0^1(1 - \cos(s\theta))\,\mathrm{d}s = 1 - \sin\theta/\theta$, so $J_r = I - \frac{1-\cos\theta}{\theta}[u]_\times + \big(1 - \frac{\sin\theta}{\theta}\big)[u]_\times^2$; substituting $[u]_\times = [\phi]_\times/\theta$ gives (8.20).

Step 4: the inverse. Write $K = [\phi]_\times$, $J_r = I - BK + CK^2$ with the coefficients B and C of (8.20), and look for the inverse in the same form, $I + aK + bK^2$. Multiplying out and reducing $K^3 = -\theta^2 K$ and $K^4 = -\theta^2 K^2$, the product equals I exactly when

$$a - B + \theta^2(bB - aC) = 0, \qquad b - aB + C - \theta^2 bC = 0.$$

These two linear equations have the solution $a = \frac{1}{2}$ and $b = D(\theta)$ (substitute the trigonometric expressions and simplify, or check numerically), which is (8.21); the identity $\frac{1+\cos\theta}{\sin\theta} = \cot\frac{\theta}{2}$ gives the form of D.

Step 5: the left Jacobian. By Proposition 8.6, $\operatorname{Exp}(\phi)\operatorname{Exp}(J_r\delta) = \operatorname{Exp}(\operatorname{Exp}(\phi)J_r\delta)\operatorname{Exp}(\phi)$, so $J_l = \operatorname{Exp}(\phi)J_r$. Repeating steps 1–3 with the perturbation on the left gives $J_l = \int_0^1 \operatorname{Exp}(s\phi)\,\mathrm{d}s$, which is $J_r(-\phi)$ and, since $\operatorname{Exp}(s\phi)^\mathsf{T} = \operatorname{Exp}(-s\phi)$, also $J_r(\phi)^\mathsf{T}$. ∎

The small-angle expansions are $J_r(\phi) = I - \frac{1}{2}[\phi]_\times + \frac{1}{6}[\phi]_\times^2 + O(\theta^3)$ and $J_r^{-1}(\phi) = I + \frac{1}{2}[\phi]_\times + \frac{1}{12}[\phi]_\times^2 + O(\theta^4)$, with the coefficient series of (8.8). Equations (8.14) and (8.15) follow from (8.19) by inverting the relation.

8.10.2 Elementary Jacobians

The entries of Table 8.1 are derived in two or three lines each. The method is always the same: **perturb, expand to first order, isolate the perturbation, and read off the coefficient.**

Function	Jacobian w.r.t. local perturbation	Note
$f(R) = Rv$	$\dfrac{\partial f}{\partial \delta\theta} = -R[v]_\times$	rotate a body vector to world
$f(R) = R^\mathsf{T} v$	$\dfrac{\partial f}{\partial \delta\theta} = [R^\mathsf{T} v]_\times$	express a world vector in body
$f(R) = R^{-1}$	$J = -R$	output on SO(3) (local)
$f(R_1, R_2) = R_1 R_2$	$J_{R_1} = R_2^\mathsf{T}, \quad J_{R_2} = I$	output on SO(3) (local)
$f(\phi) = \mathrm{Exp}(\phi)$	$J = J_r(\phi)$	vector to SO(3)
$f(R) = \mathrm{Log}(R)$	$J = J_r^{-1}(\mathrm{Log}\, R)$	SO(3) to vector
$f(\phi) = \mathrm{Exp}(\phi)\, v$	$\dfrac{\partial f}{\partial \phi} = -\mathrm{Exp}(\phi)[v]_\times J_r(\phi)$	rotation vector acting on v
$f(v) = Rv$	$J = R$	ordinary linear map

Table 8.1: Elementary Jacobians with respect to the local (right) perturbation $R \to R\,\mathrm{Exp}(\delta\theta)$. All other Jacobians in these notes are products of these by the chain rule. Each entry has been checked numerically by central differences.

Example 8.10 (Rotation action)**.** For $f(R) = Rv$: $f(R\,\mathrm{Exp}(\delta\theta)) = R(I + [\delta\theta]_\times)v + O(\|\delta\theta\|^2) = Rv + R[\delta\theta]_\times v$. By (2.8), $[\delta\theta]_\times v = -[v]_\times \delta\theta$, so the Jacobian is $-R[v]_\times$.

Example 8.11 (Inverse action)**.** For $f(R) = R^\mathsf{T} v$: $(R\,\mathrm{Exp}(\delta\theta))^\mathsf{T} v = \mathrm{Exp}(-\delta\theta)R^\mathsf{T} v \approx (I - [\delta\theta]_\times)R^\mathsf{T} v = R^\mathsf{T} v + [R^\mathsf{T} v]_\times\, \delta\theta$.

Example 8.12 (Inverse and composition)**.** For $f(R) = R^{-1}$: $(R\,\mathrm{Exp}(\delta\theta))^{-1} = \mathrm{Exp}(-\delta\theta)R^\mathsf{T} = R^\mathsf{T}\,\mathrm{Exp}(-R\delta\theta)$ by (8.11), so $f(R)^{-1} f(R \boxplus \delta\theta) = \mathrm{Exp}(-R\delta\theta)$ and the Jacobian is $-R$. For $f = R_1 R_2$ perturbed in R_1: $R_1\,\mathrm{Exp}(\delta\theta)R_2 = R_1 R_2\,\mathrm{Exp}(R_2^\mathsf{T}\delta\theta)$, so $J_{R_1} = R_2^\mathsf{T}$. Perturbing R_2 appends $\mathrm{Exp}(\delta\theta)$ directly, so $J_{R_2} = I$.

Example 8.13 (Logarithm)**.** For $f(R) = \mathrm{Log}(R)$ with $\phi = \mathrm{Log}\, R$: $\mathrm{Log}(R\,\mathrm{Exp}(\delta\theta)) = \mathrm{Log}(\mathrm{Exp}(\phi)\,\mathrm{Exp}(\delta\theta)) \approx \phi + J_r^{-1}(\phi)\delta\theta$ by (8.14).

Example 8.14 (A chain: a rotation vector acting on a vector)**.** For $f(\phi) = \mathrm{Exp}(\phi)v$, with ϕ an ordinary vector: $\mathrm{Exp}(\phi + \delta)v = \mathrm{Exp}(\phi)\,\mathrm{Exp}(J_r\delta)v \approx \mathrm{Exp}(\phi)(v + [J_r\delta]_\times v) = \mathrm{Exp}(\phi)v - \mathrm{Exp}(\phi)[v]_\times J_r(\phi)\delta$. This is the chain rule: the Jacobian of Exp (namely J_r) followed by the Jacobian of the rotation action (namely $-R[v]_\times$). It is the Jacobian one needs when a filter uses a rotation vector, rather than a perturbation of a rotation, as its state.

Pitfall: first-order expansions and products

The expansion $\mathrm{Exp}(\delta\theta) \approx I + [\delta\theta]_\times$ is valid only when $\delta\theta$ is the *only* small quantity being differentiated. Products of two small quantities ($[\delta\theta]_\times \delta b$, or $[\delta\theta]_\times n$ with noise n) are second order and are dropped. Large quantities such as the nominal rotation, the angular rate or the specific force are never approximated.

8.11 Uncertainty on SO(3)

With the local perturbation, an uncertain attitude is the random rotation

$$R = \hat{R}\,\mathrm{Exp}(\delta\theta), \qquad \delta\theta \sim \mathcal{N}(0, P_\theta), \tag{8.23}$$

a "Gaussian on the manifold": the mean $\hat{R}$ lives on SO(3) and the covariance P_θ lives in the tangent space at $\hat{R}$, in body-frame coordinates. Its eigenvectors are body axes about which the attitude is most or least certain. This distribution is not a Gaussian in any fixed coordinates of SO(3), which is exactly why it captures curved distributions such as the banana of Figure 5.4 better than a Gaussian on Euler angles or matrix entries.

Propagation. If $y = f(R)$ and J is its manifold Jacobian, then to first order $y = f(\hat{R}) \boxplus \delta y$ with $\delta y \approx J\delta\theta \sim \mathcal{N}(0, JP_\theta J^\mathsf{T})$.

Compounding. For independent uncertain rotations $R_1 = \hat{R}_1\,\mathrm{Exp}(\delta_1)$ and $R_2 = \hat{R}_2\,\mathrm{Exp}(\delta_2)$, the composition is $R_1R_2 = \hat{R}_1\hat{R}_2\,\mathrm{Exp}(\hat{R}_2^\mathsf{T}\delta_1)\,\mathrm{Exp}(\delta_2) \approx \hat{R}_1\hat{R}_2\,\mathrm{Exp}(\hat{R}_2^\mathsf{T}\delta_1 + \delta_2)$, so

$$P_{12} = \hat{R}_2^\mathsf{T} P_1 \hat{R}_2 + P_2. \tag{8.24}$$

***Remark* 8.15** (covariance entries rotate between axes)**.** Because P_θ is expressed in body axes, it changes when the body rotates even if no information is gained or lost. Suppose the world yaw is uncertain and roll and pitch are perfectly known: in world coordinates $P_w = \sigma^2 e_z e_z^\mathsf{T}$. In body coordinates $P_b = \hat{R}^\mathsf{T} P_w \hat{R} = \sigma^2(\hat{R}^\mathsf{T}e_z)(\hat{R}^\mathsf{T}e_z)^\mathsf{T}$. If the vehicle rolls by 90°, the world vertical is the body y-axis, and the yaw uncertainty moves from the (z, z) to the (y, y) entry of P. Individual diagonal entries of the attitude covariance are therefore not monotonic during propagation (a Milestone 5 concept), although its trace and eigenvalues behave as expected.

8.11.1 Averaging rotations

How should several noisy measurements $R_1, \ldots, R_N$ of the same attitude be averaged? The arithmetic mean of matrices (or of quaternions) is not a rotation, and the average of Euler angles is meaningless near wrap-around. The natural definition is the rotation that minimizes the sum of squared tangent-space distances, $\bar{R} = \arg\min \sum_i \|\,\mathrm{Log}(\bar{R}^\mathsf{T}R_i)\|^2$. It is computed by the fixed-point iteration

$$\bar{R} \leftarrow \bar{R}\,\mathrm{Exp}\Big(\frac{1}{N}\sum_{i=1}^{N}\mathrm{Log}\left(\bar{R}^\mathsf{T}R_i\right)\Big), \tag{8.25}$$

which averages the residuals in the tangent space at the current estimate and moves the estimate by that average, exactly as one would average vectors, but on the manifold. It converges in a few iterations when the rotations are within a few tens of degrees of each other. A faster approximation projects the arithmetic mean $M = \frac{1}{N}\sum_i R_i$ onto SO(3) with the SVD: if $M = U\Sigma V^\mathsf{T}$, the closest rotation is $U\,\mathrm{diag}(1, 1, \det(UV^\mathsf{T}))V^\mathsf{T}$ (the same construction as in Proposition 19.3).

8.12 SE(3) and the choice of state manifold

The rigid transformations $T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$ of Chapter 7 also form a Lie group, SE(3), of dimension six.

Lie algebra. Differentiating a curve $T(t)$ through the identity gives elements of the form $\xi^\wedge = \begin{bmatrix} [\phi]_\times & \rho \\ 0 & 0 \end{bmatrix}$, parameterized by $\xi = (\rho, \phi) \in \mathbb{R}^6$: an angular velocity ϕ and a linear velocity ρ, together a *twist*.

Exponential. The powers of $\xi^\wedge$ are $\begin{bmatrix} K^k & K^{k-1}\rho \\ 0 & 0 \end{bmatrix}$ for $k \geq 1$, with $K = [\phi]_\times$, so the exponential series gives

$$\operatorname{Exp}(\rho, \phi) = \begin{bmatrix} \operatorname{Exp}(\phi) & J_l(\phi)\,\rho \\ 0 & 1 \end{bmatrix}, \qquad J_l(\phi) = \sum_{k \geq 0} \frac{K^k}{(k+1)!}, \tag{8.26}$$

where the series is the left Jacobian of Theorem 8.9 (expand $\int_0^1 \operatorname{Exp}(s\phi)\,\mathrm{d}s$ term by term). Moving with a constant twist traces a *screw motion*: rotation about an axis combined with translation along it. The logarithm inverts the two blocks: $\phi = \operatorname{Log}(R)$ and $\rho = J_l(\phi)^{-1} p$.

Adjoint. Conjugation $T \operatorname{Exp}(\xi) T^{-1} = \operatorname{Exp}(\operatorname{Ad}_T \xi)$ holds with the 6×6 matrix

$$\operatorname{Ad}_T = \begin{bmatrix} R & [p]_\times R \\ 0 & R \end{bmatrix}, \tag{8.27}$$

obtained by multiplying out $T\xi^\wedge T^{-1}$. It transforms twists between frames, and plays the same role for poses as R does for rotation vectors.

Which manifold for the state? Perturbing a pose on SE(3), $T = \hat{T} \operatorname{Exp}(\delta\xi)$, expresses the translation error in the body frame and couples it to the rotation error through J_l. The project does *not* do this: its state is the product manifold of Section 8.9, in which the position error is a plain world-frame vector and the attitude error is a local rotation, independently. To first order the two are related by a rotation of the translation error ($\delta p_w \approx \hat{R}\,\delta\rho$); they differ at second order. Both choices are valid, but they produce different Jacobians and different covariances, and must never be mixed. A third choice, the extended pose group $\mathrm{SE}_2(3)$ of 5×5 matrices $\begin{bmatrix} R & v & p \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ collecting attitude, velocity and position, underlies the *invariant* EKF, whose error dynamics have the striking property of not depending on the state at all (Chapter 29).

8.13 Verifying manifold Jacobians numerically

Every analytic Jacobian in the project is checked against a central difference built on the $\boxplus/\boxminus$ operators (AGENTS.md E6). Column j of the numerical Jacobian is

$$J_{:,j} \approx \frac{f(x \boxplus he_j) \boxminus f(x \boxplus (-he_j))}{2h}, \qquad h \approx 10^{-6}. \tag{8.28}$$

The central difference has truncation error $O(h^2)$ and rounding error $O(u/h)$ (Section 4.4), so analytic and numerical Jacobians should agree to about 10^{-8} relative to the magnitude of the entries.

Pitfall: tests that cannot fail

A Jacobian test evaluated at a trivial point proves little. At $R = I$, R and R^T are equal, so a transpose bug is invisible; with zero angular rate, terms multiplied by $\hat{\omega}$ vanish; with zero biases, bias-sign errors may hide. Always test at a random non-identity attitude, non-zero rates, accelerations and biases, and ask of every test: *would this still pass if I flipped the sign of X?* A good habit is to temporarily break the analytic Jacobian and confirm that the test fails.

Project link: Milestone 2

`so3.hpp` implements `skew`, `Exp/Log`, `Exp_q/Log_q`, `J_r` and `J_r_inv` as derived here, and `numerical_jacobian.hpp` implements (8.28) generically over $\boxplus/\boxminus$. The exit criteria test exactly the properties of this chapter: $\mathrm{Exp}(\phi) \in \mathrm{SO}(3)$, $\mathrm{Log} \circ \mathrm{Exp} = \mathrm{id}$ at zero, tiny, generic and near-π angles, agreement of Exp_q and Exp, $\mathrm{Log}_q(q) = \mathrm{Log}_q(-q)$, the defining relation (8.19) of J_r, and $J_r^{-1} J_r = I$, all without allocation.

Key takeaways

- Attitude lives on the curved manifold SO(3); its errors live in the flat tangent space $\mathbb{R}^3$. Exp and Log connect them.
- Compute angles with atan2, never with arccos. Guard every coefficient function against 0/0, and use Taylor branches wherever cancellation is amplified (notably in J_r).
- Local (right) perturbation: $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$, error in body axes. Global errors are $\hat{R}\,\delta\theta$.
- The adjoint identity $R\,\mathrm{Exp}(\phi)R^\mathsf{T} = \mathrm{Exp}(R\phi)$ moves perturbations through rotations.
- $\mathrm{Exp}(\phi + \delta) \approx \mathrm{Exp}(\phi)\,\mathrm{Exp}(J_r(\phi)\delta)$ and $\mathrm{Log}(\mathrm{Exp}(\phi)\,\mathrm{Exp}(\delta)) \approx \phi + J_r^{-1}(\phi)\delta$.
- Every project Jacobian is a chain-rule product of the entries of Table 8.1, and is verified by central differences at a non-trivial point.

Exercises

8.1 Show that $\mathfrak{so}(3)$ is closed under the commutator $[X, Y] = XY - YX$ and that $[[a]_\times, [b]_\times] = [a \times b]_\times$.

8.2 Prove $\mathrm{Exp}(-\phi) = \mathrm{Exp}(\phi)^\mathsf{T}$ and $\mathrm{Exp}(s\phi)\,\mathrm{Exp}(t\phi) = \mathrm{Exp}((s+t)\phi)$. Why does the second identity not contradict non-commutativity?

8.3 Implement Log with both arccos and atan2 and reproduce Figure 8.2(b). Then construct a rotation within 10^{-6} rad of π and test the near-π branch.

8.4 Using the coefficient series of (8.8), choose a threshold and a number of terms for J_r so that its absolute error stays below 10^{-14} for all $\theta \in [0, 0.1]$. Justify your choice with the error model of Section 8.5.1.

8.5 Derive the Jacobian of $f(R) = \mathrm{Log}(\hat{R}^\mathsf{T} R)$ with respect to a local perturbation of R, and of $g(\hat{R}) = \mathrm{Log}(\hat{R}^\mathsf{T} R)$ with respect to a local perturbation of $\hat{R}$. Evaluate both at $R = \hat{R}$. (These are the building blocks of attitude residuals.)

8.6 Show that $J_l(\phi) = \mathrm{Exp}(\phi) J_r(\phi)$ and use it to prove (8.15) from (8.14).

8.7 A vehicle has attitude covariance $P_b = \mathrm{diag}(\sigma_r^2, \sigma_r^2, \sigma_y^2)$ in body axes while level. Compute P_b after it pitches by 90° with no new information, and interpret.

8.8 Verify every row of Table 8.1 with a 20-line Python script using central differences at a random point. Then flip one sign and confirm your script detects it.

Part II

Estimation Theory

Chapter 9

Estimation Fundamentals

In this chapter

- Define estimators, bias, variance and mean-square error.
- Derive linear and weighted least squares, the covariance of their estimates, and the Gauss–Markov theorem.
- Understand maximum likelihood, Fisher information and the Cramér–Rao bound: what "information" means.
- Derive the minimum mean-square-error and linear minimum mean-square-error estimators, and the orthogonality principle.
- Derive recursive least squares, the Kalman filter without dynamics.
- See that, for linear-Gaussian problems, all of these approaches give the same answer.

Before filtering, which estimates a changing state from a stream of measurements, comes *estimation*: inferring unknown quantities from noisy data. This chapter develops the classical theory. Its punchline is that several quite different philosophies (minimize squared errors; maximize the probability of the data; compute the posterior mean) lead to the same formulas for linear-Gaussian problems, and those formulas are the Kalman update.

9.1 Estimators and their errors

9.1.1 The setting

An unknown quantity $x \in \mathbb{R}^n$ is to be inferred from data $z \in \mathbb{R}^m$ related to it by a model, typically

$$z = h(x) + n, \qquad n \text{ random, zero mean, covariance } R. \tag{9.1}$$

An *estimator* is a function $\hat{x}(z)$ of the data. Because the data are random, the estimate is random too, and its quality is judged by the statistics of its *error* $e = \hat{x} - x$ over repetitions of the experiment.

Definition 9.1 (Bias, covariance and mean-square error)**.** The *bias* of an estimator is $b = \mathbb{E}[\hat{x}] - x$; it is *unbiased* if $b = 0$. Its *mean-square error* matrix is $\mathbb{E}[ee^{\mathsf{T}}]$, and its scalar mean-square error is $\mathbb{E}[\|e\|^2] = \operatorname{tr} \mathbb{E}[ee^{\mathsf{T}}]$.

Proposition 9.2 (Bias–variance decomposition)**.** $\mathbb{E}[ee^{\mathsf{T}}] = \operatorname{Cov}(\hat{x}) + bb^{\mathsf{T}}$, *and hence* $\mathbb{E}\|e\|^2 = \operatorname{tr}\operatorname{Cov}(\hat{x}) + \|b\|^2$.

Proof. Write $e = (\hat{x} - \mathbb{E}\hat{x}) + b$ and expand; the cross terms vanish because $\mathbb{E}[\hat{x} - \mathbb{E}\hat{x}] = 0$. ∎

An estimator can be accurate in two ways: by being unbiased, and by having a small covariance. A filter reports a covariance P along with its estimate, and a *consistent* filter is one whose reported P equals its actual mean-square error matrix (Chapter 19).

9.2 Linear least squares

9.2.1 The estimate

Consider the linear model

$$z = Hx + n, \qquad \mathbb{E}[n] = 0, \quad \operatorname{Cov}(n) = R, \tag{9.2}$$

with $H \in \mathbb{R}^{m \times n}$ of full column rank ($m \geq n$). The *weighted least-squares* (WLS) estimate minimizes the residuals weighted by the inverse noise covariance, so that reliable measurements count more:

$$\hat{x} = \arg\min_{x} (z - Hx)^{\mathsf{T}} R^{-1} (z - Hx) = \left(H^{\mathsf{T}} R^{-1} H\right)^{-1} H^{\mathsf{T}} R^{-1} z. \tag{9.3}$$

The formula comes from setting the gradient $-2H^{\mathsf{T}}R^{-1}(z - Hx)$ to zero (Section 3.5).

Proposition 9.3 (Statistics of the WLS estimate)**.** *The WLS estimate is unbiased, and its covariance is*

$$\operatorname{Cov}(\hat{x}) = \left(H^{\mathsf{T}} R^{-1} H\right)^{-1}. \tag{9.4}$$

Proof. Write $\hat{x} = Lz$ with $L = (H^{\mathsf{T}}R^{-1}H)^{-1}H^{\mathsf{T}}R^{-1}$. Then $LH = I$, so $\hat{x} = x + Ln$ and $\mathbb{E}\hat{x} = x$. By Proposition 5.7, $\operatorname{Cov}(\hat{x}) = LRL^{\mathsf{T}} = (H^{\mathsf{T}}R^{-1}H)^{-1}H^{\mathsf{T}}R^{-1}RR^{-1}H(H^{\mathsf{T}}R^{-1}H)^{-1} = (H^{\mathsf{T}}R^{-1}H)^{-1}$. ∎

The matrix $H^{\mathsf{T}}R^{-1}H$ is the *information* that the measurements carry about x, and its inverse is the uncertainty of the estimate. Information from independent measurements adds: with stacked blocks $z_i = H_i x + n_i$ and block-diagonal R, $H^{\mathsf{T}}R^{-1}H = \sum_i H_i^{\mathsf{T}} R_i^{-1} H_i$.

Worked example: fitting a line

A target moves along a line with unknown initial position p_0 and velocity v, and its position is measured at times $t_1, \dots, t_m$ with independent noise of standard deviation σ: $z_i = p_0 + v t_i + n_i$. With $x = (p_0, v)$, H has rows $(1, t_i)$ and $R = \sigma^2 I$, so

$$H^{\mathsf{T}}R^{-1}H = \frac{1}{\sigma^2}\begin{bmatrix} m & \sum t_i \\ \sum t_i & \sum t_i^2 \end{bmatrix}.$$

For m measurements equally spaced over a duration T with times centered on zero, the off-diagonal terms vanish and the velocity variance is $\sigma^2 / \sum t_i^2 \approx 12\sigma^2/(mT^2)$: velocity is determined more precisely when the measurements span a longer time, a first glimpse of how dynamics turn position measurements into velocity information (Figure 9.1).

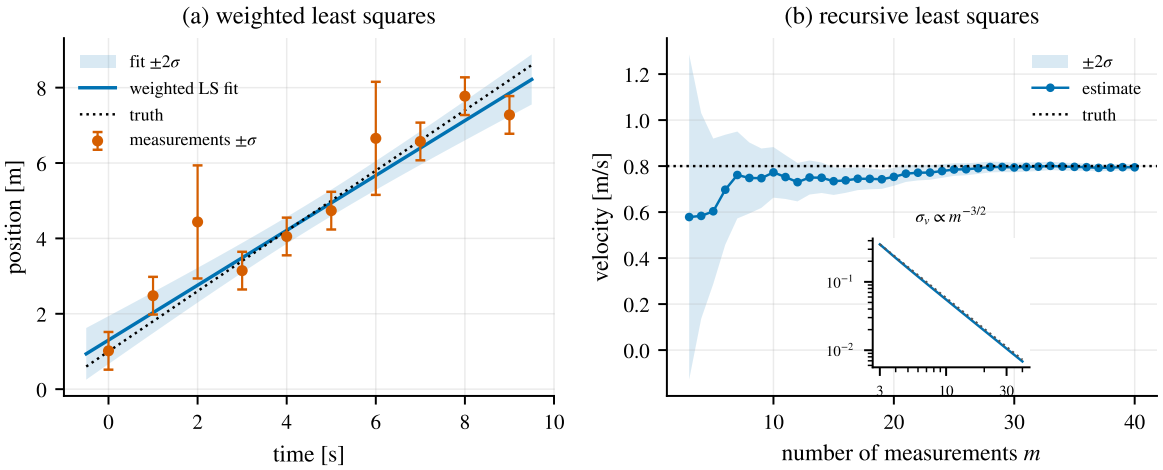


Figure 9.1: (a) Weighted least-squares fit of a line to noisy positions, with the $\pm 2\sigma$ band of the fitted line from $(H^\mathsf{T} R^{-1} H)^{-1}$. (b) Recursive least squares: the velocity estimate and its $\pm 2\sigma$ bound as measurements arrive one at a time. The bound shrinks like $m^{-3/2}$ because later measurements also lengthen the time baseline.

9.2.2 The Gauss–Markov theorem

Why weight by R^{-1}? Because it is the best possible linear choice.

> **Theorem 9.4** (Gauss–Markov). *Among all linear unbiased estimators $\tilde{x} = Lz$ of x in the model (9.2), the WLS estimate (9.3) has the smallest covariance:* $\mathrm{Cov}(\tilde{x}) \succeq (H^\mathsf{T} R^{-1} H)^{-1}$. *It is the* best linear unbiased estimator *(BLUE).*

Proof. Unbiasedness for every x requires $LH = I$. Write $L = L_\star + D$ with $L_\star$ the WLS matrix; then $DH = 0$. The covariance is $LRL^\mathsf{T} = L_\star R L_\star^\mathsf{T} + DRD^\mathsf{T} + L_\star R D^\mathsf{T} + D R L_\star^\mathsf{T}$, and the cross terms vanish: $L_\star R D^\mathsf{T} = (H^\mathsf{T} R^{-1} H)^{-1} H^\mathsf{T} R^{-1} R D^\mathsf{T} = (H^\mathsf{T} R^{-1} H)^{-1} (DH)^\mathsf{T} = 0$. So $\mathrm{Cov}(\tilde{x}) = \mathrm{Cov}(\hat{x}) + DRD^\mathsf{T} \succeq \mathrm{Cov}(\hat{x})$. ∎

The theorem needs no Gaussian assumption, only the first two moments of the noise. If the noise is Gaussian, more is true: WLS is the best of *all* unbiased estimators, linear or not, as the Cramér–Rao bound below shows.

9.3 Maximum likelihood

> **Definition 9.5** (Likelihood). For a model with density $p(z \mid x)$, the *likelihood* of a value x given observed data z is $L(x) = p(z \mid x)$, viewed as a function of x. The *maximum-likelihood* (ML) estimate maximizes it, equivalently minimizes the negative log-likelihood $-\log p(z \mid x)$.

For Gaussian noise $n \sim \mathcal{N}(0, R)$ in $z = h(x) + n$,

$$-\log p(z \mid x) = \tfrac{1}{2} \big(z - h(x)\big)^\mathsf{T} R^{-1} \big(z - h(x)\big) + \text{const}, \tag{9.5}$$

so ML estimation with Gaussian noise *is* weighted least squares, linear or nonlinear. This is the justification of the least-squares costs that pervade estimation, including the reprojection

error of computer vision (Chapter 23). For non-Gaussian noise, ML gives other costs; heavy-tailed noise, for example, leads to the robust costs of Chapter 10.

9.3.1 Fisher information and the Cramér–Rao bound

Definition 9.6 (Fisher information)**.** For a model $p(z \mid x)$, the *Fisher information matrix* is $\mathcal{I}(x) = \mathbb{E}\big[\nabla_x \log p(z \mid x)\, \nabla_x \log p(z \mid x)^{\mathsf{T}}\big] = -\mathbb{E}\big[\nabla_x^2 \log p(z \mid x)\big]$.

The Fisher information measures how sharply the likelihood peaks around the true value: a likelihood that changes rapidly with x pins x down precisely. For the linear-Gaussian model, $\nabla_x^2 \log p = -H^{\mathsf{T}} R^{-1} H$, so $\mathcal{I} = H^{\mathsf{T}} R^{-1} H$: the "information matrix" of WLS is the Fisher information.

Theorem 9.7 (Cramér–Rao lower bound)**.** *Under mild regularity conditions, every unbiased estimator satisfies* $\operatorname{Cov}(\hat{x}) \succeq \mathcal{I}(x)^{-1}$.

Proof for a scalar parameter. Let $s = \frac{\partial}{\partial x} \log p(z \mid x)$ (the *score*). It has zero mean, since $\mathbb{E}[s] = \int \partial_x p \, \mathrm{d}z = \partial_x \int p \, \mathrm{d}z = 0$. Differentiating the unbiasedness condition $\int \hat{x}(z) p(z \mid x) \, \mathrm{d}z = x$ with respect to x gives $\mathbb{E}[\hat{x}\, s] = 1$, hence $\operatorname{Cov}(\hat{x}, s) = 1$. By the Cauchy–Schwarz inequality for covariances, $1 = \operatorname{Cov}(\hat{x}, s)^2 \le \operatorname{Var}(\hat{x}) \operatorname{Var}(s) = \operatorname{Var}(\hat{x})\, \mathcal{I}(x)$. ∎

An unbiased estimator that attains the bound is *efficient*. For the linear-Gaussian model, WLS attains it: $\operatorname{Cov}(\hat{x}) = (H^{\mathsf{T}} R^{-1} H)^{-1} = \mathcal{I}^{-1}$. No unbiased estimator can do better. This gives a precise meaning to the phrase used throughout these notes: *information is inverse covariance*, and independent measurements add their information.

9.4 Bayesian estimation

9.4.1 Priors and posteriors

In the Bayesian view, the unknown x is itself a random variable with a *prior* density $p(x)$ that summarizes what is known before the data. Bayes' rule gives the *posterior* $p(x \mid z) \propto p(z \mid x) p(x)$, the complete description of what is known afterwards. Point estimates summarize the posterior:

- the *minimum mean-square error* (MMSE) estimate $\hat{x} = \mathbb{E}[x \mid z]$, the posterior mean;
- the *maximum a posteriori* (MAP) estimate, the posterior mode;
- and, ignoring the prior, the ML estimate.

Theorem 9.8 (The conditional mean is optimal)**.** *Among all estimators $g(z)$, the conditional mean $\mathbb{E}[x \mid z]$ minimizes the Bayesian mean-square error $\mathbb{E}\|x - g(z)\|^2$, and indeed minimizes the error matrix $\mathbb{E}[(x-g)(x-g)^{\mathsf{T}}]$ in the Loewner order.*

Proof. Fix z and write $m = \mathbb{E}[x \mid z]$. For any value g, $\mathbb{E}\big[(x-g)(x-g)^{\mathsf{T}} \mid z\big] = \operatorname{Cov}(x \mid z) + (m-g)(m-g)^{\mathsf{T}}$ by the bias–variance decomposition applied to the conditional distribution. The second term is PSD and vanishes exactly for $g = m$. Taking the expectation over z preserves the inequality. ∎

For a linear-Gaussian model the posterior is Gaussian (Proposition 5.12), so its mean and mode coincide: MMSE and MAP are the same estimate,

$$\hat{x} = \left(P^{-1} + H^{\mathsf{T}}R^{-1}H\right)^{-1}\left(P^{-1}\mu + H^{\mathsf{T}}R^{-1}z\right), \qquad \operatorname{Cov}(x \mid z) = \left(P^{-1} + H^{\mathsf{T}}R^{-1}H\right)^{-1}, \tag{9.6}$$

for a prior $\mathcal{N}(\mu, P)$. This is weighted least squares with one extra "measurement": the prior acts like a direct observation of x with value μ and covariance P. With an uninformative prior ($P^{-1} \to 0$) it reduces to ML/WLS.

9.4.2 Linear MMSE and the orthogonality principle

The conditional mean can be hard to compute when distributions are not Gaussian. The best *linear* (affine) estimator needs only means and covariances.

Theorem 9.9 (Linear minimum mean-square error estimator)**.** *Let x and z be random vectors with means μ_x, μ_z and covariances Σ_{xx}, Σ_{xz}, $\Sigma_{zz} \succ 0$. Among estimators of the form $\hat{x} = Az + b$, the mean-square error is minimized by*

$$\hat{x} = \mu_x + \Sigma_{xz}\Sigma_{zz}^{-1}(z - \mu_z), \qquad \mathbb{E}[(x - \hat{x})(x - \hat{x})^{\mathsf{T}}] = \Sigma_{xx} - \Sigma_{xz}\Sigma_{zz}^{-1}\Sigma_{zx}. \tag{9.7}$$

Its error is uncorrelated with the data: $\mathbb{E}\left[(x - \hat{x})(z - \mu_z)^{\mathsf{T}}\right] = 0$ (the orthogonality principle*).*

Proof. The error $e = x - Az - b$ has mean $\mu_x - A\mu_z - b$, which must vanish at the optimum (a non-zero mean only adds bb^{T}), so $b = \mu_x - A\mu_z$. Then $e = (x - \mu_x) - A(z - \mu_z)$ and $\mathbb{E}[ee^{\mathsf{T}}] = \Sigma_{xx} - A\Sigma_{zx} - \Sigma_{xz}A^{\mathsf{T}} + A\Sigma_{zz}A^{\mathsf{T}}$. Completing the square, $\mathbb{E}[ee^{\mathsf{T}}] = (A - A_\star)\Sigma_{zz}(A - A_\star)^{\mathsf{T}} + \Sigma_{xx} - \Sigma_{xz}\Sigma_{zz}^{-1}\Sigma_{zx}$ with $A_\star = \Sigma_{xz}\Sigma_{zz}^{-1}$, which is minimal for $A = A_\star$. At the optimum, $\mathbb{E}[e(z - \mu_z)^{\mathsf{T}}] = \Sigma_{xz} - A_\star\Sigma_{zz} = 0$. ∎

The formulas are identical to Gaussian conditioning (Theorem 5.11): for jointly Gaussian variables the best linear estimator is the best estimator of any kind, and for other distributions it is still the best linear one. The orthogonality principle has a geometric reading: the estimate is the projection of x onto the space of (affine) functions of the data, and the error is orthogonal to that space, exactly as a least-squares residual is orthogonal to the column space (Section 3.5).

Application to the linear model. For $z = Hx + n$ with prior mean μ and covariance P for x, and noise uncorrelated with x: $\mu_z = H\mu$, $\Sigma_{xz} = PH^{\mathsf{T}}$, $\Sigma_{zz} = HPH^{\mathsf{T}} + R$. Theorem 9.9 gives

$$\hat{x} = \mu + K(z - H\mu), \qquad K = PH^{\mathsf{T}}(HPH^{\mathsf{T}} + R)^{-1}, \qquad \mathbb{E}[ee^{\mathsf{T}}] = P - KHP. \tag{9.8}$$

This is the Kalman update (Chapter 12). By the Woodbury identity (Theorem 3.12) it equals (9.6), but it requires inverting an $m \times m$ matrix instead of an $n \times n$ one.

9.5 Recursive least squares

Measurements often arrive one at a time, and recomputing (9.3) from scratch at each arrival would waste work. Since information adds, the batch solution after k measurements satisfies

$$P_k^{-1} = P_{k-1}^{-1} + H_k^\mathsf{T} R_k^{-1} H_k, \qquad P_k^{-1}\hat{x}_k = P_{k-1}^{-1}\hat{x}_{k-1} + H_k^\mathsf{T} R_k^{-1} z_k, \tag{9.9}$$

where P_k is the covariance of $\hat{x}_k$. Rewriting this with the Woodbury identity gives the *recursive least squares* (RLS) update

$$K_k = P_{k-1}H_k^\mathsf{T}\left(H_k P_{k-1} H_k^\mathsf{T} + R_k\right)^{-1}, \qquad \hat{x}_k = \hat{x}_{k-1} + K_k\left(z_k - H_k\hat{x}_{k-1}\right), \qquad P_k = (I - K_k H_k)P_{k-1}. \tag{9.10}$$

Each step corrects the previous estimate by a gain times the *innovation* $z_k - H_k\hat{x}_{k-1}$, the part of the new measurement that the old estimate did not predict. It produces exactly the batch estimate at every step (Figure 9.1b), at a cost that does not grow with the number of measurements. RLS is the Kalman filter for a state that does not change; Chapter 12 adds dynamics.

9.6 One answer, many derivations

Approach	Assumptions	Result for $z = Hx + n$
weighted least squares	second moments of the noise	$\hat{x} = (H^\mathsf{T}R^{-1}H)^{-1}H^\mathsf{T}R^{-1}z$
Gauss–Markov (BLUE)	linear, unbiased	same
maximum likelihood	Gaussian noise	same
MAP / MMSE	Gaussian prior and noise	(9.6): WLS with the prior as a measurement
linear MMSE	means and covariances only	(9.8), equal to (9.6)
recursive LS	sequential data	same as batch, one measurement at a time

The Kalman filter inherits all of these interpretations. It is the exact Bayesian posterior for linear-Gaussian systems, the best linear estimator when noises are merely uncorrelated, and a recursive least-squares solver. When models are nonlinear, these interpretations separate, and the resulting approximations (Chapters 10 and 13) differ in which interpretation they try to preserve.

Key takeaways

- An estimator's error splits into bias and covariance: $\mathbb{E}[ee^\mathsf{T}] = \mathrm{Cov}(\hat{x}) + bb^\mathsf{T}$.
- Weighted least squares $\hat{x} = (H^\mathsf{T}R^{-1}H)^{-1}H^\mathsf{T}R^{-1}z$ has covariance $(H^\mathsf{T}R^{-1}H)^{-1}$; information adds across independent measurements.
- Gauss–Markov: WLS with R^{-1} weights is the best linear unbiased estimator.
- ML with Gaussian noise is least squares; the Fisher information bounds every unbiased estimator; information is inverse covariance.
- The conditional mean minimizes mean-square error; for Gaussians it equals the MAP estimate.

- The linear MMSE estimator $\mu + \Sigma_{xz}\Sigma_{zz}^{-1}(z - \mu_z)$ needs only second moments; its error is orthogonal to the data.
- Recursive least squares is the Kalman update without dynamics.

Exercises

9.1 Show that the sample mean of m independent measurements of a scalar with variances σ_i^2 is not the BLUE unless all σ_i are equal, and find the BLUE.

9.2 For the line-fitting example with $m = 11$ measurements at $t = -5, \dots, 5$ and $\sigma = 1$, compute the covariance of (p_0, v). How does it change if the times are $0, \dots, 10$? Interpret the off-diagonal term.

9.3 Prove the matrix version of the Cramér–Rao bound for a linear-Gaussian model by computing $\nabla_x^2 \log p$.

9.4 Show that (9.6) and (9.8) agree, using Theorem 3.12.

9.5 Derive the RLS update from the batch information form, and verify numerically that it reproduces the batch estimate at every step.

9.6 Let $x \sim \mathcal{U}[-1, 1]$ and $z = x^3 + n$ with small Gaussian n. Compare the LMMSE estimate with the conditional mean, qualitatively. Why does the LMMSE estimate do poorly?

9.7 A sensor reports a measurement with a bias β that is not modeled. Show that the WLS estimate is biased by $(H^\top R^{-1} H)^{-1} H^\top R^{-1} \beta$, and explain why unmodeled biases are worse than large noise.

Chapter 10

Nonlinear Least Squares and Optimization

In this chapter

- Formulate estimation problems with nonlinear models as nonlinear least-squares problems.
- Derive Newton, Gauss–Newton and Levenberg–Marquardt iterations, and understand when each converges.
- Optimize over manifolds such as SO(3) by iterating in the tangent space.
- Make least squares robust to outliers with M-estimators and iteratively reweighted least squares.
- Exploit the structure of problems with poses and landmarks: the Schur complement and marginalization.
- See that an extended Kalman update is one Gauss–Newton step, the bridge between filtering and optimization.

Most measurement models in VIO are nonlinear: projections divide by depth, rotations enter through trigonometric functions. Maximum likelihood with Gaussian noise then becomes a *nonlinear* least-squares problem with no closed-form solution. This chapter develops the iterative methods that solve such problems. They are used directly in this project for triangulation, and they explain what the extended Kalman filter and the MSCKF are doing.

10.1 The problem

Given residual functions $r_i(x)$ (measurement minus prediction) with noise covariances R_i, the maximum-likelihood estimate minimizes

$$F(x) = \frac{1}{2}\sum_i r_i(x)^\mathsf{T} R_i^{-1} r_i(x) = \frac{1}{2}\, r(x)^\mathsf{T} W\, r(x), \tag{10.1}$$

where r stacks the residuals and $W = \operatorname{blkdiag}(R_i^{-1})$. Examples in these notes:

- *Triangulation*: find a 3D point minimizing the reprojection errors in several images (Chapter 23).
- *Relative pose*: find the motion between two images that best explains point correspondences.
- *Bundle adjustment*: find all camera poses and all points at once.

- *Sliding-window VIO*: find a window of states minimizing visual and inertial residuals together.

Writing $W = L^\mathsf{T} L$ (for instance L the inverse of a Cholesky factor of the noise covariance) and replacing r by Lr *whitens* the residuals, so that $W = I$. We do this silently when convenient.

10.2 Descent methods

The gradient and Hessian of F follow from the chain rule. With $J = \partial r / \partial x$,

$$\nabla F = J^\mathsf{T} W r, \qquad \nabla^2 F = J^\mathsf{T} W J + \sum_k (Wr)_k \, \nabla^2 r_k. \tag{10.2}$$

Gradient descent. Step downhill, $x \leftarrow x - \alpha \nabla F$, with a step length α chosen by a line search. It always makes progress for small enough α, but it ignores curvature and zig-zags slowly along narrow valleys.

Newton's method. Minimize the quadratic Taylor model $F(x + \delta) \approx F + \nabla F^\mathsf{T} \delta + \frac{1}{2} \delta^\mathsf{T} \nabla^2 F \, \delta$ exactly:

$$\nabla^2 F \, \delta = -\nabla F. \tag{10.3}$$

Near a minimum with positive-definite Hessian, Newton's method converges *quadratically*: the number of correct digits roughly doubles each iteration. But it needs second derivatives of every residual, and far from the minimum the Hessian may not be positive definite, in which case the step can go uphill.

10.3 Gauss–Newton

Least-squares problems allow a better compromise. Linearize the *residuals* rather than the cost: $r(x + \delta) \approx r + J\delta$. The cost then becomes a linear least-squares problem in δ,

$$\min_\delta \ \tfrac{1}{2} \| r + J\delta \|_W^2 \quad \Longrightarrow \quad \left(J^\mathsf{T} W J \right) \delta = -J^\mathsf{T} W r, \tag{10.4}$$

the *Gauss–Newton* step. Comparing with (10.2), Gauss–Newton is Newton's method with the second term of the Hessian dropped. That term is small when the residuals are small at the solution, or when the residuals are nearly linear, which is the common case for well-modeled sensors. Then Gauss–Newton converges almost as fast as Newton, without second derivatives. The approximate Hessian $J^\mathsf{T} W J$ is always positive semi-definite, so the step is always a descent direction (if J has full column rank).

The linear system is the normal equation of a linear least-squares problem, so all the numerical advice of Section 3.5 applies: for small, well-conditioned problems Cholesky of $J^\mathsf{T} W J$ is fine; for ill-conditioned ones, QR of the whitened Jacobian is safer.

Algorithm 1 Gauss–Newton

Input: initial guess x, residual function $r(x)$ and its Jacobian $J(x)$

1: **repeat**
2: evaluate r and J at x
3: solve $(J^\mathsf{T} W J)\,\delta = -J^\mathsf{T} W r$ ▷ Cholesky of $J^\mathsf{T} W J$, or QR of the whitened J
4: $x \leftarrow x + \delta$
5: **until** $\|\delta\|$ or the decrease of F is below a tolerance

When Gauss–Newton fails. With a poor initial guess, large residuals or strong nonlinearity, the linear model can be a bad approximation, and a full step can increase the cost. If $J^\mathsf{T} W J$ is singular, some direction is not determined by the data (a *gauge freedom* or an unobservable direction), and the step is undefined.

10.4 Levenberg–Marquardt

The *Levenberg–Marquardt* (LM) method damps the Gauss–Newton step:

$$\left(J^\mathsf{T} W J + \lambda D\right)\delta = -J^\mathsf{T} W r, \qquad \lambda \geq 0, \tag{10.5}$$

with $D = I$ or $D = \operatorname{diag}(J^\mathsf{T} W J)$. For $\lambda \to 0$ it is Gauss–Newton; for large λ the step becomes a short gradient-descent step. The damping is adapted to how well the linear model predicted the actual change of cost. With the *gain ratio*

$$\rho = \frac{F(x) - F(x+\delta)}{F(x) - \frac{1}{2}\|r + J\delta\|_W^2}, \tag{10.6}$$

a step with ρ close to one is accepted and λ decreased (trust the model more), while a step with small or negative ρ is rejected and λ increased. LM is equivalent to a *trust-region* method that restricts the step to a region where the linearization is trusted. It is the default solver of most computer-vision and robotics libraries.

10.5 Uncertainty of the solution

At the minimum $\hat{x}$, the cost is approximately quadratic, $F(\hat{x}+\delta) \approx F(\hat{x}) + \frac{1}{2}\delta^\mathsf{T}(J^\mathsf{T} W J)\delta$, and with W equal to the inverse noise covariance the estimate has approximately the WLS covariance of the linearized problem (Proposition 9.3):

$$\operatorname{Cov}(\hat{x}) \approx \left(J^\mathsf{T} W J\right)^{-1}\Big|_{\hat{x}}. \tag{10.7}$$

This is the *Laplace approximation*: the posterior is replaced by a Gaussian fitted to its peak. It is accurate when the problem is nearly linear over the region of uncertainty. A triangulated point, for example, gets a covariance from the Jacobians of its reprojection errors.

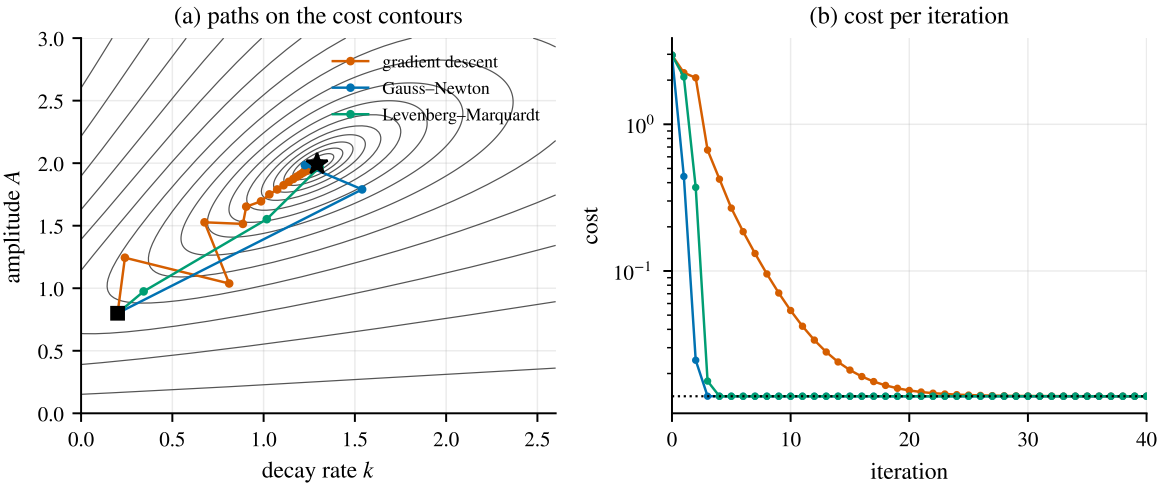


Figure 10.1: Fitting the decay rate and amplitude of an exponential to noisy data. (a) Paths of gradient descent, Gauss–Newton and Levenberg–Marquardt on the contours of the cost, from the same starting point. (b) Cost versus iteration. Gradient descent crawls along the curved valley; Gauss–Newton and LM reach the minimum in a handful of iterations.

10.6 Optimization on manifolds

When the unknown includes rotations, $x + \delta$ is replaced by the retraction $x \boxplus \delta$ of Chapter 8. Linearize the residual in the tangent space, $r(x \boxplus \delta) \approx r(x) + J\delta$ with J the manifold Jacobian (Definition 8.7), solve the same Gauss–Newton equation for δ, and update $x \leftarrow x \boxplus \delta$. The step is computed in a flat space and applied on the curved one, so the iterate never leaves the manifold and no constraint ever needs enforcing. Covariances are then expressed in the tangent space at the solution, exactly as in a filter.

Example 10.1 (Rotation averaging as Gauss–Newton). Average rotations $R_1, \dots, R_N$ (Chapter 8) by minimizing $\frac{1}{2}\sum_i \|r_i\|^2$ with $r_i(\bar{R}) = \mathrm{Log}(\bar{R}^\mathsf{T} R_i)$. Perturbing $\bar{R} \boxplus \delta = \bar{R}\,\mathrm{Exp}(\delta)$ gives $r_i = \mathrm{Log}(\mathrm{Exp}(-\delta)\bar{R}^\mathsf{T} R_i) \approx r_i(\bar{R}) - J_l^{-1}(r_i)\,\delta$ (Equation (8.15)). For small residuals $J_l^{-1} \approx I$, so $J_i \approx -I$, and the Gauss–Newton step is $\delta = \frac{1}{N}\sum_i r_i$: exactly the averaging iteration of Chapter 8.

10.7 Robust estimation

10.7.1 Outliers break least squares

A squared cost gives enormous influence to large residuals: a single wrong measurement, far from the others, can drag a least-squares estimate arbitrarily far. Real data contain such *outliers*: misassociated features, reflections, moving objects. Two defenses exist: detect and remove them (gating, RANSAC), or reduce their influence with a robust cost.

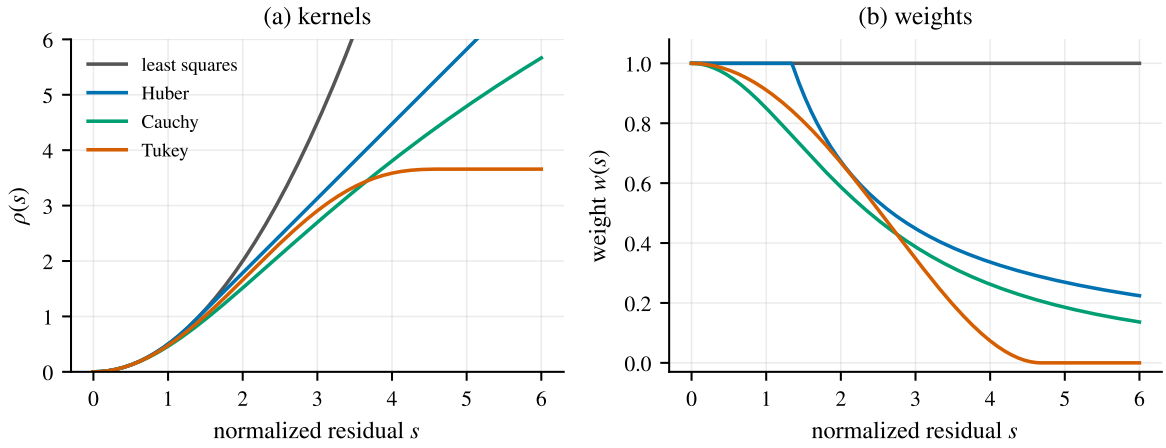


Figure 10.2: Robust kernels (left) and their weights (right) as functions of the normalized residual, for $c = 1.345$ (Huber), 2.385 (Cauchy) and 4.685 (Tukey). Huber grows linearly instead of quadratically for large residuals; Cauchy grows logarithmically; Tukey ignores residuals beyond c entirely.

10.7.2 M-estimators

An *M-estimator* replaces the quadratic by a function ρ of the whitened residual norm $s_i = \|r_i\|_{R_i^{-1}}$:

$$\min_x \sum_i \rho(s_i). \tag{10.8}$$

Kernel	$\rho(s)$	weight $w(s) = \rho'(s)/s$
least squares	$\frac{1}{2}s^2$	1
Huber	$\frac{1}{2}s^2$ for $s \le c$; $c(s - \frac{c}{2})$ otherwise	1 for $s \le c$; c/s otherwise
Cauchy	$\frac{c^2}{2}\log(1 + s^2/c^2)$	$1/(1 + s^2/c^2)$
Tukey	$\frac{c^2}{6}\left(1 - (1 - s^2/c^2)^3\right)$ for $s \le c$; $c^2/6$ otherwise	$(1 - s^2/c^2)^2$ for $s \le c$; 0 otherwise

The derivative $\psi = \rho'$ (the *influence function*) measures how hard a residual pulls on the estimate: unbounded and linear for least squares, bounded for Huber, decreasing to zero for Cauchy and Tukey (Figure 10.2). The constants c trade efficiency on clean Gaussian data against robustness; the values in the figure keep 95% of the efficiency of least squares on Gaussian noise.

10.7.3 Iteratively reweighted least squares

Setting the gradient of $\sum_i \rho(s_i)$ to zero gives $\sum_i w(s_i) J_i^\mathsf{T} R_i^{-1} r_i = 0$ with $w(s) = \rho'(s)/s$: the normal equations of a *weighted* least-squares problem whose weights depend on the residuals. *Iteratively reweighted least squares* (IRLS) alternates: compute the weights from the current residuals, take a Gauss–Newton step with those weights, repeat. Outliers receive small weights and stop dominating. Robust kernels need a reasonable initial guess, because with large initial errors every residual looks like an outlier; RANSAC (Chapter 23) provides such guesses.

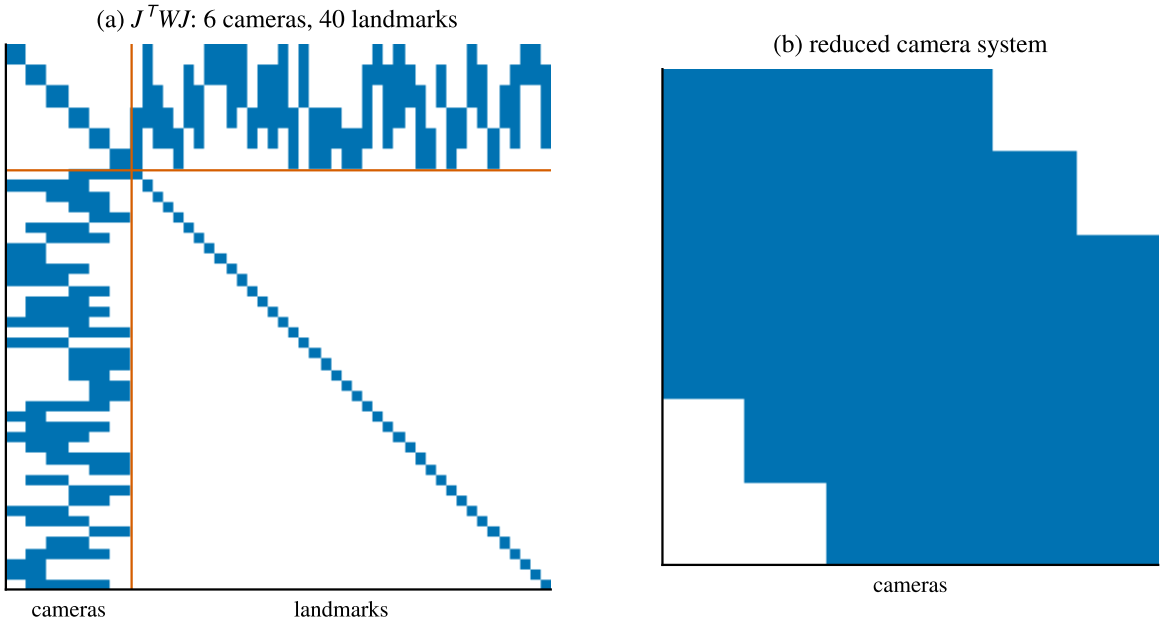


Figure 10.3: Sparsity of the normal-equation matrix $J^\mathsf{T}WJ$ for a toy bundle adjustment with 6 cameras and 40 landmarks (left), and of the reduced camera system after eliminating the landmarks (right). The landmark block is block-diagonal and cheap to eliminate; the reduced system is small but dense.

Gating, used in filters, is the hard-decision version of the same idea: a measurement whose normalized residual exceeds a threshold gets weight zero, all others weight one, a "truncated quadratic" kernel.

10.8 Structure: poses, landmarks and the Schur complement

Many estimation problems have two kinds of unknowns. In bundle adjustment, the cameras c and the landmarks l; in VIO smoothing, the states in a window and the features. Each residual involves one camera and one landmark, so the normal equations have a special block structure:

$$\begin{bmatrix} A & B \\ B^\mathsf{T} & C \end{bmatrix} \begin{bmatrix} \delta c \\ \delta l \end{bmatrix} = \begin{bmatrix} g_c \\ g_l \end{bmatrix}, \tag{10.9}$$

where C, the landmark block of $J^\mathsf{T}WJ$, is block diagonal with one 3×3 block per landmark (no residual involves two landmarks). Inverting C is therefore cheap. Eliminating the landmarks with the Schur complement (Section 3.8),

$$\left(A - BC^{-1}B^\mathsf{T}\right)\delta c = g_c - BC^{-1}g_l, \qquad \delta l = C^{-1}(g_l - B^\mathsf{T}\delta c), \tag{10.10}$$

leaves a *reduced camera system* whose size depends only on the number of cameras, often a hundred times smaller than the full problem.

The same operation has a probabilistic meaning. $J^\mathsf{T}WJ$ is the information matrix of the linearized problem, and the Schur complement is *marginalization in information form* (Theorem 5.11): the reduced system is the information that the measurements carry about the

cameras alone, after the landmarks have been integrated out. Sliding-window estimators use exactly this to discard old states, keeping their information as a Gaussian prior on the remaining ones; the price is *fill-in*, since the reduced matrix is dense (Figure 10.3). The MSCKF performs the same elimination of each landmark, by a different algebraic route: the left-nullspace projection (Chapter 26).

10.9 Filtering as optimization

Proposition 10.2 (The EKF update is one Gauss–Newton step)**.** *Consider the MAP cost of one measurement $z = h(x) + n$, $n \sim \mathcal{N}(0, R)$, with prior $\mathcal{N}(\mu, P)$:*

$$F(x) = \tfrac{1}{2}\|x - \mu\|^2_{P^{-1}} + \tfrac{1}{2}\|z - h(x)\|^2_{R^{-1}}.$$

One Gauss–Newton step from $x = \mu$ gives $x^+ = \mu + K(z - h(\mu))$ with $K = PH^\mathsf{T}(HPH^\mathsf{T} + R)^{-1}$ and $H = \partial h/\partial x|_\mu$: the extended Kalman update (Chapter 13).

Proof. The stacked residual is $r(x) = \big(L_P(x - \mu),\ L_R(h(x) - z)\big)$ with $L_P^\mathsf{T} L_P = P^{-1}$ and $L_R^\mathsf{T} L_R = R^{-1}$. At $x = \mu$, $r = (0,\ L_R(h(\mu) - z))$ and $J = (L_P,\ L_R H)$, so the Gauss–Newton equation (10.4) reads $(P^{-1} + H^\mathsf{T} R^{-1} H)\delta = H^\mathsf{T} R^{-1}(z - h(\mu))$. By the Woodbury identity, $(P^{-1} + H^\mathsf{T} R^{-1} H)^{-1} H^\mathsf{T} R^{-1} = PH^\mathsf{T}(HPH^\mathsf{T} + R)^{-1} = K$. ∎

Iterating the Gauss–Newton step, re-linearizing h at each new estimate while keeping the prior term, gives the *iterated* EKF, which handles strong measurement nonlinearities better. Sliding-window smoothers go further and re-linearize past measurements as well. The MSCKF update is one Gauss–Newton step of a sliding-window problem whose landmarks have been eliminated (Chapter 26): filtering and optimization are two ends of one spectrum, distinguished by how often they re-linearize and which variables they keep.

Key takeaways

- ML with Gaussian noise means minimizing $\frac{1}{2} r^\mathsf{T} W r$; the gradient is $J^\mathsf{T} W r$ and $J^\mathsf{T} W J$ approximates the Hessian.
- Gauss–Newton solves $(J^\mathsf{T} W J)\delta = -J^\mathsf{T} W r$ repeatedly; it converges fast when residuals are small or nearly linear.
- Levenberg–Marquardt damps the step adaptively and is robust to poor initial guesses.
- On manifolds, compute the step in the tangent space and apply it with $\boxplus$.
- $(J^\mathsf{T} W J)^{-1}$ at the optimum approximates the covariance of the estimate.
- Robust kernels bound the influence of outliers; IRLS solves them; gating is the hard version.
- Eliminating landmarks with the Schur complement is marginalization in information form.
- An EKF update is one Gauss–Newton step on the MAP cost; iterating gives the iterated EKF.

Exercises

10.1 Derive the gradient and Hessian (10.2) of the least-squares cost.

10.2 Implement Gauss–Newton and LM for fitting $y = ae^{-bt}$ to noisy data and reproduce Figure 10.1. Start from a poor initial guess and compare robustness.

10.3 Show that for a zero-residual problem Gauss–Newton converges quadratically near the solution.

10.4 Fit a line to data containing 20% gross outliers with least squares, Huber IRLS and Cauchy IRLS. Compare the results.

10.5 Implement the Schur-complement solution (10.10) for a random sparse problem and verify that it matches the full solve.

10.6 Prove that the reduced camera system equals the inverse of the camera block of $(J^\top W J)^{-1}$, and interpret this as marginalization.

10.7 Write out one iteration of the iterated EKF and show that it reduces to the standard EKF update after the first iteration.

Chapter 11

Linear Dynamical Systems

In this chapter

- Write physical systems in continuous- and discrete-time state-space form.
- Discretize deterministic inputs and white process noise exactly and approximately, including Van Loan's method.
- Understand stability, and why inertial errors grow polynomially.
- Define observability, test it with the observability matrix, and compute unobservable directions.
- Understand what observability means for the covariance of a Kalman filter.

The Kalman filter is built on a linear state-space model. This chapter collects the linear systems theory it needs: how to write a model, how to turn a continuous-time model into the discrete-time form a computer processes, and how to decide whether the measurements can determine the state at all.

11.1 State-space models

A continuous-time linear *state-space model* has the form

$$\dot{x}(t) = F(t)\,x(t) + B(t)\,u(t) + G(t)\,w(t), \qquad z(t) = H(t)\,x(t) + n(t), \tag{11.1}$$

with state x, known input u, white process noise w (density Q_c), measurement z and measurement noise n. The state is the smallest set of numbers whose present value, together with future inputs and noises, determines the future. Its discrete-time counterpart is

$$x_{k+1} = \Phi_k\,x_k + \Gamma_k\,u_k + w_k, \qquad z_k = H_k\,x_k + n_k, \qquad w_k \sim \mathcal{N}(0, Q_k),\ n_k \sim \mathcal{N}(0, R_k). \tag{11.2}$$

Example 11.1 (Models used in these notes).

- *Constant velocity*: $x = (p, v)$, $\dot{p} = v$, $\dot{v} = w$ (white acceleration). $F = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, $G = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.
- *Random-walk bias*: $\dot{b} = w_b$. $F = 0$, $G = 1$.
- *Inertial error model* (one axis, preview of Chapter 17): $\delta\dot{p} = \delta v$, $\delta\dot{v} = g\,\delta\theta - \delta b_a - n_a$, $\delta\dot{\theta} = -\delta b_g - n_g$, plus random-walk biases. A chain of integrators driven by noise.

The discrete-time solution follows by repeated substitution:

$$x_k = \Phi(k,0)\,x_0 + \sum_{j=0}^{k-1} \Phi(k, j+1)\big(\Gamma_j u_j + w_j\big), \qquad \Phi(k,j) = \Phi_{k-1}\Phi_{k-2}\cdots\Phi_j, \tag{11.3}$$

the discrete analog of the variation-of-constants formula (Proposition 4.7).

11.2 Discretization

Physical laws are continuous, computers are discrete. Over a step Δt in which F, B and G are constant, the solution of (11.1) (Theorem 4.5) is

$$x(t+\Delta t) = \underbrace{e^{F\Delta t}}_{\Phi}\, x(t) + \int_0^{\Delta t} e^{Fs} B\, u(t+\Delta t - s)\,\mathrm{d}s + \underbrace{\int_0^{\Delta t} e^{Fs} G\, w(t+\Delta t - s)\,\mathrm{d}s}_{w_k}, \tag{11.4}$$

which has exactly the discrete form (11.2).

Deterministic inputs. If the input is held constant over the step (a *zero-order hold*, which is what a sampled IMU reading means), the input term is Γu_k with $\Gamma = \int_0^{\Delta t} e^{Fs}\,\mathrm{d}s\, B$. Both Φ and Γ come from one matrix exponential,

$$\exp\!\left(\begin{bmatrix} F & B \\ 0 & 0 \end{bmatrix} \Delta t\right) = \begin{bmatrix} \Phi & \Gamma \\ 0 & I \end{bmatrix}, \tag{11.5}$$

which can be verified by noting that both sides solve the same matrix ODE in Δt.

Process noise. The discrete noise w_k is a weighted integral of white noise, so it is zero-mean Gaussian with covariance (by the same calculation as in Proposition 6.5)

$$Q_d = \int_0^{\Delta t} e^{Fs}\, G Q_c G^{\mathsf{T}}\, e^{F^{\mathsf{T}} s}\,\mathrm{d}s. \tag{11.6}$$

11.2.1 Approximations

Expanding to first order in Δt gives the approximations used by most real-time filters and frozen in AGENTS.md F2:

$$\Phi \approx I + F\Delta t, \qquad Q_d \approx G Q_c G^{\mathsf{T}}\, \Delta t. \tag{11.7}$$

Both are accurate to $O(\Delta t^2)$ per step. The units confirm the second: a noise density σ_a in $\mathrm{m/s^2/\sqrt{Hz}}$ gives $\sigma_a^2 \Delta t$ in $\mathrm{m^2/s^2}$, a velocity variance. A second-order option adds $\frac{1}{2}F^2\Delta t^2$ to Φ and $\frac{1}{2}(FGQ_cG^{\mathsf{T}} + GQ_cG^{\mathsf{T}}F^{\mathsf{T}})\Delta t^2$ to Q_d.

Example 11.2 (Double integrator)**.** For position and velocity driven by white acceleration of density σ, $F = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, $G = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $Q_c = \sigma^2$. Example 3.16 gives $e^{Fs}G = [s;\ 1]$, so (11.6) evaluates exactly to

$$Q_d = \sigma^2 \begin{bmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{bmatrix}. \tag{11.8}$$

The first-order approximation $Q_d \approx \mathrm{diag}(0, \sigma^2\Delta t)$ puts no noise on position within a single

step. Yet after n steps of exact Φ, the accumulated position variance is $\sum_{k=1}^{n}(k\Delta t)^2\sigma^2\Delta t \to \sigma^2 t^3/3$ as $\Delta t \to 0$ (a Riemann sum), which is the exact answer of Proposition 6.4. The per-step error of the approximation does not accumulate into a wrong growth law; it only introduces an $O(\Delta t)$ relative error, negligible at 200 Hz.

11.2.2 Van Loan's method

When higher accuracy is wanted, (11.6) can be computed exactly with a single matrix exponential [54].

Proposition 11.3 (Van Loan). *Let* $C = \exp\left(\begin{bmatrix} -F & GQ_cG^{\mathsf{T}} \\ 0 & F^{\mathsf{T}} \end{bmatrix}\Delta t\right) = \begin{bmatrix} C_{11} & C_{12} \\ 0 & C_{22} \end{bmatrix}$. *Then* $\Phi = C_{22}^{\mathsf{T}}$ *and* $Q_d = \Phi\, C_{12}$.

Proof. For a block upper-triangular generator, the diagonal blocks of the exponential are the exponentials of the diagonal blocks, so $C_{22} = e^{F^{\mathsf{T}}\Delta t}$. The off-diagonal block, as a function of Δt, satisfies $\dot{C}_{12} = -FC_{12} + GQ_cG^{\mathsf{T}}C_{22}$ with $C_{12}(0) = 0$, whose solution (variation of constants) is $C_{12} = \int_0^{\Delta t} e^{-F(\Delta t - s)}\, GQ_cG^{\mathsf{T}}\, e^{F^{\mathsf{T}}s}\,\mathrm{d}s$. Multiplying by $\Phi = e^{F\Delta t}$ on the left, and using $e^{F\Delta t}e^{-F(\Delta t-s)} = e^{Fs}$, gives $\Phi C_{12} = \int_0^{\Delta t} e^{Fs}\, GQ_cG^{\mathsf{T}}\, e^{F^{\mathsf{T}}s}\,\mathrm{d}s$, which is (11.6). ∎

For the 15-state inertial error model this costs a 30×30 matrix exponential per step, too expensive at 200 Hz but an excellent test oracle for the closed forms of Chapter 17.

11.3 Stability

A linear time-invariant system is *stable* if its free response decays. For $\dot{x} = Fx$ the modes are $e^{\lambda_i t}$ (Theorem 4.5), so the system is asymptotically stable when all eigenvalues of F have negative real parts. For $x_{k+1} = \Phi x_k$ the modes are λ_i^k, so all eigenvalues of Φ must lie strictly inside the unit circle; the two conditions correspond through $\lambda_\Phi = e^{\lambda_F \Delta t}$.

Integrators have eigenvalues at zero (continuous) or one (discrete): they are *marginally* stable. A single integrator holds its state; a chain of integrators (a nilpotent F) makes initial errors grow *polynomially*, like t, t^2, t^3. The error dynamics of inertial navigation are of this kind (Example 11.1), which is why IMU errors grow polynomially rather than exponentially, and why they never stop growing without aiding.

11.4 Observability

11.4.1 Definition and the observability matrix

Definition 11.4 (Observability). A system is *observable* if the initial state x_0 can be determined from the inputs and the noise-free outputs over a finite time.

Because the inputs are known, their contribution to the outputs can be subtracted, so observability depends only on (Φ, H). For a discrete time-invariant system without noise,

$z_k = H\Phi^k x_0$. Stacking the first n outputs,

$$\begin{bmatrix} z_0 \\ z_1 \\ \vdots \\ z_{n-1} \end{bmatrix} = \underbrace{\begin{bmatrix} H \\ H\Phi \\ \vdots \\ H\Phi^{n-1} \end{bmatrix}}_{\mathcal{O}} x_0. \tag{11.9}$$

Theorem 11.5 (Observability rank test)**.** *The system is observable if and only if the* observability matrix $\mathcal{O}$ *has rank n. Its nullspace is the* unobservable subspace*: the initial states that produce identically zero output forever.*

Proof. If $\mathcal{O}$ has full column rank, x_0 is the unique solution of the stacked equations. If $\mathcal{O}x_0 = 0$ for some $x_0 \neq 0$, then $H\Phi^k x_0 = 0$ for $k < n$, and by the Cayley–Hamilton theorem (Φ^n is a linear combination of $I, \Phi, \ldots, \Phi^{n-1}$) also for all $k \geq n$; such an x_0 is indistinguishable from zero. More outputs never add rank beyond the first n. ∎

The unobservable subspace is invariant under Φ: an unobservable error stays unobservable as the system evolves. For time-varying systems, the same reasoning gives $\mathcal{O} = [H_0;\ H_1\Phi(1,0);\ H_2\Phi(2,0);\ \ldots]$, and observability can depend on the trajectory, a fact that matters greatly for VIO (Chapter 29).

11.4.2 Examples

Example 11.6 (Position versus velocity measurements)**.** For the discretized constant-velocity model, $\Phi = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$.

- Measuring position, $H = [1\ \ 0]$: $\mathcal{O} = \begin{bmatrix} 1 & 0 \\ 1 & \Delta t \end{bmatrix}$ has rank 2. Velocity is observable, through the change of position.
- Measuring velocity, $H = [0\ \ 1]$: $\mathcal{O} = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$ has rank 1, with nullspace $(1, 0)$. Position is unobservable: shifting the whole trajectory changes no measurement. This is dead reckoning in one line.

Example 11.7 (A bias that looks like the state)**.** Let $x = (p, b)$ with $\dot{p} = 0$, $\dot{b} = 0$ and $z = p + b$. Then $H = [1\ \ 1]$, $\Phi = I$, and $\mathcal{O} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ has nullspace $(1, -1)$: a change of position compensated by an opposite change of bias is invisible. A sensor bias can be estimated only if something else breaks this symmetry. In inertial navigation that something is motion: an accelerometer bias becomes separable from tilt once the vehicle rotates (Chapter 16).

Example 11.8 (An IMU along a line)**.** Let $x = (p, v, b_a)$ with $\dot{p} = v$, $\dot{v} = a_m - b_a$ (a_m known) and $\dot{b}_a = 0$, and measure p. In continuous time, $F = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix}$ and $H = [1\ 0\ 0]$,

and the continuous-time observability matrix $[H;\ HF;\ HF^2]$ is $\begin{bmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & -1\end{bmatrix}$, of full rank. Position measurements reveal velocity (one differentiation) and the accelerometer bias (two differentiations): the reason a pose sensor makes all IMU states observable in Milestone 6.

11.4.3 Duality and controllability

The dual notion, *controllability*, asks whether the inputs (or noises) can drive the state anywhere; its test matrix is $[\Gamma,\ \Phi\Gamma,\ \ldots,\ \Phi^{n-1}\Gamma]$, and (Φ, Γ) is controllable exactly when $(\Phi^\mathsf{T}, \Gamma^\mathsf{T})$ is observable. For a Kalman filter, controllability by the process noise means that every state is continually excited by noise, which keeps the filter from becoming certain of anything and ignoring new data.

11.4.4 What observability means for a filter

For a time-invariant system that is observable from its measurements and controllable from its process noise, the Kalman filter covariance converges to a unique, bounded steady state, whatever the initial covariance (Chapter 12). Along unobservable directions there is no such bound: the covariance there can only grow with process noise or stay at its prior value, because no measurement reduces it. A filter must respect this. If its linearized model makes an unobservable direction appear observable, it will *reduce* the covariance along that direction without justification, and become overconfident. This is precisely what happens to the standard EKF in VIO (Chapter 29).

Key takeaways

- State-space models collect everything needed to predict the future; discrete models follow from continuous ones by integrating over a step.
- $\Phi = e^{F\Delta t}$, $Q_d = \int e^{Fs} G Q_c G^\mathsf{T} e^{F^\mathsf{T} s} \mathrm{d}s$; to first order $\Phi \approx I + F\Delta t$ and $Q_d \approx G Q_c G^\mathsf{T} \Delta t$. Van Loan computes both exactly.
- Chains of integrators are marginally stable: their errors grow polynomially.
- Observability: $\mathcal{O} = [H;\ H\Phi;\ \ldots]$ must have full rank; its nullspace contains the directions the outputs cannot see.
- A filter may reduce uncertainty only in observable directions.

Exercises

11.1 Write the mass-spring-damper $m\ddot{y} + c\dot{y} + ky = u$ in state-space form and find its eigenvalues. When is it stable?

11.2 Compute Γ for the double integrator with a zero-order-hold acceleration input, and interpret its entries.

11.3 Verify Proposition 11.3 numerically for the double integrator.

11.4 Find the unobservable subspace of the constant-velocity model with measurements of $p_1 - p_2$ for two independent targets.

11.5 Show that the continuous-time system $\dot{x} = Fx$, $z = Hx$ is observable if and only if $[H;\ HF;\ \ldots;\ HF^{n-1}]$ has full rank.

11.6 A gyroscope measures $\omega + b_g$, and an inclinometer measures the integrated angle. Is the gyro bias observable? What if the inclinometer is replaced by a second gyroscope?

Chapter 12

The Kalman Filter

In this chapter

- Derive the Kalman filter from Bayes' rule and Gaussian conditioning, and recognize it as linear MMSE estimation and recursive least squares with dynamics.
- Understand the gain, the innovation and its covariance, the several equivalent forms of the covariance update, and the information form.
- Prove that the innovations of a correct filter are white, and use this to test filters without ground truth.
- Know why the Joseph form is used and how to implement the update without explicit inverses.
- Understand steady-state behavior, sequential updates, colored noise, continuous-time filtering and smoothing.
- Work a filter through by hand.

Rudolf Kálmán's 1960 paper [32] gave a recursive solution to linear estimation that needs only the current estimate and covariance. It flew on Apollo and remains the backbone of navigation. This chapter derives it for linear systems; Chapters 13 and 14 extend it to the nonlinear, manifold-valued state of VIO.

12.1 The model

A discrete-time linear system with Gaussian noise is described by (Chapter 11)

$$x_k = \Phi_k\, x_{k-1} + w_k, \qquad w_k \sim \mathcal{N}(0, Q_k), \tag{12.1}$$

$$z_k = H_k\, x_k + n_k, \qquad n_k \sim \mathcal{N}(0, R_k), \tag{12.2}$$

with $x_0 \sim \mathcal{N}(\hat{x}_0, P_0)$, and with x_0, $\{w_k\}$ and $\{n_k\}$ mutually independent and white. Φ_k is the state transition matrix, Q_k the process noise covariance, H_k the measurement matrix and R_k the measurement noise covariance; known inputs add a term $\Gamma_k u_k$ that only shifts the mean. Continuous-time models are discretized as in Section 11.2. The goal is the posterior $p(x_k \mid z_{1:k})$ given all measurements up to time k.

12.2 The Bayes filter

Two structural assumptions make recursion possible. The state is *Markov*: given x_{k-1}, the earlier states and measurements carry no further information about x_k. And each

measurement depends only on the current state. Then the posterior can be computed recursively in two steps:

$$\textbf{predict:} \quad p(x_k \mid z_{1:k-1}) = \int p(x_k \mid x_{k-1})\, p(x_{k-1} \mid z_{1:k-1})\, \mathrm{d}x_{k-1}, \tag{12.3}$$

$$\textbf{update:} \quad p(x_k \mid z_{1:k}) = \frac{p(z_k \mid x_k)\, p(x_k \mid z_{1:k-1})}{p(z_k \mid z_{1:k-1})}. \tag{12.4}$$

The prediction marginalizes the previous state out of the joint density $p(x_k, x_{k-1} \mid z_{1:k-1}) = p(x_k \mid x_{k-1}) p(x_{k-1} \mid z_{1:k-1})$ (using the Markov property); the update is Bayes' rule, with the new measurement independent of the past given the current state. For a linear-Gaussian model all of these densities are Gaussian, so each step reduces to updating a mean and a covariance. We write $\hat{x}_k^-, P_k^-$ for the predicted (prior) moments and $\hat{x}_k^+, P_k^+$ for the updated (posterior) moments.

12.3 Derivation

12.3.1 Prediction

By Proposition 5.7 applied to (12.1), with x_{k-1} and w_k independent,

$$\hat{x}_k^- = \Phi_k\, \hat{x}_{k-1}^+, \qquad P_k^- = \Phi_k P_{k-1}^+ \Phi_k^\mathsf{T} + Q_k. \tag{12.5}$$

Prediction moves the error with the dynamics and adds process noise; it never decreases uncertainty in the directions excited by Q_k.

12.3.2 Update

Before the measurement arrives, the state and the measurement are jointly Gaussian. From (12.2), dropping the time index,

$$\begin{bmatrix} x \\ z \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \hat{x}^- \\ H\hat{x}^- \end{bmatrix}, \begin{bmatrix} P^- & P^- H^\mathsf{T} \\ HP^- & HP^- H^\mathsf{T} + R \end{bmatrix} \right). \tag{12.6}$$

Conditioning on the observed z with Theorem 5.11 gives the update directly.

Theorem 12.1 (Kalman update). *Define the* innovation *(residual)* $r = z - H\hat{x}^-$*, its covariance* $S = HP^-H^\mathsf{T} + R$*, and the* Kalman gain $K = P^-H^\mathsf{T}S^{-1}$*. Then*

$$\hat{x}^+ = \hat{x}^- + Kr, \qquad P^+ = P^- - KSK^\mathsf{T} = (I - KH)\, P^-. \tag{12.7}$$

Proof. Equation (5.10) with $\Sigma_{ab} = P^-H^\mathsf{T}$ and $\Sigma_{bb} = S$ gives $\hat{x}^+ = \hat{x}^- + P^-H^\mathsf{T}S^{-1}(z - H\hat{x}^-)$ and $P^+ = P^- - P^-H^\mathsf{T}S^{-1}HP^-$. Since $KSK^\mathsf{T} = P^-H^\mathsf{T}S^{-1}HP^- = KHP^-$, both covariance forms agree. ∎

The same formulas arise from the other routes of Chapter 9: they are the linear MMSE estimate (9.8) (so the Kalman filter is the best *linear* filter even for non-Gaussian noise with

the same second moments), the MAP estimate of Proposition 5.12, and recursive least squares with a prediction step inserted between measurements.

Key idea: the gain is a ratio of uncertainties

For a scalar state measured directly ($H = 1$), $K = \frac{P^-}{P^- + R}$ and $P^+ = \frac{P^- R}{P^- + R}$. If the prior is much more uncertain than the sensor ($P^- \gg R$), $K \to 1$ and the estimate jumps to the measurement. If the sensor is much worse ($R \gg P^-$), $K \to 0$ and the measurement is nearly ignored. The posterior variance is always smaller than both. In general, K maps the innovation, which lives in measurement space, into a correction in state space, including corrections to states that are not measured directly but are correlated with measured ones through P^-. This is how a pose measurement corrects velocity and biases.

12.3.3 The information form

Applying the Woodbury identity (Theorem 3.12) to $P^+ = P^- - P^- H^\mathsf{T}(HP^-H^\mathsf{T} + R)^{-1}HP^-$ gives

$$(P^+)^{-1} = (P^-)^{-1} + H^\mathsf{T} R^{-1} H, \qquad K = P^+ H^\mathsf{T} R^{-1}. \tag{12.8}$$

Information adds: each independent measurement contributes $H^\mathsf{T} R^{-1} H$. The covariance form (12.7) inverts an $m \times m$ matrix (m = measurement dimension); the information form inverts an $n \times n$ matrix (n = state dimension). Filters use the former when $m < n$; the *information filter*, which propagates $(P^{-1}, P^{-1}\hat{x})$ instead, is preferred when measurements are many and large and the prediction step is simple.

12.4 A worked example by hand

Worked example: a random walk observed directly

A scalar state follows $x_k = x_{k-1} + w_k$ with $Q = 1$ and is measured as $z_k = x_k + n_k$ with $R = 2$. Start from $\hat{x}_0 = 0$, $P_0 = 4$, and receive $z_1 = 1.5$, $z_2 = 2.0$.

	$k = 1$	$k = 2$
predict	$\hat{x}_1^- = 0$, $P_1^- = 4 + 1 = 5$	$\hat{x}_2^- = 1.071$, $P_2^- = 1.429 + 1 = 2.429$
innovation	$r_1 = 1.5$, $S_1 = 5 + 2 = 7$	$r_2 = 2.0 - 1.071 = 0.929$, $S_2 = 4.429$
gain	$K_1 = 5/7 = 0.714$	$K_2 = 2.429/4.429 = 0.548$
update	$\hat{x}_1^+ = 1.071$, $P_1^+ = 5 \cdot 2/7 = 1.429$	$\hat{x}_2^+ = 1.071 + 0.548 \cdot 0.929 = 1.581$, $P_2^+ = 1.097$

The variance falls from 4 to about 1.1 in two steps and then settles where the information gained per measurement balances the uncertainty added per step: solving $P^- = P^- R/(P^- + R) + Q$ gives $P_\infty^- = \frac{1}{2}(Q + \sqrt{Q^2 + 4QR}) = 2$, $K_\infty = 0.5$, $P_\infty^+ = 1$.

12.5 Properties

Proposition 12.2 (Optimality). *For the linear-Gaussian model, $\hat{x}_k^+ = \mathbb{E}[x_k \mid z_{1:k}]$: the Kalman filter is the minimum mean-square error estimator among* all *estimators, linear or not (Theorem 9.8). If the noises are not Gaussian but have the stated means and covariances, it remains the best* linear *estimator (Theorem 9.9).*

Proposition 12.3 (Innovation properties). *If the model is correct, the innovations r_k are zero-mean, Gaussian, with covariance S_k, and uncorrelated across time. Consequently $r_k^\mathsf{T} S_k^{-1} r_k \sim \chi^2_{m_k}$.*

Proof. $r_k = H_k(x_k - \hat{x}_k^-) + n_k$ is a linear combination of zero-mean Gaussians; its two terms are independent, giving covariance $H_k P_k^- H_k^\mathsf{T} + R_k = S_k$. For whiteness, take $j < k$. The innovation r_j is a linear function of $z_1, \dots, z_j$. The prediction error $x_k - \hat{x}_k^-$ is uncorrelated with every past measurement by the orthogonality principle (Theorem 9.9), and n_k is independent of the past; hence $\mathbb{E}[r_k r_j^\mathsf{T}] = H_k \mathbb{E}[(x_k - \hat{x}_k^-) r_j^\mathsf{T}] + \mathbb{E}[n_k r_j^\mathsf{T}] = 0$. The χ^2 statement is Theorem 5.17. ∎

Proposition 12.3 is the most useful diagnostic in filtering, because it can be checked on real data without ground truth. The *normalized innovation squared* (NIS) $r^\mathsf{T} S^{-1} r$ should look like a χ^2_m sample, and the innovation sequence should look white. Values that are systematically too large mean the filter is overconfident or the model is wrong; systematically too small means it is underconfident; correlation in time reveals unmodeled dynamics. Gating (Chapter 18) uses the same statistic one measurement at a time.

12.5.1 Steady state

For a time-invariant model, the covariance recursion does not depend on the measurements at all: it can be computed in advance. Substituting the update into the prediction gives the *discrete Riccati equation*

$$P_{k+1}^- = \Phi\big(P_k^- - P_k^- H^\mathsf{T} (H P_k^- H^\mathsf{T} + R)^{-1} H P_k^-\big)\Phi^\mathsf{T} + Q. \tag{12.9}$$

If (Φ, H) is observable (more precisely, detectable) and $(\Phi, Q^{1/2})$ is controllable (stabilizable), P_k^- converges to the unique positive semi-definite solution of the algebraic Riccati equation obtained by setting $P_{k+1}^- = P_k^-$, from any initial P_0, and the gain converges to a constant. The worked example shows this convergence. Navigation filters are not time-invariant, but the same balance between injected and gained information governs them: in observable directions the covariance settles to a level set by the noise; in unobservable directions it does not (Section 11.4).

12.6 The Joseph form

Proposition 12.4 (Joseph form). *For* any *gain K (optimal or not), the error covariance after the update $\hat{x}^+ = \hat{x}^- + K(z - H\hat{x}^-)$ is*

$$P^+ = (I - KH)\, P^- \,(I - KH)^\mathsf{T} + K R K^\mathsf{T}. \tag{12.10}$$

For the optimal gain it equals $(I - KH)P^-$.

Algorithm 2 Numerically careful Kalman update

Input: prior $(\hat{x}, P)$, measurement z, model (H, R)

$r \leftarrow z - H\hat{x}$ ▷ innovation
$S \leftarrow HPH^\mathsf{T} + R; \quad S \leftarrow \frac{1}{2}(S + S^\mathsf{T})$
$L \leftarrow \mathrm{chol}(S)$ ▷ or LDL^T; failure means S is not PD
$\gamma \leftarrow \|L^{-1}r\|^2$ ▷ NIS, for gating and logging
if $\gamma > \chi^2_{0.95}(\dim r)$ **then**
    return reject (and count it)
solve $S\,K^\mathsf{T} = HP$ for K^T using L ▷ never form S^{-1}
$\hat{x} \leftarrow \hat{x} + Kr$
$P \leftarrow (I - KH)P(I - KH)^\mathsf{T} + KRK^\mathsf{T}; \quad P \leftarrow \frac{1}{2}(P + P^\mathsf{T})$

Proof. The posterior error is $e^+ = x - \hat{x}^+ = (I - KH)(x - \hat{x}^-) - Kn$. Its two terms are independent, so by Proposition 5.7 its covariance is (12.10). Expanding with $K = P^-H^\mathsf{T}S^{-1}$: $(I - KH)P^-(I - KH)^\mathsf{T} + KRK^\mathsf{T} = (I - KH)P^- - (I - KH)P^-H^\mathsf{T}K^\mathsf{T} + KRK^\mathsf{T} = (I - KH)P^- - P^-H^\mathsf{T}K^\mathsf{T} + KSK^\mathsf{T}$, and the last two terms cancel because $KSK^\mathsf{T} = P^-H^\mathsf{T}K^\mathsf{T}$. ∎

Why use the longer formula? In floating point, $(I - KH)P^-$ is the difference of two nearly equal matrices when the update is informative, and it is not symmetric. Rounding can push small eigenvalues negative, after which the filter is broken: variances become negative, Cholesky fails, gates misbehave. The Joseph form is a sum of a congruence of P^- and a congruence of R, so by Proposition 2.29 it is PSD whenever P^- and R are, regardless of rounding in K. It also remains correct when K is deliberately suboptimal (for example, a gain computed with a slightly stale P). AGENTS.md F3 mandates it.

Implementation note: cost of the Joseph form

With n states and m measurements, $(I - KH)P(I - KH)^\mathsf{T}$ costs $O(n^3)$ if done naively. For the MSCKF ($n = 75$) this is fine. If it ever matters, note that the update can be rearranged as $P^+ = P - KHP - (KHP)^\mathsf{T} + KSK^\mathsf{T}$, which costs $O(n^2m)$ and is algebraically identical to the Joseph form for any gain. It no longer has the sum-of-congruences structure that guarantees positive semi-definiteness under rounding, though, so keep the symmetrization and monitor the smallest eigenvalue in tests. Profile before optimizing.

12.7 Implementing the update

Algorithm 2 collects the numerical rules of the previous sections. Two details deserve comment. The gain is obtained by solving $SK^\mathsf{T} = HP^\mathsf{T} = HP$ (since P is symmetric), so the Cholesky factor of S is reused for both the gate and the gain. And symmetrization after each step removes the asymmetry that rounding introduces, which otherwise slowly accumulates (AGENTS.md F2).

12.7.1 Sequential updates

Proposition 12.5 (Sequential processing). *If a stacked measurement $z = (z_1, z_2)$ has block-diagonal noise covariance $R = \mathrm{blkdiag}(R_1, R_2)$, updating with z_1 and then with z_2 gives exactly the same posterior as a single update with z.*

Proof. With independent noise blocks, the likelihood factorizes, $p(z \mid x) = p(z_1 \mid x)p(z_2 \mid x)$, so $p(x \mid z) \propto p(z_2 \mid x)\left[p(z_1 \mid x)p(x)\right] \propto p(z_2 \mid x)\, p(x \mid z_1)$: updating with z_2 the posterior after z_1. Both updates are exact for linear-Gaussian models. (In information form: the information contributions $H_1^\mathsf{T} R_1^{-1} H_1 + H_2^\mathsf{T} R_2^{-1} H_2$ simply add.) ∎

Sequential processing can reduce cost (several small inversions instead of one large one) and allows per-block gating. If the blocks share errors (correlated R), sequential processing without accounting for the correlation is wrong. For nonlinear models the two orders differ slightly, because the second update is linearized at the estimate produced by the first.

12.8 Extensions

Colored noise. The derivation assumed white process and measurement noise. Time-correlated disturbances are handled by adding the state of a shaping filter to the filter state (Chapter 6): this is why IMU biases, which are random walks, are state variables.

Continuous time: the Kalman–Bucy filter. For continuous measurements $z(t) = Hx + n$ with white n of density R_c, the limit of the discrete filter as the sampling period goes to zero is

$$\dot{\hat{x}} = F\hat{x} + K\big(z - H\hat{x}\big), \qquad K = PH^\mathsf{T} R_c^{-1}, \qquad \dot{P} = FP + PF^\mathsf{T} + GQ_cG^\mathsf{T} - PH^\mathsf{T} R_c^{-1} HP, \tag{12.11}$$

a Lyapunov equation (Proposition 6.5) with an information term subtracted. Navigation filters process sampled measurements and use the discrete form, but the continuous form gives insight into steady-state behavior.

Smoothing. A filter estimates x_k from measurements up to time k. A *smoother* also uses later measurements, which can only improve the estimate. The Rauch–Tung–Striebel smoother runs a Kalman filter forward, storing $\hat{x}_k^\pm$ and $P_k^\pm$, and then sweeps backward:

$$C_k = P_k^+ \Phi_{k+1}^\mathsf{T} (P_{k+1}^-)^{-1}, \qquad \hat{x}_{k|N} = \hat{x}_k^+ + C_k(\hat{x}_{k+1|N} - \hat{x}_{k+1}^-), \qquad P_{k|N} = P_k^+ + C_k(P_{k+1|N} - P_{k+1}^-)C_k^\mathsf{T}. \tag{12.12}$$

Real-time VIO is a filtering problem, but the MSCKF smooths in a limited sense: the past poses kept as clones in its sliding window are corrected by every later update that involves them.

12.9 A simulated example

Figure 12.1 shows the filter on the constant-velocity model (Example 11.2), with position measurements at 10 Hz. Three behaviors generalize to VIO. The covariance converges from

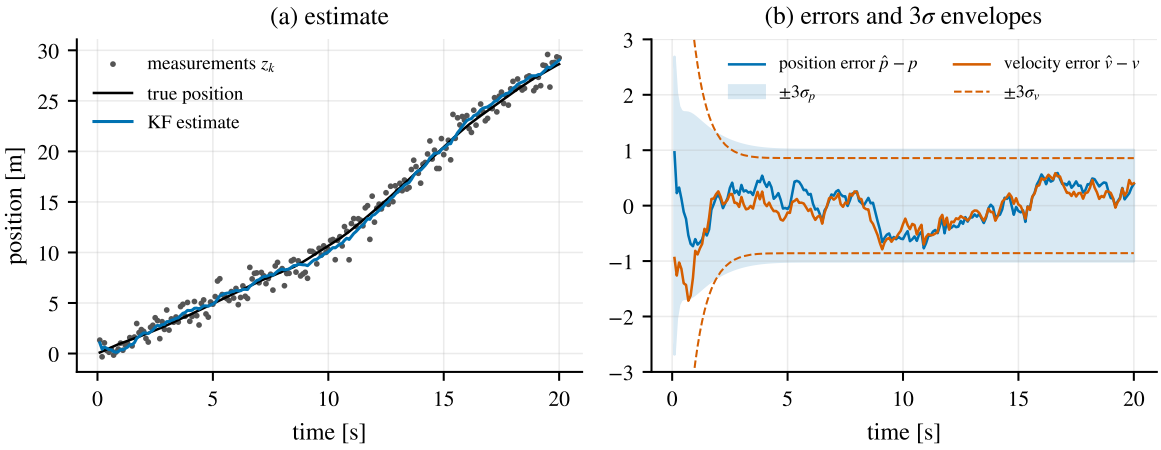


Figure 12.1: A constant-velocity Kalman filter tracking a 1D target from noisy position measurements. (a) The estimate smooths the measurements. (b) The errors stay inside the $\pm 3\sigma$ envelopes, which shrink quickly from the uninformative prior and settle at the steady-state values set by the balance between process noise and measurement noise. The velocity is never measured, yet it is estimated through its correlation with position.

its initial value to a steady state determined only by Q, R and the geometry, not by the data. The unmeasured state (velocity) is estimated through the correlation that the dynamics build up between position and velocity. And the errors stay inside the 3σ envelope about 99.7% of the time, the visual signature of a consistent filter.

Key takeaways

- Predict: $\hat{x} \leftarrow \Phi\hat{x}$, $P \leftarrow \Phi P \Phi^{\mathsf{T}} + Q$. Update: condition the joint Gaussian of state and measurement.
- $K = PH^{\mathsf{T}}S^{-1}$ maps innovations into state corrections, including corrections to unmeasured but correlated states.
- The Kalman filter is the exact Bayesian filter for linear-Gaussian models and the best linear filter otherwise.
- Innovations of a correct filter are white with covariance S; their NIS is χ^2. This is testable without ground truth.
- Time-invariant observable systems reach a steady state set by the balance of process and measurement noise.
- Use the Joseph form, solve rather than invert, and symmetrize. Independent measurement blocks may be processed sequentially.

Exercises

12.1 Derive (12.7) for a scalar state measured twice by independent sensors of variances R_1 and R_2, first sequentially, then as a stacked 2D measurement. Show that the results agree.

12.2 Show that $P^{+} \preceq P^{-}$ for the optimal gain. Is this still true for an arbitrary gain in the Joseph form?

12.3 Construct a 2×2 example in which $(I - KH)P^{-}$ computed in single precision is not positive semi-definite while the Joseph form is.

12.4 Continue the worked example for five more steps and verify that P_k^{-} converges to 2.

12.5 For the scalar random walk with process variance q and measurement variance r, derive $P_\infty^{-} = \tfrac{1}{2}(q + \sqrt{q^2 + 4qr})$ and the steady-state gain. What happens as $q/r \to 0$ and $q/r \to \infty$?

12.6 Implement Algorithm 2 in Python, reproduce Figure 12.1, and compute the fraction of time steps where the position error lies outside $\pm 3\sigma$.

12.7 Test the whiteness of the innovations of your filter by computing their sample autocorrelation. Then mis-tune Q and repeat.

Chapter 13

Nonlinear Filtering

In this chapter

- Derive the extended Kalman filter by linearization, and identify every approximation it makes.
- Understand the iterated EKF as Gauss–Newton.
- Understand the unscented transform and the unscented Kalman filter.
- Understand particle filters and why they do not scale to VIO.
- Know why VIO filters are EKFs, and what that implies for how they must be designed.

Real systems are nonlinear: rotations enter through trigonometric functions, projections divide by depth. A Gaussian pushed through a nonlinear function is no longer Gaussian (Section 5.10), so the exact Bayes filter of Chapter 12 no longer has a finite-dimensional form. This chapter presents the three main families of approximations. The extended Kalman filter, the basis of this project, is treated in detail; the others are presented so that the choice is understood.

13.1 The extended Kalman filter

13.1.1 The model

A nonlinear discrete-time model reads

$$x_k = f(x_{k-1}, u_k, w_k), \qquad z_k = h(x_k) + n_k, \tag{13.1}$$

with known input u_k (an IMU reading, for instance), process noise $w_k \sim \mathcal{N}(0, Q_k)$ and measurement noise $n_k \sim \mathcal{N}(0, R_k)$.

13.1.2 Derivation

The *extended Kalman filter* (EKF) applies the Kalman filter to a linearization of the model about the current estimate.

Prediction. Expand f to first order around the previous estimate and zero noise:

$$x_k \approx f(\hat{x}^+_{k-1}, u_k, 0) + F_k\,(x_{k-1} - \hat{x}^+_{k-1}) + L_k w_k, \qquad F_k = \left.\frac{\partial f}{\partial x}\right|_{\hat{x}^+_{k-1}}, \quad L_k = \left.\frac{\partial f}{\partial w}\right|_{\hat{x}^+_{k-1}}. \tag{13.2}$$

Taking the mean and covariance of the right-hand side (Proposition 5.7):

$$\hat{x}_k^- = f(\hat{x}_{k-1}^+, u_k, 0), \qquad P_k^- = F_k P_{k-1}^+ F_k^\mathsf{T} + L_k Q_k L_k^\mathsf{T}. \tag{13.3}$$

The mean is propagated through the *nonlinear* function, the covariance through its Jacobian.

Update. Expand h around the prediction, $h(x_k) \approx h(\hat{x}_k^-) + H_k(x_k - \hat{x}_k^-)$ with $H_k = \partial h/\partial x|_{\hat{x}_k^-}$, and apply the linear update with the innovation computed from the nonlinear prediction:

$$\begin{aligned} r_k &= z_k - h(\hat{x}_k^-), & S_k &= H_k P_k^- H_k^\mathsf{T} + R_k, & K_k &= P_k^- H_k^\mathsf{T} S_k^{-1}, \\ \hat{x}_k^+ &= \hat{x}_k^- + K_k r_k, & P_k^+ &= (I - K_k H_k) P_k^- (I - K_k H_k)^\mathsf{T} + K_k R_k K_k^\mathsf{T}. \end{aligned} \tag{13.4}$$

13.1.3 What the EKF gets wrong

Every step above is exact for linear models. For nonlinear models the EKF makes three approximations:

1. *Gaussianity*: the true prior and posterior are not Gaussian, but are represented by a mean and a covariance.
2. *Mean bias*: the predicted mean is $f(\hat{x})$, not $\mathbb{E}[f(x)]$. By Section 5.10 the difference is about $\frac{1}{2}\sum_i e_i \operatorname{tr}(\nabla^2 f_i\, P)$, proportional to the curvature times the covariance.
3. *Covariance error*: FPF^T ignores higher-order terms, and the Jacobians are evaluated at the *estimate*, not at the unknown true state.

All three errors grow with the size of the uncertainty relative to the scale over which the functions curve. When errors are small, the EKF is excellent; when they are large, it can be badly wrong, and in particular it can become *inconsistent*: report a covariance smaller than its actual errors. The third approximation has a subtle long-term consequence in VIO, a spurious gain of information about unobservable directions (Chapter 29).

13.1.4 The iterated EKF

By Proposition 10.2, the EKF update is one Gauss–Newton step on the MAP cost $\frac{1}{2}\|x - \hat{x}^-\|^2_{(P^-)^{-1}} + \frac{1}{2}\|z - h(x)\|^2_{R^{-1}}$, linearized at the prior mean. When the measurement function is strongly nonlinear, that single step can be inaccurate. The *iterated EKF* (IEKF) repeats it, re-linearizing at the latest iterate $x^{(i)}$ while keeping the prior term:

$$x^{(i+1)} = \hat{x}^- + K^{(i)}\big(z - h(x^{(i)}) - H^{(i)}(\hat{x}^- - x^{(i)})\big), \qquad K^{(i)} = P^- H^{(i)\mathsf{T}}\big(H^{(i)} P^- H^{(i)\mathsf{T}} + R\big)^{-1}, \tag{13.5}$$

with $H^{(i)}$ evaluated at $x^{(i)}$ and $x^{(0)} = \hat{x}^-$. The first iteration is the EKF. The covariance is computed with the last Jacobian. Iteration helps when the measurement is very informative relative to the prior, so that the posterior lies far from the prior mean, where the first linearization is poor.

13.2 Sigma-point filters

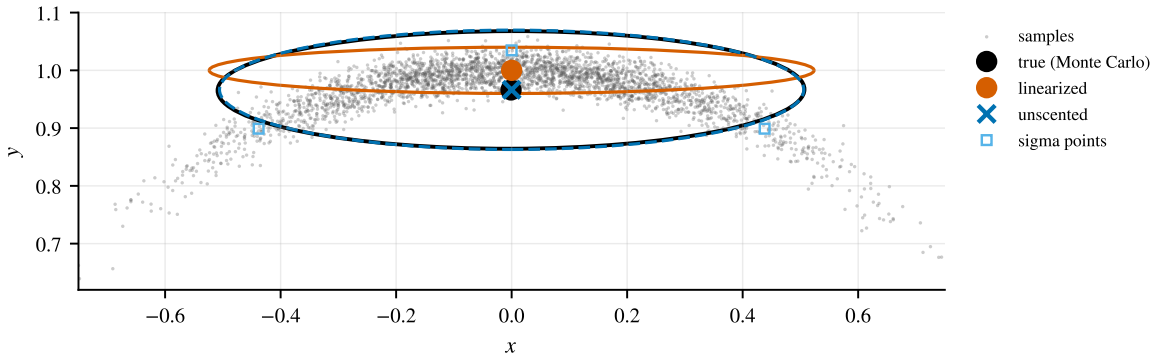


Figure 13.1: A range-bearing measurement converted to Cartesian coordinates, with $\sigma_r = 0.02$ and $\sigma_\phi = 15°$. Monte Carlo samples of the true distribution (dots), its true mean and covariance (black), the linearized approximation (orange), and the unscented transform (blue, dashed), with the five sigma points; ellipses are 2σ. Linearization places the mean on the circle, where it should not be; the unscented transform recovers the inward shift of the mean and a better covariance.

13.2.1 The unscented transform

Linearization approximates the *function*. The *unscented transform* approximates the *distribution* instead, by a small set of deterministically chosen *sigma points* that have the right mean and covariance, and pushes each point through the exact nonlinear function.

For $x \in \mathbb{R}^n$ with mean μ and covariance $P = LL^\mathsf{T}$ (Cholesky), choose $\lambda = \alpha^2(n + \kappa) - n$ and the $2n + 1$ points

$$\chi_0 = \mu, \qquad \chi_i = \mu + \sqrt{n + \lambda}\,\ell_i, \qquad \chi_{n+i} = \mu - \sqrt{n + \lambda}\,\ell_i, \qquad i = 1, \dots, n, \tag{13.6}$$

where ℓ_i is the i-th column of L, with weights

$$W_0^{(m)} = \frac{\lambda}{n + \lambda}, \quad W_0^{(c)} = \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta), \quad W_i^{(m)} = W_i^{(c)} = \frac{1}{2(n + \lambda)}. \tag{13.7}$$

Propagate $\gamma_i = f(\chi_i)$ and recombine:

$$\bar{y} = \sum_i W_i^{(m)} \gamma_i, \qquad P_y = \sum_i W_i^{(c)} (\gamma_i - \bar{y})(\gamma_i - \bar{y})^\mathsf{T}. \tag{13.8}$$

The points are placed symmetrically along the principal axes of the uncertainty, so the weighted points reproduce μ and P exactly. Expanding f in a Taylor series about μ shows that $\bar{y}$ captures the second-order (curvature) term of the mean that linearization misses, and that P_y is correct to second order, without computing any derivatives. The parameters α, β, κ control the spread of the points ($\beta = 2$ is optimal for Gaussians).

13.2.2 The unscented Kalman filter

The *unscented Kalman filter* (UKF) uses the unscented transform for both steps. Prediction: generate sigma points from $(\hat{x}^+, P^+)$, propagate them through f, and recombine into $(\hat{x}^-, P^-)$,

adding Q. Update: generate sigma points from $(\hat{x}^-, P^-)$, push them through h to get the predicted measurement $\bar{z}$ and its covariance S (adding R), and the cross-covariance $P_{xz} = \sum_i W_i^{(c)}(\chi_i - \hat{x}^-)(\gamma_i - \bar{z})^\mathsf{T}$; then $K = P_{xz}S^{-1}$, $\hat{x}^+ = \hat{x}^- + K(z - \bar{z})$ and $P^+ = P^- - KSK^\mathsf{T}$. These are the LMMSE formulas of Theorem 9.9 with moments estimated by the sigma points rather than by linearization. On manifolds, sigma points are generated in the tangent space and mapped with $\boxplus$, and their mean is computed iteratively as in Chapter 8.

The UKF costs about $2n + 1$ evaluations of f and h per step instead of Jacobians, which is attractive when Jacobians are hard to derive. For VIO with a 75-dimensional state, the 151 propagations of the full state per IMU sample are expensive, and the accuracy gain over a well-designed error-state EKF is small, because the error-state formulation already keeps the nonlinearity mild.

13.3 Particle filters

A *particle filter* represents the posterior by N weighted samples (particles) $\{x^{(j)}, w^{(j)}\}$, which can represent any distribution, multi-modal ones included. The basic *sequential importance resampling* (SIR) algorithm:

1. *Predict*: move each particle through the dynamics with a random noise sample, $x_k^{(j)} = f(x_{k-1}^{(j)}, u_k, w^{(j)})$.
2. *Weight*: multiply each weight by the likelihood of the measurement, $w^{(j)} \leftarrow w^{(j)} p(z_k \mid x_k^{(j)})$, and normalize.
3. *Resample*: when the effective sample size $N_{\text{eff}} = 1/\sum_j (w^{(j)})^2$ falls too low, draw N new particles with probabilities $w^{(j)}$ and reset the weights to $1/N$, so that particles concentrate where the posterior mass is.

Particle filters are exact as $N \to \infty$ and excel at low-dimensional problems with multi-modal posteriors, such as localizing a robot in a known map. Their weakness is dimension: the number of particles needed to cover a distribution grows roughly exponentially with the state dimension. For the 15 to 75 dimensions of VIO they are impractical, except in hybrid forms that sample only a few variables.

13.4 Why VIO uses the EKF

	EKF	UKF	particle filter
cost per step	Jacobians, $O(n^3)$	$2n + 1$ propagations, $O(n^3)$	N propagations, N huge for large n
derivatives needed	yes	no	no
non-Gaussian posteriors	no	no	yes
accuracy for small errors	excellent	excellent	good (noisy)
used in VIO	MSCKF, OpenVINS, ROVIO (iterated)	rarely	no

Visual-inertial odometry has a high-dimensional state, a fast prediction rate (hundreds of Hz), and uncertainties that, once the filter is initialized, are small compared with the curvature of its models. Those are exactly the conditions under which the EKF is accurate and the alternatives are expensive. The design principle that makes this work is to keep the linearized errors small: estimate the *error* of a nominal state that absorbs the large nonlinear motion, and reset it after every correction. That is the error-state Kalman filter of the next chapter.

Key takeaways

- The EKF propagates the mean through the nonlinear model and the covariance through its Jacobians, evaluated at the estimate.
- Its errors (non-Gaussianity, mean bias, covariance error, estimate-dependent Jacobians) grow with uncertainty relative to curvature.
- The iterated EKF is Gauss–Newton on the MAP cost; it helps when measurements are very informative.
- The unscented transform propagates sigma points through the exact function and captures second-order effects without derivatives.
- Particle filters handle any distribution but scale exponentially with dimension.
- VIO uses EKFs with small, frequently reset errors: the error-state formulation.

Exercises

13.1 Write the EKF for a planar robot with state (x, y, ψ) driven by speed and turn rate and measured by range to a known beacon. Identify where linearization error enters.

13.2 Reproduce Figure 13.1. Increase σ_ϕ until the linearized mean lies outside the 1σ ellipse of the true distribution.

13.3 Show that the unscented transform reproduces the mean and covariance exactly for a linear function.

13.4 Show that for a scalar $x \sim \mathcal{N}(\mu, \sigma^2)$ and $f(x) = x^2$, the unscented transform with $n = 1$, $\kappa = 2$, $\alpha = 1$ gives the exact mean $\mu^2 + \sigma^2$.

13.5 Implement a particle filter for a 1D robot localizing itself from range measurements to two landmarks, and observe a bimodal posterior.

Chapter 14

The Error-State Kalman Filter

In this chapter

- Understand the three-way split into true, nominal and error state, and why it is the right way to filter on a manifold.
- Derive the linearized error dynamics with a mechanical recipe.
- Define measurement residuals and Jacobians on the manifold with the correct sign convention.
- Derive the injection and reset steps, including the reset Jacobian G.
- Assemble the complete error-state Kalman filter loop, and follow it through a complete attitude-estimation example.

14.1 Filtering on a manifold

The EKF of Chapter 13 assumes the state lives in $\mathbb{R}^n$, so that $\hat{x} + Kr$ makes sense. For VIO, the state contains a rotation, which does not live in a vector space (Chapter 8). Three quantities must therefore be distinguished.

Definition 14.1 (True, nominal and error state).

- The **true state** x is the unknown physical state, a point on the state manifold $\mathcal{M}$.
- The **nominal state** $\hat{x} \in \mathcal{M}$ is the current best estimate, propagated with the full nonlinear model and no noise.
- The **error state** $\delta x = x \boxminus \hat{x} \in \mathbb{R}^n$ is the small, minimal-dimension difference, defined with the operators of Section 8.9.

The filter's Gaussian belief is over the error state: $\delta x \sim \mathcal{N}(\delta\hat{x}, P)$, so that $x = \hat{x} \boxplus \delta x$.

For the IMU state this means 16 nominal numbers (quaternion included) but a 15-dimensional error and a 15×15 covariance, with no redundant directions and no constraints. The Kalman filter operates only on the error state, which is a vector, so all of Chapter 12 applies to it. This design is called the *error-state Kalman filter* (ESKF), or *indirect* Kalman filter. In aerospace the same idea for attitude is the *multiplicative* EKF.

Key idea: why the error state

- **Minimal and unconstrained.** The error has exactly as many components as degrees of freedom, so its covariance is non-singular and needs no constraint handling.
- **Always near the origin.** After every update the error is reset to zero, so it never approaches the singularities of three-parameter rotations, and the small-angle

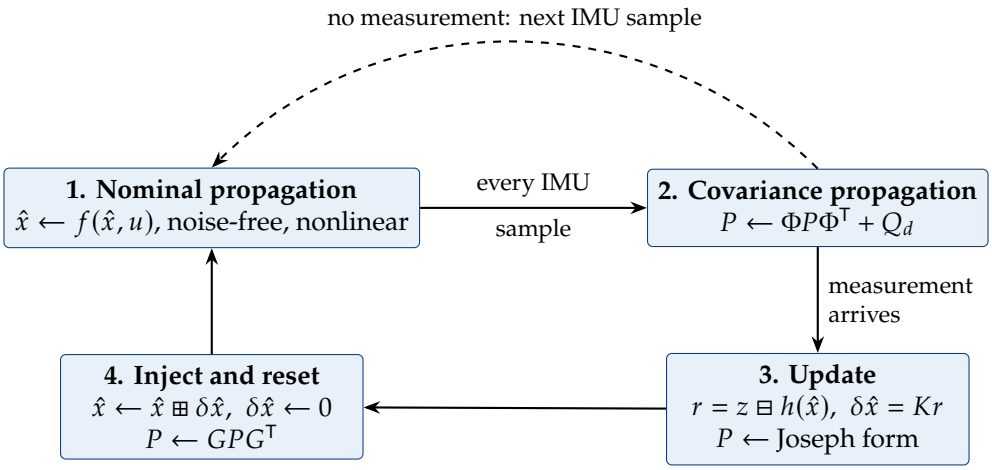


Figure 14.1: The error-state Kalman filter cycle. The error-state mean is zero throughout propagation, so only the covariance is propagated in step 2. It becomes non-zero in step 3 and is transferred to the nominal state and zeroed in step 4.

approximations used to derive its dynamics remain valid.

- **Nearly linear.** The large, fast, nonlinear motion is carried by the nominal state, which is integrated exactly. The error evolves slowly and is small, so its dynamics are almost linear and linearization is accurate (Chapter 13).
- **Slow.** Error dynamics are governed by sensor errors, not by vehicle motion, so the covariance could even be propagated at a lower rate than the nominal state if computation were scarce.

14.2 The ESKF cycle

Figure 14.1 shows the four steps.

1. **Nominal propagation.** At each IMU sample, integrate the nonlinear kinematics with the measured inputs, ignoring noise (Chapter 16).
2. **Error covariance propagation.** Propagate P with the linearized error dynamics (Chapter 17).
3. **Update.** When a measurement arrives, compute the residual and its Jacobian on the manifold, and compute the Kalman estimate of the error, $\delta\hat{x} = Kr$, and the posterior covariance.
4. **Injection and reset.** Move the estimated error into the nominal state with $\boxplus$, set the error mean back to zero, and correct the covariance for the change of reference point.

Proposition 14.2 (The error mean needs no propagation). *If $\delta\hat{x} = 0$ after a reset, and the error obeys linear dynamics $\dot{\delta x} = F_c \delta x + G_c w$ with zero-mean noise, then $\mathbb{E}[\delta x] = 0$ until the next update.*

Proof. The mean obeys $\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}[\delta x] = F_c\, \mathbb{E}[\delta x]$ (Proposition 6.5) with initial value zero, whose unique solution is zero. ∎

This is why step 2 propagates only the covariance: all the knowledge of the error state's mean has been transferred into the nominal state.

14.3 Linearized error dynamics: the recipe

Suppose the true state obeys $\dot{x} = f(x, u, w)$ with noise w, and the nominal state obeys $\dot{\hat{x}} = f(\hat{x}, u, 0)$. The error dynamics are derived in four mechanical steps.

1. Write the true dynamics block by block.
2. Substitute $x = \hat{x} \boxplus \delta x$ in every block. For the attitude, $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$ (or $\mathrm{Exp}(\delta\theta)\hat{R}$ for a global perturbation).
3. Subtract the nominal dynamics.
4. Keep only terms of first order in the small quantities (δx and w), using $\mathrm{Exp}(\delta\theta) \approx I + [\delta\theta]_\times$ and dropping their products.

The result is a linear time-varying system

$$\dot{\delta x} = F_c(\hat{x}, u)\,\delta x + G_c(\hat{x})\,w, \tag{14.1}$$

which Section 11.2 turns into Φ and Q_d. The attitude block needs the time derivative of the perturbation itself, worked out in Lemma 17.1. Section 14.7 applies the recipe to a complete small filter, and Chapter 17 to the inertial navigation equations.

14.4 Measurement updates on the manifold

Let a sensor produce a measurement z on a manifold $\mathcal{Z}$ (a vector space, SO(3), a pose, ...) according to the generative model

$$z = h(x) \boxplus n, \qquad n \sim \mathcal{N}(0, R_z). \tag{14.2}$$

The ESKF compares the measurement with its prediction from the nominal state,

$$r = z \boxminus h(\hat{x}), \tag{14.3}$$

and models how the residual depends on the unknown true error δx. Substituting $z = h(\hat{x} \boxplus \delta x) \boxplus n$ and linearizing,

$$r \approx H\,\delta x + n, \qquad H = \frac{\partial}{\partial \delta x}\Big[h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})\Big]_{\delta x = 0}. \tag{14.4}$$

H is the manifold Jacobian of h (Definition 8.7). With r and H, the update is the ordinary Kalman update of the error state with prior $\mathcal{N}(0, P)$:

$$S = HPH^\mathsf{T} + R_z, \qquad K = PH^\mathsf{T}S^{-1}, \qquad \delta\hat{x} = Kr, \qquad P^+ = (I - KH)P(I - KH)^\mathsf{T} + KR_zK^\mathsf{T}. \tag{14.5}$$

Pitfall: the sign of H

There are two ways to set up the linearization, and they differ by a sign. The generative view above differentiates the *prediction* with respect to the *true* error: $r \approx +H\delta x$. An optimization view instead writes the residual as a function of a correction δ to the estimate, $e(\delta) = z \boxminus h(\hat{x} \boxplus \delta)$, whose Jacobian is $-H$ (to first order). Both lead to the same correction if used consistently. Mixing them flips every correction. In this project, H is always (14.4), and its finite-difference test perturbs the nominal state: $r(\delta x) := h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})$ and $H \approx \partial r / \partial \delta x$.

For a vector-valued measurement, $\boxminus$ is subtraction and this reduces to the EKF. For a rotation-valued measurement such as the pose measurement of Milestone 6, the attitude part of the residual is $\mathrm{Log}_q(\hat{q}^{-1} \otimes q_{meas})$ (AGENTS.md F3).

14.5 Injection and reset

After the update, the error estimate $\delta\hat{x} = Kr$ is non-zero. Injection moves it into the nominal state:

$$\hat{x}^+ = \hat{x} \boxplus \delta\hat{x} : \quad \hat{p} \leftarrow \hat{p} + \delta\hat{p}, \quad \hat{v} \leftarrow \hat{v} + \delta\hat{v}, \quad \hat{q} \leftarrow \hat{q} \otimes \mathrm{Exp}_q(\delta\hat{\theta}), \quad \hat{b} \leftarrow \hat{b} + \delta\hat{b}, \tag{14.6}$$

followed by normalization of every quaternion that was touched, for the IMU and every clone (AGENTS.md F4).

The error is now defined relative to the new nominal state, $\delta x^+ = x \boxminus \hat{x}^+$, and its mean is zero. But its covariance is not exactly P^+, because the reference point of the tangent space has moved.

Proposition 14.3 (Reset Jacobian). *Let $\delta x \sim \mathcal{N}(\delta\hat{x}, P^+)$ be the error relative to $\hat{x}$ after the update, and $\delta x^+ = x \boxminus (\hat{x} \boxplus \delta\hat{x})$ the error relative to the injected nominal state. To first order, $\delta x^+ \sim \mathcal{N}(0, GP^+G^\mathsf{T})$ with $G = \partial\delta x^+/\partial\delta x$ evaluated at $\delta\hat{x}$. For the vector blocks $G = I$, and for each attitude block*

$$G_\theta = I - \tfrac{1}{2}[\delta\hat{\theta}]_\times. \tag{14.7}$$

Proof. For vector blocks $\delta x^+ = \delta x - \delta\hat{x}$, whose Jacobian is I. For the attitude, $q = \hat{q} \otimes \mathrm{Exp}_q(\delta\theta)$ and $q = \hat{q}^+ \otimes \mathrm{Exp}_q(\delta\theta^+)$ with $\hat{q}^+ = \hat{q} \otimes \mathrm{Exp}_q(\delta\hat{\theta})$, so

$$\mathrm{Exp}(\delta\theta^+) = \mathrm{Exp}(-\delta\hat{\theta})\ \mathrm{Exp}(\delta\theta).$$

By the BCH formula (8.13), $\delta\theta^+ = \delta\theta - \delta\hat{\theta} - \frac{1}{2}\,\delta\hat{\theta} \times \delta\theta + \cdots$, whose derivative with respect to $\delta\theta$ is $I - \frac{1}{2}[\delta\hat{\theta}]_\times$. At $\delta\theta = \delta\hat{\theta}$ the value is exactly zero, so the new error has zero mean. Linearizing δx^+ about $\delta\hat{x}$ and applying Proposition 5.7 gives the covariance. ∎

***Remark* 14.4** (relation to the right Jacobian). The derivative of $\mathrm{Log}(\mathrm{Exp}(-\delta\hat{\theta})\,\mathrm{Exp}(\delta\theta))$ at $\delta\theta = \delta\hat{\theta}$ can be computed exactly with (8.19): it is $J_r(\delta\hat{\theta}) = I - \frac{1}{2}[\delta\hat{\theta}]_\times + O(\|\delta\hat{\theta}\|^2)$. The frozen specification (AGENTS.md F4) uses the first-order form (14.7). For a typical correction of a few milliradians the two differ by about 10^{-6} and the difference is irrelevant. Changing it would be a logged convention change, not a silent substitution.

Algorithm 3 Error-state Kalman filter (one IMU step and an optional update)

Input: nominal $\hat{x}$, covariance P, IMU sample at t_k, optional measurement z at t_k

$\Delta t \leftarrow t_k - t_{k-1}$ ▷ from integer nanosecond timestamps
$\hat{x} \leftarrow f(\hat{x}, u_k, \Delta t)$; normalize $\hat{q}$ ▷ nominal propagation
$\Phi \leftarrow I + F_c \Delta t$; $\quad Q_d \leftarrow G_c Q_c G_c^\mathsf{T} \Delta t$
$P \leftarrow \Phi P \Phi^\mathsf{T} + Q_d$; $\quad P \leftarrow \frac{1}{2}(P + P^\mathsf{T})$
if measurement z available **then**
  $r \leftarrow z \boxminus h(\hat{x})$; $\quad H \leftarrow \partial[h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})]/\partial \delta x$
  $S \leftarrow HPH^\mathsf{T} + R_z$; factor S; gate on $r^\mathsf{T} S^{-1} r$
  $K \leftarrow PH^\mathsf{T} S^{-1}$ (by solving); $\quad \delta\hat{x} \leftarrow Kr$
  $P \leftarrow (I - KH)P(I - KH)^\mathsf{T} + KR_z K^\mathsf{T}$
  $\hat{x} \leftarrow \hat{x} \boxplus \delta\hat{x}$; normalize quaternions ▷ inject
  $P \leftarrow GPG^\mathsf{T}$; $\quad P \leftarrow \frac{1}{2}(P + P^\mathsf{T})$ ▷ reset

The covariance reset is therefore

$$P \leftarrow GPG^\mathsf{T}, \qquad G = \operatorname{blkdiag}\left(I_6,\ I - \tfrac{1}{2}[\delta\hat{\theta}]_\times,\ I_6,\ \ldots\right), \tag{14.8}$$

with one $I - \frac{1}{2}[\delta\hat{\theta}_i]_\times$ block for each clone's attitude and identity elsewhere.

Frozen convention: injection and reset, AGENTS.md F4

Inject with $\boxplus$ into the IMU block and every clone; normalize every touched quaternion; reset $\delta\hat{x} \leftarrow 0$ and $P \leftarrow GPG^\mathsf{T}$ with G implemented exactly as above. Using $G \approx I$ is a deviation that must be logged.

How much does G matter? It rotates the attitude covariance by half the injected correction, a first-order effect in $\|\delta\hat{\theta}\|$. With small corrections at high rate the effect is tiny, and many implementations use $G = I$. Large corrections (after a long outage, or a loop closure) make it noticeable. It costs almost nothing, and implementing it exactly removes one approximation from the consistency analysis of Milestone 7.

14.6 The complete error-state filter

Algorithm 3 assembles the pieces. It is generic: Chapters 16 and 17 supply f, F_c and G_c for the IMU, Chapter 18 supplies h and H for a pose sensor, and Chapter 26 supplies them for a camera. When measurements arrive between IMU samples, the filter first propagates to the measurement time by splitting the IMU interval (Chapter 16).

14.7 A complete example: attitude estimation

To see every step of Algorithm 3 at work on a problem small enough to follow completely, consider an *attitude and heading reference system* (AHRS) without a magnetometer: estimate the attitude $R = R_{wb}$ of a slowly moving device and the bias b of its gyroscope, using the gyroscope for propagation and the accelerometer as an inclinometer (it measures the reaction to gravity when linear accelerations are small).

To avoid solving the project's own derivation exercise, this example uses the **global** perturbation $R = \mathrm{Exp}(\delta\theta)\hat{R}$, with the error expressed in the world frame. The project's state uses the local perturbation; its forms are the subject of the derivation exercises of Chapter 17 (solutions in Appendix E).

State and error state. Nominal $(\hat{R}, \hat{b})$; error $\delta x = (\delta\theta, \delta b) \in \mathbb{R}^6$ with $R = \mathrm{Exp}(\delta\theta)\hat{R}$ and $b = \hat{b} + \delta b$.

Models. Gyroscope $\omega_m = \omega + b + n_g$; bias random walk $\dot{b} = n_b$; accelerometer at rest $a_m = R^\mathsf{T} v + n_a$ with $v = -g_w = (0, 0, 9.81)$.

Nominal propagation. $\hat{R} \leftarrow \hat{R}\,\mathrm{Exp}\big((\omega_m - \hat{b})\Delta t\big)$, $\hat{b}$ unchanged.

Error dynamics. By Example 17.2 (in Chapter 17), $\dot{\delta\theta} = -\hat{R}\,\delta b - \hat{R}\,n_g$, and $\dot{\delta b} = n_b$. Hence

$$F_c = \begin{bmatrix} 0 & -\hat{R} \\ 0 & 0 \end{bmatrix}, \qquad G_c = \begin{bmatrix} -\hat{R} & 0 \\ 0 & I \end{bmatrix}, \qquad Q_c = \mathrm{diag}(\sigma_g^2 I, \sigma_b^2 I). \tag{14.9}$$

Measurement Jacobian. $h(x) = R^\mathsf{T} v$. Perturbing, $h(\mathrm{Exp}(\delta\theta)\hat{R}) = \hat{R}^\mathsf{T}\,\mathrm{Exp}(-\delta\theta)v \approx \hat{R}^\mathsf{T}(v - \delta\theta \times v) = \hat{R}^\mathsf{T} v + \hat{R}^\mathsf{T}[v]_\times\,\delta\theta$, so

$$H = \begin{bmatrix} \hat{R}^\mathsf{T}[v]_\times & 0 \end{bmatrix}, \qquad r = a_m - \hat{R}^\mathsf{T} v. \tag{14.10}$$

$[v]_\times$ has rank two: the measurement observes the two components of $\delta\theta$ perpendicular to gravity (roll and pitch) but not the component along it (yaw). It does not observe the bias directly at all.

Injection and reset. $\hat{R} \leftarrow \mathrm{Exp}(\delta\hat{\theta})\hat{R}$, $\hat{b} \leftarrow \hat{b} + \delta\hat{b}$. Repeating the reset derivation for the global perturbation, $\mathrm{Exp}(\delta\theta^+) = \mathrm{Exp}(\delta\theta)\,\mathrm{Exp}(-\delta\hat{\theta})$, gives $G_\theta = I + \frac{1}{2}[\delta\hat{\theta}]_\times$: the same magnitude as (14.7), opposite sign.

Figure 14.2 shows the filter on simulated data. Several lessons of this chapter are visible in it. The errors stay inside their 3σ bounds: the filter is consistent. Tilt becomes accurate within seconds, through the accelerometer. Yaw is unobservable without a magnetometer or vision, so its uncertainty only grows. And the gyro bias, although never measured, is estimated through its correlation with the tilt error it causes: the cross-covariance built by propagation lets the gain correct it. This is exactly how the full VIO filter estimates IMU biases from camera measurements.

Project link: Milestones 4–6

M4 implements step 2 of Algorithm 3, M5 steps 3–4, and M6 the update, injection and reset with a pose measurement. Testing them separately localizes bugs: if propagation is exact on synthetic data, and the covariance matches Monte Carlo statistics, then any later inconsistency is in the update.

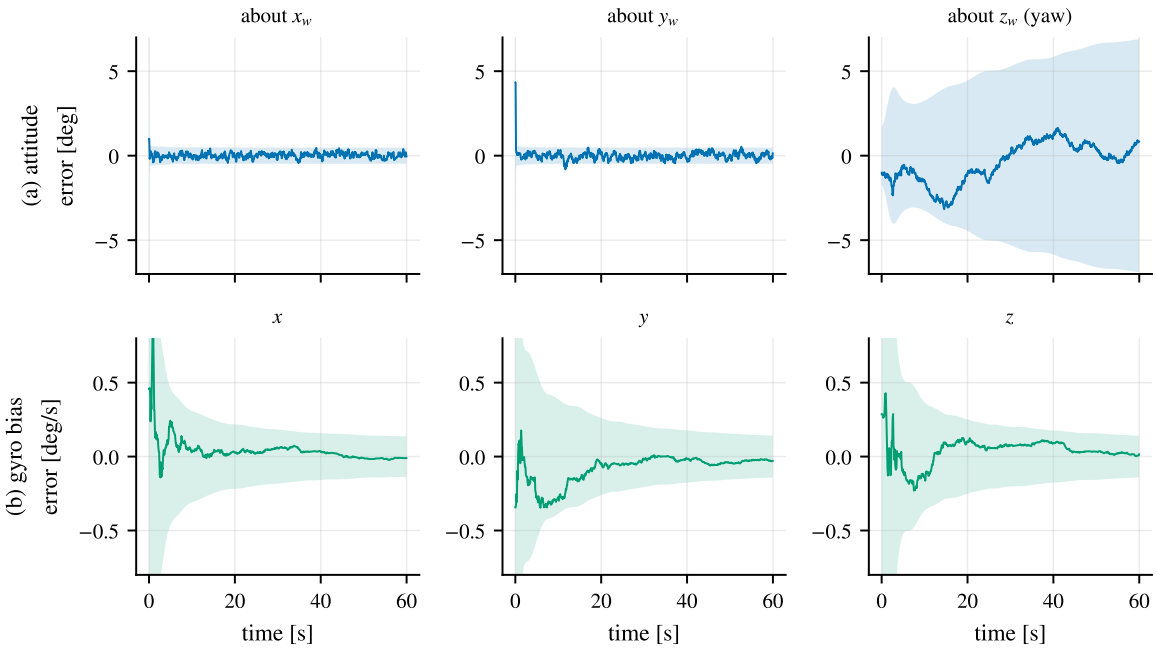


Figure 14.2: The attitude ESKF of Section 14.7 on a simulated tumbling device: gyroscope at 100 Hz with a constant bias, accelerometer used as an inclinometer at 10 Hz. (a) Attitude errors about the world axes with their $\pm 3\sigma$ bounds. Tilt (about x and y) is quickly bounded by the accelerometer; the error about the vertical (yaw) is unobservable, and its bound grows. (b) Gyro bias errors with $\pm 3\sigma$ bounds. The bias is never measured directly, yet all three components converge, because the device's rotation makes each body axis horizontal at some time.

Key takeaways

- The ESKF splits the state into a nonlinear nominal part and a small, minimal, nearly linear error part. The Kalman filter only ever sees the error.
- The error mean is zero during propagation; only P is propagated.
- Residual $r = z \boxminus h(\hat{x}) \approx H\delta x + n$ with H the manifold Jacobian of h at $\hat{x}$.
- After injection, reset $P \leftarrow GPG^\mathsf{T}$ with $G_\theta = I - \frac{1}{2}[\delta\hat{\theta}]_\times$ for every (local) attitude block.
- The attitude example shows observability at work: gravity bounds tilt, yaw drifts, and unmeasured biases are estimated through correlations.

Exercises

14.1 Derive the reset Jacobian for the global perturbation $R = \mathrm{Exp}(\delta\theta)\hat{R}$ and confirm $G_\theta = I + \frac{1}{2}[\delta\hat{\theta}]_\times$.

14.2 Suppose $\delta\hat{\theta} = (0, 0, 0.1)$ rad and $P_{\theta\theta} = \mathrm{diag}(10^{-4}, 10^{-4}, 10^{-2})$. Compute $GP_{\theta\theta}G^\mathsf{T}$ and compare with $J_r(\delta\hat{\theta})P_{\theta\theta}J_r(\delta\hat{\theta})^\mathsf{T}$. When is the difference noticeable?

14.3 An iterated ESKF repeats the update with H re-evaluated at $\hat{x} \boxplus \delta\hat{x}$. Show that one

iteration is a Gauss–Newton step on the MAP cost $\|\delta x\|^2_{P^{-1}} + \|z \boxminus h(\hat{x} \boxplus \delta x)\|^2_{R_z^{-1}}$.

14.4 Implement the attitude filter of Section 14.7 and reproduce Figure 14.2. Then hold the device still and observe which bias component stops converging. Explain why.

14.5 Rewrite the attitude filter with the local perturbation $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$: derive F_c, G_c, H and G. (This is a small version of the project's derivation exercise.)

Part III

Inertial Navigation

Chapter 15

Inertial Sensors and Their Errors

In this chapter

- Understand how MEMS accelerometers and gyroscopes work, from Newton's law and the Coriolis effect.
- Understand specific force: what an accelerometer measures, and why it reads $+g$ at rest.
- State the IMU measurement model of the project, and the fuller error model of which it is a simplification.
- Know how IMUs are calibrated and how their noise parameters are measured and tuned.
- Convert between continuous-time noise densities, discrete per-sample noise, and datasheet units.

15.1 How a MEMS accelerometer works

A MEMS (micro-electro-mechanical system) *accelerometer* is a tiny proof mass, a few micrograms of silicon, suspended on silicon springs inside a case, with capacitive electrodes that measure its displacement relative to the case.

15.1.1 The spring-mass model

Let the case have acceleration a along the sensitive axis, let g be the component of gravitational acceleration along that axis, and let x be the displacement of the mass relative to the case. The absolute acceleration of the mass is $a + \ddot{x}$, and the forces on it are the spring, the damper and gravity (Figure 15.1). Newton's law gives

$$m(a + \ddot{x}) = -kx - c\dot{x} + mg \quad \Longrightarrow \quad \ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2 x = -(a - g), \tag{15.1}$$

with natural frequency $\omega_n = \sqrt{k/m}$ and damping ratio $\zeta = c/(2\sqrt{km})$. At frequencies well below ω_n the derivative terms are negligible and

$$x \approx -\frac{a - g}{\omega_n^2}. \tag{15.2}$$

The deflection measures $a - g$, not a. Its frequency response, the ratio of x to $-(a - g)/\omega_n^2$ for a sinusoidal input at angular frequency ω, is $1/(1 - (\omega/\omega_n)^2 + 2i\zeta\omega/\omega_n)$: flat below the resonance, peaked at it for small damping, falling off above it (Figure 15.2). MEMS accelerometers resonate in the kHz range, and their electronics low-pass filter the output well below that, which also serves as the anti-aliasing filter (Chapter 6).

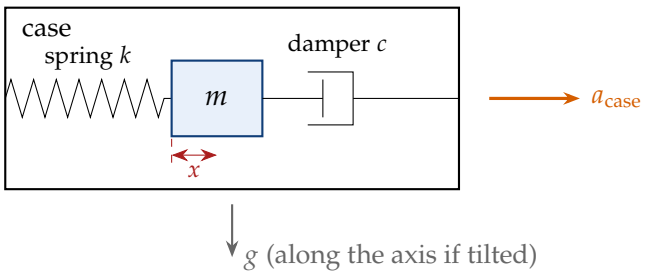


Figure 15.1: The spring-mass-damper model of an accelerometer. The displacement x of the proof mass relative to the case is proportional, at low frequencies, to the specific force along the sensitive axis.

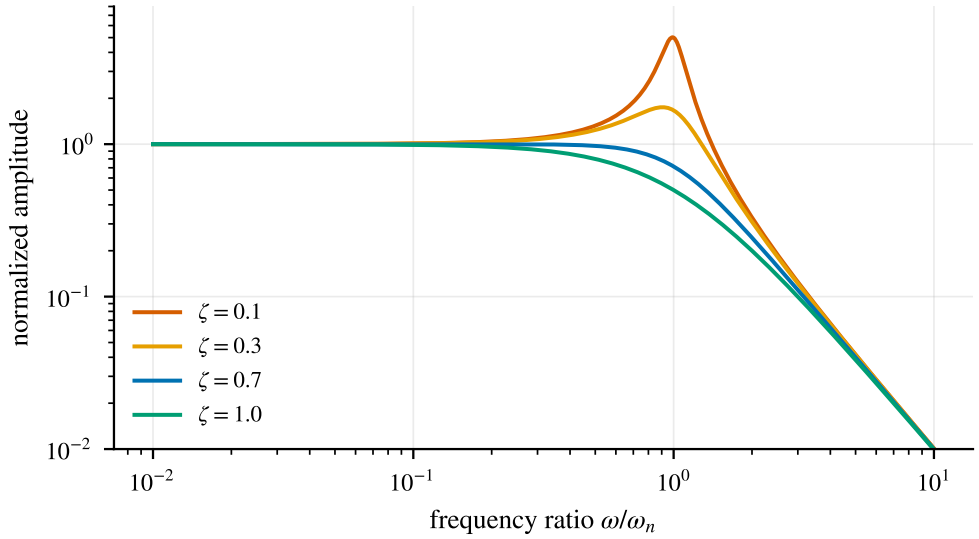


Figure 15.2: Frequency response of the spring-mass accelerometer model for several damping ratios. Below the natural frequency the response is flat: the deflection is proportional to specific force. A damping ratio near 0.7 gives the widest flat band without a resonant peak.

15.1.2 Specific force

The quantity measured is the *specific force*

$$f = a - g, \tag{15.3}$$

the non-gravitational force per unit mass acting on the sensor (the spring force divided by m). Gravity acts on the mass and the case equally and cannot be felt directly: this is Einstein's equivalence principle. Two cases build intuition.

- **At rest on a table**, $a = 0$, so $f = -g$. With $g_w = [0, 0, -9.81]^{\mathsf{T}}$, the accelerometer reads $+9.81$ m/s^2 *upward*: it senses the table pushing it up.
- **In free fall**, $a = g$, so $f = 0$. A falling IMU reads zero, whatever its velocity.

To recover the kinematic acceleration from the measurement, the estimator rotates the specific force into the world frame and *adds gravity back*: $a_w = R_{wb} f_b + g_w$. A sign error here produces a vertical acceleration error of $2g \approx 20$ m/s^2 and an obviously diverging altitude, which is fortunately easy to recognize.

Noise. Even a perfect mechanical sensor is noisy: the proof mass is jostled by the thermal motion of the surrounding gas, a *thermo-mechanical* noise whose acceleration density is about $\sqrt{4k_B T\omega_n/(mQ)}$ (with k_B Boltzmann's constant, T the temperature and Q the mechanical quality factor). Tiny masses are noisier, which is the basic trade-off of MEMS inertial sensing. Readout electronics add their own noise.

15.2 How a MEMS gyroscope works

A MEMS *gyroscope* has no spinning wheel. It keeps a proof mass vibrating and detects the Coriolis force that rotation induces. Let the mass be driven along the x-axis of the sensor at frequency ω_d, so its velocity relative to the sensor is $v = A\omega_d \cos(\omega_d t)\, e_x$. If the sensor rotates with angular velocity Ω, then in the sensor frame the mass experiences the Coriolis acceleration $-2\Omega \times v$ (Equation (7.24)). For a rotation about the z-axis this acceleration points along y:

$$a_y = -2\,\Omega_z\, A\omega_d \cos(\omega_d t). \tag{15.4}$$

The resulting vibration along y (the *sense* mode) has an amplitude proportional to the rate Ω_z and oscillates at the drive frequency, so the electronics recover Ω_z by demodulating the sense signal synchronously with the drive. The sensitive axis is perpendicular to both the drive and the sense directions.

Real designs use two or four masses vibrating in anti-phase (a *tuning fork*): their Coriolis forces are opposite and add in the differential readout, while linear accelerations and vibrations push all masses the same way and cancel. Imperfect cancellation leaves a small sensitivity of the gyro output to acceleration (g-sensitivity). Manufacturing asymmetries couple the drive into the sense mode (*quadrature*), and their drift with temperature is a major source of gyro bias.

15.3 The measurement model

15.3.1 The project's model

Frozen convention: IMU model, AGENTS.md C6

$$\omega_m = \omega_b + b_g + n_g, \qquad a_m = R_{wb}^{\mathsf{T}}\big(a_w - g_w\big) + b_a + n_a, \qquad \dot{b}_g = n_{b_g}, \quad \dot{b}_a = n_{b_a}, \tag{15.5}$$

with ω_m, a_m the gyroscope and accelerometer readings in the body frame, ω_b the true angular rate, a_w the true acceleration of the body in the world frame, b_g, b_a the biases (body frame), and $n_g, n_a, n_{b_g}, n_{b_a}$ zero-mean white noises with densities $\sigma_g, \sigma_a, \sigma_{b_g}, \sigma_{b_a}$. The noise vector is ordered $w = [n_a, n_g, n_{b_a}, n_{b_g}] \in \mathbb{R}^{12}$.

The model has two kinds of error. *White noise* (n_g, n_a) is independent from sample to sample and averages out. *Bias* (b_g, b_a) is a slowly varying offset that does not average out and is therefore estimated as part of the state (a shaping filter, Chapter 6). Its slow drift is modeled as a random walk driven by n_{b_g}, n_{b_a}. Over the minutes of a EuRoC sequence the biases change little but measurably.

Error	Description	In the model?
White noise	sample-to-sample electronic and thermo-mechanical noise (angle/velocity random walk)	yes, n_g, n_a
Turn-on bias	constant offset that differs at each power-up	yes, initial b and P_0
In-run bias drift	slow wandering of the offset	yes, random walk
Bias instability	flicker ($1/f$) noise floor of the bias	approximated by the random walk
Scale factor	output = $(1 + s)\times$ input	no
Axis misalignment	sensing axes not exactly orthogonal / aligned with the case	no
g-sensitivity	gyro output depends on acceleration	no
Temperature effects	bias and scale factor change with temperature	no (absorbed by bias walk)
Quantization	finite resolution of the ADC	no (below white noise)
Saturation, vibration	clipping on impacts; aliasing of high-frequency vibration	no
Earth rotation	gyros sense 7.3×10^{-5} rad/s	no (Chapter 7)

Table 15.1: IMU error sources and how the project's model treats them.

15.3.2 A fuller error model

High-accuracy inertial systems use a richer model, of which (15.5) is the special case $S = M = G = 0$:

$$a_m = (I + S_a)M_a\, f_b + b_a + n_a, \qquad \omega_m = (I + S_g)M_g\, \omega_b + G_g\, f_b + b_g + n_g. \tag{15.6}$$

Here S is a diagonal matrix of *scale-factor* errors (the output is $(1 + s)$ times the input), M a near-identity matrix describing *axis misalignment and non-orthogonality* (the three sensing axes are neither exactly perpendicular nor exactly aligned with the body frame), and G_g the g-sensitivity of the gyroscope. Each of these parameters can be calibrated, estimated online as additional states, or ignored, at the cost of some unmodeled error.

Table 15.1 lists the main error sources. Unmodeled errors do not disappear: they appear as apparent extra noise or apparent bias drift. A filter that ignores them must compensate with larger noise parameters than the white-noise floor alone would suggest, which is why tuning is needed even with a well-calibrated sensor. The size of the effect is easy to estimate: a scale-factor error of 10^{-3} on a gyro turning at 1 rad/s is an apparent bias of 10^{-3} rad/s, as large as a poor bias and much larger than the EuRoC gyro's noise.

15.4 Calibration

Gyroscope bias. At rest, the gyroscope should read zero (or the Earth rate). Averaging a few seconds of data gives the bias, with a standard deviation of $\sigma_g/\sqrt{T}$ after T seconds (Chapter 6): for EuRoC, 10^{-4} rad/s after 3 s.

Accelerometer bias and scale: the six-position test. Place the IMU with each axis pointing straight up and then straight down. For an axis with bias b and scale factor $1 + s$, the two

readings are $(1+s)g + b$ and $-(1+s)g + b$, so

$$b = \frac{a_{\text{up}} + a_{\text{down}}}{2}, \qquad 1 + s = \frac{a_{\text{up}} - a_{\text{down}}}{2g}. \tag{15.7}$$

Using more orientations and a least-squares fit (Chapter 9) also yields the misalignments.

Gyroscope scale and misalignment. These require known rotations, from a rate table, or from a known sequence of orientations measured by an accelerometer.

Camera–IMU calibration. The extrinsic transform T_{bc}, the time offset between the sensors, and optionally the IMU's own scale and misalignment are estimated jointly by moving the rig in front of a calibration target. The Kalibr toolbox [17], which produced EuRoC's calibration, represents the trajectory as a continuous-time B-spline and solves one large nonlinear least-squares problem (Chapter 10) combining camera reprojection errors and IMU residuals.

Temperature. Biases and scale factors depend on temperature. Industrial IMUs are calibrated at several temperatures and compensated with an on-board temperature sensor; consumer IMUs often are not, and a filter must track the resulting drift as bias random walk.

15.5 Noise parameters and their units

The noise densities are continuous-time quantities (Section 6.3). For EuRoC's ADIS16448, the `imu0/sensor.yaml` file gives the values in Table 15.2.

YAML key	Symbol	Value	Units
`gyroscope_noise_density`	σ_g	1.6968×10^{-4}	$\text{rad/s}/\sqrt{\text{Hz}}$
`accelerometer_noise_density`	σ_a	2.0×10^{-3}	$\text{m/s}^2/\sqrt{\text{Hz}}$
`gyroscope_random_walk`	σ_{b_g}	1.9393×10^{-5}	$\text{rad/s}^2/\sqrt{\text{Hz}}$
`accelerometer_random_walk`	σ_{b_a}	3.0×10^{-3}	$\text{m/s}^3/\sqrt{\text{Hz}}$
`rate_hz`	$1/\Delta t$	200	Hz

Table 15.2: EuRoC IMU noise parameters (`imu0/sensor.yaml`). Verify against your copy of the dataset.

The units follow from the definitions. A gyro rate in rad/s with a white noise density has units $\text{rad/s} \cdot \sqrt{\text{s}} = \text{rad/s}/\sqrt{\text{Hz}}$. A gyro bias random walk is the integral of white noise with units of rad/s^2, so its density is in $\text{rad/s}^2/\sqrt{\text{Hz}}$, and after t seconds the bias has standard deviation $\sigma_{b_g}\sqrt{t}$ in rad/s.

Discrete equivalents. At sample period Δt (Equation (6.8)):

$$\sigma_{g,d} = \frac{\sigma_g}{\sqrt{\Delta t}}, \qquad \sigma_{a,d} = \frac{\sigma_a}{\sqrt{\Delta t}}, \qquad \sigma_{b_g,d} = \sigma_{b_g}\sqrt{\Delta t}, \qquad \sigma_{b_a,d} = \sigma_{b_a}\sqrt{\Delta t}. \tag{15.8}$$

For EuRoC: $\sigma_{g,d} = 2.4 \times 10^{-3}$ rad/s per sample, $\sigma_{a,d} = 2.8 \times 10^{-2}$ m/s^2 per sample, and the gyro bias diffuses by 1.4×10^{-6} rad/s per sample. In the filter you never need these directly, because $Q_d = G_c Q_c G_c^\mathsf{T} \Delta t$ uses the densities, but they are what a synthetic data generator must add to each sample.

Datasheet units. Manufacturers often quote angle random walk (ARW) in $^\circ/\sqrt{\text{h}}$ and velocity random walk (VRW) in m/s/$\sqrt{\text{h}}$. Since $1\ ^\circ/\sqrt{\text{h}} = \frac{\pi/180}{60}$ rad/$\sqrt{\text{s}} = 2.909 \times 10^{-4}$ rad/s/$\sqrt{\text{Hz}}$, EuRoC's σ_g is an ARW of $0.58\ ^\circ/\sqrt{\text{h}}$, and its σ_a is a VRW of 0.12 m/s/$\sqrt{\text{h}}$. Bias instability is quoted in $^\circ$/h and is *not* the same thing as the random walk density.

Measuring them. The densities are measured from a long static recording with the Allan variance (Section 6.8 and Figure 6.3): white noise is read off the slope $-\frac{1}{2}$ at $\tau = 1$ s, and the bias random walk off the slope $+\frac{1}{2}$ at $\tau = 3$ s.

15.6 Tuning in practice

The densities from an Allan analysis describe a stationary sensor in a quiet lab. On a flying vehicle, vibration, temperature changes, scale-factor and misalignment errors, timing jitter and integration error all add to the effective noise. Common practice is to start from the Allan values (or the dataset's values) and inflate them by a modest factor until the filter is consistent, as judged by the statistical tests of Chapter 19. Two rules keep tuning honest.

1. **Change physically meaningful parameters only**, and keep them within a plausible range of their measured values.
2. **Judge on consistency statistics** (NEES on synthetic data, NIS and 3σ coverage on real data), not on trajectory error alone. Tuning for the lowest trajectory error on the test sequence is overfitting.

Pitfall: mixing up the two discretizations

The white-noise variance per sample is $\sigma^2/\Delta t$ (larger at higher rate), while the random-walk variance per step is $\sigma_b^2 \Delta t$ (smaller at higher rate). Swapping them is a common bug in synthetic data generators, and it produces noise that is wrong by a factor of $1/\Delta t^2 = 40\,000$ at 200 Hz.

15.7 The EuRoC IMU stream

EuRoC's `imu0/data.csv` contains one row per sample: `timestamp [ns], w_x, w_y, w_z [rad/s], a_x, a_y, a_z [m/s^2]`, i.e. ω_m and a_m in B (AGENTS.md C7). Samples arrive at nominally 200 Hz, hardware-synchronized with the cameras. The filter must nevertheless take Δt from consecutive timestamps (AGENTS.md C5): real data has jitter and occasional gaps, which Milestone 3 logs. The ground-truth file starts later than the IMU file, so the filter is initialized at the first ground-truth sample at or after the stream start.

Project link: Milestones 3 and 4

M3 parses `sensor.yaml` into exactly the parameters of Table 15.2 and builds the event stream. M4's synthetic trajectory generator produces noise-free IMU data from (15.5); adding noise with the discrete equivalents above turns it into the Monte Carlo generator of M5.

Key takeaways

- An accelerometer is a spring-mass system whose deflection, below resonance, is proportional to specific force $f = a - g$: it reads $+g$ upward at rest and zero in free fall.
- A MEMS gyroscope measures the Coriolis force on a vibrating mass.
- $\omega_m = \omega_b + b_g + n_g$ and $a_m = R_{wb}^{\mathsf{T}}(a_w - g_w) + b_a + n_a$, with random-walk biases; scale factors and misalignments are neglected.
- Biases and scale factors can be calibrated with static multi-position tests; camera–IMU extrinsics with a target and a continuous-time solver.
- Noise densities have units of $[u]/\sqrt{\text{Hz}}$; per-sample white noise is $\sigma/\sqrt{\Delta t}$, per-step bias diffusion is $\sigma_b\sqrt{\Delta t}$.
- Unmodeled errors force noise inflation; tune on consistency, not on accuracy.

Exercises

15.1 An IMU lies on a table tilted by 10° about its x-axis. Compute the ideal accelerometer reading. Which component reveals the tilt, and with what sign?

15.2 A MEMS accelerometer has a resonance at 5 kHz. How large is its static deflection per m/s^2? Compare with the size of a silicon atom (≈ 0.2 nm) and comment.

15.3 Derive the gyroscope's Coriolis acceleration from (7.24), and explain why a tuning-fork design rejects linear acceleration.

15.4 A six-position test gives $a_{\text{up}} = 9.83$ and $a_{\text{down}} = -9.79\ \text{m/s}^2$ on one axis. Compute the bias and scale factor.

15.5 A gyroscope datasheet gives ARW $= 0.2\ ^\circ/\sqrt{\text{h}}$. Convert it to $\text{rad/s}/\sqrt{\text{Hz}}$ and to the per-sample standard deviation at 400 Hz.

15.6 A scale-factor error of $s = 10^{-3}$ affects a gyro on a vehicle spinning at 1 rad/s. What equivalent bias does it produce? Why can't a constant-bias model absorb it during maneuvers?

Chapter 16

Strapdown Propagation of the Nominal State

In this chapter

- State the continuous-time nominal kinematics and understand each term.
- Derive the exact solution of the kinematics for constant inputs over a step.
- Compare Euler, midpoint and Runge–Kutta integration, and predict their convergence orders.
- Propagate to arbitrary timestamps, handle gaps, and initialize the nominal state.
- Understand IMU preintegration as used by optimization-based estimators.

A *strapdown* inertial system has its sensors rigidly attached (strapped down) to the vehicle, as opposed to mounted on a stabilized gimbal platform. All the work of keeping track of orientation is done in software, by integrating the gyroscope. This chapter is about that integration.

16.1 Continuous-time kinematics

16.1.1 The navigation equations

The *mechanization equations* follow from three facts established earlier. The position changes with the velocity, $\dot{p}_{wb} = v_{wb}$. The velocity changes with the acceleration, which by the definition of specific force (Chapter 15) is the measured specific force rotated into the world frame plus gravity, $\dot{v}_{wb} = a_w = R_{wb} f_b + g_w$. And the attitude changes with the body angular velocity, $\dot{R}_{wb} = R_{wb}[\omega_b]_\times$ (Section 7.8). Here the world frame is treated as inertial, with the effects of the Earth's rotation neglected or absorbed into gravity and the gyro bias; Chapter 7 showed that they are tiny over the scales of a EuRoC flight. Substituting the IMU model, $f_b = a_m - b_a - n_a$ and $\omega_b = \omega_m - b_g - n_g$, gives the true kinematics of Chapter 17, and replacing the unknown biases by their estimates and setting the noise to zero gives the nominal kinematics.

Frozen convention: nominal propagation, AGENTS.md F1

$$\hat{a} = a_m - \hat{b}_a, \quad \hat{\omega} = \omega_m - \hat{b}_g; \qquad \dot{\hat{p}} = \hat{v}, \quad \dot{\hat{v}} = \hat{R}_{wb}\,\hat{a} + g_w, \quad \dot{\hat{q}} = \tfrac{1}{2}\,\hat{q} \otimes [0,\, \hat{\omega}], \quad \dot{\hat{b}}_a = \dot{\hat{b}}_g = 0. \tag{16.1}$$

The attitude update is always multiplicative, $\hat{q} \leftarrow \hat{q} \otimes \mathrm{Exp}_q(\hat{\omega}\Delta t)$, followed by normalization. The integration scheme for p and v is chosen and justified in Milestone 4 and

logged in `PROGRESS.md`.

The equations form a cascade: attitude depends on the gyroscope only; velocity depends on attitude and the accelerometer; position depends on velocity. Errors cascade the same way, which is the origin of the t^2 and t^3 growth laws of Chapter 1.

16.2 Exact solution for constant inputs

Suppose $\hat{\omega}$ and $\hat{a}$ are constant over $[t_k, t_k + \Delta t]$, which is what a zero-order hold of one IMU sample assumes. Then the kinematics can be solved in closed form.

Proposition 16.1 (Constant-input propagation). *With $\hat{\omega}$, $\hat{a}$ constant over the step and $\phi = \hat{\omega}\Delta t$,*

$$\hat{R}_{k+1} = \hat{R}_k \operatorname{Exp}(\phi), \tag{16.2}$$

$$\hat{v}_{k+1} = \hat{v}_k + g_w \Delta t + \hat{R}_k \, \Delta t \, J_l(\phi) \, \hat{a}, \tag{16.3}$$

$$\hat{p}_{k+1} = \hat{p}_k + \hat{v}_k \Delta t + \tfrac{1}{2} g_w \Delta t^2 + \hat{R}_k \, \Delta t^2 \, N(\phi) \, \hat{a}, \tag{16.4}$$

where $N(\phi) = \sum_{j \geq 0} [\phi]_\times^j / (j+2)! = \frac{1}{2} I + \frac{\theta - \sin\theta}{\theta^3} [\phi]_\times + \frac{\theta^2/2 + \cos\theta - 1}{\theta^4} [\phi]_\times^2$ and $\theta = \|\phi\|$.

Proof. The attitude is Proposition 7.4. During the step $\hat{R}(t_k + s) = \hat{R}_k \operatorname{Exp}(\hat{\omega} s)$, so $\hat{v}_{k+1} - \hat{v}_k = g_w \Delta t + \hat{R}_k \int_0^{\Delta t} \operatorname{Exp}(\hat{\omega} s) \, \mathrm{d}s \, \hat{a}$, and $\int_0^{\Delta t} \operatorname{Exp}(\hat{\omega} s) \, \mathrm{d}s = \Delta t \int_0^1 \operatorname{Exp}(u \phi) \, \mathrm{d}u = \Delta t \, J_l(\phi)$ by Theorem 8.9. Integrating once more gives the position, with $\int_0^{\Delta t} \int_0^s \operatorname{Exp}(\hat{\omega} u) \, \mathrm{d}u \, \mathrm{d}s = \Delta t^2 N(\phi)$ by term-wise integration of the exponential series, and the closed form follows from $[\phi]_\times^3 = -\theta^2 [\phi]_\times$. ∎

For small ϕ, $J_l(\phi) \approx I + \frac{1}{2}[\phi]_\times$ and $N(\phi) \approx \frac{1}{2} I + \frac{1}{6}[\phi]_\times$. The correction terms account for the rotation of the specific-force vector during the step. At 200 Hz and 1 rad/s, $\|\phi\| = 0.005$, so they change the velocity increment by about 0.25%: small, but systematic.

16.3 Numerical integration schemes

Real inputs are not constant over a step: the IMU provides samples at t_k and t_{k+1}, and the true signal varies between them. Three families of schemes are common. All of them update the attitude multiplicatively.

Euler with zero-order hold. Use the sample at the start of the step for the whole step:

$$\hat{R}_{k+1} = \hat{R}_k \operatorname{Exp}(\hat{\omega}_k \Delta t), \quad \hat{v}_{k+1} = \hat{v}_k + (\hat{R}_k \hat{a}_k + g_w)\Delta t, \quad \hat{p}_{k+1} = \hat{p}_k + \hat{v}_k \Delta t + \tfrac{1}{2}(\hat{R}_k \hat{a}_k + g_w)\Delta t^2. \tag{16.5}$$

Local error $O(\Delta t^2)$, global error $O(\Delta t)$. The dominant error is essentially a *half-sample time shift*: using the sample from the start of the interval for the whole interval delays the signal by $\Delta t/2$ on average. To see the local error, expand the true velocity increment $\int_{t_k}^{t_{k+1}} a_w(t) \, \mathrm{d}t = a_w(t_k)\Delta t + \frac{1}{2}\dot{a}_w(t_k)\Delta t^2 + O(\Delta t^3)$; Euler keeps only the first term, so each step errs by $\frac{1}{2}\dot{a}_w \Delta t^2$, and the errors of the $T/\Delta t$ steps accumulate to $O(\Delta t)$.

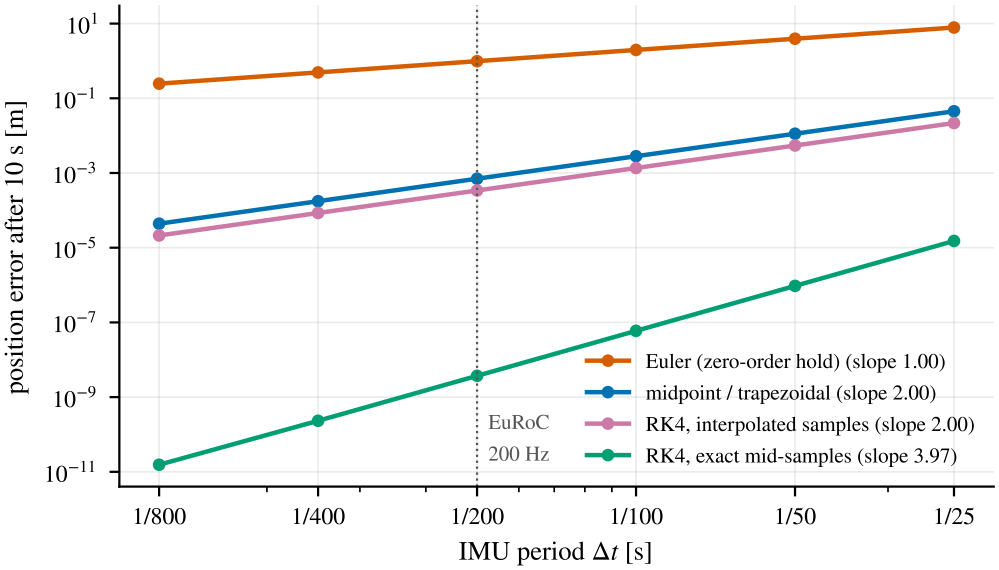


Figure 16.1: Position error after 10 s of noise-free integration on a synthetic trajectory (2 m circle at 0.8 rad/s with ±23° roll oscillations), as a function of the IMU period. The measured slopes match the predicted global orders. RK4 reaches fourth order only with exact mid-step inputs, which a real IMU does not provide; with interpolated inputs it is no better than the midpoint rule.

Midpoint (trapezoidal). Use the average of the two bounding samples:

$$\hat{R}_{k+1} = \hat{R}_k \operatorname{Exp}\left(\tfrac{1}{2}(\hat{\omega}_k + \hat{\omega}_{k+1})\Delta t\right), \quad \bar{a}_w = \tfrac{1}{2}\left(\hat{R}_k \hat{a}_k + \hat{R}_{k+1}\hat{a}_{k+1}\right) + g_w, \tag{16.6}$$

$$\hat{v}_{k+1} = \hat{v}_k + \bar{a}_w \Delta t, \qquad \hat{p}_{k+1} = \hat{p}_k + \hat{v}_k \Delta t + \tfrac{1}{2}\bar{a}_w \Delta t^2.$$

Local error $O(\Delta t^3)$, global $O(\Delta t^2)$, at the cost of having to wait for the next sample before completing the step.

Runge–Kutta 4. RK4 evaluates the dynamics at the start, twice at the middle, and at the end of the step. Its global error is $O(\Delta t^4)$ *if* the inputs at mid-step are known exactly. With an IMU they are not, and must be interpolated from the bounding samples. Linear interpolation is only second-order accurate, which caps the whole scheme at $O(\Delta t^2)$.

Figure 16.1 confirms the predicted orders. Halving Δt divides the error by 2 for Euler and by 4 for the midpoint rule, which is exactly the convergence-order test of Milestone 4. The magnitudes matter as much as the slopes. At 200 Hz on this fairly dynamic trajectory, the Euler scheme accumulates about a meter of error in 10 s, far more than the sensor noise would cause (Figure 17.1), because the half-sample delay of the attitude creates a tilt error of order $\frac{1}{2}\omega\Delta t$, which leaks gravity. The midpoint rule is three orders of magnitude better. On gentler motion the difference shrinks, and frequent visual updates bound integration error in any case, but the integration error should be well below the noise-induced error, or it becomes an unmodeled error that the filter cannot account for.

Key idea: choosing a scheme

The integration error is a deterministic, unmodeled error. The scheme should make it small compared with the stochastic error the filter does model. Whichever scheme Milestone 4 adopts, document its order and verify it with the halving test on a trajectory with closed-form ground truth.

16.4 Coning, sculling and the rotation-vector equation

16.4.1 The rotation vector of a step

The attitude update over a step is a rotation $\Delta R = R(t_k)^{\mathsf{T}} R(t_{k+1})$, which we can write as $\operatorname{Exp}(\phi)$ for a rotation vector ϕ. If the angular velocity keeps a fixed direction during the step, $\phi = \int \omega \, \mathrm{d}t$. If its direction changes, this is wrong (Proposition 4.7), and the correct ϕ obeys a differential equation of its own.

Proposition 16.2 (Bortz equation). *The rotation vector $\phi(t)$ with $\operatorname{Exp}(\phi(t)) = R(t_k)^{\mathsf{T}} R(t)$ satisfies*

$$\dot{\phi} = J_r^{-1}(\phi)\,\omega = \omega + \tfrac{1}{2}\,\phi \times \omega + D(\|\phi\|)\,\phi \times (\phi \times \omega), \tag{16.7}$$

with D the coefficient function of Theorem 8.9.

Proof. Over an instant $\mathrm{d}t$, $\operatorname{Exp}(\phi + \dot{\phi}\,\mathrm{d}t) = \operatorname{Exp}(\phi)\operatorname{Exp}(\omega\,\mathrm{d}t)$. By the definition of the right Jacobian (8.19), the left side is $\operatorname{Exp}(\phi)\operatorname{Exp}(J_r(\phi)\dot{\phi}\,\mathrm{d}t)$, so $J_r(\phi)\dot{\phi} = \omega$. Expand J_r^{-1} with (8.21). ∎

The term $\frac{1}{2}\phi \times \omega$ is non-zero only when ω is not parallel to the rotation already accumulated: it is the *non-commutativity correction*. Under *coning motion*, in which the angular velocity vector itself rotates (for example $\omega = (a\cos\Omega t,\ a\sin\Omega t,\ 0)$, the motion of a vibrating or wobbling body), this correction does not average to zero. Integrating the rate vector alone then produces a steady, spurious rotation about the cone axis, a drift that no amount of averaging removes.

16.4.2 The two-sample coning correction

Given the integrated gyro increments α_1 and α_2 over the two halves of a step (which a high-rate IMU, or two consecutive samples, provides), the rotation vector of the whole step is, to third order,

$$\phi \approx \alpha_1 + \alpha_2 + \tfrac{2}{3}\,\alpha_1 \times \alpha_2. \tag{16.8}$$

Derivation. Keep the first-order Bortz equation $\dot{\phi} \approx \omega + \frac{1}{2}\phi \times \omega$ and suppose the rate varies linearly over the step, $\omega(s) = a + bs$ for $s \in [0, h]$. To the needed order $\phi(s) \approx \int_0^s \omega = as + \frac{1}{2}bs^2$, so

$$\begin{aligned}\phi(h) &\approx \int_0^h \omega\,\mathrm{d}s + \frac{1}{2}\int_0^h \left(as + \tfrac{1}{2}bs^2\right) \times (a + bs)\,\mathrm{d}s \\ &= \int_0^h \omega\,\mathrm{d}s + \frac{1}{2}(a \times b)\int_0^h \left(s^2 - \tfrac{1}{2}s^2\right)\mathrm{d}s = \alpha_1 + \alpha_2 + \frac{h^3}{12}\,a \times b.\end{aligned}$$

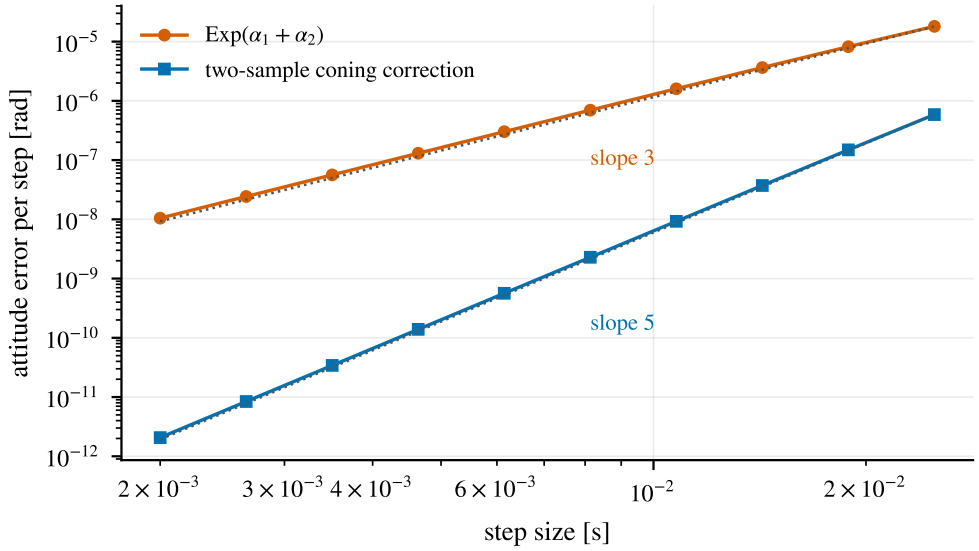


Figure 16.2: Attitude error over one step under coning motion (rate amplitude 0.5 rad/s, coning frequency 10 Hz), as a function of the step size. Exponentiating the integrated rate errs at third order; the two-sample correction (16.8) reduces the error to fifth order.

The half-step increments are $\alpha_1 = \frac{h}{2}a + \frac{h^2}{8}b$ and $\alpha_2 = \frac{h}{2}a + \frac{3h^2}{8}b$, whose cross product is $\alpha_1 \times \alpha_2 = \frac{h^3}{8}\, a \times b$. Hence $\frac{h^3}{12}a \times b = \frac{2}{3}\alpha_1 \times \alpha_2$. ∎

Figure 16.2 confirms the orders. *Sculling* is the analogous effect for velocity: when rotation and acceleration oscillate in phase, the specific force is rotated while it is being integrated, and the error rectifies into a velocity drift; it has similar multi-sample corrections. High-end inertial systems, with gyros sampled at kHz rates on vibrating platforms, rely on such algorithms [20]. For a MEMS IMU sampled at 200 Hz, with vibration filtered by the sensor's own electronics, the effects are small compared with the sensor noise, and the project's single-sample schemes suffice. The analysis still matters: it tells you what you are neglecting, and how to check that it is negligible.

16.5 Timing

16.5.1 Propagating to an arbitrary time

Images arrive between IMU samples. To clone the state at an image time $t_c \in (t_k, t_{k+1})$ (AGENTS.md F5), the filter must propagate exactly to t_c: interpolate the IMU reading at t_c linearly between samples k and $k+1$, propagate from t_k to t_c with the interpolated input, clone, and then propagate from t_c to t_{k+1}. The covariance follows the same split, with Δt replaced by the two partial intervals.

16.5.2 Event ordering and gaps

Measurements must be processed in timestamp order, and the state must be propagated to a measurement's time before the measurement is applied (Chapter 28). Milestone 3 fixes

the tie rule: an IMU sample precedes a measurement with an equal timestamp, so the state is propagated to that time first. Gaps in the IMU stream (intervals larger than about 1.5 nominal periods) must not be assumed away. Δt always comes from the timestamps, and a long gap simply produces a long step with a correspondingly large Q_d.

16.6 Initialization

16.6.1 From ground truth

The project initializes the nominal state from the first ground-truth sample at or after the start of the stream (Milestone 4): p, v, q, b_a and b_g all come from the ground-truth columns, mapped with the column order of AGENTS.md C7. This isolates propagation from the initialization problem. The use of ground truth for initialization is logged when the full VIO runs (Milestone 10b).

16.6.2 Static initialization

A real system initializes from its own sensors. If the vehicle is at rest for about a second,

$$\bar{a}_m = \mathrm{mean}(a_m) \approx R_{wb}^{\mathsf{T}}(-g_w) + b_a, \qquad \bar{\omega}_m = \mathrm{mean}(\omega_m) \approx b_g. \tag{16.9}$$

The gyro bias is the mean rate. Neglecting b_a, the unit vector $u_b = \bar{a}_m / \|\bar{a}_m\|$ is the world "up" direction expressed in the body frame, $u_b = R_{wb}^{\mathsf{T}} e_z$. Any rotation with $R_{wb} u_b = e_z$ is consistent with it. One such rotation turns u_b onto e_z about the axis perpendicular to both:

$$R_{wb} = \mathrm{Exp}\Big(\operatorname{atan2}\left(\|u_b \times e_z\|,\ u_b^{\mathsf{T}} e_z\right) \frac{u_b \times e_z}{\|u_b \times e_z\|} \Big), \tag{16.10}$$

(with $R_{wb} = I$ if $u_b = e_z$ already), and composing it on the left with any rotation $R_z(\psi)$ about the vertical gives all the others. Roll and pitch are determined; yaw is arbitrary, and is set to zero (or to the ground-truth yaw for evaluation).

Pitfall: tilt and accelerometer bias are indistinguishable at rest

A horizontal accelerometer bias b_a produces the same reading as a tilt of $\|b_a\|/g$: a bias of 10 mm/s^2 looks exactly like a tilt of 1 mrad. At rest the filter cannot separate them, and the initial covariance must say so (large σ_{b_a}, correlated with the tilt). They become separable once the vehicle rotates, because the bias rotates with the body while gravity does not.

Initial covariances should reflect real uncertainty: a few milliradians in roll and pitch, a large or unknown yaw, the turn-on bias range from the datasheet, and a small but non-zero velocity uncertainty at rest. *Dynamic* initialization from a moving start, as in VINS-Mono [42], first solves a vision-only structure-from-motion problem and then aligns it with preintegrated IMU data to recover scale, gravity and velocity.

16.7 Sidebar: IMU preintegration

An optimization-based estimator may relinearize a window of states many times. Re-integrating every IMU sample between keyframes i and j each time the estimate of state i changes would be wasteful. *Preintegration* [37, 16] integrates the measurements in the body frame of keyframe i, so that the result does not depend on the state at i:

$$\Delta R_{ij} = \prod_{k=i}^{j-1} \operatorname{Exp}(\hat{\omega}_k \Delta t), \quad \Delta v_{ij} = \sum_{k=i}^{j-1} \Delta R_{ik}\, \hat{a}_k \Delta t, \quad \Delta p_{ij} = \sum_{k=i}^{j-1} \left(\Delta v_{ik} \Delta t + \tfrac{1}{2} \Delta R_{ik}\, \hat{a}_k \Delta t^2 \right). \tag{16.11}$$

These relate the two keyframe states through $R_j = R_i \Delta R_{ij}$, $v_j = v_i + g\Delta t_{ij} + R_i \Delta v_{ij}$ and $p_j = p_i + v_i \Delta t_{ij} + \frac{1}{2} g \Delta t_{ij}^2 + R_i \Delta p_{ij}$. The preintegrated quantities depend on the bias estimate, and first-order bias Jacobians avoid re-integration when the bias estimate changes. A filter like the MSCKF does not need preintegration, because it integrates each sample exactly once.

Project link: Milestone 4

Exit criteria: a stationary input keeps the state constant to 10^{-9} for 60 s; a constant rate reproduces $q_0 \otimes \operatorname{Exp}_q(\omega t)$; a circular trajectory converges at the predicted order; the quaternion norm stays within 10^{-12} of one after 10^6 steps; propagation does not allocate; and 10 s of dead reckoning on MH_01 produce no NaN.

Key takeaways

- $\dot{p} = v$, $\dot{v} = R\hat{a} + g$, $\dot{q} = \frac{1}{2} q \otimes [0, \hat{\omega}]$; attitude updates are multiplicative.
- For constant inputs the solution is exact: $R \operatorname{Exp}(\phi)$, $v + g\Delta t + R\Delta t J_l(\phi)\hat{a}$, and a similar position formula.
- Euler is first order (a half-sample delay), midpoint second order. RK4 is limited to second order by input interpolation.
- Integration error is unmodeled error; keep it well below sensor-noise error and verify the order.
- Propagate exactly to measurement times by splitting the IMU interval.
- At rest, gravity gives roll and pitch, the mean gyro gives b_g, and tilt and horizontal b_a are indistinguishable.

Exercises

16.1 Verify Proposition 16.1 numerically by comparing with an ODE solver at tight tolerance.

16.2 Show that the Euler scheme applied to a constant rotation rate is exact for the attitude, but not for the velocity, and quantify the velocity error per step in terms of $\|\hat{\omega}\|\Delta t$.

16.3 Reproduce Figure 16.1. Then add gyro noise and plot the noise-induced position error on the same axes. At what IMU rate does integration error fall below noise error for each scheme?

16.4 Implement the static initializer and test it on synthetic data with a known tilt. Add a horizontal accelerometer bias and show that it biases the estimated tilt by $\|b_a\|/g$.

16.5 Why must the propagation to an image timestamp interpolate the IMU sample instead of using the nearest sample? Estimate the attitude error from using the nearest sample at 1 rad/s and 200 Hz.

Chapter 17

Error-State Dynamics and Covariance Propagation

In this chapter

- Set up the 15-dimensional IMU error state with the frozen index layout.
- Derive linearized error dynamics with the four-step recipe, including the attitude error under a perturbation convention.
- Discretize into Φ and Q_d and propagate the covariance of the IMU state and its clones.
- Predict analytically how fast each noise source inflates the position uncertainty.
- Validate the propagation by finite differences and by Monte Carlo simulation.

17.1 True and nominal systems

The true state evolves under the IMU model (15.5) solved for the physical quantities. With $\omega = \omega_m - b_g - n_g$ and $a = a_m - b_a - n_a$ the true body rate and specific force,

$$\dot{p} = v, \qquad \dot{v} = R\,(a_m - b_a - n_a) + g_w, \qquad \dot{R} = R\,[\omega_m - b_g - n_g]_\times, \qquad \dot{b}_a = n_{b_a}, \qquad \dot{b}_g = n_{b_g}, \tag{17.1}$$

where $R = R_{wb}$. The nominal state follows (16.1), which is (17.1) with the biases replaced by their estimates and the noise set to zero. The error state measures their difference.

Frozen convention: error-state layout, AGENTS.md C3

Indices	Block	Meaning
0–2	δp	position error, world frame
3–5	δv	velocity error, world frame
6–8	$\delta\theta$	attitude error, **body frame (local)**: $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$
9–11	δb_a	accelerometer bias error
12–14	δb_g	gyroscope bias error
$15 + 6i \ldots +2$	δp_i	clone i position error
$18 + 6i \ldots +2$	$\delta\theta_i$	clone i attitude error (local)

Noise ordering: $w = [n_a, n_g, n_{b_a}, n_{b_g}] \in \mathbb{R}^{12}$.

Index constants belong in one place in the code (`constexpr` offsets in `types.hpp`), and every block access goes through them. Hard-coded literals such as `P.block<3,3>(6,6)` scattered through the code are how layout bugs are born.

17.2 Deriving the error dynamics

The goal is the linear system

$$\dot{\delta x}_I = F_c\, \delta x_I + G_c\, w, \qquad F_c \in \mathbb{R}^{15\times 15}, \quad G_c \in \mathbb{R}^{15\times 12}, \tag{17.2}$$

obtained with the recipe of Section 14.3. Two of the five blocks are immediate.

Position. Substituting $p = \hat{p} + \delta p$ and $v = \hat{v} + \delta v$ into $\dot{p} = v$ and subtracting $\dot{\hat{p}} = \hat{v}$ gives $\dot{\delta p} = \delta v$ exactly. No linearization is needed.

Biases. $\dot{b}_a = n_{b_a}$ and $\dot{\hat{b}}_a = 0$ give $\dot{\delta b}_a = n_{b_a}$, and likewise $\dot{\delta b}_g = n_{b_g}$.

The velocity and attitude blocks involve the rotation and require the Lie tools of Chapter 8. One lemma handles the time derivative of the perturbation itself.

> **Lemma 17.1** (Time derivative of a perturbation). *For a differentiable $\delta\theta(t)$, $\frac{\mathrm{d}}{\mathrm{d}t}\operatorname{Exp}(\delta\theta) = \operatorname{Exp}(\delta\theta)[J_r(\delta\theta)\,\dot{\delta\theta}]_\times = [J_l(\delta\theta)\,\dot{\delta\theta}]_\times \operatorname{Exp}(\delta\theta)$. To first order in $\delta\theta$, both reduce to $[\dot{\delta\theta}]_\times$ multiplying $\operatorname{Exp}(\delta\theta)$ on the corresponding side.*

Proof. Apply the definition (8.19) of J_r and J_l with $\delta = \dot{\delta\theta}\,\mathrm{d}t$. ∎

17.2.1 Worked example: attitude error under the global perturbation

To demonstrate the method without solving the project's exercise, we first derive the attitude error dynamics for the *global* perturbation $R = \operatorname{Exp}(\delta\theta_g)\hat{R}$. This is **not** the project's convention; it is here for contrast.

> **Example 17.2** (Global attitude error dynamics). Differentiate both sides of $R = \operatorname{Exp}(\delta\theta_g)\hat{R}$ using Lemma 17.1 to first order and the kinematics $\dot{R} = R[\omega]_\times$, $\dot{\hat{R}} = \hat{R}[\hat{\omega}]_\times$:
>
> $$\underbrace{\operatorname{Exp}(\delta\theta_g)\hat{R}[\omega]_\times}_{\dot{R}} \approx [\dot{\delta\theta}_g]_\times \operatorname{Exp}(\delta\theta_g)\hat{R} + \operatorname{Exp}(\delta\theta_g)\hat{R}[\hat{\omega}]_\times.$$
>
> Move the last term to the left and use $\omega - \hat{\omega} = -\delta b_g - n_g$:
>
> $$[\dot{\delta\theta}_g]_\times \operatorname{Exp}(\delta\theta_g)\hat{R} \approx \operatorname{Exp}(\delta\theta_g)\hat{R}[-\delta b_g - n_g]_\times.$$
>
> Multiply on the right by $(\operatorname{Exp}(\delta\theta_g)\hat{R})^{\mathsf{T}}$ and apply (2.14) with the rotation $\operatorname{Exp}(\delta\theta_g)\hat{R}$:
>
> $$[\dot{\delta\theta}_g]_\times \approx [\operatorname{Exp}(\delta\theta_g)\hat{R}(-\delta b_g - n_g)]_\times \approx [-\hat{R}\,\delta b_g - \hat{R}\,n_g]_\times,$$
>
> where the last step drops the product of $\delta\theta_g$ with δb_g and n_g, both small. Hence
>
> $$\dot{\delta\theta}_g = -\hat{R}\,\delta b_g - \hat{R}\,n_g. \tag{17.3}$$
>
> The global attitude error does not depend on itself at all: the world-frame error is constant unless driven by gyro errors, which are rotated into the world by $\hat{R}$.

17.2.2 Your turn: the local (right) perturbation

Derivation exercise (project Jacobian): attitude and velocity rows of F_c and G_c

Derive $\dot{\delta\theta}$ and $\dot{\delta v}$ under the project's local perturbation $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$.

1. **Attitude.** Differentiate $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$ with Lemma 17.1 (right form). Cancel the nominal term, then use $\mathrm{Exp}(-\delta\theta)[\hat{\omega}]_\times \mathrm{Exp}(\delta\theta) = [\mathrm{Exp}(-\delta\theta)\hat{\omega}]_\times$ and $\mathrm{Exp}(-\delta\theta)\hat{\omega} \approx \hat{\omega} - \delta\theta \times \hat{\omega}$. You should find a term proportional to $\delta\theta$ that the global version lacks. Explain it physically: what happens to a body-frame error when the body rotates?
2. **Velocity.** Substitute $R = \hat{R}\,\mathrm{Exp}(\delta\theta) \approx \hat{R}(I + [\delta\theta]_\times)$ and $a_m - b_a - n_a = \hat{a} - \delta b_a - n_a$ into $\dot{v} = R(a_m - b_a - n_a) + g_w$, subtract $\dot{\hat{v}} = \hat{R}\hat{a} + g_w$, drop second-order products, and use (2.8) to move $\delta\theta$ to the right.
3. **Assemble** F_c and G_c in the frozen order of AGENTS.md C3. Before filling any block, predict from the cascade structure of the kinematics which blocks must be zero.

Check: build $\Phi \approx I + F_c\Delta t$ and compare each column with $\big(f(\hat{x} \boxplus \epsilon e_i) \boxminus f(\hat{x})\big)/\epsilon$ for one step f of your nominal integrator, at a state with non-identity attitude, non-zero rates, specific force and biases (Milestone 5). Agreement must be at the level of the $O(\Delta t^2)$ terms neglected by $I + F_c\Delta t$. Then record the result in `DOCS/derivations.md`.

Worked solution: Appendix E.1 (page 280).

Physically, the velocity row you will find encodes the most important coupling in inertial navigation: an attitude error tilts the computed specific force, so a fraction of the large vector $\hat{a}$ (dominated by the reaction to gravity, 9.81 m/s^2) leaks into the velocity error. That single term is behind the t^3 law of Chapter 1.

17.3 Discretization

Over one IMU step the error dynamics are discretized as in Section 11.2 (AGENTS.md F2):

$$\Phi \approx I_{15} + F_c\Delta t, \qquad Q_c = \mathrm{diag}\left(\sigma_a^2 I_3,\ \sigma_g^2 I_3,\ \sigma_{b_a}^2 I_3,\ \sigma_{b_g}^2 I_3\right), \qquad Q_d \approx G_c Q_c G_c^{\mathsf{T}} \Delta t, \tag{17.4}$$

with F_c and G_c evaluated at the nominal state and the bias-corrected IMU inputs at the beginning of the step (or at the midpoint, consistently with the chosen integration scheme). The densities are the continuous-time values of Table 15.2; the Δt in Q_d converts them into variances accumulated during the step.

***Remark* 17.3** (higher-order options)**.** F_c is block upper-triangular with a large nilpotent part, so $\exp(F_c\Delta t)$ has a closed form. In particular the attitude block of the exact Φ is a rotation matrix, while the first-order block is not exactly orthogonal. At 200 Hz and 1 rad/s the difference per step is about $(\|\hat{\omega}\|\Delta t)^2/2 \approx 10^{-5}$, below the noise. Adopting a higher-order Φ is allowed if it is logged (AGENTS.md F2). Van Loan's method (Proposition 11.3) gives exact reference values for testing whichever form is used. The closed-form blocks are listed with the solutions in Appendix E.2.

The continuous-time view. The discrete recursion is a numerical integration of a differential equation for the covariance. For the linear stochastic system $\dot{\delta x} = F_c\,\delta x + G_c\,w$, the covariance obeys the Lyapunov differential equation of Proposition 6.5,

$$\dot{P} = F_c P + P F_c^{\mathsf{T}} + G_c Q_c G_c^{\mathsf{T}}. \tag{17.5}$$

Substituting $\Phi = I + F_c\Delta t$ and $Q_d = G_c Q_c G_c^{\mathsf{T}}\Delta t$ into one step gives $\Phi P \Phi^{\mathsf{T}} + Q_d = P + \big(F_c P + P F_c^{\mathsf{T}} + G_c Q_c G_c^{\mathsf{T}}\big)\Delta t + F_c P F_c^{\mathsf{T}}\Delta t^2$: the filter's propagation is Euler's method applied to (17.5), with a local error of $O(\Delta t^2)$. Each term has a meaning. $G_c Q_c G_c^{\mathsf{T}}$ injects uncertainty where the noise enters (velocity, attitude and the biases), and $F_c P + P F_c^{\mathsf{T}}$ transports it along the couplings of F_c: from biases into attitude and velocity, from attitude into velocity, from velocity into position. The growth laws of Proposition 17.4 below are exact solutions of (17.5) for a level, stationary IMU, and they are the reference against which the discrete propagation is tested in Milestone 5.

17.4 Propagating the full covariance

With N clones the state has dimension $n = 15 + 6N$, and the covariance is partitioned as

$$P = \begin{bmatrix} P_{II} & P_{IC} \\ P_{CI} & P_{CC} \end{bmatrix}, \qquad P_{II} \in \mathbb{R}^{15\times 15},\ P_{IC} \in \mathbb{R}^{15\times 6N},\ P_{CC} \in \mathbb{R}^{6N\times 6N}. \tag{17.6}$$

Clones are frozen copies of past poses, so their error dynamics are zero: $\dot{\delta x}_C = 0$. The full transition matrix is $\mathrm{blkdiag}(\Phi, I_{6N})$, and the full process noise is $\mathrm{blkdiag}(Q_d, 0)$. Multiplying out:

Frozen convention: covariance propagation, AGENTS.md F2

$$P_{II} \leftarrow \Phi P_{II}\Phi^{\mathsf{T}} + Q_d, \qquad P_{IC} \leftarrow \Phi P_{IC}, \qquad P_{CI} = P_{IC}^{\mathsf{T}}, \qquad P_{CC} \text{ unchanged}, \tag{17.7}$$

followed by $P \leftarrow \frac{1}{2}(P + P^{\mathsf{T}})$.

The block form is not only a convenience. Updating only the affected blocks costs $O(15^2 \cdot 6N)$ per step instead of $O(n^3)$, and it never touches the large P_{CC} block between images. The cross-covariance P_{IC} is what remembers that the clones and the current state share a history. It is what lets a visual constraint between two past poses correct the current velocity and biases.

Implementation note: allocation-free propagation

P is a fixed-*max*-size Eigen matrix (Chapter 30). Write the block updates with `.noalias()` into preallocated member workspaces, e.g. a 15×15 temporary for ΦP_{II} and a $15\times 6N_{max}$ temporary for ΦP_{IC}, because the product reads the block it overwrites. Avoid `auto` with Eigen expressions.

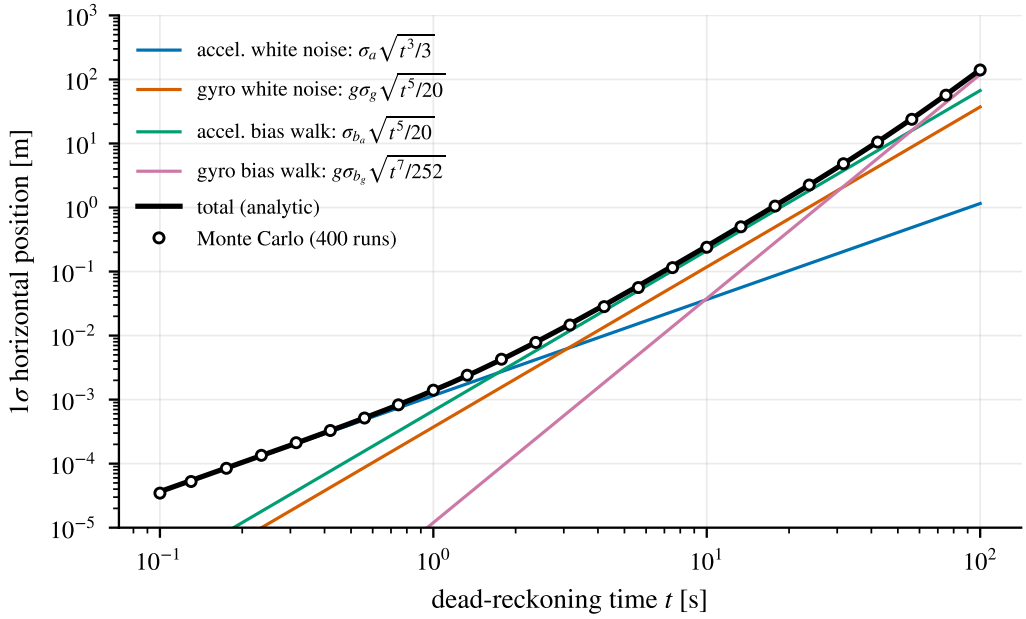


Figure 17.1: Horizontal position standard deviation of a dead-reckoning IMU with EuRoC noise parameters, from (17.8), with a Monte Carlo check (400 runs of the discretized error equations at 200 Hz). Accelerometer white noise dominates for the first two seconds, the accelerometer bias walk from there to about a minute, and then the gyro bias walk takes over with its $t^{7/2}$ growth.

17.5 How uncertainty grows

The structure of F_c explains how quickly an IMU-only solution degrades. Consider a level, stationary IMU, and one horizontal axis. Three couplings act in cascade.

- A gyro error (white noise or bias error) integrates into a tilt error $\delta\theta$.
- A tilt of $\delta\theta$ rotates the measured reaction to gravity, producing a horizontal specific-force error of magnitude $g\,\delta\theta$, which integrates into velocity error.
- Velocity error integrates into position error.

Each noise source therefore reaches position after a known number of integrations, and Proposition 6.4 gives its contribution to the position variance directly.

Proposition 17.4 (Dead-reckoning growth laws). *For a level, stationary IMU with zero initial uncertainty, the horizontal position variance grows as*

$$\sigma_p^2(t) = \underbrace{\frac{\sigma_a^2 t^3}{3}}_{\text{accel. white}} + \underbrace{\frac{g^2\sigma_g^2 t^5}{20}}_{\text{gyro white}} + \underbrace{\frac{\sigma_{b_a}^2\, t^5}{20}}_{\text{accel. bias walk}} + \underbrace{\frac{g^2\sigma_{b_g}^2\, t^7}{252}}_{\text{gyro bias walk}}. \tag{17.8}$$

Proof. Accelerometer white noise is integrated twice ($m = 2$ in Proposition 6.4). Gyro white noise is integrated once into tilt, scaled by g, and integrated twice more ($m = 3$). The accelerometer bias random walk is itself integrated white noise, then integrated twice ($m = 3$). The gyro bias random walk adds one more integration ($m = 4$). The sources are independent,

so their variances add. ■

Figure 17.1 evaluates (17.8) with EuRoC's parameters. After one second of blind flight the position uncertainty is about 1 mm, after ten seconds about 25 cm, and after a minute more than 10 m. The camera's job is to keep the filter permanently in the left part of this plot.

Project link: Milestone 5

The analytic check of M5 isolates the first term of (17.8): with $P_0 = 0$, only accelerometer noise and a fixed orientation, the horizontal position variance must equal $\sigma_a^2 t^3/3$. Because Φ and Q_d are first-order approximations, the agreement improves as $\Delta t \to 0$ (Example 11.2), and the test tolerance should reflect that.

Why diagonal entries need not grow monotonically. The attitude covariance is expressed in body axes (Remark 8.15). When the vehicle rotates, the attitude block of Φ rotates the covariance between axes, so an individual diagonal entry can shrink during pure propagation even though no information is gained. Monotonicity holds for invariant quantities, such as the eigenvalues of $P_{\theta\theta}$ when the vehicle is not rotating, but not for individual entries.

17.6 Validating covariance propagation

Two independent tests catch nearly all propagation bugs.

Finite differences on the discrete propagation. The Jacobian of one step of the nominal integrator, computed numerically with $\boxplus/\boxminus$, must match Φ column by column (the derivation exercise above). This catches sign errors, transposes, and blocks placed at wrong indices. The test must use a non-trivial state.

Monte Carlo. Simulate M noisy IMU sequences from a known trajectory, integrate each with the nominal equations from a perturbed initial state drawn from P_0, and compute the errors $e^{(j)} = x_{\text{true}} \boxminus \hat{x}^{(j)}$. Their sample covariance must agree with the propagated P. If the filter is correct, the ratio of a sample variance to the predicted variance is distributed as $\chi^2_{M-1}/(M-1)$, so for $M = 500$ it should lie in $[0.88, 1.13]$ with 95% probability for each diagonal entry. Larger deviations, or many entries near the bounds in the same direction, indicate an error in G_c, Q_c or the units. This catches errors that finite differences cannot, such as a wrong noise density or a missing Δt.

Key takeaways

- The error state is 15-dimensional with the frozen layout $[\delta p, \delta v, \delta\theta, \delta b_a, \delta b_g]$.
- Error dynamics follow from substituting $x = \hat{x} \boxplus \delta x$, subtracting the nominal dynamics, and keeping first-order terms. The perturbation convention changes the result.
- $\Phi \approx I + F_c \Delta t$, $Q_d \approx G_c Q_c G_c^{\mathsf{T}} \Delta t$ with continuous densities; clones have identity dynamics.

- Position variance grows like t^3 (accel. noise), t^5 (gyro noise, accel. bias walk) and t^7 (gyro bias walk).
- Validate Φ by finite differences and P by Monte Carlo with statistically justified bounds.

Exercises

17.1 Derive the velocity error dynamics for the global perturbation, and compare its structure with your local result. Which one depends on the vehicle's heading?

17.2 Using your F_c, show that $\delta\theta$ is decoupled from δp and δv, and explain this from the cascade structure.

17.3 Starting from (17.4), write Q_d block by block and check its units.

17.4 Implement covariance propagation for the 15-state system in Python and reproduce Figure 17.1, including the Monte Carlo points.

17.5 For a vehicle rotating about its z-axis at 1 rad/s with $P_{\theta\theta} = \mathrm{diag}(10^{-4}, 10^{-6}, 10^{-6})$, propagate $P_{\theta\theta}$ for a quarter turn with no noise and observe the diagonal entries.

17.6 How large must M be for a Monte Carlo variance check to detect a 10% error in σ_a with high probability?

Part IV

The Error-State Filter in Practice

Chapter 18

Measurement Updates, Gating and the Pose Sensor

In this chapter

- Apply the generic manifold update to concrete sensors, deriving H by the elementary Jacobians.
- Understand which state directions a measurement observes, from the nullspace of H.
- Implement the mock pose sensor of Milestone 6 and derive its Jacobian.
- Gate measurements with the χ^2 test, and understand how gating fails in an overconfident filter.
- Understand how a pose sensor makes velocity and biases observable.

18.1 The generic update

Frozen convention: measurement update, AGENTS.md F3

1. Residual $r = z \boxminus h(\hat{x})$: plain subtraction for vector measurements; for the pose measurement $r = [\, p_{meas} - \hat{p};\ \mathrm{Log}_q(\hat{q}^{-1} \otimes q_{meas}) \,]$.
2. $H = \partial r / \partial \delta x$ at $\delta x = 0$ using C4 (Section 14.4), derived by hand and verified by finite differences.
3. $S = HPH^\mathsf{T} + R_z$; $K = PH^\mathsf{T}S^{-1}$ by a Cholesky/LDL$^\mathsf{T}$ solve, never an explicit inverse.
4. Mahalanobis gate $r^\mathsf{T}S^{-1}r < \chi^2_{0.95}(\dim r)$ from a `constexpr` table. Rejections are counted and logged, never silently dropped.
5. Joseph form $P \leftarrow (I - KH)P(I - KH)^\mathsf{T} + KR_zK^\mathsf{T}$, then injection and reset (Section 14.5).

Everything specific to a sensor is in h and H. The rest of this section derives them for two sensors that the project does not use, to demonstrate the technique, before turning to the pose sensor it does use.

18.2 Worked example: position with a lever arm

A GNSS antenna mounted at a known offset ℓ_b from the IMU, expressed in the body frame, measures the antenna position:

$$z = h(x) + n = p_{wb} + R_{wb}\,\ell_b + n, \qquad n \sim \mathcal{N}(0, \sigma^2 I_3). \tag{18.1}$$

The residual is a plain difference, $r = z - (\hat{p} + \hat{R}\ell_b)$. To find H, perturb the nominal state with $\boxplus$ and expand:

$$h(\hat{x} \boxplus \delta x) = \hat{p} + \delta p + \hat{R}\,\mathrm{Exp}(\delta\theta)\ell_b \approx \hat{p} + \delta p + \hat{R}\ell_b + \hat{R}[\delta\theta]_\times \ell_b = h(\hat{x}) + \delta p - \hat{R}[\ell_b]_\times \delta\theta.$$

Reading off the coefficients (the rotation-action row of Table 8.1),

$$H = \begin{bmatrix} I_3 & 0_3 & -\hat{R}[\ell_b]_\times & 0_3 & 0_3 \end{bmatrix}. \tag{18.2}$$

The attitude column is non-zero because the antenna swings around the IMU when the body rotates. A lever-arm position measurement therefore carries attitude information, but only about rotations that move the antenna: the nullspace of $-\hat{R}[\ell_b]_\times$ is the direction ℓ_b, since rotating about the lever arm itself does not move the antenna.

18.3 Worked example: a direction sensor

A magnetometer, or an accelerometer used as an inclinometer at rest, measures a known world vector m_w in body coordinates:

$$z = R_{wb}^\mathsf{T} m_w + n. \tag{18.3}$$

With the inverse-action row of Table 8.1, $h(\hat{x} \boxplus \delta x) = \mathrm{Exp}(-\delta\theta)\hat{R}^\mathsf{T} m_w \approx \hat{R}^\mathsf{T} m_w + [\hat{R}^\mathsf{T} m_w]_\times \delta\theta$, so

$$H = \begin{bmatrix} 0_3 & 0_3 & [\hat{R}^\mathsf{T} m_w]_\times & 0_3 & 0_3 \end{bmatrix}. \tag{18.4}$$

$[v]_\times$ has rank two and nullspace $\mathrm{span}(v)$, so a direction measurement observes two of the three attitude degrees of freedom. The rotation about the measured direction is unobservable. For gravity this is the familiar statement that an accelerometer at rest gives roll and pitch but not yaw.

Key idea: the nullspace of H tells you what a sensor cannot see

Any error direction $\delta x \in \mathrm{null}(H)$ produces no change in the predicted measurement, so the update cannot correct it. Reading the nullspace of each new H is a quick sanity check of both the derivation and your intuition about the sensor.

18.4 The pose sensor of Milestone 6

Before introducing a camera, the project tests the whole ESKF with a "mock sensor" that measures the pose directly. Ground-truth poses, decimated to about 10 Hz, are corrupted with synthetic noise of known statistics.

Frozen convention: pose measurement, AGENTS.md C6

$$p_{meas} = p_{wb} + n_p, \qquad q_{meas} = q_{wb} \otimes \mathrm{Exp}_q(n_\theta), \qquad R_z = \mathrm{diag}(\sigma_p^2 I_3,\ \sigma_\theta^2 I_3). \tag{18.5}$$

The attitude noise is applied on the right, in the body frame, which is exactly the local perturbation. That choice makes the residual (14.3) statistically clean: if the nominal state were exact, the attitude residual would be exactly n_θ. To generate the noise, draw n_p and n_θ from seeded `std::mt19937_64` generators (AGENTS.md E5) with the same σ_p, σ_θ that appear in R_z.

Derivation exercise (project Jacobian): the pose measurement Jacobian $H \in \mathbb{R}^{6\times 15}$

Using (14.4), derive H for $h(x) = (p_{wb}, q_{wb})$ with the residual of AGENTS.md F3.

1. The position rows are a plain difference. Which state blocks does $\hat{p} + \delta p$ depend on?
2. For the attitude rows, evaluate $\mathrm{Log}_q\left(\hat{q}^{-1} \otimes (\hat{q} \otimes \mathrm{Exp}_q(\delta\theta))\right)$ and simplify. Which elementary Jacobian applies if you instead linearize around a non-zero residual?
3. Write H in the frozen column order and state its nullspace. Which states are *not* directly measured, and how can they still be corrected?

Check: the finite-difference test of $r(\delta x) = h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})$ at a non-trivial state, as required by Milestone 6.

Worked solution: Appendix E.3 (page 281).

How unmeasured states get corrected. The pose sensor measures neither velocity nor biases, yet the update corrects both. The mechanism is the cross-covariance built up by propagation. Between updates, a velocity error produces a growing position error, so P_{pv} becomes large; an accelerometer bias produces a velocity error and then a position error, so P_{pb_a} grows; a gyro bias produces an attitude error, so $P_{\theta b_g}$ grows. The gain $K = PH^\mathsf{T}S^{-1}$ has non-zero rows for every state correlated with the measured ones, so each pose update corrects them in proportion to those correlations.

Why the biases converge. Observability here has an intuitive reading. The gyro bias is the slope of the difference between the integrated gyro and the measured attitude, which a sequence of attitude measurements reveals. The velocity is the slope of the measured position. The accelerometer bias is the difference between the acceleration implied by successive velocities and the specific force rotated with the (measured) attitude, so it too becomes observable. Even at rest it is separable from tilt, because the attitude is measured directly. With pose measurements every state of the IMU error state is observable, and their covariances converge.

18.5 Gating

Real measurements include outliers: a misassociated feature, a reflection, a moving object. A single outlier with a large residual can drag the estimate far away and corrupt the covariance.

The χ^2 test of Proposition 12.3 rejects measurements that are statistically implausible given the filter's own uncertainty:

$$\gamma = r^{\mathsf{T}} S^{-1} r \overset{?}{<} \chi^2_{1-\alpha}(\dim r). \tag{18.6}$$

$\dim r$	1	2	3	4	5	6
$\chi^2_{0.95}$	3.841	5.991	7.815	9.488	11.070	12.592
$\chi^2_{0.99}$	6.635	9.210	11.345	13.277	15.086	16.812

Table 18.1: Gate thresholds. A longer table, up to 600 degrees of freedom, is in Appendix C.

With a 95% gate, a consistent filter rejects 5% of perfectly good measurements. That is the price of rejecting most outliers, and it is why rejections are counted and logged: a rejection rate close to 5% is a sign of health, a rate near zero suggests an underconfident filter (or a gate that is too loose to be useful), and a rate far above 5% indicates overconfidence or a model error.

Pitfall: the death spiral

Gating assumes the filter is consistent. If P is too small, good measurements produce large γ and are rejected; with no corrections P should grow, but if the model's process noise is also too small it grows slowly while the true error grows quickly, and more measurements are rejected. The filter then dead-reckons with false confidence until it diverges. When the rejection rate jumps, suspect overconfidence before suspecting outliers.

The threshold depends on the dimension of r. A pose measurement uses $\chi^2_{0.95}(6)$, a single image observation $\chi^2_{0.95}(2)$, and a projected MSCKF feature residual $\chi^2_{0.95}(2M-3)$, where M is the number of observations (Chapter 26). Because the quantiles grow with the dimension, the table is generated offline and stored as a `constexpr` array, which avoids depending on a statistics library in the core.

18.6 Pseudo-measurements and timing

Pseudo-measurements. Knowledge that is not a sensor reading can be fed to the filter as if it were one. The most useful is the *zero-velocity update* (ZUPT): when the vehicle is known to be at rest (detected, for example, from a very small variance of the accelerometer and gyroscope readings over a window), apply the measurement $z = 0 = v_{wb} + n$ with $H = [0 \;\; I_3 \;\; 0 \;\; 0 \;\; 0]$ and a small noise covariance. It stops velocity drift, and through the correlations it also refines tilt and biases. A ZUPT applied when the vehicle is in fact moving is a large, confident, wrong measurement; the detection must be conservative. Similar pseudo-measurements express a known constant altitude, a non-holonomic constraint (a car does not slide sideways), or a known landmark.

Measurement timing. A measurement taken at time t_z must be applied to the state at t_z, after propagating exactly to that time (Chapter 16). Measurements that arrive *late*, after the

filter has already propagated past their timestamp, are a problem for every filter: either the filter buffers the IMU data and re-propagates from t_z, or it uses a stochastic clone of the state at t_z (Chapter 25), which is exactly the MSCKF's approach for camera measurements. On EuRoC, data are processed offline in timestamp order, so late measurements do not occur.

18.7 Testing the update

Project link: Milestone 6

The exit criteria test properties that hold for any correct implementation:

- H passes the finite-difference test at a non-trivial state;
- $P_{prior} - P_{post}$ is positive semi-definite (an update never adds uncertainty);
- after injection and reset all quaternions are unit, and a second update with the same z produces a residual consistent with the first correction;
- as $R_z \to 0$ the posterior pose converges to z;
- with constant synthetic biases, the bias estimates converge to within their own 3σ;
- update, injection and reset do not allocate;
- a full MH_01 run with noisy ground-truth updates neither diverges nor produces NaN, and the χ^2 rejection count is logged.

The "second update" test is subtle and worth understanding. After the first update, the nominal pose has moved toward z by Kr, so the second residual should be approximately $(I - HK)r$, which is smaller. If it is instead larger, or has the opposite sign, the injection used the wrong side of the quaternion product or the wrong sign of H.

Key takeaways

- Derive H by perturbing $h(\hat{x} \boxplus \delta x)$, expanding to first order and reading off coefficients.
- The nullspace of H lists the error directions a sensor cannot see.
- The pose sensor's attitude noise is a local perturbation, matching the residual $\mathrm{Log}_q(\hat{q}^{-1} \otimes q_{meas})$.
- Unmeasured states are corrected through cross-covariances built up by propagation.
- Gate with $\chi^2_{0.95}(\dim r)$, log the rejections, and treat a high rejection rate as a symptom of overconfidence.

Exercises

18.1 For the lever-arm example, compute S and K for a state with

$$P = \mathrm{diag}(\sigma_p^2 I,\ \sigma_v^2 I,\ \sigma_\theta^2 I,\ \sigma_{b_a}^2 I,\ \sigma_{b_g}^2 I).$$

Which components of $\delta\theta$ does the update correct?

18.2 Derive H for a range measurement to a known beacon b_w: $z = \|p_{wb} - b_w\| + n$. What is its nullspace?

18.3 Simulate the death spiral: run a 1D constant-velocity filter with Q underestimated by a factor of 100 and a 95% gate. Plot the rejection rate and the error over time.

18.4 Show that with a pose measurement of a static vehicle, the horizontal accelerometer bias is observable. (Hint: what does the attitude measurement remove from the tilt/bias ambiguity of Chapter 16?)

18.5 Prove that $P_{prior} - P_{post} = KSK^\mathsf{T} \succeq 0$ for the optimal gain, and explain why this test can fail in floating point if the simple covariance update is used.

Chapter 19

Consistency and Evaluation

In this chapter

- Test whether a filter's covariance tells the truth, with NEES, NIS and 3σ coverage.
- Compute exact statistical bounds for single-run and Monte Carlo tests.
- Diagnose over- and under-confidence and tune noise parameters without cheating.
- Measure trajectory accuracy with ATE and RPE, using the alignment appropriate for VIO.

A filter produces two things: an estimate and a covariance. Most evaluations look only at the first. But a filter with good average accuracy and a covariance that is too small is dangerous: it rejects valid measurements, it reports false confidence to whatever consumes its output, and it eventually diverges. This chapter is about checking the second output.

19.1 What consistency means

Recall from Chapter 5 that a filter reporting $(\hat{x}_k, P_k)$ is consistent if its error $e_k = x_k \boxminus \hat{x}_k$ has zero mean and covariance P_k. Together with Proposition 12.3, a consistent, correctly modeled filter satisfies three testable properties:

1. estimation errors are compatible with P_k (tested with NEES, needs ground truth);
2. innovations are compatible with S_k (tested with NIS, needs no ground truth);
3. innovations are white (tested with autocorrelation, needs no ground truth).

The error must be computed with the manifold difference $\boxminus$, using exactly the same convention as the filter's covariance. Comparing a local-perturbation covariance with a global attitude error, or with an Euler angle difference, produces meaningless statistics.

19.2 Normalized estimation error squared

Definition 19.1 (NEES). The normalized estimation error squared at time k is

$$\epsilon_k = e_k^\mathsf{T} P_k^{-1} e_k, \qquad e_k = x_k \boxminus \hat{x}_k. \tag{19.1}$$

If the filter is consistent and its errors are Gaussian, $\epsilon_k \sim \chi^2_n$ with $n = \dim e$ by Theorem 5.17: its expected value is n. A single run gives one sample per time step, which is noisy. Monte Carlo runs average it out. With N independent runs, the average NEES $\bar{\epsilon}_k = \frac{1}{N}\sum_{j=1}^{N} \epsilon_k^{(j)}$

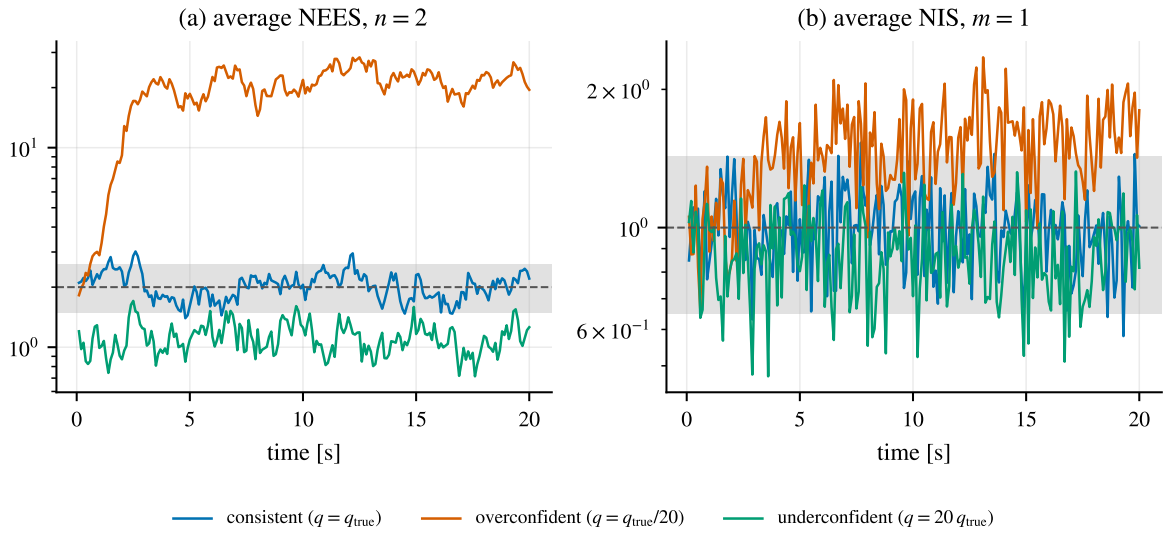


Figure 19.1: Consistency statistics of a constant-velocity filter over 50 Monte Carlo runs, with the 95% acceptance region (gray) of (19.2). Underestimating the process noise by a factor of 20 makes the filter overconfident: the NEES leaves the region by an order of magnitude, and the NIS rises above it. Overestimating it makes the filter underconfident: the NEES falls below the region. Note that the NIS of the underconfident filter is less clearly abnormal: NIS alone is a weak detector of underconfidence.

satisfies $N\bar{\epsilon}_k \sim \chi^2_{Nn}$, so the two-sided 95% interval is

$$\left[\frac{\chi^2_{0.025}(Nn)}{N}, \frac{\chi^2_{0.975}(Nn)}{N} \right]. \tag{19.2}$$

For the 15-state IMU filter with $N = 50$ runs this is $[13.52, 16.56]$; for $N = 100$, $[13.95, 16.09]$. Appendix C lists more. Average NEES above the interval means overconfidence (errors larger than claimed); below means underconfidence.

Block NEES. The NEES of the full state mixes well-estimated and poorly estimated blocks. Computing it separately for position, velocity, attitude and biases, each with its own 3×3 block of P and χ^2_3 bounds, points directly at the misbehaving part.

19.3 Normalized innovation squared

Definition 19.2 (NIS). $\nu_k = r_k^{\mathsf{T}} S_k^{-1} r_k$, the same quantity the gate computes.

Under consistency $\nu_k \sim \chi^2_m$ with $m = \dim r_k$. NIS needs no ground truth, so it is the main consistency monitor on real data. Two summaries are useful: the fraction of updates with ν_k below its 95% quantile, which should be close to 95% (Milestone 7), and the time-average of ν_k, which should be close to m. If measurements have different dimensions, normalize by m_k or compare each against its own quantile.

NEES	NIS	Likely causes
high	high	process noise too small; wrong noise units (missing Δt or $\sqrt{\Delta t}$); unmodeled IMU errors; integration error
high	normal	errors in directions the measurement does not see (unobservable states, e.g. yaw in VIO); wrong $\boxminus$ convention in the evaluation
normal	high	measurement noise too small; outliers passing the gate; wrong H
low	low	process or measurement noise too large
high, growing	normal	spurious information along unobservable directions (EKF inconsistency, Chapter 29)
spikes	spikes	time-synchronization errors; wrong extrinsics; bad initialization

Table 19.1: Reading consistency statistics. Combinations narrow the search for the bug.

3σ **coverage.** For each scalar component i, a consistent Gaussian error lies within $\pm 3\sqrt{P_{ii}}$ 99.7% of the time. Plotting errors against their 3σ envelopes is the most readable diagnostic, and the coverage fraction is a simple scalar summary. On EuRoC, Milestone 7 requires at least 95% coverage for velocity and biases. These states are not measured by the pose sensor, so their consistency is not circular.

19.4 Diagnosing inconsistency

Table 19.1 is a starting point for debugging. Two additional checks are valuable. First, plot each residual component over time: a non-zero mean reveals a bias (wrong extrinsics, a wrong sign in h); slow oscillation reveals correlated errors (a coloured noise the filter assumes white). Second, compare the covariance with Monte Carlo statistics on synthetic data, where the truth is known exactly and the noise is exactly as modeled. If the filter is inconsistent on synthetic data, the bug is in the filter. If it is consistent there but not on real data, the problem is the model.

19.4.1 Tuning without cheating

1. **Synthetic first.** With synthetic data generated with exactly the filter's noise parameters, there is nothing to tune: a consistent implementation must pass. Any failure is a bug.
2. **Physically meaningful knobs only.** On real data, adjust the IMU noise densities (start from Table 15.2, inflate modestly for unmodeled effects) and the measurement noise (start from the pixel noise of the detector). Do not add arbitrary noise to arbitrary states.
3. **Judge on consistency, not accuracy.** Choose parameters that make NIS and coverage statistics correct. Lower trajectory error obtained by making the filter inconsistent will not generalize.
4. **Separate tuning from evaluation.** Tune on some sequences and report results on others.
5. **Ground truth never enters the filter** except where explicitly allowed and logged (initialization).

19.5 Trajectory accuracy

Consistency says whether the covariance is honest; accuracy says how small the errors are. VIO accuracy is reported with two metrics [50, 56].

19.5.1 Absolute trajectory error

The estimated trajectory lives in a world frame that is only defined up to the unobservable directions, so it must be aligned with the ground truth before comparing. Given time-associated positions p_i (ground truth) and $\hat{p}_i$ (estimate), find the transformation that best maps the estimate onto the truth, then compute

$$\mathrm{ATE}_{\mathrm{RMSE}} = \left(\frac{1}{N} \sum_{i=1}^{N} \left\| p_i - (R^\star \hat{p}_i + t^\star) \right\|^2 \right)^{1/2}. \tag{19.3}$$

For rigid alignment, the optimal transformation has a closed form (Kabsch/Umeyama [53]).

Proposition 19.3 (Optimal rigid alignment). *Let μ and $\hat{\mu}$ be the means of the two point sets, $a_i = p_i - \mu$ and $b_i = \hat{p}_i - \hat{\mu}$ the centered points, and $\Sigma_{ab} = \frac{1}{N} \sum_i a_i b_i^\mathsf{T} = U D V^\mathsf{T}$ an SVD. The minimizer of $\sum_i \|p_i - (R\hat{p}_i + t)\|^2$ over rotations R and translations t is*

$$R^\star = U \operatorname{diag}\left(1, 1, \det(UV^\mathsf{T})\right) V^\mathsf{T}, \qquad t^\star = \mu - R^\star \hat{\mu}. \tag{19.4}$$

Proof. For fixed R, the cost is a quadratic in t minimized where its gradient vanishes, $\sum_i (p_i - R\hat{p}_i - t) = 0$, i.e. $t = \mu - R\hat{\mu}$: the alignment maps centroid onto centroid. Substituting, the cost becomes $\sum_i \|a_i - R b_i\|^2 = \sum_i (\|a_i\|^2 + \|b_i\|^2) - 2 \sum_i a_i^\mathsf{T} R b_i$, so we must maximize $\sum_i a_i^\mathsf{T} R b_i = \operatorname{tr}\left(R \sum_i b_i a_i^\mathsf{T}\right) = N \operatorname{tr}(R\, V D U^\mathsf{T}) = N \operatorname{tr}(D\, Z)$ with $Z = U^\mathsf{T} R V$. Z is orthogonal, so its diagonal entries are at most one in magnitude, and $\operatorname{tr}(DZ) = \sum_i d_i Z_{ii} \le \sum_i d_i$, with equality for $Z = I$, i.e. $R = UV^\mathsf{T}$. If $\det(UV^\mathsf{T}) = -1$ this is a reflection, not a rotation; the best rotation then flips the axis with the smallest singular value, $Z = \operatorname{diag}(1, 1, -1)$, which gives the stated formula. ∎

If the scale is also free (a similarity transform, needed for monocular vision without an IMU), the optimal scale is $s^\star = \operatorname{tr}(D \operatorname{diag}(1, 1, \det(UV^\mathsf{T}))) \big/ \frac{1}{N} \sum_i \|b_i\|^2$, applied as $s^\star R^\star \hat{p}_i + t^\star$ with $t^\star = \mu - s^\star R^\star \hat{\mu}$.

Key idea: align only what is unobservable

Which transformation to allow depends on the estimator. Monocular vision alone has a free scale, so it uses a similarity transform (7 DOF). VIO has metric scale and observable roll and pitch: its unobservable directions are exactly translation and yaw. The appropriate alignment is therefore 4-DOF (translation plus a rotation about gravity) [56]. A full 6-DOF alignment would also remove roll and pitch errors that the estimator was responsible for.

The 4-DOF alignment follows the same proof with R restricted to rotations $R_z(\psi)$ about the vertical. For centered points $a = (x, y, z)$ (truth) and $b = (\hat{x}, \hat{y}, \hat{z})$ (estimate), $a^\mathsf{T} R_z(\psi) b =$

$(x\hat{x} + y\hat{y})\cos\psi + (y\hat{x} - x\hat{y})\sin\psi + z\hat{z}$, whose sum over the points is maximized by

$$\psi^\star = \operatorname{atan2}\Big(\sum_i (y_i\hat{x}_i - x_i\hat{y}_i),\ \sum_i (x_i\hat{x}_i + y_i\hat{y}_i)\Big), \tag{19.5}$$

again with $t^\star = \mu - R_z(\psi^\star)\hat{\mu}$.

Milestone 10b reports ATE after SE(3) alignment, as in most published benchmarks; reporting the 4-DOF result alongside it is more faithful to what VIO can know.

19.5.2 Relative pose error

ATE is dominated by the largest drift and depends on trajectory length and shape. The relative pose error (RPE) compares relative motions over a fixed distance or time Δ:

$$E_i = \left(T_i^{-1}T_{i+\Delta}\right)^{-1}\left(\hat{T}_i^{-1}\hat{T}_{i+\Delta}\right), \tag{19.6}$$

and reports statistics of the translation and rotation parts of E_i, often as a percentage of distance traveled. It needs no global alignment and measures local drift, which is what odometry is about. The evo package [21] computes both metrics with all the alignment variants.

Project link: Milestone 7

M7 logs $\hat{x}$, diag(P), ground truth, NIS and NEES to a preallocated buffer, flushes it outside the hot loop, and plots 3σ envelopes and NEES/NIS against their χ^2 bounds from a single script. Exit criteria: average NEES inside the 95% interval for 15 DOF over most of a $\geq$ 50-run synthetic Monte Carlo; NIS within its per-update bound for about 95% of updates; at least 95% 3σ coverage of velocity and biases on MH_01; zero allocations while logging.

Key takeaways

- A filter must be judged on the honesty of its covariance as well as its accuracy.
- NEES needs ground truth and tests P; NIS needs none and tests S. Both are χ^2 under consistency.
- Monte Carlo averages have exact bounds $[\chi^2_{0.025}(Nn), \chi^2_{0.975}(Nn)]/N$.
- Overconfidence (NEES too high) is the dangerous failure; NIS alone detects underconfidence poorly.
- Tune physically meaningful parameters for consistency, on synthetic data first.
- Report ATE after an alignment matched to the unobservable directions, and RPE for local drift.

Exercises

19.1 Show that the single-run NEES of a consistent 15-state filter exceeds its 95% quantile 5% of the time. Why is the Monte Carlo average a much sharper test?

19.2 Reproduce Figure 19.1. Then make only the measurement noise wrong instead of the process noise and compare the signatures.

19.3 Derive the closed-form 4-DOF alignment above by maximizing $\sum_i p_i^{\mathsf{T}} R_z(\psi) \hat{p}_i$ over ψ.

19.4 Explain why the NEES of yaw in a VIO run on real data tends to grow over time even when the NIS looks healthy. (Revisit after Chapter 29.)

19.5 Compute the RPE over 10 m segments for a trajectory with a constant yaw drift of 1° per minute flying at 1 m/s. How does it compare with the ATE after a 10-minute flight?

Part V

Computer Vision

Chapter 20

Image Formation and Camera Models

In this chapter

- Understand how a camera forms an image: pinholes, lenses, focus, sensors, exposure and shutters.
- Derive the pinhole projection, the intrinsic matrix and normalized image coordinates.
- Use homogeneous coordinates and the basic projective geometry of points, lines and homographies.
- Model and invert the radial-tangential lens distortion used by EuRoC, and know the fisheye alternative.
- Understand how cameras are calibrated.
- State the project's camera measurement model with its noise, derive the projection Jacobian, and understand extrinsics and time offsets.

20.1 How a camera forms an image

20.1.1 From pinhole to lens

A camera measures the *directions* of light rays. In a *camera obscura*, light from the scene enters a dark box through a tiny hole and forms an inverted image on the opposite wall: every point of the wall receives light only from the scene point on the straight line through the hole. This is the *pinhole model*, and it is geometrically perfect. It is also impractical: a hole small enough to give a sharp image lets in very little light.

A lens gathers light through a large aperture and bends it so that all rays from one scene point converge again at one image point. For an ideal *thin lens* of focal length f_L, a point at distance z_o in front of the lens is imaged sharply at distance z_i behind it, where

$$\frac{1}{f_L} = \frac{1}{z_o} + \frac{1}{z_i}. \tag{20.1}$$

For distant scenes ($z_o \gg f_L$) the image forms at $z_i \approx f_L$, which is where the sensor sits in a camera focused at infinity. Points at other distances form slightly blurred spots; the range of distances that stays acceptably sharp is the *depth of field*, larger for small apertures and short focal lengths. The central ray through the lens center is not bent, so the geometry of a focused lens camera is exactly that of a pinhole at the lens center: the pinhole model describes it, and lens imperfections are added as *distortion* (Section 20.4).

20.1.2 The image sensor

The sensor is a grid of light-sensitive *pixels*. Each accumulates charge in proportion to the light it receives during the *exposure time*, and the charge is converted into a number, typically 8 bits per pixel for the monochrome cameras of EuRoC. Several properties of the sensor matter to an estimator.

- *Exposure and motion blur.* A camera rotating at ω rad/s during an exposure of T seconds smears each point over about $f\omega T$ pixels, with f the focal length in pixels. At 1 rad/s, 5 ms and $f = 458$ px this is 2.3 px, enough to degrade feature tracking. EuRoC's difficult sequences contain exactly such blur.
- *Global versus rolling shutter.* A global shutter exposes all pixels at the same instant, as EuRoC's cameras do. A rolling shutter (common in phones) exposes rows one after another, so each row is captured at a different time and, on a moving camera, from a different pose. Rolling-shutter images need an extended measurement model.
- *Noise and dynamic range.* Pixel values carry photon and electronic noise, and scenes brighter or darker than the sensor's range saturate. Auto exposure changes the brightness of the same scene point between frames, which matters for intensity-based tracking (Chapter 22).

Pixel coordinates. We write pixel coordinates as (u, v), with u increasing to the right and v increasing downward, and the origin at the top-left of the image. In OpenCV's convention, integer coordinates are pixel *centers*. The camera frame C is aligned with this: x_c to the right, y_c down, z_c along the optical axis into the scene.

20.2 The pinhole projection

An ideal pinhole camera lets light through a single point, the optical center. A scene point with camera-frame coordinates $p_c = (X, Y, Z)$, with Z along the optical axis, projects through the center onto an image plane. Placing a virtual image plane at unit distance *in front of* the center (which avoids the inversion of the physical image) and using similar triangles, the image of the point is

$$\pi(p_c) = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} X/Z \\ Y/Z \end{bmatrix}. \tag{20.2}$$

These are the *normalized image coordinates*: dimensionless, independent of the lens, and equal to the tangents of the ray's angles from the optical axis. The vector $\bar{x} = (x, y, 1)$ is the *bearing* of the point, defined up to scale.

20.2.1 Intrinsic parameters

Pixels are obtained by scaling and shifting the normalized coordinates with the *intrinsic matrix*:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \qquad K = \begin{bmatrix} f_u & s & c_u \\ 0 & f_v & c_v \\ 0 & 0 & 1 \end{bmatrix}, \tag{20.3}$$

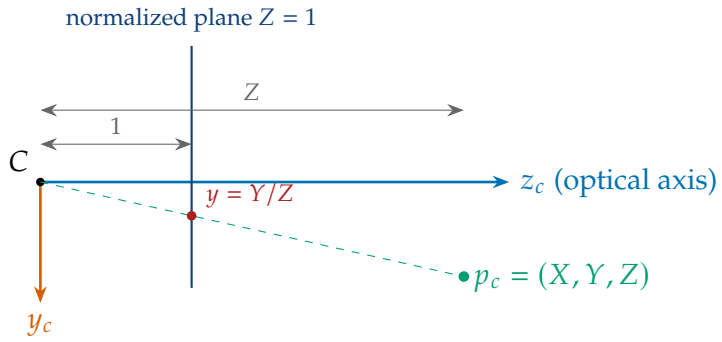


Figure 20.1: Pinhole projection in the y-z plane. The ray from the optical center C to the point intersects the plane $Z = 1$ at height Y/Z. All points along the ray project to the same image point, so depth is lost.

with focal lengths f_u, f_v in pixels, principal point (c_u, c_v) (where the optical axis pierces the image), and skew s (zero for modern sensors). The focal length in pixels is the physical focal length divided by the pixel pitch; $f_u \neq f_v$ if pixels are not square. Inverting K turns a pixel into a bearing, $\bar{x} = K^{-1}(u, v, 1)^\mathsf{T}$. The horizontal *field of view* of a W-pixel-wide image is $2 \arctan\left(W/(2f_u)\right)$, ignoring distortion. EuRoC's `cam0` has $f_u = 458.654$, $f_v = 457.296$, $c_u = 367.215$, $c_v = 248.375$ on a 752×480 sensor: a horizontal field of view of about 79°.

20.3 Homogeneous coordinates and projective geometry

20.3.1 Points and lines in the image

The image point (x, y) corresponds to the whole ray of 3D vectors $\lambda(x, y, 1)$, $\lambda \neq 0$. *Homogeneous coordinates* make this explicit: a point of the plane is represented by any non-zero 3-vector $\tilde{x} = (x_1, x_2, x_3)$, with $\tilde{x}$ and $\lambda\tilde{x}$ representing the same point, and the ordinary coordinates are recovered as $(x_1/x_3, x_2/x_3)$. Vectors with $x_3 = 0$ represent *points at infinity*: directions, where parallel lines meet.

A line $ax + by + c = 0$ is represented by the 3-vector $l = (a, b, c)$, also up to scale, and a point lies on it exactly when

$$l^\mathsf{T}\tilde{x} = 0. \tag{20.4}$$

This symmetric relation makes two constructions one-liners:

- the line through two points is $l = \tilde{x}_1 \times \tilde{x}_2$ (it is orthogonal to both, so both lie on it);
- the intersection of two lines is $\tilde{x} = l_1 \times l_2$ (by the same argument).

Two parallel lines (a, b, c_1) and (a, b, c_2) intersect at $(b, -a, 0)$ times a constant: a point at infinity in their common direction. In an image of a 3D scene, parallel 3D lines are generally not parallel in the image and meet at a finite *vanishing point*, the image of their common direction.

20.3.2 Homographies

Definition 20.1 (Homography)**.** A *homography* (projective transformation) of the plane is a map $\tilde{x} \mapsto H\tilde{x}$ for an invertible 3×3 matrix H, defined up to scale (eight degrees of freedom).

Homographies map lines to lines (if $l^{\mathsf{T}}\tilde{x} = 0$ then $(H^{-\mathsf{T}}l)^{\mathsf{T}}(H\tilde{x}) = 0$) but do not preserve parallelism, lengths or angles. Two situations in which they arise are central to vision:

- the images of a *plane* in two views are related by a homography (Chapter 23), which is the basis of camera calibration below;
- two images taken by a camera that only *rotates* are related by the homography KRK^{-1}, independently of the scene (Chapters 22 and 23).

20.3.3 The projection matrix

In homogeneous coordinates the complete projection of a world point p_w (with homogeneous coordinates $\tilde{p}_w = (p_w, 1)$) into pixels is a single linear map,

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R_{cw} & p_{cw} \end{bmatrix} \tilde{p}_w = P\,\tilde{p}_w, \qquad P = K\,[R_{cw} \;\; p_{cw}] \in \mathbb{R}^{3\times 4}, \tag{20.5}$$

where (R_{cw}, p_{cw}) maps world coordinates into the camera frame and $\lambda = Z$ is the depth. The *camera matrix* P has eleven degrees of freedom (twelve entries up to scale): five intrinsic and six extrinsic. Linear methods that estimate P, H or the essential matrix from correspondences (Chapter 23) all exploit the fact that projection is linear in homogeneous coordinates.

20.4 Lens distortion

Real lenses bend rays, most strongly near the image edges, so the image of a straight line is curved. The *radial-tangential* (Brown–Conrady) model used by EuRoC maps ideal normalized coordinates (x, y) to distorted ones (x_d, y_d):

$$x_d = x\,(1 + k_1 r^2 + k_2 r^4) + 2p_1 xy + p_2(r^2 + 2x^2), \tag{20.6}$$

$$y_d = y\,(1 + k_1 r^2 + k_2 r^4) + p_1(r^2 + 2y^2) + 2p_2 xy, \qquad r^2 = x^2 + y^2, \tag{20.7}$$

after which K is applied to (x_d, y_d). The *radial* terms (k_1, k_2) model the dominant symmetric distortion of a lens; the *tangential* terms (p_1, p_2) model a lens slightly tilted or decentered relative to the sensor. For `cam0`, $(k_1, k_2, p_1, p_2) = (-0.2834, 0.07396, 1.94 \times 10^{-4}, 1.76 \times 10^{-5})$. The negative k_1 produces *barrel* distortion (Figure 20.2). At the middle of the left and right image edges the radial factor $1 + k_1 r^2 + k_2 r^4$ is about 0.79, which moves points by about 100 pixels; at the corners it is about 0.73, a shift of about 160 pixels. Ignoring distortion is therefore not an option.

Undistortion. The front end measures distorted pixels and needs ideal normalized coordinates. The model has no closed-form inverse, but a fixed-point iteration converges quickly

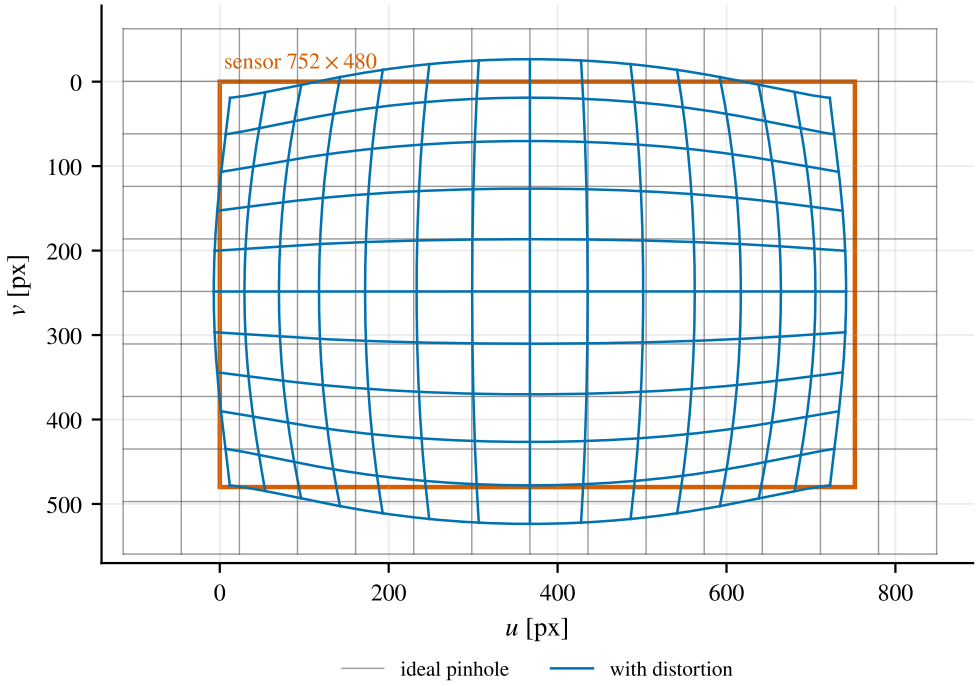


Figure 20.2: EuRoC `cam0` lens distortion. A regular grid in normalized coordinates (gray) is imaged by the real lens as the curved blue grid. Distortion pulls points toward the center, increasingly toward the edges, so the sensor (orange) sees a wider field than an ideal pinhole with the same focal length would.

for moderate distortion: starting from $x^{(0)} = x_d$,

$$x^{(j+1)} = \frac{x_d - \Delta_x(x^{(j)}, y^{(j)})}{1 + k_1 r_j^2 + k_2 r_j^4}, \qquad y^{(j+1)} = \frac{y_d - \Delta_y(x^{(j)}, y^{(j)})}{1 + k_1 r_j^2 + k_2 r_j^4}, \tag{20.8}$$

where Δ denotes the tangential terms. A Gauss–Newton iteration (Chapter 10) with the 2×2 Jacobian of the distortion converges faster and more reliably near the edges. OpenCV's `cv::undistortPoints` implements the iterative scheme. Milestone 8 checks undistortion by distorting and undistorting a grid of points.

Key idea: undistort points, not images

The estimator needs a few hundred feature locations, not a rectified image. Undistorting only the tracked points costs microseconds; remapping a full image costs milliseconds, resamples the pixels (blurring them), and changes the noise of the feature detector in a position-dependent way. Track in the raw image and undistort the tracks.

Wide-angle lenses. Beyond about 120° of field of view, the pinhole model itself breaks down (a ray at 90° from the axis would have infinite normalized coordinates). *Fisheye* models parameterize the image radius as a function of the angle ϑ of the incoming ray, for example the equidistant (Kannala–Brandt) model $r_d = \vartheta(1 + k_1\vartheta^2 + k_2\vartheta^4 + \cdots)$. The rest of the pipeline is unchanged: everything downstream of undistortion works with bearings, which can be represented as unit vectors when they are far from the optical axis.

20.5 Camera calibration

How were EuRoC's intrinsics and distortion coefficients obtained? By observing a known calibration target (a checkerboard or grid of markers) in many images and fitting the camera model to the observed corner positions.

Zhang's method. Place the target in the plane $Z = 0$ of its own frame. A target point $(X, Y, 0)$ then projects as $\lambda\tilde{u} = K[r_1\ r_2\ r_3\ t](X, Y, 0, 1)^\mathsf{T} = K[r_1\ r_2\ t](X, Y, 1)^\mathsf{T}$: each image of the target is a homography $H = [h_1\ h_2\ h_3] \propto K[r_1\ r_2\ t]$, estimable from four or more corner correspondences (Chapter 23). Because r_1 and r_2 are orthonormal, $r_1 = \lambda K^{-1}h_1$ and $r_2 = \lambda K^{-1}h_2$ give two constraints on the matrix $\omega = K^{-\mathsf{T}}K^{-1}$:

$$h_1^\mathsf{T}\omega h_2 = 0, \qquad h_1^\mathsf{T}\omega h_1 = h_2^\mathsf{T}\omega h_2. \tag{20.9}$$

These are linear in the entries of the symmetric ω, so three or more images determine it; K follows from a Cholesky factorization of ω. The linear estimate is then refined, together with the distortion coefficients and each image's pose, by Levenberg–Marquardt (Chapter 10) minimizing the *reprojection error*, the distance between observed and predicted corners. The residual reprojection error after calibration, typically a few tenths of a pixel, is the standard measure of calibration quality.

Camera–IMU calibration. The extrinsics T_{bc} and the time offset are calibrated jointly with the IMU, by moving the rig in front of the target (Chapter 15).

20.6 The measurement model

Frozen convention: camera model, AGENTS.md C6

In normalized, undistorted image coordinates,

$$p_c = R_{bc}^\mathsf{T}\big(R_{wb}^\mathsf{T}(p_f - p_{wb}) - p_{bc}\big), \qquad z = \begin{bmatrix} p_{c,x}/p_{c,z} \\ p_{c,y}/p_{c,z} \end{bmatrix} + n_{uv}, \qquad \sigma_{uv} \approx \sigma_{px}/f. \tag{20.10}$$

The first equation chains the world-to-body and body-to-camera transformations of (7.20). The noise model follows from $u = f_u x + c_u$: a pixel error δu corresponds to a normalized error $\delta u / f_u$. With $\sigma_{px} = 1$ px and $f \approx 458$ px, $\sigma_{uv} \approx 2.2 \times 10^{-3}$. Strictly, distortion makes the mapping from pixel noise to normalized noise position-dependent, by the inverse of the distortion Jacobian, and $f_u \neq f_v$. Both effects are small for EuRoC and conventionally ignored, but they are a legitimate refinement.

20.7 The projection Jacobian

Proposition 20.2 (Jacobian of the pinhole projection)**.**

$$\frac{\partial \pi}{\partial p_c} = \frac{1}{Z}\begin{bmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{bmatrix} = \frac{1}{Z}\begin{bmatrix} I_2 & -\pi(p_c) \end{bmatrix}. \tag{20.11}$$

Proof. Differentiate X/Z and Y/Z with the quotient rule: $\partial(X/Z)/\partial X = 1/Z$, $\partial(X/Z)/\partial Z = -X/Z^2$, and similarly for Y/Z. ∎

The Jacobian has rank two, and its nullspace is spanned by p_c itself: moving the point along its viewing ray does not change its image. This is the single-view loss of depth in differential form, and it is why a feature must be observed from at least two sufficiently different viewpoints before it constrains anything (Chapter 23). The factor $1/Z$ says that distant points have small image motion for a given change in position. They constrain rotation well and translation poorly.

The full measurement Jacobian with respect to the filter state is the chain-rule product of this matrix with the Jacobian of p_c with respect to the clone pose and the feature position. Deriving those is part of Chapter 26.

Pitfall: points behind the camera

(20.2) happily projects points with $Z < 0$, flipping their image through the center. A triangulation or linearization that produces a negative depth must be rejected (the *cheirality* check), not projected. Points with very small positive Z produce huge Jacobians and should be rejected too.

20.8 Extrinsics and time offset

Extrinsic calibration. The transformation T_{bc} between the IMU and the camera is estimated offline, for EuRoC with Kalibr [17], and stored as `T_BS` in `cam0/sensor.yaml`. Its rotation is close to a permutation of axes, and its translation is a few centimeters (Appendix D). An error of $\delta\phi$ in the extrinsic rotation biases every predicted bearing by about $\delta\phi$, which looks to the filter like a systematic residual. A translation error matters mostly for near points. The project treats T_{bc} as exact (AGENTS.md C3). Estimating it online is a stretch goal.

Time offset. If the camera timestamp differs from the IMU clock by t_d, a feature moving across the image at velocity $\dot{z}$ is measured with an error of about $\dot{z}\, t_d$. A 1 ms offset at an image-plane velocity of 0.5 normalized units per second (fast rotation) is an error of 5×10^{-4}, a quarter of a pixel. EuRoC is hardware synchronized, so $t_d \approx 0$; consumer devices can have offsets of tens of milliseconds and must estimate it online [35].

Key takeaways

- A focused lens camera has the geometry of a pinhole at the lens center; lens imperfections are modeled as distortion.

- Exposure time causes motion blur of about $f\omega T$ pixels; EuRoC's global shutter exposes all pixels at once.
- Normalized coordinates $(X/Z, Y/Z)$ are lens-independent bearings; K maps them to pixels.
- Homogeneous coordinates make projection linear; lines through points and intersections of lines are cross products; homographies map planes and pure rotations between views.
- EuRoC's lens has strong barrel distortion; undistort the tracked points, not the images.
- Cameras are calibrated by fitting the model to images of a known target, minimizing reprojection error.
- $\partial\pi/\partial p_c = \frac{1}{Z}[I_2 \;\; -\pi(p_c)]$; its nullspace is the viewing ray.

Exercises

20.1 A lens with $f_L = 4$ mm is focused at infinity. How far must the sensor move to focus on an object at 0.5 m?

20.2 Compute the horizontal and vertical fields of view of `cam0` with and without distortion.

20.3 Compute the intersection of the lines $x + y = 1$ and $x - y = 0$ with homogeneous coordinates, and the line through $(1, 2)$ and $(3, 1)$.

20.4 Show that two parallel lines intersect at a point at infinity, and find the vanishing point of the image of a family of parallel 3D lines with direction d under the camera $P = K[I \;\; 0]$.

20.5 Implement distortion and fixed-point undistortion for the EuRoC model. How many iterations are needed for 10^{-6} accuracy at the image corners? Compare with Gauss–Newton.

20.6 Derive the two linear constraints of Zhang's method from the orthonormality of r_1 and r_2.

20.7 A point at depth 10 m is observed by a camera translating sideways at 1 m/s and rotating at 1 rad/s. Which motion dominates its image velocity? What does this imply for the observability of translation from distant points?

Chapter 21

Digital Image Processing

In this chapter

- Treat an image as a sampled function, and filter it by convolution.
- Smooth images with Gaussian kernels, and understand separability and cost.
- Compute image gradients robustly with derivative-of-smoothing kernels such as Sobel and Scharr.
- Understand why images must be blurred before they are down-sampled, and build Gaussian pyramids.
- Interpolate images at sub-pixel positions, find local maxima, and refine them to sub-pixel accuracy.
- Know how photometric changes affect intensity-based methods.

Feature detection and tracking (Chapter 22) operate on raw pixel arrays: they smooth them, differentiate them, shrink them and sample them between pixels. This chapter develops these operations from first principles, as far as the front end needs them.

21.1 Images as functions

A grayscale image is a function $I(u, v)$ giving the brightness at image position (u, v), sampled on the integer grid of pixel centers and quantized, typically to integers between 0 and 255. In memory it is a 2D array indexed by row and column, $I[\text{row}, \text{col}] = I(u = \text{col}, v = \text{row})$; mixing up the two orders is a classic bug. Color images have three such channels; a monochrome image can be formed as a weighted sum of them. EuRoC's cameras are monochrome.

Between pixels, I is undefined until we decide how to interpolate it (Section 21.5). Its derivatives must be approximated by differences of nearby pixels (Section 21.3). Both operations are sensitive to noise and to aliasing, which is what the rest of this chapter is about.

21.2 Linear filtering

21.2.1 Correlation and convolution

Linear filtering replaces each pixel by a weighted sum of its neighbors, with the weights given by a small array h, the *kernel*. Two closely related forms exist:

$$\begin{aligned} \text{correlation:} \quad & (h \star I)[u,v] = \sum_{i,j} h[i,j]\, I[u+i, v+j], \\ \text{convolution:} \quad & (h * I)[u,v] = \sum_{i,j} h[i,j]\, I[u-i, v-j]. \end{aligned} \tag{21.1}$$

Convolution flips the kernel; for symmetric kernels the two coincide. Both are *linear* and *shift-invariant*: filtering a shifted image gives the shifted filtered image. Convolution is also commutative and associative, so a cascade of filters equals a single filter, $h_2 * (h_1 * I) = (h_2 * h_1) * I$. Near the image border the kernel extends past the edge, and the missing pixels must be invented: by zeros, by replicating the border, or by reflecting the image. Results within a kernel radius of the border are always less reliable, which is why detectors ignore a margin there.

A $k \times k$ kernel costs k^2 multiplications per pixel. A kernel that factors as an outer product of a column and a row, $h = h_v h_u^{\mathsf{T}}$, is *separable*: filtering with h equals filtering the rows with h_u and then the columns with h_v, at a cost of $2k$ per pixel.

21.2.2 Smoothing

Averaging neighbors reduces noise. The simplest kernel, the *box filter*, averages a $k \times k$ square. The *Gaussian filter* weights neighbors by

$$G_\sigma(u,v) = \frac{1}{2\pi\sigma^2} \exp\!\Big(-\frac{u^2+v^2}{2\sigma^2}\Big) = g_\sigma(u)\, g_\sigma(v), \qquad g_\sigma(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-x^2/2\sigma^2}, \tag{21.2}$$

sampled at integer offsets, truncated at about $\pm 3\sigma$ and renormalized to sum to one. The factorization shows that it is separable. The Gaussian is preferred over the box because it has no sharp edges: its frequency response is itself a Gaussian, which removes high frequencies smoothly without the ripples of the box filter. Cascading two Gaussian filters gives a Gaussian whose variances add, $G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2+\sigma_2^2}}$. Small Gaussians are well approximated by binomial kernels, $\frac{1}{4}[1,2,1]$ and $\frac{1}{16}[1,4,6,4,1]$ in each direction.

21.3 Image derivatives

The *image gradient* $\nabla I = (I_u, I_v)$ points in the direction of fastest brightness increase, and its magnitude measures the local contrast. It is approximated by differences, for example the central difference $I_u[u,v] \approx \frac{1}{2}(I[u+1,v] - I[u-1,v])$, a correlation with $\frac{1}{2}[-1,0,1]$.

Differentiation amplifies noise: a derivative multiplies each frequency component by its frequency, so pixel-to-pixel noise, the highest frequency present, is amplified most. The remedy is to smooth before differentiating, and since both are convolutions, to combine them into one kernel. The *Sobel* operator smooths across the derivative direction with $[1,2,1]$ and

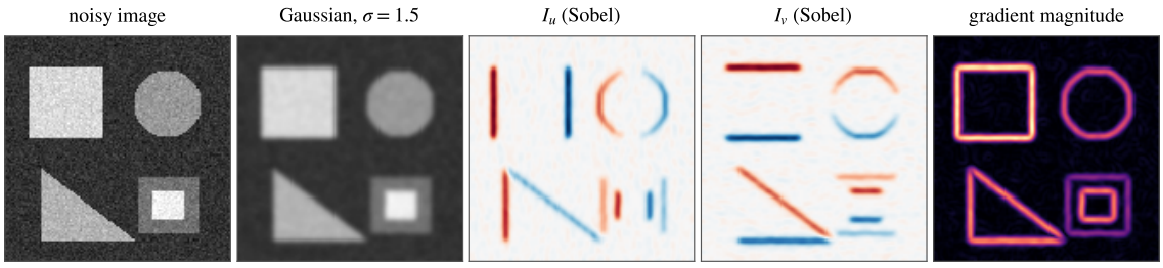


Figure 21.1: A synthetic noisy image, its Gaussian smoothing ($\sigma = 1.5$ px), its Sobel derivatives I_u and I_v, and the gradient magnitude. Vertical edges respond in I_u, horizontal edges in I_v; corners respond in both, which is what corner detectors look for.

differentiates along it:

$$S_u = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad S_v = S_u^{\mathsf{T}}. \tag{21.3}$$

The *Scharr* operator uses $[3, 10, 3]$ instead of $[1, 2, 1]$, which gives gradient directions that are more accurate for all orientations; OpenCV's Lucas–Kanade tracker uses it. More generally, the *derivative of a Gaussian*, $\partial G_\sigma / \partial u$, smooths and differentiates at a chosen scale σ. Whatever the kernel, it is a weighted difference: its entries sum to zero, so it ignores uniform brightness, and its scale must be divided out when absolute gradient values matter (Sobel's sum of positive weights is 4 times the step, so $S_u \star I \approx 8 I_u$).

21.4 Sampling, aliasing and pyramids

21.4.1 Aliasing in images

An image is a sampled signal, and the sampling theorem (Chapter 6) applies in each direction: a pixel grid can represent spatial frequencies up to half a cycle per pixel. Shrinking an image by keeping every other pixel halves the representable frequency. Any finer detail does not vanish: it *aliases* into false coarse patterns (moiré). The cure is the same as for any sampled signal: low-pass filter, by blurring, before down-sampling.

21.4.2 Gaussian pyramids

A *Gaussian pyramid* is a sequence of images, each a blurred and half-resolution copy of the previous one:

$$I_{\ell+1} = \downarrow_2 \left(G * I_\ell \right), \tag{21.4}$$

where G is a small Gaussian (OpenCV's `pyrDown` uses the binomial kernel $\frac{1}{16}[1, 4, 6, 4, 1]$ in each direction) and $\downarrow_2$ keeps every second row and column. Level ℓ has $2^{-\ell}$ of the original resolution in each direction, so a pixel position u_0 at level 0 corresponds to $u_\ell \approx u_0 / 2^\ell$ at level ℓ (up to half-pixel conventions). The whole pyramid costs only a third more memory than the original image ($1 + \frac{1}{4} + \frac{1}{16} + \cdots = \frac{4}{3}$).

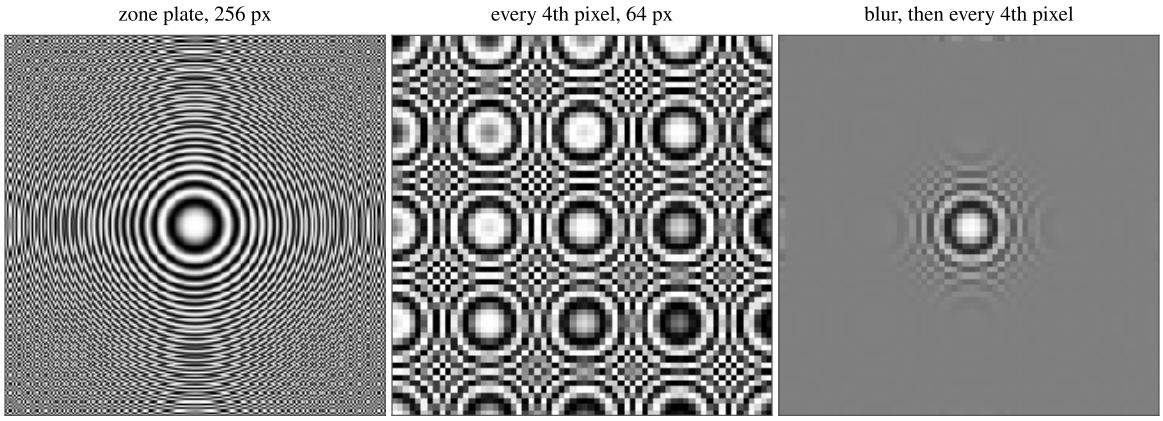


Figure 21.2: Aliasing in images. A "zone plate", whose spatial frequency increases away from the center (left), is shrunk four times by keeping every fourth pixel (middle) and by Gaussian blurring first (right). Naive sub-sampling creates false ring patterns; blurring removes the frequencies that cannot be represented.

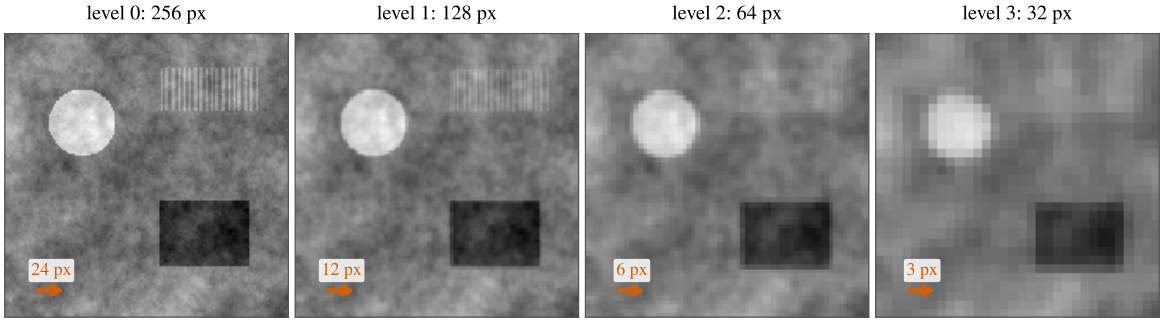


Figure 21.3: Four levels of a Gaussian pyramid of a textured synthetic image, displayed at a common size. Fine texture disappears level by level; coarse structure survives. A displacement of 24 px at level 0 is a displacement of 3 px at level 3.

Pyramids serve two purposes in the front end. Motion that is large in pixels at full resolution is small at coarse levels, where an iterative tracker can capture it (Chapter 22). And structures of different sizes are best detected at different scales.

21.5 Interpolation

Sub-pixel tracking needs image values and gradients at non-integer positions. Writing a position as $(u_0 + a, v_0 + b)$ with integer (u_0, v_0) and fractional parts $a, b \in [0, 1)$:

- *nearest neighbor* takes the closest pixel: fast, but the result jumps as the position moves, which ruins sub-pixel accuracy;
- *bilinear interpolation* interpolates linearly in u along the two neighboring rows and then linearly in v between the results:

$$I(u_0 + a, v_0 + b) = (1 - a)(1 - b)\, I_{00} + a(1 - b)\, I_{10} + (1 - a)b\, I_{01} + ab\, I_{11}, \tag{21.5}$$

with $I_{ij} = I[u_0 + i, v_0 + j]$. The result is continuous, and the weights are the areas of the

rectangles opposite each corner;

- *bicubic* interpolation fits a smooth cubic patch to a 4×4 neighborhood; it has continuous derivatives at the cost of 16 pixels per sample.

Lucas–Kanade trackers use bilinear interpolation for both the image and its gradients. With images represented as 8-bit integers, implementations use fixed-point weights for speed, which limits the attainable sub-pixel resolution to a few hundredths of a pixel, well below the noise of real tracking.

21.6 Finding local maxima

Detectors compute a response map (corner strength, for example) and select the pixels where it is large *and* locally maximal. *Non-maximum suppression* keeps a pixel only if its response exceeds a threshold and is the largest in its neighborhood (a 3×3 window, or a larger radius to enforce spacing between detections). The location of a maximum can then be refined below the pixel grid by fitting a parabola through the response r_{-1}, r_0, r_1 at the maximum and its two neighbors in each direction; the vertex lies at the offset

$$\delta = \frac{r_{-1} - r_1}{2\,(r_{-1} - 2r_0 + r_1)} \tag{21.6}$$

pixels from the center. Sub-pixel accuracy at the detection stage is less important for tracking-based front ends, whose tracker refines positions anyway.

21.7 Photometric changes

Intensity-based methods assume that a scene point keeps its brightness between frames (Chapter 22). Auto-exposure, lighting changes and non-Lambertian surfaces violate this. A common model is an affine change, $I_2 \approx \alpha I_1 + \beta$. Gradients are unaffected by β, and comparing patches after subtracting their means and dividing by their standard deviations (*zero-mean normalized cross-correlation*) is unaffected by both. Some trackers estimate α and β along with the motion; others pre-process images, for example with histogram equalization, to reduce the variation.

Key takeaways

- Filtering is convolution with a kernel; separable kernels cost $2k$ instead of k^2 per pixel.
- Gaussian smoothing reduces noise without ringing; variances of cascaded Gaussians add.
- Derivatives amplify noise; Sobel, Scharr and derivative-of-Gaussian kernels smooth and differentiate at once.
- Down-sampling without blurring aliases; a Gaussian pyramid blurs and halves the resolution at each level.
- Bilinear interpolation gives continuous sub-pixel samples; non-maximum sup-

pression and parabola fits localize detections.

- Normalize patches when brightness may change between frames.

Exercises

21.1 Show that the 2D Gaussian kernel is separable, and that filtering with it costs $2k$ operations per pixel for a k-tap kernel.

21.2 Show that convolving $\frac{1}{2}[1, 1]$ with itself repeatedly produces the binomial kernels, and that their variance grows linearly with the number of convolutions.

21.3 Compute the response of the Sobel kernel S_u to a linear ramp $I(u, v) = cu$, and explain the factor 8.

21.4 Generate a zone plate and reproduce Figure 21.2. How large must the blur be before shrinking by 4?

21.5 Derive the bilinear interpolation formula by interpolating first along u and then along v, and show that the result does not depend on the order.

21.6 Derive the parabola-vertex formula for sub-pixel peak refinement.

Chapter 22

Feature Detection and Tracking

In this chapter

- Derive the structure tensor from the auto-correlation of an image patch, and understand corner detectors (Harris, Shi–Tomasi, FAST).
- Derive Lucas–Kanade optical flow from brightness constancy, as a Gauss–Newton problem, including the aperture problem.
- Implement iterative, pyramidal Lucas–Kanade tracking, and know its failure modes.
- Build a robust tracking pipeline: bucketing, forward-backward checks, IMU-aided prediction and geometric verification.
- Manage feature tracks through their life cycle with bounded memory.

The front end turns a stream of images into *feature tracks*: for each tracked scene point, the list of its undistorted normalized coordinates in the images where it was seen, with the image timestamps. Everything the MSCKF knows about the scene comes from these tracks, so their quality bounds the quality of the estimator.

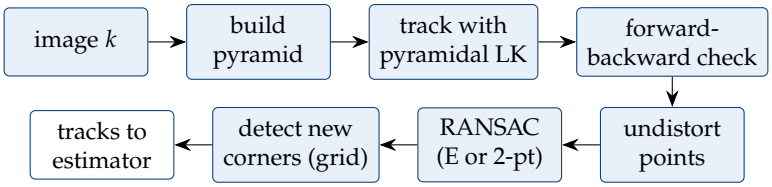


Figure 22.1: A KLT-based front end, as used by MSCKF-style estimators.

22.1 What makes a good feature

22.1.1 The auto-correlation of a patch

A feature is useful if it can be found again, precisely, in the next image. Consider a small window W around a point and ask how much the window changes when shifted by a small displacement d. Measure the change by the weighted sum of squared differences

$$E(d) = \sum_{x \in W} w(x) \left(I(x+d) - I(x)\right)^2, \tag{22.1}$$

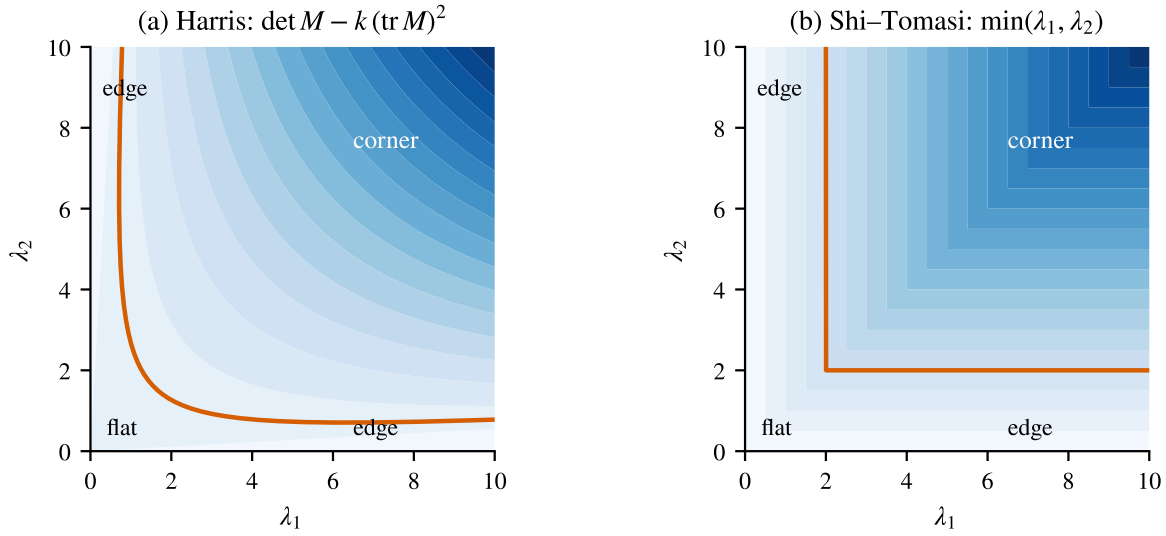


Figure 22.2: Corner scores over the plane of structure-tensor eigenvalues, with a threshold contour (orange). Both scores accept only windows with two large eigenvalues. Shi–Tomasi's $\min(\lambda_1, \lambda_2)$ is exactly the quantity that makes the Lucas–Kanade system well conditioned.

with a window function w (uniform, or a Gaussian that emphasizes the center). By a first-order Taylor expansion $I(x + d) \approx I(x) + \nabla I(x)^\top d$, so

$$E(d) \approx \sum_{x \in W} w(x) \left(\nabla I(x)^\top d\right)^2 = d^\top M\, d, \qquad M = \sum_{x \in W} w(x) \begin{bmatrix} I_u^2 & I_u I_v \\ I_u I_v & I_v^2 \end{bmatrix}, \tag{22.2}$$

with the gradients I_u, I_v computed as in Section 21.3. M is the *structure tensor* (second-moment matrix) of the window: a symmetric PSD 2×2 matrix, so $E(d)$ is a quadratic form whose level sets are ellipses (Figure 2.4). Its eigenvalues $\lambda_1 \geq \lambda_2 \geq 0$ describe how distinctive the window is.

- Both small: the window is *flat*; it looks the same after any small shift.
- One large, one small: an *edge*; shifting along the edge (the eigenvector of λ_2) changes nothing.
- Both large: a *corner*; every shift changes the window, so its position is well defined in two dimensions.

By the Rayleigh quotient (Proposition 2.30), λ_2 is the smallest change $E(d)$ caused by any unit shift: the worst-case distinctiveness.

22.1.2 Corner detectors

The *Harris detector* [22] scores each pixel with $\det M - k(\operatorname{tr} M)^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$, with $k \approx 0.04$–0.06, which avoids computing eigenvalues. The *Shi–Tomasi detector* [45] scores $\lambda_2 = \min(\lambda_1, \lambda_2)$ directly (Figure 22.2), computed in closed form for a 2×2 matrix as $\frac{1}{2}\big(\operatorname{tr} M - \sqrt{(\operatorname{tr} M)^2 - 4 \det M}\big)$. Both are followed by thresholding and non-maximum suppression. Shi–Tomasi has a deeper justification: it selects exactly the windows for which the tracker of the next section is well conditioned.

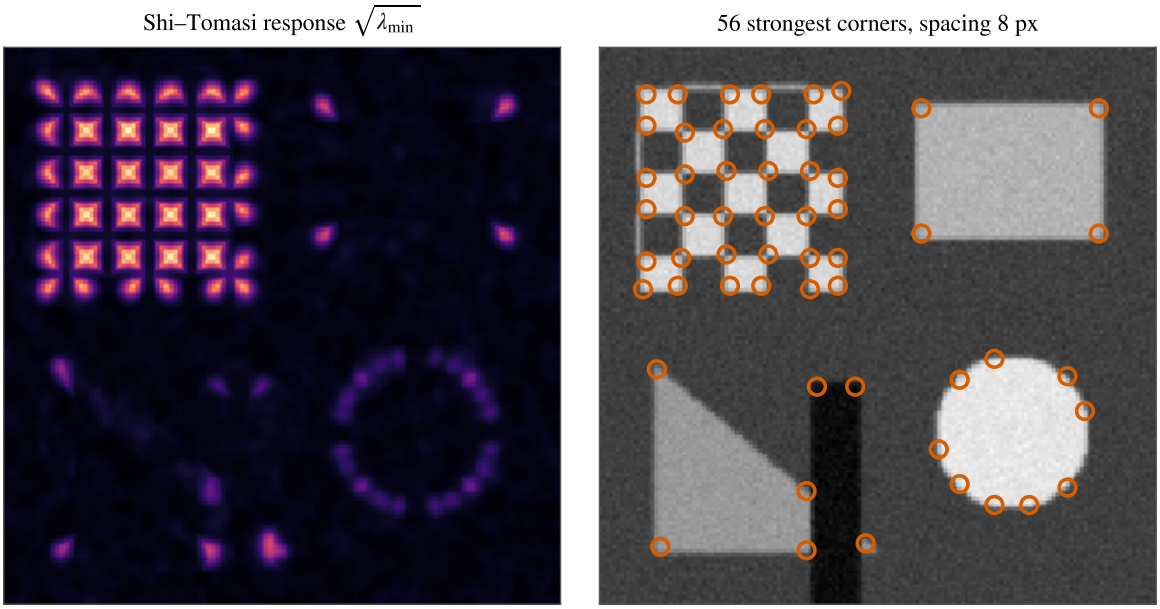


Figure 22.3: Shi–Tomasi corner response on a synthetic image (left), and the strongest corners after non-maximum suppression with a minimum spacing of 8 px (right). Edges and flat regions get low scores; corners and blob-like texture get high ones.

Other detectors trade quality for speed. FAST classifies a pixel as a corner if a long enough arc of the 16 pixels on a circle around it is uniformly brighter or darker than the center, using only comparisons; it is very fast but noisier. Detectors used for matching across large viewpoint changes add *scale* selection (detecting at several pyramid levels) and an *orientation* for each feature, as ORB does. A frame-to-frame tracker needs neither.

22.2 Lucas–Kanade optical flow

22.2.1 Brightness constancy and the aperture problem

Assume that a scene point keeps its brightness between consecutive frames, $I_{k+1}(x+d) = I_k(x)$, where d is the image motion of the point. Linearizing in d,

$$I_{k+1}(x) + \nabla I^{\mathsf{T}} d \approx I_k(x) \quad \Longleftrightarrow \quad I_u\, d_u + I_v\, d_v + I_t = 0, \qquad I_t = I_{k+1}(x) - I_k(x). \tag{22.3}$$

This *optical-flow constraint* is one equation in two unknowns. It determines only the component of d along the gradient: motion along an edge is invisible. This is the *aperture problem*, the same phenomenon seen through a small hole when a long line slides past: only its perpendicular motion is visible.

22.2.2 The Lucas–Kanade solution

Lucas and Kanade [36] assume the displacement is the same for all pixels of a small window, which provides many equations for the two unknowns, and solve (22.3) for all of them in the

least-squares sense:

$$\min_{d} \sum_{x \in W} w(x) \big(\nabla I(x)^{\mathsf{T}} d + I_t(x) \big)^2 \quad \Longrightarrow \quad M\, d = - \sum_{x \in W} w(x)\, I_t(x)\, \nabla I(x), \tag{22.4}$$

with the structure tensor M of (22.2) as the normal matrix (Section 3.5). The system is solvable and well conditioned exactly when both eigenvalues of M are large compared with the image noise: Shi and Tomasi's insight was to select features by the very criterion that makes them trackable. With image noise of variance σ_I^2 per pixel, the covariance of the estimated displacement is approximately $\sigma_I^2 M^{-1}$ (Proposition 9.3), so $1/\sqrt{\lambda_2}$ measures how precisely the worst direction is tracked.

22.2.3 Iterative refinement

Equation (22.4) rests on a linearization that holds only for small d. The full problem, minimizing $\sum_x w(x)(I_{k+1}(x + d) - I_k(x))^2$ over d, is a nonlinear least-squares problem, and Lucas–Kanade is its Gauss–Newton solution (Chapter 10):

Algorithm 4 Iterative Lucas–Kanade for one feature

Input: images I_k, I_{k+1}; feature position x_0; initial guess d (zero, or a prediction)

1: compute the template $T(x) = I_k(x)$ and its gradients over the window around x_0; compute M and factor it
2: **repeat**
3: sample $I_{k+1}(x + d)$ over the window with bilinear interpolation (Section 21.5)
4: $e(x) \leftarrow I_{k+1}(x + d) - T(x)$ ▷ residual image
5: solve $M\, \Delta d = - \sum_x w(x)\, e(x)\, \nabla T(x)$
6: $d \leftarrow d + \Delta d$
7: **until** $\|\Delta d\| < 0.01$ px or the iteration limit is reached

Using the gradients of the *template* $T = I_k$ rather than of the warped image lets M be computed and factored once per feature (the *inverse compositional* idea, analyzed by Baker and Matthews [2]). Sub-pixel accuracy comes from the bilinear interpolation. A few iterations usually suffice when the initial guess is within the window radius.

22.2.4 Pyramids

The linearization is valid only for displacements smaller than the scale over which the image gradient is roughly constant, about the window radius. Fast motion violates this: at 1 rad/s and 20 Hz, the image moves by about 23 px between frames. The remedy is a Gaussian pyramid (Section 21.4). Tracking starts at the coarsest level, where a 24-pixel motion becomes a 3-pixel motion; the result, doubled, initializes the next finer level, and so on down to full resolution (Figure 22.4) [7]. With L levels above the base, the trackable displacement grows by about 2^L.

22.2.5 When tracking fails

Lucas–Kanade fails when its assumptions fail: on edges and flat regions (an ill-conditioned M, the aperture problem); under occlusion, when the point disappears behind another object;

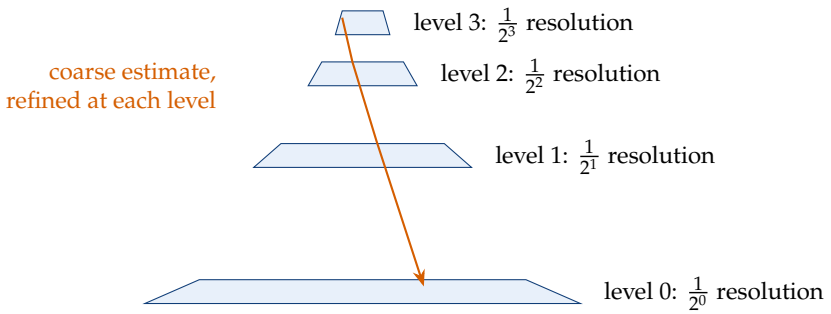


Figure 22.4: Pyramidal tracking. A displacement that is too large to linearize at full resolution is small at the top level; each level doubles the estimate and refines it.

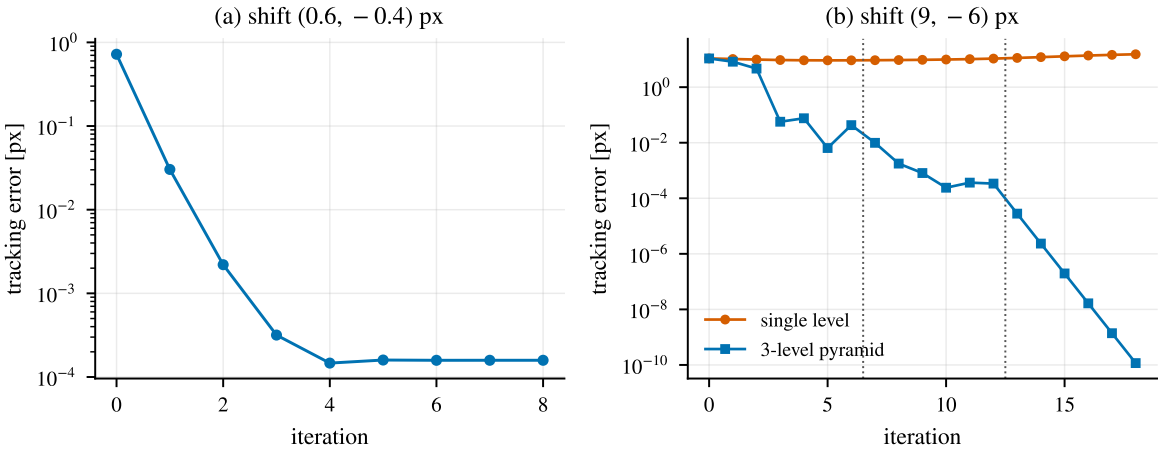


Figure 22.5: Lucas–Kanade on a synthetic textured image. (a) For a small shift of (0.6, −0.4) px, the tracking error falls by orders of magnitude in a few Gauss–Newton iterations. (b) For a shift of (9, −6) px, single-level tracking stalls in a wrong local minimum, while a three-level pyramid converges.

under strong brightness changes (Chapter 21); under motion blur, which smears the texture the tracker relies on; and when the window straddles a depth discontinuity, where the two halves move differently. Most failures go undetected by the tracker itself, which returns a confident but wrong position. Hence the checks of the next section.

22.3 Making tracking robust

Optical flow optimizes photometric similarity, not physical correctness. It will track the shadow of a moving object, a specular highlight, or a T-junction formed by two objects at different depths. Several layers of checks remove such tracks.

1. **Forward-backward consistency.** Track each point from frame k to $k+1$, then back to k. A correct track returns to its starting point; reject it if the round-trip error exceeds a fraction of a pixel.
2. **IMU-aided prediction.** The gyroscope predicts the rotation between frames, $R_{c_2c_1} = R_{bc}^{\mathsf{T}} R_{b_1b_2}^{\mathsf{T}} R_{bc}$, where $R_{b_1b_2}$ is the integrated gyro rotation. For distant points, or a purely

rotating camera, image points move by the infinite homography (Chapter 20): in pixels, $\tilde{u}_2 \propto K R_{c_2 c_1} K^{-1} \tilde{u}_1$ (after undistortion). Starting LK from this prediction gives it a much better initial guess, and allows smaller windows and fewer pyramid levels.

3. **Geometric verification.** Reject tracks that violate the epipolar constraint, with essential-matrix or two-point RANSAC, or the rotation-only constraint under low parallax (Chapter 23).
4. **Statistical gating in the filter.** Tracks that pass all of the above are still gated by the χ^2 test of the MSCKF update (Chapter 26).

22.3.1 Feature distribution

Asking a detector for the 200 strongest corners typically returns clusters on the most textured object in view. Clustered features constrain the motion poorly: features in a small patch of the image make rotation and translation hard to separate. The standard remedy is *grid bucketing*: divide the image into cells (for example 8×6) and cap the number of features per cell, plus a minimum distance between features. New features are detected only in cells that have fallen below their quota, using a mask around existing tracks. A uniform distribution of bearings makes the stacked measurement Jacobian better conditioned, which is exactly what the estimator needs.

22.4 The feature life cycle

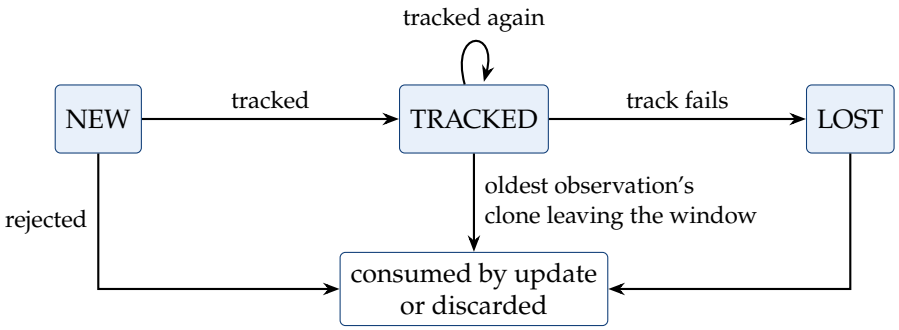


Figure 22.6: Feature life cycle. A track is handed to the MSCKF update when it is lost, or when its oldest observation is about to leave the sliding window (AGENTS.md F5). Its observations are used once and then discarded.

Each feature has a persistent ID and a history of observations: (clone index or image timestamp, normalized coordinates). Since the estimator can only use observations made at times that still have a clone in the window, the history never needs to exceed `kMaxClones` entries. That makes it possible to store all tracks in a fixed-capacity pool with fixed-capacity histories, so the steady state of the front end performs no allocation for its bookkeeping (Milestone 9).

Pitfall: using a measurement twice

Once a track's observations have been used in an MSCKF update, they must never be used again, not even partially. Re-using them double counts the same information and makes the filter overconfident. If a track continues after being used (because its oldest clone was marginalized), its subsequent observations start a fresh track.

22.5 Engineering the front end

The front end is the one component allowed to allocate (AGENTS.md E4), because OpenCV allocates internally, but it must reuse its buffers. In practice: keep the previous pyramid and swap it with the new one instead of rebuilding storage; preallocate point vectors with `reserve`; keep the `cv::Mat` objects as members so that their memory is reused when sizes do not change; and *measure* the remaining allocations with the allocation guard rather than claiming they are zero. Feature tracking is typically the most expensive part of the pipeline in time, too, so bucketing parameters, window size and pyramid depth are the main performance knobs.

Descriptor-based front ends (ORB, BRIEF) describe each feature by a binary string of intensity comparisons in its neighborhood and match features across frames by the Hamming distance between descriptors, rejecting ambiguous matches whose best and second-best distances are similar (the *ratio test*). They handle large viewpoint changes and re-detection after occlusion, which matters for loop closure, but for frame-to-frame VIO at 20 Hz KLT tracking is cheaper and more accurate. Direct methods skip features altogether and use photometric error in the estimator itself.

Project link: Milestones 8 and 9

M8 builds detection, bucketing, pyramidal LK with forward-backward checks, persistent IDs and point undistortion, and tests them on synthetic images with known sub-pixel motion. M9 adds the life-cycle manager with a fixed-capacity pool and essential-matrix RANSAC in normalized coordinates, including the documented pure-rotation behavior.

Key takeaways

- The structure tensor M, from a first-order expansion of the patch auto-correlation, measures how well a window's position is defined; corners have two large eigenvalues.
- Lucas–Kanade is Gauss–Newton on the patch alignment error; its normal matrix is M, so good corners are exactly the trackable windows.
- The aperture problem: a single pixel constrains only the flow along the gradient.
- Pyramids extend the trackable displacement; gyro prediction makes tracking cheaper and more robust.
- Bucketing spreads features for good geometry; forward-backward checks and RANSAC remove bad tracks.
- Tracks are consumed exactly once and their histories are bounded by the window size.

Exercises

22.1 Compute the structure tensor of an ideal step edge and of an ideal corner, and verify the eigenvalue picture.

22.2 Derive the closed-form smallest eigenvalue of a symmetric 2×2 matrix used by the Shi–Tomasi detector.

22.3 Implement Algorithm 4 for a synthetic textured image translated by $(0.3, -0.7)$ pixels and reproduce Figure 22.5(a). What happens with a translation of 8 pixels, and how many pyramid levels fix it?

22.4 Show that the forward-backward error of a correct track is due only to noise and interpolation, and estimate a sensible threshold.

22.5 Compute the pixel displacement predicted by the gyroscope for a feature at the image center when the camera rotates by 0.05 rad about its y-axis between frames.

22.6 Implement grid bucketing and plot the feature distribution with and without it on a EuRoC image.

22.7 Design the data structure for the feature pool with fixed capacity and per-feature histories bounded by `kMaxClones`. What happens when the pool is full?

Chapter 23

Multiple-View Geometry

In this chapter

- Derive the epipolar constraint, the essential and fundamental matrices, epipoles and epipolar lines.
- Recover relative motion from correspondences: the eight-point algorithm with normalization, the five-point and two-point methods, and the decomposition of the essential matrix.
- Understand homographies between views, the pure-rotation case, and the degeneracies of relative-pose estimation.
- Use RANSAC to reject outliers, size its iteration count, and set thresholds in the right units.
- Triangulate points by midpoint, linear and Gauss–Newton methods, and quantify triangulation accuracy.
- Understand bundle adjustment, its sparsity and its gauge freedom: the batch problem the MSCKF approximates.

23.1 Epipolar geometry

23.1.1 The epipolar constraint

Two cameras observe the same 3D point. Let the coordinates of the point in the two camera frames be X_1 and X_2, related by the rigid motion

$$X_2 = R\,X_1 + t, \qquad R = R_{c_2c_1}, \quad t = p_{c_2c_1}, \tag{23.1}$$

and let $\bar{x}_1 = X_1/Z_1$ and $\bar{x}_2 = X_2/Z_2$ be the bearings (homogeneous normalized coordinates).

Theorem 23.1 (Epipolar constraint). *Every correspondence of a rigid point satisfies*

$$\bar{x}_2^{\mathsf{T}}\, E\, \bar{x}_1 = 0, \qquad E = [t]_\times R. \tag{23.2}$$

Proof. Cross both sides of $X_2 = RX_1 + t$ with t: $[t]_\times X_2 = [t]_\times R X_1$. The left side is orthogonal to X_2, so $X_2^{\mathsf{T}}[t]_\times R X_1 = 0$. Dividing by $Z_1 Z_2 > 0$ gives (23.2). ∎

Geometrically, the two camera centers and the 3D point span a plane, the *epipolar plane*, and the constraint states that the two viewing rays and the baseline are coplanar (Figure 23.1). The constraint involves no depth: it is a statement about bearings only, which is why it is the natural tool for checking tracks before any 3D reconstruction exists.

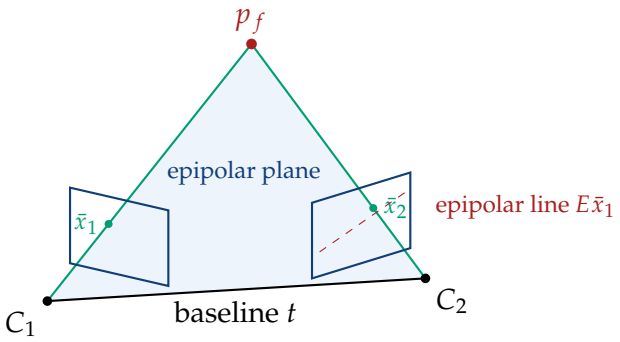


Figure 23.1: Epipolar geometry. The camera centers, the point and both viewing rays lie in one plane. The image of the point in the second camera must therefore lie on the line where that plane cuts the second image.

23.1.2 Epipolar lines and epipoles

Given $\bar{x}_1$, the vector $\ell_2 = E\bar{x}_1$ represents a line in the second image (Section 20.3), the *epipolar line*: the corresponding point must lie on it. Searching for a match can be restricted to this line. All epipolar lines in the second image pass through the *epipole* e_2, the image of the first camera center, which satisfies $e_2^\top E = 0$ (it is proportional to t); symmetrically $Ee_1 = 0$ with $e_1 \propto R^\top t$. When the camera moves forward, the epipole is the *focus of expansion* from which image points flow outward.

> **Proposition 23.2** (Properties of the essential matrix)**.** *For $t \neq 0$, $E = [t]_\times R$ has rank two and two equal non-zero singular values, $(\|t\|, \|t\|, 0)$. It has five degrees of freedom: three for the rotation and two for the direction of t. Its overall scale, like the scale of t, is unobservable.*

Proof. $EE^\top = [t]_\times RR^\top [t]_\times^\top = -[t]_\times^2 = \|t\|^2 I - tt^\top$ by (2.10), whose eigenvalues are $\|t\|^2$ (twice, on the plane orthogonal to t) and 0 (along t). The singular values of E are their square roots. ∎

In pixel coordinates, with $\tilde{u}_i = K\bar{x}_i$, the constraint reads $\tilde{u}_2^\top F \tilde{u}_1 = 0$ with the *fundamental matrix* $F = K^{-\top} E K^{-1}$ (seven degrees of freedom, rank two). The project works with undistorted normalized coordinates throughout, so it uses E directly (Milestone 9).

23.2 Estimating relative motion

23.2.1 The eight-point algorithm

Equation (23.2) is linear in the nine entries of E. Writing $\bar{x}_1 = (x_1, y_1, 1)$ and $\bar{x}_2 = (x_2, y_2, 1)$, each correspondence gives one equation

$$a^\top e = 0, \qquad a = (x_2x_1,\ x_2y_1,\ x_2,\ y_2x_1,\ y_2y_1,\ y_2,\ x_1,\ y_1,\ 1), \qquad e = \mathrm{vec}_{\mathrm{rows}}(E). \tag{23.3}$$

With eight or more correspondences, stack them into $Ae = 0$ and take the unit vector minimizing $\|Ae\|$: the right singular vector of A with the smallest singular value (Proposition 3.7). Noise makes the result an invalid essential matrix, which is projected onto the nearest valid

one by replacing its singular values with $(1, 1, 0)$ (Theorem 3.6 and Eckart–Young). When the algorithm is applied to pixel coordinates (to estimate F), the coordinates must first be *normalized* (translated to zero mean and scaled to unit average distance from the origin), or the matrix A, whose entries range from 1 to 10^5, is so badly conditioned that the result is unusable; this is Hartley's normalization.

23.2.2 Decomposing the essential matrix

Proposition 23.3 (Motion from the essential matrix)**.** *Let $E = U\,\mathrm{diag}(1, 1, 0)V^{\mathsf{T}}$ with U, V rotations (flip their signs if needed), and $W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. The rotation and translation direction are one of the four combinations*

$$R \in \{UWV^{\mathsf{T}},\ UW^{\mathsf{T}}V^{\mathsf{T}}\}, \qquad t \in \{+u_3,\ -u_3\}, \tag{23.4}$$

where u_3 is the third column of U. Exactly one of the four places triangulated points in front of both cameras.

The proof checks that $[u_3]_\times UWV^{\mathsf{T}}$ and $[u_3]_\times UW^{\mathsf{T}}V^{\mathsf{T}}$ both equal E up to sign, and that no other factorizations exist. The four solutions correspond to the second camera being on either side of the first, facing either forward or backward; the *cheirality* test (positive depth of a triangulated point in both views) selects the physical one.

23.2.3 The five-point and two-point methods

Five correspondences are the minimum, since E has five degrees of freedom. Nistér's algorithm [41] combines the linear constraints of five correspondences with the cubic constraints that characterize essential matrices ($\det E = 0$ and $2EE^{\mathsf{T}}E - \mathrm{tr}(EE^{\mathsf{T}})E = 0$) and solves the resulting polynomial system, returning up to ten candidates. Fewer points per hypothesis means fewer RANSAC iterations, which is why `cv::findEssentialMat` uses it.

In VIO the gyroscope supplies the relative rotation R between consecutive frames with high accuracy. With R known, (23.2) becomes linear in t:

$$\bar{x}_2^{\mathsf{T}}[t]_\times R\bar{x}_1 = t^{\mathsf{T}}\big(R\bar{x}_1 \times \bar{x}_2\big) = 0. \tag{23.5}$$

Each correspondence says that t is orthogonal to the vector $c_i = R\bar{x}_1^{(i)} \times \bar{x}_2^{(i)}$, so two correspondences determine the direction of t as $c_1 \times c_2$. A two-point hypothesis needs far fewer RANSAC iterations than a five-point one, and it is the outlier-rejection strategy of MSCKF-VIO [51].

23.2.4 Measuring the residual

The algebraic residual $\bar{x}_2^{\mathsf{T}}E\bar{x}_1$ has no geometric meaning. The geometric error of a correspondence is the smallest displacement of $\bar{x}_1$ and $\bar{x}_2$ that makes the constraint hold exactly.

Linearizing the constraint in that displacement gives the *Sampson distance*

$$d_S = \frac{(\bar{x}_2^\mathsf{T} E \bar{x}_1)^2}{(E\bar{x}_1)_1^2 + (E\bar{x}_1)_2^2 + (E^\mathsf{T}\bar{x}_2)_1^2 + (E^\mathsf{T}\bar{x}_2)_2^2}, \tag{23.6}$$

the squared residual divided by the squared norm of its gradient with respect to the four image coordinates: the first-order squared distance to the constraint surface. It is in the units of the coordinates used. With normalized coordinates, a pixel threshold τ_{px} must be converted to τ_{px}/f (Milestone 9).

Pitfall: thresholds in the wrong units

Calling `cv::findEssentialMat` on normalized coordinates with an identity camera matrix and a threshold of "1.0" accepts residuals of 1 normalized unit, about 450 pixels: every track becomes an inlier. The threshold must be τ_{px}/f, e.g. $1/458 \approx 2.2 \times 10^{-3}$.

23.3 Homographies between views

Planar scenes. If all points lie on a plane $n^\mathsf{T} X_1 = d$ (unit normal n and distance d in the first camera's frame), then $1 = n^\mathsf{T} X_1/d$ and $X_2 = RX_1 + t\, n^\mathsf{T} X_1/d$, so

$$X_2 = \left(R + \frac{t\, n^\mathsf{T}}{d}\right) X_1, \qquad \bar{x}_2 \propto H\bar{x}_1, \quad H = R + \frac{t n^\mathsf{T}}{d}. \tag{23.7}$$

Pure rotation. If $t = 0$, $X_2 = RX_1$ for every point, at any depth: $\bar{x}_2 \propto R\bar{x}_1$, the *infinite homography*. In pixels it is KRK^{-1}.

A homography has eight degrees of freedom and is estimated from four or more correspondences by a direct linear transform: each correspondence gives two linear equations in the nine entries of H (from $\tilde{x}_2 \times H\tilde{x}_1 = 0$), solved by Proposition 3.7.

23.4 RANSAC

Random Sample Consensus [15] estimates a model from data containing outliers by hypothesis and verification.

Algorithm 5 RANSAC

Input: data, minimal sample size s, threshold τ, confidence p

1: best $\leftarrow \emptyset$; $N \leftarrow \infty$; $k \leftarrow 0$
2: **while** $k < N$ **do**
3: draw s data points at random; fit candidate models
4: count inliers with residual $< \tau$; keep the model with the most inliers
5: update N from the current inlier ratio w with (23.8); $k \leftarrow k + 1$
6: refit the model on all inliers of the best hypothesis (least squares)

If a fraction w of the data are inliers, a random minimal sample is outlier-free with probability w^s, and N independent samples all contain an outlier with probability $(1 - w^s)^N$. Requiring

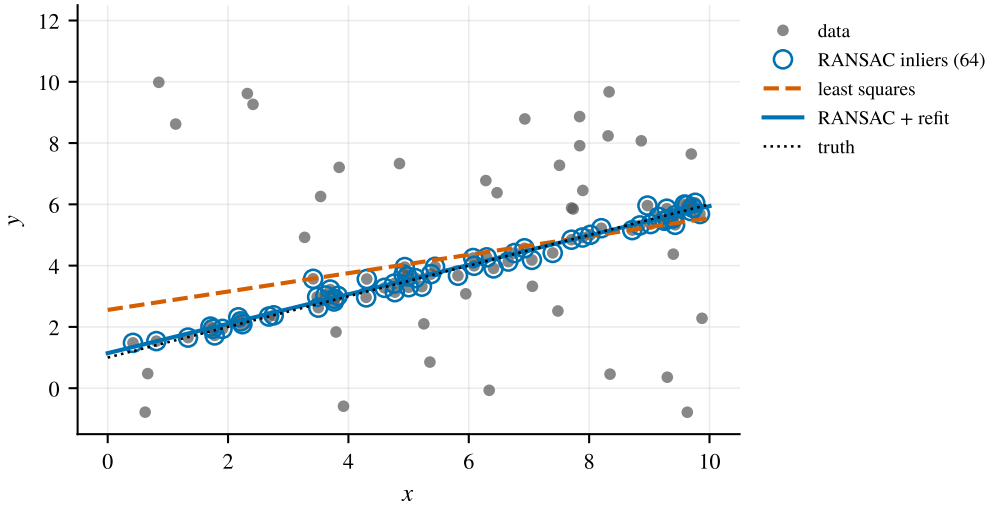


Figure 23.2: Fitting a line to data with 40% gross outliers. Least squares is dragged away by the outliers; RANSAC finds the line supported by the most inliers (circled) and refits it to them.

this to be at most $1 - p$ gives the number of iterations

$$N = \frac{\log(1 - p)}{\log(1 - w^s)}. \tag{23.8}$$

$p = 0.99$	$w = 0.9$	$w = 0.7$	$w = 0.5$
8-point ($s = 8$)	9	78	1177
5-point ($s = 5$)	6	26	146
2-point ($s = 2$)	3	7	17

The exponential dependence on s is why minimal solvers matter, and why using the gyroscope to remove three unknowns pays off. Since w is unknown in advance, the algorithm starts pessimistic and updates N from the best inlier ratio found so far (adaptive termination). Variants score hypotheses by the sum of truncated residuals (MSAC) rather than the inlier count, or refine the best hypothesis locally (LO-RANSAC).

23.5 Degeneracies

23.5.1 Pure rotation

If the camera only rotates, $t = 0$ and $E = [0]_\times R = 0$: the epipolar constraint is satisfied by every pair of points, inlier or not. Under small translation the problem is not degenerate but badly conditioned: t is poorly determined, inlier sets fluctuate, and RANSAC may reject good tracks or accept bad ones. Hovering drones rotate in place often, so this case must be handled (Milestone 9).

Under pure rotation the correspondences satisfy the stronger constraint $\bar{x}_2 \propto R\bar{x}_1$ (Section 23.3), which does not involve t at all. A robust front end measures the parallax remaining after rotation compensation, for example the median of $\|\pi(R\bar{x}_1) - \pi(\bar{x}_2)\|$ over the tracks:

- if it is large, run essential-matrix (or two-point) RANSAC;
- if it is tiny, skip epipolar RANSAC and reject only tracks that violate the rotation-only prediction $\bar{x}_2 \propto R\bar{x}_1$, with R from the gyroscope.

Either way the system keeps tracking: low parallax means the features carry little information about translation, not that they are wrong. The MSCKF update handles them gracefully, because their nullspace-projected residuals simply contribute little.

23.5.2 Other degeneracies

The eight-point algorithm is degenerate when all points lie on a plane, a common situation (floors, walls), because a homography then explains the data equally well; the five-point algorithm is not. Very distant points behave like points at infinity and carry only rotational information. Moving objects form a second, consistent rigid motion that RANSAC can mistake for the dominant one if they fill the image.

23.6 Triangulation

Given observations z_i of a point in cameras with known poses (R_{wc_i}, p_{wc_i}), triangulation estimates its world position p_f.

23.6.1 The midpoint method

Each observation defines a ray $p_{wc_i} + \lambda_i r_i$ with direction $r_i = R_{wc_i}\bar{x}_i$. With noise the rays do not intersect. The midpoint method finds the point closest to all rays in the least-squares sense: the distance of p from ray i is $\|(I - \hat{r}_i\hat{r}_i^\mathsf{T})(p - p_{wc_i})\|$ with $\hat{r}_i$ the unit direction, so

$$\Big(\sum_i (I - \hat{r}_i\hat{r}_i^\mathsf{T})\Big)\, p = \sum_i (I - \hat{r}_i\hat{r}_i^\mathsf{T})\, p_{wc_i}. \tag{23.9}$$

It is simple and geometric, but it minimizes 3D distances rather than image errors, which weights distant views wrongly.

23.6.2 Linear triangulation

In camera i the point has coordinates $X_i = R_{wc_i}^\mathsf{T}(p_f - p_{wc_i})$, which must be parallel to the bearing $\bar{x}_i = (z_i, 1)$: $[\bar{x}_i]_\times X_i = 0$. Substituting,

$$[\bar{x}_i]_\times R_{wc_i}^\mathsf{T}\, p_f = [\bar{x}_i]_\times R_{wc_i}^\mathsf{T}\, p_{wc_i}, \qquad i = 1, \dots, M. \tag{23.10}$$

Each view contributes three equations of rank two. Stacking all views gives an overdetermined linear system $Ap_f = b$ that is solved in the least-squares sense with QR or SVD. With two views and noise-free data the solution is exact. This is the linear initialization of AGENTS.md F5. It minimizes an algebraic error, not the reprojection error, so it is refined next.

23.6.3 Nonlinear refinement with inverse depth

The maximum-likelihood estimate minimizes the reprojection error (Chapter 10)

$$\min_{p_f} \sum_{i=1}^{M} \left\| z_i - \pi(X_i(p_f)) \right\|^2. \tag{23.11}$$

The MSCKF [40] parameterizes the point in an *anchor* camera A (for example the first observation) by inverse depth [10]:

$$X_A = \frac{1}{\rho}\begin{bmatrix}\alpha\\ \beta\\ 1\end{bmatrix}, \qquad X_i = R_{iA}X_A + p_{iA} \quad \Longrightarrow \quad h_i := \rho X_i = R_{iA}\begin{bmatrix}\alpha\\ \beta\\ 1\end{bmatrix} + \rho\, p_{iA}, \quad z_i = \pi(h_i), \tag{23.12}$$

where (R_{iA}, p_{iA}) is the transformation from the anchor camera to camera i, computed from the clone poses, and $\pi(h_i) = \pi(X_i)$ because projection ignores scale. The parameters (α, β) are the normalized coordinates in the anchor view and $\rho = 1/Z_A$ is the inverse depth. This parameterization behaves well for distant points ($\rho \to 0$ is a regular point), and the measurement function is nearly linear in (α, β, ρ). Its Jacobian follows from the chain rule:

$$\frac{\partial z_i}{\partial(\alpha, \beta, \rho)} = \left.\frac{\partial \pi}{\partial h}\right|_{h_i} \begin{bmatrix} r_1 & r_2 & p_{iA}\end{bmatrix}, \tag{23.13}$$

where r_1, r_2 are the first two columns of R_{iA} and $\partial\pi/\partial h$ is Proposition 20.2. Gauss–Newton (Algorithm 1) iterates $\delta y = -(J^\top J)^{-1} J^\top e$ with the stacked Jacobian J and residual e, usually converging in a few iterations from the linear initialization. Levenberg–Marquardt damping adds robustness.

23.6.4 How accurate is triangulation?

For two views separated by a baseline b perpendicular to the viewing direction, the depth is $Z = b/d$ with d the disparity in normalized coordinates. With independent noise σ_{uv} in each view, $\sigma_d = \sqrt{2}\sigma_{uv}$, and since $\partial Z/\partial d = -b/d^2 = -Z^2/b$, to first order

$$\sigma_Z \approx \frac{Z^2}{b}\sqrt{2}\,\sigma_{uv}. \tag{23.14}$$

Depth error grows quadratically with depth and inversely with baseline. Equivalently, the relative depth error is inversely proportional to the parallax angle $\approx b/Z$.

Figure 23.3 shows that 10% depth accuracy needs about 2° of parallax, a baseline of 17 cm at 5 m. Below about 1° the linearization behind (23.14) breaks down entirely. A triangulation module must therefore check its own result before handing it to the filter:

- positive depth in every view (cheirality);
- sufficient parallax (or baseline relative to depth), or equivalently a well-conditioned Gauss–Newton Hessian;
- small reprojection error in every view after refinement;
- a plausible depth range.

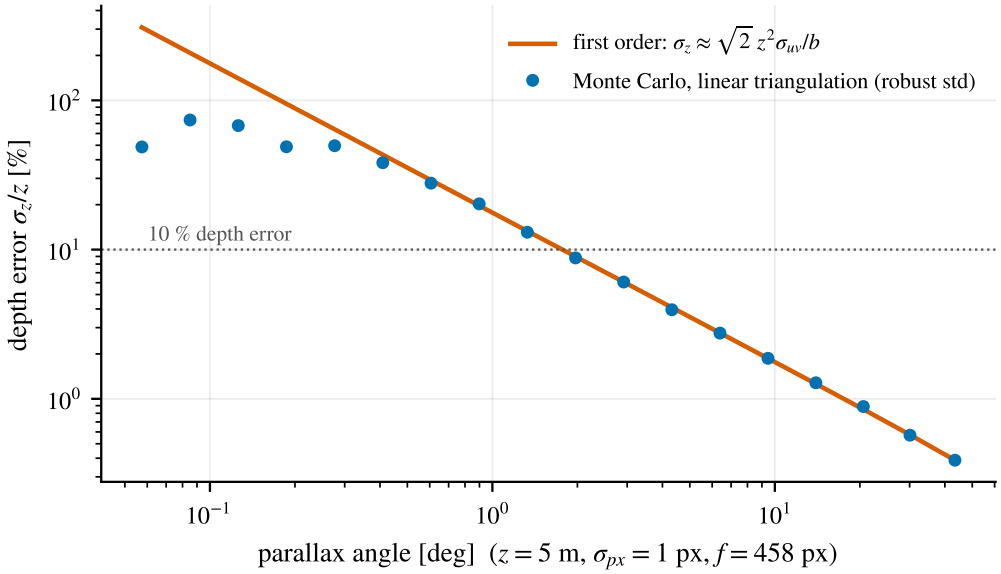


Figure 23.3: Relative depth error of two-view triangulation at $Z = 5$ m with 1-pixel noise, versus parallax angle. Above about 2° the first-order formula (23.14) is accurate. Below about 1° the error distribution becomes heavy-tailed (depths can even become negative), and the estimate is unusable.

Features that fail are discarded, not fed to the update (AGENTS.md F5, Milestone 10b).

23.7 Bundle adjustment

Bundle adjustment estimates all camera poses and all 3D points at once by minimizing the total reprojection error,

$$\min_{\{T_i\},\{p_j\}} \sum_{(i,j)\ \text{observed}} \left\| z_{ij} - \pi\left(T_i^{-1} p_j\right) \right\|^2_{R^{-1}}, \tag{23.15}$$

with Gauss–Newton or Levenberg–Marquardt on the manifold of poses (Chapter 10). It is the gold standard of geometric vision. Two of its properties illuminate VIO.

- *Structure.* Each residual involves one camera and one point, so the normal equations have the block structure of Section 10.8: eliminating the points with the Schur complement leaves a small *reduced camera system* (Figure 10.3).
- *Gauge freedom.* Moving and rotating everything together, or (for a monocular camera) scaling everything, leaves all reprojection errors unchanged: the problem has seven unobservable directions, and $J^\mathsf{T} W J$ is singular along them. Solvers fix the gauge by holding the first camera fixed and the scale constant, or by adding a weak prior. With an IMU, scale and two rotation directions (roll and pitch) become observable, and the gauge shrinks to the four directions of Chapter 29.

The MSCKF can be understood as a fast, filtered approximation of bundle adjustment over a sliding window: it eliminates each point exactly as the Schur complement does (Chapter 26), takes a single Gauss–Newton step, and summarizes the past by the filter's Gaussian prior

instead of relinearizing it.

Pose from known points. Finally, when the 3D points are known (a map), a single image determines the camera pose: the *perspective-n-point* (PnP) problem, solved minimally from three points and refined by Gauss–Newton. It is the core of relocalization against a map, a capability that odometry lacks.

Key takeaways

- Rigid correspondences satisfy $\bar{x}_2^\mathsf{T}[t]_\times R\bar{x}_1 = 0$; E has 5 DOF, singular values $(\|t\|, \|t\|, 0)$, and an unobservable scale.
- The eight-point algorithm is linear (normalize pixel coordinates first); E decomposes into four motions, one of which passes the cheirality test.
- With the rotation from the gyroscope, two correspondences determine the translation direction.
- Planes and pure rotations relate views by homographies; under pure rotation E vanishes and the rotation-only constraint takes over.
- RANSAC iterations grow as w^{-s}: prefer minimal solvers. Thresholds in normalized units are τ_{px}/f.
- Triangulate linearly, refine by Gauss–Newton with inverse depth, and reject low-parallax or negative-depth points; depth error grows as Z^2/b.
- Bundle adjustment is the batch problem; the MSCKF approximates it with exact landmark elimination and a single Gauss–Newton step.

Exercises

23.1 Verify Theorem 23.1 numerically for random points and a random relative pose, then show that adding an outlier breaks it.

23.2 Prove Proposition 23.2 and show that $e_2 \propto t$ is the left null vector of E.

23.3 Implement the eight-point algorithm with and without Hartley normalization on pixel coordinates, and compare the errors.

23.4 Implement the decomposition of Proposition 23.3 and the cheirality test, and verify on synthetic data that the correct solution is selected.

23.5 Implement the two-point method with known rotation inside RANSAC on synthetic data with 30% outliers. Compare the number of iterations needed with the five-point method.

23.6 Generate a pure-rotation sequence, observe what `cv::findEssentialMat` returns, and implement the parallax test.

23.7 Show that (23.10) has rank two per view and that two views with non-parallel rays give a unique solution.

23.8 Derive (23.14), reproduce Figure 23.3, and repeat with three views over the same baseline.

23.9 Count the gauge freedoms of monocular bundle adjustment and of visual-inertial bundle adjustment, and explain the difference.

Part VI

The Multi-State Constraint Kalman Filter

Chapter 24

From EKF-SLAM to the MSCKF

In this chapter

- Understand how EKF-SLAM uses visual landmarks, and why it scales poorly.
- Understand the central idea of the MSCKF: constraints between multiple past poses instead of landmarks.
- See why past poses must be kept in the state, with their correlations.
- Relate the MSCKF to sliding-window optimization.

24.1 EKF-SLAM

The most direct way to use a camera in an EKF is to put the landmarks in the state. With L landmarks,

$$x = \left(x_I,\ p_{f_1},\ \ldots,\ p_{f_L}\right), \qquad \dim \delta x = 15 + 3L, \tag{24.1}$$

and each image observation $z = \pi(p_c(x_I, p_{f_j})) + n$ is an ordinary EKF measurement of the IMU state and one landmark. This is EKF-SLAM, and MonoSLAM [11] showed it can run in real time with a single camera.

24.1.1 How it works

Prediction. Landmarks are static, so only the IMU block is propagated: exactly the block structure of clones in Chapter 17, $P_{II} \leftarrow \Phi P_{II}\Phi^\mathsf{T} + Q_d$, $P_{IL} \leftarrow \Phi P_{IL}$, with the landmark block unchanged.

Update. An observation of landmark j has a Jacobian with two non-zero blocks: one for the IMU pose and one for landmark j, $H = [H_I\ \ 0\ \cdots\ 0\ \ H_{f_j}\ \ 0\ \cdots]$. The gain $K = PH^\mathsf{T}S^{-1}$ is nevertheless dense: every landmark correlated with the robot or with landmark j is corrected. This is how EKF-SLAM "closes loops": re-observing an old landmark corrects the robot, which through the correlations corrects every other landmark. It is also what makes it expensive.

Landmark initialization. A new landmark enters the state as a function of the current state and the first measurement, $p_f = g(x_I, z)$. This is the state augmentation of Proposition 25.1: $P \leftarrow JPJ^\mathsf{T} + J_z R J_z^\mathsf{T}$ with $J = [I;\ \partial g/\partial x]$ and $J_z = [0;\ \partial g/\partial z]$. For a monocular camera, g needs a depth that a single measurement does not provide, which leads to the initialization problem below.

24.1.2 Why it scales poorly

EKF-SLAM has three structural problems.

1. **Cost.** The covariance has $(15 + 3L)^2$ entries, and every update touches all of them: the Joseph update costs $O(n^2)$ per scalar measurement at best. Tracking 200 landmarks gives a 615×615 covariance; updating it at 20 Hz with hundreds of observations per frame is expensive, and the cost grows quadratically with the map.
2. **Initialization.** A single monocular observation says nothing about depth. A new landmark must be initialized with a huge depth uncertainty, which is highly non-Gaussian in Euclidean coordinates, or its initialization must be delayed until it has been seen from several viewpoints. Inverse-depth parameterizations [10] help, but add complexity.
3. **Housekeeping.** Landmarks that leave the field of view must either stay in the state, growing it, or be marginalized, which in covariance form is cheap but discards their future value.

24.2 The multi-state constraint

Consider a feature observed in M images taken at times $t_1, \ldots, t_M$. Each observation constrains the pose at its time and the feature position. Taken together, after the unknown feature position is eliminated, the $2M$ observations impose $2M - 3$ constraints *on the M poses alone*. This is the *multi-state constraint*: geometric information about how the camera moved between the images, with the scene point serving only as an intermediary.

Key idea: the MSCKF in one sentence

Keep a sliding window of past poses in the state instead of landmarks; when a feature track is complete, triangulate it, linearize its observations, and eliminate the feature by projecting onto the left nullspace of its Jacobian, leaving a residual that depends only on the poses; then update.

Mourikis and Roumeliotis [40] showed that this uses all the information that the observations carry about the poses (under linearization), at a cost linear in the number of features. Feature positions are never part of the state, so there is no landmark initialization problem inside the filter, and features leaving the view simply trigger an update.

24.3 Why past poses must be in the state

Why not keep only the current IMU state and use the constraint between the current pose and the past ones? Because the past poses are *estimated* quantities, with errors correlated with the current state's error: they were all produced by the same propagation from the same measurements. A measurement relating the current pose to a past pose estimate, which treats that past estimate as fixed or independent, double counts information and makes the filter overconfident.

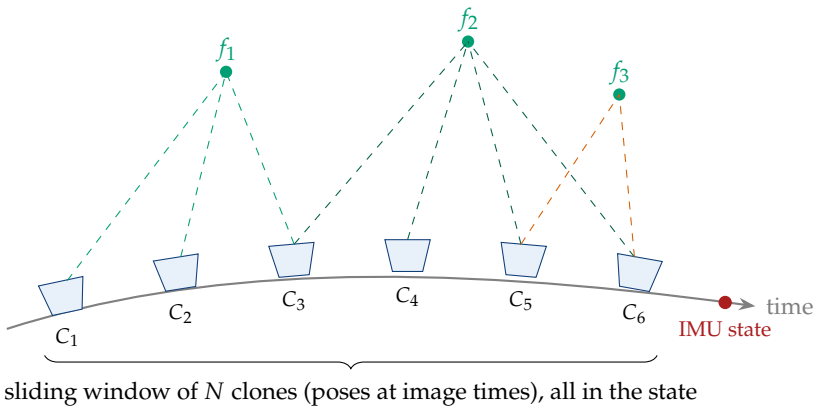


Figure 24.1: The MSCKF state: the current IMU state plus clones of past camera poses. Feature f_1 is seen by clones 1–3 and constrains them; f_2 by clones 3–6. Neither feature is in the state. Feature f_3 has only two observations so far and keeps being tracked.

The cure is *stochastic cloning* [43]. When an image is captured, a copy of the current pose (a clone) is appended to the state, together with its exact correlations to everything else. From then on the clone is a state variable whose estimate can be corrected by later measurements, and whose correlation with the evolving IMU state is propagated exactly. Chapter 25 develops this in detail.

24.4 The MSCKF state

The project's state (AGENTS.md C3) is the IMU state plus $N \le$ `kMaxClones` clones, each holding the IMU pose (p_{wb}, q_{wb}) at an image timestamp:

$$x = \big(\, p,\ v,\ q,\ b_a,\ b_g,\ \ p_1, q_1,\ \ldots,\ p_N, q_N \,\big), \qquad \dim \delta x = 15 + 6N. \tag{24.2}$$

The camera pose of clone i is derived through the fixed extrinsics, $R_{wc_i} = R_{wb_i} R_{bc}$ and $p_{wc_i} = p_{wb_i} + R_{wb_i} p_{bc}$. With `kMaxClones` $= 10$, the error state has 75 dimensions, independent of how many features are tracked.

24.5 Relation to sliding-window optimization

Optimization-based VIO (OKVIS [33], VINS-Mono [42]) keeps a similar window of poses and minimizes the sum of squared reprojection and IMU residuals over poses *and* landmarks, relinearizing at every iteration, then marginalizes old states into a Gaussian prior. The MSCKF update is equivalent to a single Gauss–Newton iteration of such a problem, with the landmarks eliminated by a Schur complement and the prior given by the filter's covariance. Eliminating a landmark by nullspace projection and by Schur complement give the same information about the poses (Theorem 26.1). The differences are that the MSCKF never relinearizes past measurements, and processes each feature track exactly once, which makes it cheaper and more predictable, at some cost in accuracy under strong nonlinearity.

24.6 Variants

The basic MSCKF has been refined in many directions. MSCKF 2.0 [34] added First-Estimates Jacobians for consistency and online calibration of extrinsics and time offset. Hybrid filters keep long-lived features in the state as SLAM landmarks while processing short tracks MSCKF-style. MSCKF-VIO [51] uses a stereo camera and two-point RANSAC for fast flight, and OpenVINS [18] is a modular research platform that implements most of these options and documents them carefully.

Key takeaways

- EKF-SLAM keeps landmarks in the state; its cost grows quadratically with the map.
- The MSCKF keeps past poses (clones) instead; each feature track is turned into constraints on those poses.
- Past poses must be in the state, with their correlations, or information is double counted.
- The MSCKF update is equivalent to one Gauss–Newton step of sliding-window bundle adjustment with landmarks marginalized.

Exercises

24.1 Estimate the memory and the per-update flop count of EKF-SLAM with 200 landmarks and of the MSCKF with 10 clones.

24.2 A feature is observed in M images. How many scalar constraints does it impose on the poses? Why does the count start being useful only at $M = 2$, and why does the project require $M \geq 3$?

24.3 Explain in your own words why using a past pose estimate as if it were known exactly makes the filter overconfident. Construct a scalar example.

Chapter 25

Stochastic Cloning and the Sliding Window

In this chapter

- See, on a scalar example, why relative measurements require cloned states with correlations.
- Derive the general state-augmentation formula $P \leftarrow JPJ^{\mathsf{T}}$.
- Derive the project's cloning Jacobian and the invariance property of cloning.
- Marginalize the oldest clone in place without allocation, and choose a window policy and size.

25.1 A scalar example

Let x be a scalar position that performs a random walk, $x_{k+1} = x_k + w_k$ with $\mathrm{Var}(w_k) = Q$, and suppose its current estimate has variance P. At time 1 we *clone* it, $x_c := x_1$; between times 1 and 2 the state moves; then a sensor measures the displacement:

$$z = x_2 - x_c + n, \qquad \mathrm{Var}(n) = R. \tag{25.1}$$

Augmenting the state with the clone gives the vector (x, x_c). At the moment of cloning both entries are the *same* random variable, so their joint covariance is $\begin{bmatrix} P & P \\ P & P \end{bmatrix}$, which is singular. Propagating x but not x_c gives

$$P_{\text{aug}} = \begin{bmatrix} P+Q & P \\ P & P \end{bmatrix}, \qquad H = \begin{bmatrix} 1 & -1 \end{bmatrix}. \tag{25.2}$$

The innovation covariance and gain are

$$S = (P+Q) - 2P + P + R = Q + R, \qquad K = \frac{1}{Q+R}\begin{bmatrix} Q \\ 0 \end{bmatrix}. \tag{25.3}$$

The result is exactly right: a displacement measurement is informative only about what happened *between* the two times, so S involves only Q and R. The update corrects the current state by the part of its uncertainty that arose after the clone, and leaves the clone untouched. The posterior variance of x_2 is $P + Q - Q^2/(Q+R)$, which stays large if P is large, as it must: a relative measurement cannot reveal absolute position.

Now suppose we had treated the past estimate as known (no clone, zero variance). Then $S = P + Q + R$ and the posterior variance of x_2 would be $(P+Q)R/(P+Q+R)$, which tends

to R when P is large. The filter would believe it had learned the absolute position to within the sensor noise from a displacement measurement: a severe overconfidence. Treating the past estimate as an independent random variable is wrong in a different way. Only the exact cross-covariance gives the right answer, and stochastic cloning [43] is the mechanism that keeps it.

25.2 State augmentation

Proposition 25.1 (Augmentation with a function of the state)**.** *Let x have error covariance P, and append a new variable $y = g(x)$ whose error satisfies $\delta y \approx J_g\,\delta x$. The augmented error $(\delta x, \delta y) = J\,\delta x$ with $J = \begin{bmatrix} I \\ J_g \end{bmatrix}$ has covariance*

$$P \leftarrow JPJ^\mathsf{T} = \begin{bmatrix} P & PJ_g^\mathsf{T} \\ J_gP & J_gPJ_g^\mathsf{T} \end{bmatrix}. \tag{25.4}$$

Proof. Proposition 5.7 with the linear map J. ∎

The augmented covariance has rank $\dim x$: it is singular, because y is a deterministic function of x at the moment of augmentation. This is correct and harmless. Propagation adds process noise to the IMU block only, after which the clone and the current state are no longer perfectly correlated. The Joseph form and symmetrization keep the matrix positive semi-definite throughout.

25.3 The project's clone

At an image timestamp, after propagating the IMU state to exactly that time, the filter appends a clone holding the current pose (p_{wb}, q_{wb}), and augments the covariance with the cloning Jacobian (AGENTS.md F5).

Derivation exercise (project Jacobian): the cloning Jacobian J

The clone's error is defined with the same retraction as the IMU pose (AGENTS.md C4): $p_c = \hat{p}_c + \delta p_c$ and $q_c = \hat{q}_c \otimes \mathrm{Exp}_q(\delta\theta_c)$, with $\hat{p}_c = \hat{p}$ and $\hat{q}_c = \hat{q}$ at cloning time.

1. At the moment of cloning the true clone equals the true IMU pose. Express δp_c and $\delta\theta_c$ in terms of the IMU error state. Is any linearization needed?
2. Write $J_g \in \mathbb{R}^{6\times(15+6N)}$ and then $J \in \mathbb{R}^{(21+6N)\times(15+6N)}$, with the new clone appended at the end of the state.
3. What would change if the clone stored the *camera* pose (p_{wc}, q_{wc}) instead of the IMU pose? (The project deliberately does not do this.)

Check: Milestone 10a requires that after cloning, the new clone's covariance block and its cross-covariances equal the corresponding IMU pose blocks *exactly*, and that a run with cloning enabled but only pose updates reproduces the IMU estimate and covariance of a run without cloning to 10^{-10}.

Worked solution: Appendix E.4 (page 282).

Proposition 25.2 (Cloning does not change the IMU marginal)**.** *Augmentation, propagation with static clones, and marginalization of clones leave the marginal distribution of the IMU state unchanged, provided no measurement involves the clones.*

Proof. Augmentation leaves the P_{II} block unchanged by (25.4). Propagation updates P_{II} by $\Phi P_{II}\Phi^\mathsf{T} + Q_d$ regardless of the clones. Marginalization deletes clone rows and columns. A measurement that involves only the IMU state has a Jacobian with zero clone columns; its gain for the IMU block, $P_{II}H_I^\mathsf{T}S^{-1}$, and $S = H_I P_{II} H_I^\mathsf{T} + R_z$ do not involve the clones either. ■

This is the invariance test of Milestone 10a. It is a powerful end-to-end check of the augmentation, propagation and marginalization code, because any indexing error in the clone blocks leaks into the IMU block and breaks the equality.

25.4 Clones in propagation and updates

Clones are static (Chapter 17): propagation multiplies their cross-covariance with the IMU state by Φ and leaves the clone-clone block alone. Visual measurements depend only on clone states, so their Jacobians are zero in the IMU columns. The update nevertheless corrects the IMU state, through the gain rows $P_{IC}H_C^\mathsf{T}S^{-1}$: information about relative motion between past poses corrects the current velocity, attitude and biases through the correlations built up by propagation. Injection and reset (Section 14.5) apply to every clone, each with its own $I - \frac{1}{2}[\delta\hat{\theta}_i]_\times$ block.

25.5 Marginalizing the oldest clone

When the window holds `kMaxClones` clones and a new image arrives, the oldest clone must go. In covariance form, marginalization is deletion of its six rows and columns (Theorem 5.11). Before deleting it, the filter must process every feature that has an observation at that clone (AGENTS.md F5), because those observations become unusable once the clone is gone.

Implementation note: in-place deletion without allocation

P is stored in a fixed-max-size matrix whose active part is the top-left $n \times n$ corner. Deleting the block at offset o means shifting every row below it up by six and every column to its right left by six, then reducing n by six. The source and destination regions overlap. Copying toward lower indices in ascending order is safe, because each element is read before it is overwritten (`std::copy` permits overlapping ranges when the destination begins before the source), but an Eigen block assignment between overlapping blocks of the same matrix is an aliasing hazard whose traversal order is not guaranteed. Write the shift explicitly, or through a preallocated member workspace, and test it against a reference built with a dynamic matrix (Milestone 10a).

The clone states themselves are stored in a fixed-capacity container, such as a ring buffer of N_{max} poses with timestamps, so that removing the oldest is a pointer move. If the covariance blocks are kept in the same order as the container, a ring-buffer layout could even avoid the shift, at the cost of more complex index arithmetic. Simplicity is usually the better choice here: the shift costs $O(n^2)$ once per image.

25.6 Window policy and size

The project uses the simplest policy: first in, first out. Each image creates a clone, and the oldest is marginalized when the window is full. More elaborate policies keep clones that are geometrically useful: if the vehicle barely moved since the previous image, marginalize the second-newest clone instead of the oldest, so that the window spans a longer baseline.

The window size trades accuracy for cost.

- **Longer windows** let longer tracks be used in full, with more parallax, and give each update more constraints.
- **Cost** grows as $O(n^3)$ per update with $n = 15 + 6N$, and as $O(n^2)$ per IMU step for the cross-covariance propagation.
- **Memory**: a fixed-max covariance of doubles occupies $8(15 + 6N_{max})^2$ bytes: 45 kB for $N_{max} = 10$, 146 kB for 20, 304 kB for 30. Workspaces for H, S and K add similar amounts.

Pitfall: the stack budget

Fixed-max-size Eigen matrices store their maximum-size buffer *inline*, with no heap allocation. That is what makes them allocation-free, but it also means a filter object constructed on the stack carries all of its matrices on the stack. With the default 1 MB main-thread stack on Windows, a filter with a few hundred kilobytes of workspaces can overflow it. Construct the filter once, during the load phase, as a heap object (for example with `std::make_unique`), which the memory tiers allow, and keep the large matrices as members rather than function-local temporaries (AGENTS.md E4, Milestone 10a).

Key takeaways

- A relative measurement between a past and a present state needs the past state in the filter, with its exact correlations. Treating the past estimate as known makes the filter overconfident.
- Augmentation with $y = g(x)$ uses $P \leftarrow JPJ^\mathsf{T}$, $J = [I; J_g]$; the result is singular at first and becomes regular after propagation.
- Cloning, static propagation and marginalization leave the IMU marginal unchanged; this is a strong test.
- Marginalizing a clone deletes its rows and columns, after first processing its features. Shift in place carefully.
- Fixed-max matrices live inline: keep them as members of a heap-allocated filter.

Exercises

25.1 In the scalar example, compute the posterior covariance of (x_2, x_c) and verify that the clone's variance is unchanged. When would a relative measurement change the clone's estimate?

25.2 Extend the scalar example to two clones and two displacement measurements, and show that processing them gives the same result as one measurement of $x_2 - x_{c_1}$ followed by one of $x_{c_2} - x_{c_1}$, if correlations are kept.

25.3 Implement in-place deletion of a 6×6 block from a 75×75 symmetric matrix stored in a larger buffer, and test it against a reference.

25.4 For $N_{max} = 10, 20, 30$, estimate the flop count of one MSCKF update with 50 features of 6 observations each, with and without QR compression (Chapter 26).

Chapter 26

The MSCKF Visual Update

In this chapter

- Build the linearized measurement model of a feature track over the clones that observed it.
- Eliminate the feature by left-nullspace projection, and prove that no information about the poses is lost.
- Compute the projection efficiently with Householder reflections.
- Gate features, stack them, compress the stack with QR, and apply the update to the IMU state and all clones.

26.1 A feature track

A feature j has been observed at the clones $\mathcal{S}_j = \{i_1, \dots, i_M\}$, with undistorted normalized observations $z_{j,i}$. The update is triggered when the track is lost or when its oldest observation's clone is about to be marginalized, and requires $M \geq 3$ (AGENTS.md F5). The first step is to estimate the feature position $\hat{p}_f$ in the world frame by triangulation from the clone camera poses

$$\hat{R}_{wc_i} = \hat{R}_{wb_i} R_{bc}, \qquad \hat{p}_{wc_i} = \hat{p}_{wb_i} + \hat{R}_{wb_i} p_{bc}, \tag{26.1}$$

using the linear initialization and Gauss–Newton refinement of Section 23.6, with the positive-depth, parallax and reprojection checks. Features that fail are discarded.

26.2 Residuals and Jacobians

For each observation, the predicted measurement uses the clone estimate and the triangulated point,

$$\hat{p}_{c_i} = R_{bc}^{\mathsf{T}} \big(\hat{R}_{wb_i}^{\mathsf{T}} (\hat{p}_f - \hat{p}_{wb_i}) - p_{bc} \big), \qquad r_{j,i} = z_{j,i} - \pi(\hat{p}_{c_i}). \tag{26.2}$$

Linearizing with respect to the error state δx (IMU and clones) and the feature position error $\delta p_f = p_f - \hat{p}_f$,

$$r_{j,i} \approx H_{x,ji}\, \delta x + H_{f,ji}\, \delta p_f + n_{j,i}, \qquad n_{j,i} \sim \mathcal{N}(0, \sigma_{uv}^2 I_2). \tag{26.3}$$

The structure of these Jacobians follows from the chain rule

$$H_{x,ji} = \frac{\partial \pi}{\partial p_c}\Big|_{\hat{p}_{c_i}} \frac{\partial p_{c_i}}{\partial \delta x}, \qquad H_{f,ji} = \frac{\partial \pi}{\partial p_c}\Big|_{\hat{p}_{c_i}} \frac{\partial p_{c_i}}{\partial p_f}, \tag{26.4}$$

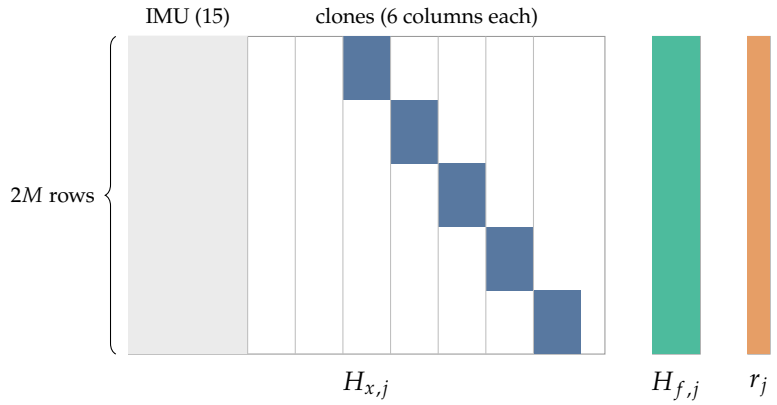


Figure 26.1: Sparsity of the stacked feature Jacobian for a feature seen by five consecutive clones. Each pair of rows depends on one clone; the IMU columns are zero. $H_{f,j}$ is dense.

with the projection Jacobian of Proposition 20.2. The observation at clone i depends only on that clone's pose, so $H_{x,ji}$ is zero except in the six columns of $(\delta p_i, \delta\theta_i)$. In particular it is zero in all IMU columns.

Derivation exercise (project Jacobian): the visual measurement Jacobians $H_{x,ji}$ and $H_{f,ji}$

With $p_{c_i} = R_{bc}^{\mathsf{T}}\big(R_{wb_i}^{\mathsf{T}}(p_f - p_{wb_i}) - p_{bc}\big)$, the local perturbation $R_{wb_i} = \hat{R}_{wb_i}\,\mathrm{Exp}(\delta\theta_i)$ and $p_{wb_i} = \hat{p}_{wb_i} + \delta p_i$:

1. Derive $\partial p_{c_i}/\partial\delta\theta_i$ with the inverse-action row of Table 8.1. Be careful about which vector the rotation acts on.
2. Derive $\partial p_{c_i}/\partial\delta p_i$ and $\partial p_{c_i}/\partial p_f$. How are they related, and why does that make physical sense?
3. Multiply by $\partial\pi/\partial p_c$ and place the blocks in the frozen column layout. State the sparsity pattern of the stacked $H_{x,j}$.

Check: Milestone 10b requires finite-difference tests of the projection Jacobians with respect to the clone δp, the clone $\delta\theta$ and p_f, at a non-trivial clone pose with non-trivial extrinsics.

Worked solution: Appendix E.5 (page 282).

Stacking the M observations of feature j,

$$r_j \approx H_{x,j}\,\delta x + H_{f,j}\,\delta p_f + n_j, \qquad r_j, n_j \in \mathbb{R}^{2M}, \quad H_{x,j} \in \mathbb{R}^{2M\times(15+6N)}, \quad H_{f,j} \in \mathbb{R}^{2M\times 3}. \tag{26.5}$$

26.3 The problem with the feature

Equation (26.5) cannot be used directly in an update. The feature error δp_f is not part of the state, so there is nothing to correct. Worse, $\hat{p}_f$ was computed from the same measurements $z_{j,i}$ and the same clone estimates, so its error is correlated with both n_j and δx in a way the filter's covariance does not describe. Treating $\hat{p}_f$ as known would double count the measurements. EKF-SLAM resolves this by putting p_f in the state. The MSCKF eliminates it.

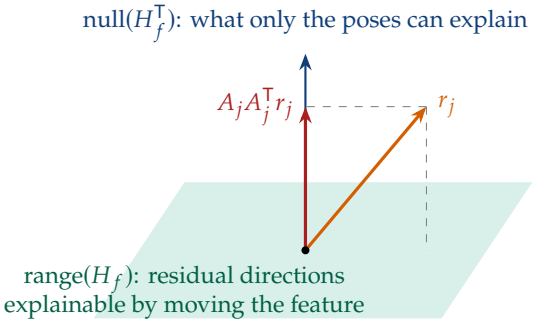


Figure 26.2: Geometry of the nullspace projection. Residual components inside the column space of H_f can be absorbed by an adjustment of the unknown feature position, so they say nothing about the poses. The component orthogonal to it, in the left nullspace, cannot be explained by any feature position and must be explained by the poses.

26.4 Left-nullspace projection

$H_{f,j}$ is a tall $2M \times 3$ matrix. If the feature is well triangulated it has full column rank 3, and its left nullspace has dimension $2M - 3$ (Section 2.5.4). Let $A_j \in \mathbb{R}^{2M\times(2M-3)}$ have orthonormal columns spanning $\operatorname{null}(H_{f,j}^\top)$, so that $A_j^\top H_{f,j} = 0$ and $A_j^\top A_j = I$. Multiplying (26.5) by $A_j^\top$,

$$r_{o,j} := A_j^\top r_j \approx \underbrace{A_j^\top H_{x,j}}_{H_{o,j}}\,\delta x + \underbrace{A_j^\top H_{f,j}}_{=0}\,\delta p_f + A_j^\top n_j = H_{o,j}\,\delta x + n_{o,j}. \tag{26.6}$$

The feature has disappeared, and the projected noise is still white with the same variance: $\operatorname{Cov}(n_{o,j}) = \sigma_{uv}^2 A_j^\top A_j = \sigma_{uv}^2 I_{2M-3}$. Equation (26.6) is a standard EKF measurement of the state alone, of dimension $2M - 3$.

Theorem 26.1 (Nullspace projection loses no information)**.** *Suppose δp_f has no prior information and $H_{f,j}$ has full column rank. Then the information about δx in the projected measurement (26.6) equals the information about δx in (26.5) after δp_f is marginalized out:*

$$H_{o,j}^\top H_{o,j} = H_{x,j}^\top \Big(I - H_{f,j}(H_{f,j}^\top H_{f,j})^{-1} H_{f,j}^\top\Big) H_{x,j}, \tag{26.7}$$

and likewise for the information vectors $H_{o,j}^\top r_{o,j}$. The two formulations therefore produce identical updates.

Proof. The joint information matrix of $(\delta x, \delta p_f)$ contributed by (26.5) is $\sigma^{-2}\begin{bmatrix} H_x^\top H_x & H_x^\top H_f \\ H_f^\top H_x & H_f^\top H_f \end{bmatrix}$. Marginalizing δp_f in information form is the Schur complement (Theorem 5.11), $\sigma^{-2}\big(H_x^\top H_x - H_x^\top H_f (H_f^\top H_f)^{-1} H_f^\top H_x\big)$, and by the projector identity (3.10), $I - H_f(H_f^\top H_f)^{-1}H_f^\top = AA^\top$. The same algebra applies to the information vector $\sigma^{-2}H_x^\top(I - H_f(H_f^\top H_f)^{-1}H_f^\top)r$. ∎

An equivalent way to see this: complete A_j to an orthogonal matrix $Q = [Q_1 \;\; A_j]$. Multiplying by $Q^\top$ splits the equations into three that involve δp_f (through the invertible 3×3 matrix

$Q_1^\top H_f$) and $2M-3$ that do not. Since δp_f is free, the first three can always be satisfied by choosing δp_f, so they carry no information about δx. The rest carry all of it.

Worked example: what survives the projection for two views

Let two cameras with identity orientation observe a point straight ahead of the first, $p_f = (0, 0, Z)$, the second camera being displaced sideways by b. In camera coordinates the point is at $(0, 0, Z)$ and $(-b, 0, Z)$, and by Proposition 20.2 the feature Jacobian (the derivative of the four image coordinates with respect to p_f) is

$$H_f = \frac{1}{Z}\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & b/Z \\ 0 & 1 & 0 \end{bmatrix}.$$

A vector a in its left nullspace must satisfy $a_1 + a_3 = 0$, $a_2 + a_4 = 0$ and $a_3 b/Z = 0$, so $a = (0, 1, 0, -1)/\sqrt{2}$. The projected residual is $r_o = (r_{1,y} - r_{2,y})/\sqrt{2}$: *the point must appear at the same height in both images*. That is the epipolar constraint for a sideways baseline, the condition that stereo rectification exploits. The two horizontal residuals are absorbed by the unknown lateral position and depth (through the disparity), and one vertical residual by the unknown height; only the difference of heights says anything about the cameras.

***Remark* 26.2** (how many observations)**.** Two observations ($M = 2$) leave $2M-3 = 1$ constraint, the epipolar constraint between two clones, as the example shows. It is weak and sensitive to triangulation error, so the project requires $M \geq 3$ (AGENTS.md F5), which gives at least three constraints and a better-conditioned triangulation. If the triangulation is poor (low parallax), $H_{f,j}$ is close to rank-deficient, the projection is ill-defined, and the feature should have been rejected by the triangulation checks.

26.4.1 Computing the projection

A_j is never formed. By Proposition 3.4, the last $2M-3$ columns of the Q factor of a full QR decomposition of $H_{f,j}$ span its left nullspace. Householder QR of the $2M \times 3$ matrix $H_{f,j}$ uses exactly three reflections. Applying the same three reflections to $H_{x,j}$ and r_j, and keeping the last $2M-3$ rows of the results, gives $H_{o,j}$ and $r_{o,j}$ directly:

Algorithm 6 Left-nullspace projection by Householder reflections

Input: $H_f \in \mathbb{R}^{m\times 3}$ ($m = 2M$), $H_x \in \mathbb{R}^{m\times k}$ (only the non-zero clone columns), $r \in \mathbb{R}^m$, stored side by side as $[H_f \mid H_x \mid r]$

1: **for** $c = 1, 2, 3$ **do**
2: $x \leftarrow$ entries $c, \ldots, m$ of column c of H_f
3: $v \leftarrow x$; $v_1 \leftarrow v_1 + \operatorname{sign}(x_1)\|x\|$ ▷ Chapter 3
4: apply $I - 2vv^\top/(v^\top v)$ to rows $c, \ldots, m$ of every column of $[H_f \mid H_x \mid r]$
5: $H_o \leftarrow$ rows $4, \ldots, m$ of H_x; $r_o \leftarrow$ rows $4, \ldots, m$ of r ▷ rows $4..m$ of H_f are now zero

The cost is $O(M \cdot k)$ for k columns, and only the $6M$ non-zero clone columns of $H_{x,j}$ need processing. Givens rotations achieve the same with finer control over sparsity and were

suggested in the original paper. Milestone 10b checks $\|A^\mathsf{T} H_f\| < 10^{-10}$ and the orthonormality of A in a test, where forming A explicitly is acceptable.

26.5 Gating, stacking and compression

Per-feature gating. Each projected residual is gated before it is accepted (AGENTS.md F5):

$$\gamma_j = r_{o,j}^\mathsf{T} \left(H_{o,j} P H_{o,j}^\mathsf{T} + \sigma_{uv}^2 I \right)^{-1} r_{o,j} < \chi^2_{0.95}(2M - 3). \tag{26.8}$$

This rejects tracks that passed the front end's checks but are inconsistent with the filter's current belief: tracks on moving objects, slowly drifting tracks, and features with wrong associations.

Stacking. The accepted features are stacked into one measurement,

$$r_o = H_o \, \delta x + n_o, \qquad r_o \in \mathbb{R}^d, \quad d = \sum_j (2M_j - 3), \tag{26.9}$$

with the number of rows capped at a compile-time maximum so that the stacked workspace is fixed-size. With 50 features of 6 observations each, $d = 450$, much larger than the state dimension $n = 75$.

QR compression. When $d > n$, the update can be made cheaper without approximation.

> **Proposition 26.3** (Measurement compression). *Let $H_o = Q_1 T_H$ be a thin QR decomposition, with $T_H \in \mathbb{R}^{n \times n}$ upper triangular, and define $r_n = Q_1^\mathsf{T} r_o$. Then the measurement $r_n = T_H \, \delta x + n_n$ with $n_n \sim \mathcal{N}(0, \sigma_{uv}^2 I_n)$ yields exactly the same update as (r_o, H_o).*

Proof. The update depends on the measurement only through the information $H_o^\mathsf{T} H_o / \sigma^2$ and the information vector $H_o^\mathsf{T} r_o / \sigma^2$ (Equation (12.8)). Since $Q_1^\mathsf{T} Q_1 = I$, $T_H^\mathsf{T} T_H = H_o^\mathsf{T} Q_1 Q_1^\mathsf{T} H_o = H_o^\mathsf{T} H_o$ (as $Q_1 Q_1^\mathsf{T}$ projects onto range(H_o), which contains the columns of H_o), and $T_H^\mathsf{T} r_n = H_o^\mathsf{T} Q_1 Q_1^\mathsf{T} r_o = H_o^\mathsf{T} r_o$. The discarded components $Q_2^\mathsf{T} r_o$ are pure noise, independent of δx. ∎

Compression reduces the innovation covariance from $d \times d$ to at most $n \times n$, and the Joseph update from $O(dn^2 + d^3)$ to $O(n^3)$ after an $O(dn^2)$ QR. Only the non-zero columns of H_o (the clone columns) need to take part in the decomposition. Compression is optional in the project (AGENTS.md F5); it is worthwhile when $d \gg n$.

26.6 The update

The stacked (and possibly compressed) measurement is applied with the generic update of Chapter 18: $S = HPH^\mathsf{T} + \sigma_{uv}^2 I$, gain by Cholesky solve, Joseph form, then injection into the IMU state and *every* clone, normalization of every touched quaternion, and the reset with one $I - \frac{1}{2}[\delta\hat{\theta}_i]_\times$ block per attitude. Corrections to the clones are real: they improve the poses that future features will be triangulated from.

Algorithm 7 MSCKF visual update

Input: filter state with clones, set of feature tracks ready for update

$d \leftarrow 0$
for each ready feature j with $M_j \geq 3$ observations **do**
  triangulate $\hat{p}_f$ (linear + Gauss–Newton); **if** checks fail **then** discard, **continue**
  build r_j, $H_{x,j}$ (clone columns only), $H_{f,j}$
  apply Householder QR of $H_{f,j}$ to $[H_{x,j} \;\; r_j]$; keep rows $4 \ldots 2M_j \rightarrow H_{o,j}$, $r_{o,j}$
  if $\gamma_j < \chi^2_{0.95}(2M_j - 3)$ **and** $d + 2M_j - 3 \leq d_{max}$ **then**
    append $H_{o,j}$, $r_{o,j}$ to the stacked workspace; $d \leftarrow d + 2M_j - 3$
  else
    count the rejection
  mark the track's observations as consumed
if $d > 0$ **then**
  optionally compress with QR if $d > n$
  EKF update (Joseph), inject into IMU and all clones, normalize, reset with G

Project link: Milestone 10b

The exit criteria test every link of Algorithm 7: projection Jacobians by finite differences; triangulation exact on noise-free data and bounded under noise, rejecting negative-depth and low-parallax features; $\|A^\top H_f\| < 10^{-10}$ with orthonormal A; a full synthetic VIO run without divergence, with NEES reported (yaw inconsistency expected and documented); zero allocations with fixed-max workspaces; and MH_01 end to end with ATE reported.

Key takeaways

- Each observation depends on one clone; $H_{x,j}$ is zero in the IMU columns and dense in the feature's $H_{f,j}$.
- Projecting onto the left nullspace of $H_{f,j}$ removes the feature, keeps the noise white, and loses no information about the poses (it equals Schur-complement marginalization).
- Three Householder reflections applied to $[H_{x,j} \; r_j]$ compute the projection without forming A_j.
- Gate each feature with $\chi^2_{0.95}(2M-3)$, stack, compress with QR if $d > n$, and update IMU and clones together.

Exercises

26.1 Verify Theorem 26.1 numerically: compare the posterior covariance of δx from the projected update with that from an update of the augmented state $(\delta x, \delta p_f)$ with a very large prior on δp_f.

26.2 Show that the three discarded rows $Q_1^\top r_j$ determine the correction to $\hat{p}_f$ given δx. How would you recover an updated feature estimate if you wanted one?

26.3 For $M = 2$, compute A_j explicitly and interpret the single projected residual geometri-

cally.

26.4 Implement the Householder application of Algorithm 7 and compare its cost and result with forming A_j from a full QR.

26.5 Prove that the projected noise vectors $n_{o,j}$ of two different features are independent if their raw noises are, so that stacking them with a block-diagonal (here scalar) noise covariance is correct. Why are the projected *residuals* themselves correlated?

Chapter 27

A Complete MSCKF in the Plane

In this chapter

- Build a complete MSCKF for a robot moving in a plane, small enough that every Jacobian can be derived by hand in a few lines.
- Meet every ingredient of the full filter in its simplest form: nominal and error state, propagation, cloning, triangulation, nullspace projection, gating, update and marginalization.
- See, in a three-view example, what the projected residual actually measures.
- Run the filter in simulation and check its accuracy and consistency.
- Identify the unobservable directions, and list exactly what changes in three dimensions.

The previous three chapters described the MSCKF in full generality. This chapter does something different: it builds the whole filter for a toy problem in which nothing is hidden. A robot drives on a plane, measures its speed and turn rate, and carries a camera that sees the world as a line of pixels. Every equation is derived, every Jacobian is checked against geometric intuition, and the result is run in a simulation whose code accompanies these notes (`scripts/planar_sims.py`, about two hundred lines of Python). If the general chapters felt abstract, work through this one with a pencil; afterwards, the three-dimensional filter differs only in details that Section 27.10 lists one by one.

Project link: what this chapter does and does not give away

The planar Jacobians below are derived in full because the planar system is not the project's system. The three-dimensional Jacobians (F_c, G_c, the pose and camera H, the cloning J) differ precisely where the difficulty lies: rotations that do not commute, the frozen local perturbation, gravity, biases, and the reset Jacobian. Deriving them remains your task (AGENTS.md Guardrail 4). Use this chapter as the template: the same procedure (perturb, expand to first order, read off the coefficient), and the same finite-difference checks.

27.1 The planar world

27.1.1 Frames and pose

The world frame W is a fixed plane with axes x_w, y_w. The robot carries a body frame B with x_b pointing forward and y_b to the left. Its pose is the position $p \in \mathbb{R}^2$ of the body origin in W

and the heading ψ, the angle from x_w to x_b. The rotation

$$R(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix} \tag{27.1}$$

maps body-frame vectors to the world frame, $v_w = R(\psi)\, v_b$: it plays the role of R_{wb} (AGENTS.md C1). A point p_f given in W has body coordinates $p_b = R(\psi)^{\mathsf{T}}(p_f - p)$.

27.1.2 The planar skew matrix

The matrix

$$\Omega = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \tag{27.2}$$

rotates a vector by $+90°$: $\Omega\,(a_1, a_2) = (-a_2, a_1)$. It is the planar version of the skew matrix: for a rotation about the z-axis, $[\omega e_z]_\times$ acts on the xy-plane exactly as $\omega\,\Omega$ (compare Section 8.2).

Lemma 27.1 (Properties of Ω). *For all ψ, δ and $a \in \mathbb{R}^2$:*

1. $\Omega^{\mathsf{T}} = -\Omega$, $\Omega^2 = -I$, *and* $a^{\mathsf{T}}\Omega a = 0$;
2. $R(\psi) = \cos\psi\, I + \sin\psi\, \Omega = \exp(\psi\,\Omega)$;
3. $R(\psi)\,\Omega = \Omega\, R(\psi)$ *and* $R(\psi)^{\mathsf{T}}\Omega = \Omega R(\psi)^{\mathsf{T}}$;
4. $\dfrac{\mathrm{d}}{\mathrm{d}\psi} R(\psi) = \Omega R(\psi)$, *and* $R(\psi + \delta) = R(\psi)\big(I + \delta\,\Omega\big) + O(\delta^2)$;
5. $R(\psi + \delta)^{\mathsf{T}} = (I - \delta\,\Omega)\, R(\psi)^{\mathsf{T}} + O(\delta^2)$.

Proof. (1) Direct multiplication; $a^{\mathsf{T}}\Omega a = -a_1 a_2 + a_2 a_1 = 0$. (2) Read off the entries of $\cos\psi\, I + \sin\psi\,\Omega$. For the exponential, $\Omega^2 = -I$ gives $\Omega^{2k} = (-1)^k I$ and $\Omega^{2k+1} = (-1)^k \Omega$, so the series splits into the cosine and sine series. (3) By (2), $R(\psi)$ is a combination of I and Ω, and both commute with Ω; transpose for the second identity. (4) Differentiate (2): $-\sin\psi\, I + \cos\psi\,\Omega = \Omega(\cos\psi\, I + \sin\psi\,\Omega)$, using $\Omega^2 = -I$. Then $R(\psi + \delta) = R(\psi)R(\delta)$ and $R(\delta) = I + \delta\,\Omega + O(\delta^2)$ by (2). (5) Transpose (4) and use $\Omega^{\mathsf{T}} = -\Omega$. ∎

Property (3) is the reason the plane is easy: rotations commute with each other and with Ω, so $R(\hat{\psi})R(\delta) = R(\delta)R(\hat{\psi})$ and a heading error can be added on either side. In three dimensions neither is true, and the frozen local perturbation of AGENTS.md C4 is a real choice with real consequences.

27.2 Sensors

27.2.1 Odometry

Wheel odometry plays the part of the IMU. It measures the forward speed v and the turn rate ω in the body frame, with white noise:

$$v_m = v + n_v, \qquad \omega_m = \omega + n_\omega, \qquad \mathbb{E}[n_v(t)n_v(s)] = \sigma_v^2\,\delta(t-s), \quad \mathbb{E}[n_\omega(t)n_\omega(s)] = \sigma_\omega^2\,\delta(t-s). \tag{27.3}$$

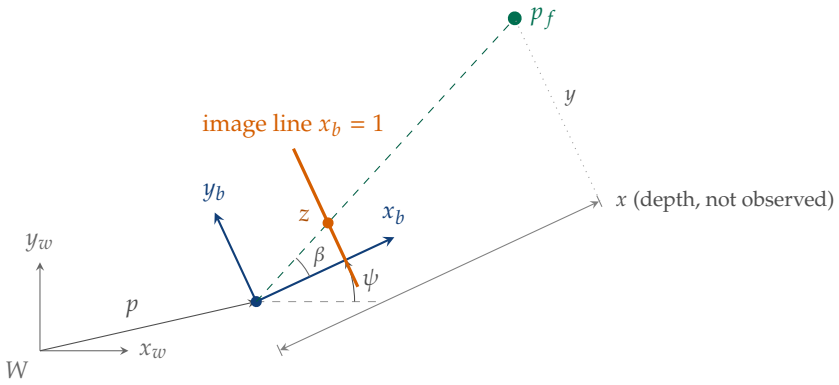


Figure 27.1: The planar robot and its one-dimensional camera. The feature p_f has body coordinates (x, y); the camera reports where the ray to it crosses the image line $x_b = 1$, the normalized coordinate $z = y/x = \tan\beta$. Moving the feature along the ray does not change z.

As for the IMU (Section 6.3), σ_v and σ_ω are continuous-time noise densities, in $\mathrm{m/s}/\sqrt{\mathrm{Hz}}$ and $\mathrm{rad/s}/\sqrt{\mathrm{Hz}}$; a sample at period Δt has standard deviation $\sigma/\sqrt{\Delta t}$. The robot does not slip sideways, so the kinematics are

$$\dot{p} = R(\psi)\, e_1\, v, \qquad \dot{\psi} = \omega, \qquad e_1 = (1, 0). \tag{27.4}$$

There are no biases and no gravity. Both appear in the exercises.

27.2.2 A one-dimensional camera

The camera sits at the body origin and looks along x_b. A planar pinhole camera has a one-dimensional image: the point $p_b = (x, y)$ in front of the camera ($x > 0$) is projected onto the *image line* $x = 1$ at the normalized coordinate

$$z = \pi(p_b) + n_z = \frac{y}{x} + n_z, \qquad n_z \sim \mathcal{N}(0, \sigma_z^2). \tag{27.5}$$

This is the planar analog of the normalized image coordinates of Chapter 20: a real 1D sensor with focal length f (in pixels) and principal point c would report $u = fz + c$, so a pixel noise σ_{px} corresponds to $\sigma_z = \sigma_{px}/f$. The camera measures only the *bearing* $\beta = \arctan z$ of the point; its range is invisible, exactly as depth is invisible to a monocular camera in space.

27.3 State, error state and propagation

27.3.1 The state

The filter state is the current robot pose followed by N clones of past poses, taken at image times:

$$x = \left(p,\ \psi,\ \ p_1, \psi_1,\ \ldots,\ p_N, \psi_N\right) \in \mathbb{R}^{3+3N}. \tag{27.6}$$

Because planar rotations commute, the retraction is ordinary addition (wrapping angles to $(-\pi, \pi]$): $p = \hat{p} + \delta p$ and $\psi = \hat{\psi} + \delta\psi$, which is simultaneously the local and the global

perturbation, and $\delta x = (\delta p, \delta\psi, \delta p_1, \delta\psi_1, \dots)$. The state has no velocity block, since speed is measured directly rather than integrated from accelerations.

27.3.2 Nominal propagation

Between odometry samples the inputs are held constant. The nominal state is integrated with the midpoint rule for the heading,

$$\hat{p}_{k+1} = \hat{p}_k + R\big(\hat{\psi}_k + \tfrac{1}{2}\omega_m \Delta t\big)\, e_1\, v_m \Delta t, \qquad \hat{\psi}_{k+1} = \hat{\psi}_k + \omega_m \Delta t, \tag{27.7}$$

which is the exact solution for an arc travelled at constant speed and turn rate, up to the factor $\sin(\omega\Delta t/2)/(\omega\Delta t/2) = 1 - O(\Delta t^2)$ on its length. Clones do not move.

27.3.3 Error dynamics

Proposition 27.2 (Planar error dynamics). *To first order in the errors and the noise, the robot error obeys $\dot{\delta x}_R = F_c\, \delta x_R + G_c\, n$ with $\delta x_R = (\delta p, \delta\psi)$, $n = (n_v, n_\omega)$ and*

$$F_c = \begin{bmatrix} 0_{2\times 2} & \Omega\, R(\hat{\psi})\, e_1\, v_m \\ 0_{1\times 2} & 0 \end{bmatrix}, \qquad G_c = \begin{bmatrix} -R(\hat{\psi})\, e_1 & 0_{2\times 1} \\ 0 & -1 \end{bmatrix}. \tag{27.8}$$

The top-right block equals $\Omega\, \dot{\hat{p}}$: a heading error $\delta\psi$ makes the position error grow perpendicular to the direction of travel.

Proof. Write the true quantities as nominal plus error, $\psi = \hat{\psi} + \delta\psi$, and the true inputs as measured minus noise, $v = v_m - n_v$, $\omega = \omega_m - n_\omega$. For the heading, $\dot{\delta\psi} = \dot{\psi} - \dot{\hat{\psi}} = (\omega_m - n_\omega) - \omega_m = -n_\omega$. For the position, by Lemma 27.1(4),

$$\begin{aligned} \dot{p} &= R(\hat{\psi} + \delta\psi)\, e_1\, (v_m - n_v) = R(\hat{\psi})(I + \delta\psi\, \Omega)\, e_1\, (v_m - n_v) + O(\delta\psi^2) \\ &= R(\hat{\psi}) e_1 v_m + \delta\psi\, R(\hat{\psi})\Omega e_1\, v_m - R(\hat{\psi}) e_1\, n_v + \dots, \end{aligned}$$

dropping the product $\delta\psi\, n_v$. Subtracting $\dot{\hat{p}} = R(\hat{\psi}) e_1 v_m$ and using $R\Omega = \Omega R$ gives $\dot{\delta p} = \Omega R(\hat{\psi}) e_1 v_m\, \delta\psi - R(\hat{\psi}) e_1\, n_v$, which is the first block row. ∎

Discretization follows Chapter 11 exactly: $\Phi \approx I + F_c \Delta t$, and $Q_d \approx G_c Q_c G_c^{\mathsf{T}} \Delta t$ with $Q_c = \operatorname{diag}(\sigma_v^2, \sigma_\omega^2)$. The covariance is propagated block by block, with static clones:

$$P_{RR} \leftarrow \Phi P_{RR} \Phi^{\mathsf{T}} + Q_d, \qquad P_{RC} \leftarrow \Phi P_{RC}, \qquad P_{CC} \text{ unchanged}, \tag{27.9}$$

followed by symmetrization.

For the observability analysis we also need the transition matrix of the discrete integrator (27.7) itself, without the $O(\Delta t^2)$ truncation of $I + F_c \Delta t$.

Proposition 27.3 (Exact discrete error transition). *Linearizing (27.7) in the errors gives $\delta p_{k+1} = \delta p_k + \Omega\, (\hat{p}_{k+1} - \hat{p}_k)\, \delta\psi_k + (\text{noise})$ and $\delta\psi_{k+1} = \delta\psi_k + (\text{noise})$, that is*

$$\Phi_k = \begin{bmatrix} I_2 & \Omega\, (\hat{p}_{k+1} - \hat{p}_k) \\ 0 & 1 \end{bmatrix}. \tag{27.10}$$

Proof. The true state obeys the same recursion with the true heading and inputs. A heading error $\delta\psi_k$ rotates the step $s = R(\hat{\psi}_k + \frac{1}{2}\omega_m\Delta t)e_1 v_m\Delta t$ into $R(\delta\psi_k)s \approx s + \delta\psi_k\,\Omega s$, and $s = \hat{p}_{k+1} - \hat{p}_k$. ∎

The transition depends on the estimates only through the displacement $\hat{p}_{k+1} - \hat{p}_k$. Over many steps the products of such matrices telescope, $\Phi_{k-1}\cdots\Phi_0 = \begin{bmatrix} I & \Omega(\hat{p}_k - \hat{p}_0) \\ 0 & 1 \end{bmatrix}$, provided each displacement is computed from *the same* estimates; Chapter 29 shows what happens when it is not.

27.4 Cloning and marginalization

At an image time the robot pose is copied into a new clone. The clone is an exact copy, so its error is the robot's error: $\delta p_{N+1} = \delta p$, $\delta\psi_{N+1} = \delta\psi$. With the robot block first, Proposition 25.1 gives

$$P \leftarrow JPJ^{\mathsf{T}}, \qquad J = \begin{bmatrix} I_{3+3N} \\ \begin{bmatrix} I_3 & 0_{3\times 3N} \end{bmatrix} \end{bmatrix}. \tag{27.11}$$

No linearization is involved: the copy is exact, which is why the new clone's covariance block and its cross-covariances equal the robot's *exactly*. When the window is full, the features that the oldest clone observed are processed first, then the clone is marginalized by deleting its three rows and columns, as in Chapter 25.

27.5 The feature measurement

27.5.1 Residual and Jacobians

Feature j, at the unknown position p_f, is observed by clone i at the coordinate z_i. With the triangulated estimate $\hat{p}_f$ and the clone estimate $(\hat{p}_i, \hat{\psi}_i)$, the predicted body-frame point and the residual are

$$\hat{p}_{b,i} = R(\hat{\psi}_i)^{\mathsf{T}}(\hat{p}_f - \hat{p}_i) = (\hat{x}_i, \hat{y}_i), \qquad r_i = z_i - \hat{y}_i/\hat{x}_i. \tag{27.12}$$

Proposition 27.4 (Planar camera Jacobians). *Let $\hat{z}_i = \hat{y}_i/\hat{x}_i$ and $J_\pi = \frac{1}{\hat{x}_i}\begin{bmatrix} -\hat{z}_i & 1 \end{bmatrix}$, the derivative of y/x with respect to (x, y). To first order, $r_i \approx H_{x,i}\,\delta x + H_{f,i}\,\delta p_f + n_{z,i}$, where $H_{x,i}$ is zero except in the three columns of clone i, and*

$$\frac{\partial r_i}{\partial \delta p_i} = -J_\pi R(\hat{\psi}_i)^{\mathsf{T}}, \qquad \frac{\partial r_i}{\partial \delta\psi_i} = -J_\pi\Omega\,\hat{p}_{b,i} = -\left(1 + \hat{z}_i^2\right), \qquad H_{f,i} = \frac{\partial r_i}{\partial \delta p_f} = J_\pi R(\hat{\psi}_i)^{\mathsf{T}}. \tag{27.13}$$

Proof. The measurement is $z_i = \pi(p_{b,i}) + n_{z,i}$ with $p_{b,i} = R(\psi_i)^{\mathsf{T}}(p_f - p_i)$, so $r_i = z_i - \pi(\hat{p}_{b,i}) \approx J_\pi\,(p_{b,i} - \hat{p}_{b,i}) + n_{z,i}$: we need the first-order change of $p_{b,i}$. Substituting $p_i = \hat{p}_i + \delta p_i$, $p_f = \hat{p}_f + \delta p_f$ and, by Lemma 27.1(5), $R(\hat{\psi}_i + \delta\psi_i)^{\mathsf{T}} \approx (I - \delta\psi_i\Omega)R(\hat{\psi}_i)^{\mathsf{T}}$,

$$p_{b,i} \approx (I - \delta\psi_i\Omega)\,R(\hat{\psi}_i)^{\mathsf{T}}(\hat{p}_f - \hat{p}_i + \delta p_f - \delta p_i) \approx \hat{p}_{b,i} - \delta\psi_i\,\Omega\,\hat{p}_{b,i} + R(\hat{\psi}_i)^{\mathsf{T}}(\delta p_f - \delta p_i).$$

Multiplying by J_π gives the three Jacobians. For the heading term, $J_\pi \Omega \hat{p}_{b,i} = \frac{1}{\hat{x}} \begin{bmatrix} -\hat{z} & 1 \end{bmatrix} \begin{bmatrix} -\hat{y} \\ \hat{x} \end{bmatrix} = \frac{\hat{z}\hat{y}+\hat{x}}{\hat{x}} = \hat{z}^2 + 1$, using $\hat{y} = \hat{z}\hat{x}$. ∎

Each block has a clear meaning.

- **Rotation** shifts every bearing by the same angle: turning the robot left by $\delta\psi$ decreases every bearing by $\delta\psi$, and since $z = \tan\beta$, $\mathrm{d}z = (1 + z^2)\,\mathrm{d}\beta$. The heading column does not depend on the depth at all. This is why a camera measures rotation so well, even from points at infinity.
- **Translation** enters through J_π, which carries the factor $1/\hat{x}$: moving the robot shifts near points a lot and far points hardly at all. This is *parallax*, the only way a camera senses translation.
- **The feature** enters with the opposite sign of the clone position: moving the feature by δp_f looks, to this camera, exactly like moving the robot by $-\delta p_f$. Only the relative position $p_f - p_i$ matters.

Implementation note: check the Jacobians numerically anyway

Even a three-line derivation deserves a finite-difference check (Section 4.4). Perturb each clone coordinate and the feature by $\varepsilon = 10^{-6}$, recompute $\pi(R(\psi_i)^\mathsf{T}(p_f - p_i))$, and compare the difference quotients with (27.13); agreement to about 10^{-8} is expected. Flip the sign of the heading term and the check fails at once, which is the point.

27.5.2 Triangulation

To linearize we need $\hat{p}_f$. Observation i says that p_f lies on the ray from $\hat{p}_i$ in the direction $d_i = R(\hat{\psi}_i)\,(1, z_i)$, that is $p_f - \hat{p}_i = \lambda_i d_i$ for some depth $\lambda_i > 0$. The depth is eliminated with the normal $n_i = \Omega d_i$, which is perpendicular to the ray (Lemma 27.1(1)):

$$n_i^\mathsf{T}(p_f - \hat{p}_i) = 0, \qquad i = 1, \dots, M \qquad \Longrightarrow \qquad \underbrace{\begin{bmatrix} n_1^\mathsf{T} \\ \vdots \\ n_M^\mathsf{T} \end{bmatrix}}_{A} p_f = \underbrace{\begin{bmatrix} n_1^\mathsf{T}\hat{p}_1 \\ \vdots \\ n_M^\mathsf{T}\hat{p}_M \end{bmatrix}}_{b}. \tag{27.14}$$

Two non-parallel rays determine p_f, and more rays give an overdetermined system solved by least squares (Section 3.5). If the rays are nearly parallel (too little parallax), A is ill-conditioned and the feature is rejected; the simulation rejects features with $\kappa(A) > 10^4$. The linear solution minimizes algebraic distances, not the measurement errors, so it is refined by a few Gauss–Newton iterations (Section 10.3) on $\sum_i (z_i - \pi(R(\hat{\psi}_i)^\mathsf{T}(p_f - \hat{p}_i)))^2$, whose Jacobian rows are exactly the $H_{f,i}$ of (27.13). Finally every depth $\hat{x}_i$ must be positive (in the simulation, at least 0.2 m): a point behind a camera is a failed triangulation, not a measurement.

27.6 Eliminating the feature

Stacking the M observations of feature j gives $r_j \approx H_{x,j}\,\delta x + H_{f,j}\,\delta p_f + n_j$ with $H_{f,j} \in \mathbb{R}^{M\times 2}$. If the rays are not parallel, $H_{f,j}$ has rank 2 and its left nullspace has dimension $M-2$: *a feature seen M times gives $M-2$ constraints on the poses*. Two observations give none, because two rays in a plane always intersect, so any pair of bearings is consistent with some feature position. With A_j an orthonormal basis of the left nullspace (computed by QR, Proposition 3.4),

$$r_{o,j} = A_j^\mathsf{T} r_j \approx A_j^\mathsf{T} H_{x,j}\,\delta x + A_j^\mathsf{T} n_j, \qquad \mathrm{Cov}(A_j^\mathsf{T} n_j) = \sigma_z^2 I_{M-2}, \tag{27.15}$$

exactly as in Chapter 26. For three observations the nullspace is a single vector, and in $\mathbb{R}^3$ there is a closed form for it.

Proposition 27.5 (Three views). *If $M = 3$ and the columns $h_1, h_2 \in \mathbb{R}^3$ of $H_{f,j}$ are linearly independent, the left nullspace of $H_{f,j}$ is spanned by the cross product $a = h_1 \times h_2$.*

Proof. $a^\mathsf{T} h_1 = (h_1 \times h_2)\cdot h_1 = 0$ and likewise for h_2 (Section 2.3), and $a \neq 0$ because the columns are independent. The nullspace has dimension $3-2=1$, so a spans it. ∎

Worked example: what three views measure

Let the robot face along x_w ($\hat\psi_i = 0$) at the three positions $\hat p_1 = (0,-1)$, $\hat p_2 = (0,0)$, $\hat p_3 = (0,1)$, a sideways motion with two equal baselines, and let the feature be at $\hat p_f = (4,0)$. The body-frame points are $(4,1)$, $(4,0)$ and $(4,-1)$, so $\hat z = (\frac14, 0, -\frac14)$ and, by (27.13),

$$H_f = \begin{bmatrix} -\frac{1}{16} & \frac14 \\ 0 & \frac14 \\ \frac{1}{16} & \frac14 \end{bmatrix}, \qquad h_1 \times h_2 = \tfrac{1}{64}(-1,2,-1), \qquad a = \tfrac{1}{\sqrt6}(1,-2,1).$$

The projected residual is $r_o = (r_1 - 2r_2 + r_3)/\sqrt6$, a *second difference*. The reason is visible in the geometry: for cameras translating sideways at a fixed heading, $z_i = (y_f - y_i)/x_f$ is a linear function of the camera position y_i, whatever the feature. A feature can absorb any offset and any slope of the three residuals; only their curvature is left for the poses to explain.

The projected Jacobian is $H_o = a^\mathsf{T} H_x$, with clone blocks $(\delta p_x, \delta p_y, \delta\psi)$

$$H_o = \tfrac{1}{\sqrt6}\Big[\ \big(\tfrac{1}{16}, -\tfrac14, -\tfrac{17}{16}\big)\ \ (0, \tfrac12, 2)\ \ \big(-\tfrac{1}{16}, -\tfrac14, -\tfrac{17}{16}\big)\ \Big],$$

since the clone blocks of H_x are $\big(\frac{1}{16}, -\frac14, -\frac{17}{16}\big)$, $\big(0, -\frac14, -1\big)$ and $\big(-\frac{1}{16}, -\frac14, -\frac{17}{16}\big)$. Now test what it can see by multiplying H_o by a pose perturbation:

- *Translate all three clones* by the same (δ_x, δ_y): $H_o\,\delta x = \frac{1}{\sqrt6}\big[\frac{1}{16} - \frac{1}{16}\big]\delta_x + \frac{1}{\sqrt6}\big[-\frac14 + \frac12 - \frac14\big]\delta_y = 0$. Invisible.
- *Rotate everything* about the origin by δ: each clone moves by $\delta\,\Omega\hat p_i$, that is $(\delta, 0)$, $(0,0)$, $(-\delta, 0)$, and turns by δ. The result is $\frac{\delta}{\sqrt6}\big[\frac{1}{16} - \frac{17}{16} + 2 + \frac{1}{16} - \frac{17}{16}\big] = 0$. Invisible.
- *Scale* the configuration about the origin: $\delta p_1 = (0, -s)$, $\delta p_3 = (0, s)$, $\delta p_2 = 0$, giving

Algorithm 8 Planar MSCKF

Input: initial pose estimate and covariance, window size $N_{\max}$

for each odometry sample (v_m, ω_m) with period Δt **do**
  propagate P with $\Phi = I + F_c\Delta t$, $Q_d = G_c Q_c G_c^\mathsf{T}\Delta t$ (Proposition 27.2); clones static
  propagate the nominal pose with (27.7)
  if an image arrives at this time **then**
    collect tracks that ended (feature not seen in this image)
    if $N = N_{\max}$ **then** also collect tracks whose first observation is the oldest clone
    for each collected track with $M \geq 3$ **do**
      triangulate (27.14) + Gauss–Newton; reject on parallax or negative depth
      build $r_j, H_{x,j}, H_{f,j}$ with (27.13); project onto the left nullspace of $H_{f,j}$
      gate with $\chi^2_{0.95}(M-2)$; keep or count the rejection
    stack the survivors; EKF update (Joseph); inject into the robot and all clones
    if $N = N_{\max}$ **then** marginalize the oldest clone
    clone the current pose; append this image's observations to their tracks

$\frac{s}{\sqrt{6}}\left[\frac{1}{4} - \frac{1}{4}\right] = 0$. Invisible: a camera cannot measure scale.

- *Move only the middle clone* sideways by δ_y: $H_o\,\delta x = \delta_y/(2\sqrt{6}) \neq 0$. Visible: the camera measures the *ratio* of the two baselines.
- *Turn only the middle clone* by δ: $H_o\,\delta x = 2\delta/\sqrt{6} \neq 0$. Visible: relative headings are measured directly.

This small example contains the whole story of monocular visual odometry: relative rotations and the shape of the trajectory are observed, while the global position, the global heading and the scale are not. In the filter, the odometry speed supplies the scale, and nothing supplies the global pose.

Gating, stacking and the update. Each projected residual is gated with $\gamma_j = r_{o,j}^\mathsf{T}(H_{o,j}PH_{o,j}^\mathsf{T} + \sigma_z^2 I)^{-1} r_{o,j} < \chi^2_{0.95}(M-2)$, the survivors are stacked into one measurement, and the update is the generic one of Chapter 18: $S = HPH^\mathsf{T} + \sigma_z^2 I$, the gain by a linear solve, the Joseph form, symmetrization. Injection adds the correction to the robot and to every clone, $\hat{p}_i \leftarrow \hat{p}_i + \delta\hat{p}_i$, $\hat{\psi}_i \leftarrow \hat{\psi}_i + \delta\hat{\psi}_i$ (wrapped). There is no reset Jacobian: after injection the new error is $\psi - (\hat{\psi} + \delta\hat{\psi}) = \delta\psi - \delta\hat{\psi}$, whose derivative with respect to $\delta\psi$ is exactly 1. In three dimensions the corresponding derivative is $I - \frac{1}{2}[\delta\hat{\theta}]_\times$ (Section 14.5), because rotations do not commute.

27.7 The complete filter

Algorithm 8 collects the steps; it is the full MSCKF of Chapter 28 with the planar models substituted.

The heart of the filter, the per-feature residual, is short enough to show in full. It mirrors (27.13) line by line; `OM` is Ω and `rot(psi)` is $R(\psi)$.

```
def feature_residual(self, obs):              # obs: list of (clone id, z)
    pf = self.triangulate(obs)                # linear + Gauss-Newton, with checks
    if pf is None:
```

Table 27.1: Planar simulation parameters.

Trajectory	circle of radius 10 m at 1 m/s and 0.1 rad/s, 90 s (1.4 laps)
Odometry	100 Hz; $\sigma_v = 0.01$ m/s/$\sqrt{\text{Hz}}$, $\sigma_\omega = 0.02$ rad/s/$\sqrt{\text{Hz}}$
Camera	10 Hz; field of view ±50°; depths 1 m–15 m; $\sigma_z = 0.003$ (1.5 px at $f = 500$ px)
Landmarks	260, in two rings (radii 3 m–7 m and 13 m–19 m)
Window	$N_{\max} = 10$ clones (1 s)
Initial uncertainty	$\sigma_p = 1$ cm per axis, $\sigma_\psi = 3$ mrad

```
        return None
    m = len(obs)
    Hx, Hf, r = np.zeros((m, self.n)), np.zeros((m, 2)), np.zeros(m)
    for row, (cid, z) in enumerate(obs):
        k = self.clone_index(cid)
        _, p_i, psi_i = self.clones[k]
        Rt = rot(psi_i).T
        pb = Rt @ (pf - p_i)                  # feature in the body frame of the clone
        Jpi = np.array([-pb[1] / pb[0]**2, 1.0 / pb[0]])   # d(y/x)/d(x, y)
        col = 3 + 3 * k                       # columns of clone k
        Hx[row, col:col + 2] = -Jpi @ Rt      # clone position
        Hx[row, col + 2] = -Jpi @ (OM @ pb)   # clone heading: -(1 + z^2)
        Hf[row] = Jpi @ Rt                    # feature position
        r[row] = z - pb[1] / pb[0]
    Q, _ = qr(Hf, mode="full")
    A = Q[:, 2:]                              # orthonormal basis of the left nullspace
    Ho, ro = A.T @ Hx, A.T @ r
    S = Ho @ self.P @ Ho.T + self.sig_z**2 * np.eye(m - 2)
    if ro @ np.linalg.solve(S, ro) > chi2.ppf(0.95, m - 2):
        self.rejected += 1                    # gated out: counted, never silent
        return None
    return Ho, ro
```

A production implementation would not form `A`; it would apply the Householder reflections of Algorithm 6 to $[H_f \mid H_x \mid r]$, and it would work only with the non-zero clone columns of H_x.

27.8 Simulation results

Table 27.1 lists the simulation set-up. The truth moves exactly on the circle; odometry samples are drawn with the noise densities of the table, and each landmark inside the field of view and depth range produces a bearing with noise σ_z. Dead reckoning, the same nominal integration without any camera update, serves as the baseline.

The results (Figures 27.2 to 27.4) illustrate four points.

1. **The camera helps, mostly through the heading.** Odometry alone accumulates a heading error with standard deviation $\sigma_\omega\sqrt{t} = 0.19$ rad after 90 s. The MSCKF ends at 0.067 rad: the camera measures the relative rotation between clones far better than the gyro-like turn rate, and because heading errors turn into cross-track position errors, the position error drops by a factor of 2.5.
2. **The error still grows.** Even with the camera, the heading and position uncertainties grow without bound (Figure 27.3). This is not a failure: global position and heading

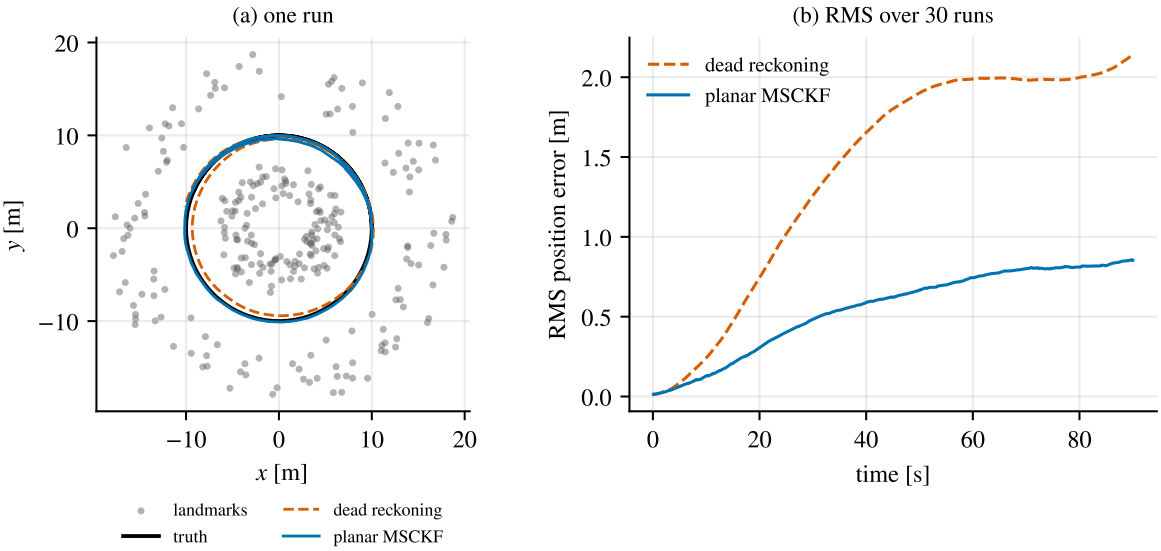


Figure 27.2: Planar MSCKF versus dead reckoning. (a) One run. (b) Root-mean-square position error over 30 Monte Carlo runs: after 90 s, 0.85 m for the MSCKF against 2.1 m for odometry alone.

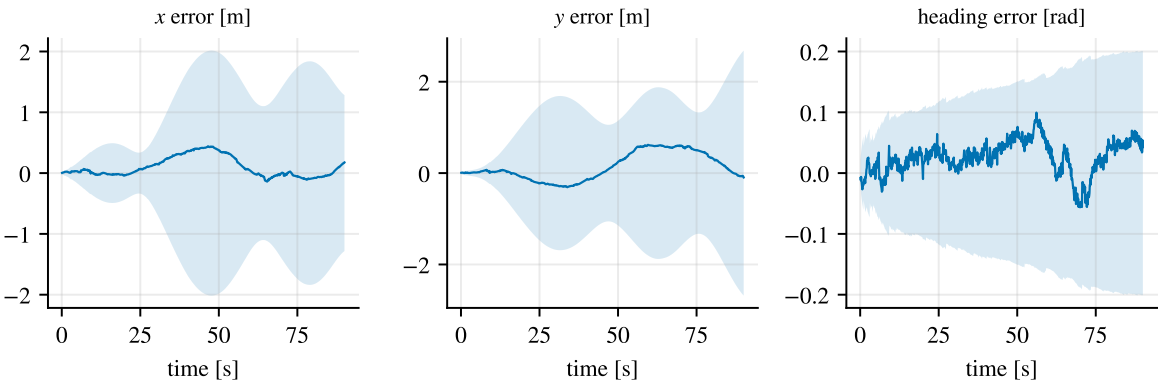


Figure 27.3: Estimation errors of one run with the filter's $\pm 3\sigma$ envelopes (shaded). The position envelopes oscillate because most of the position uncertainty is perpendicular to the direction of travel, which rotates as the robot circles.

are unobservable (Section 27.9), and an odometry filter without loop closures can only slow their drift, never stop it. The along-track error is set by the odometry speed noise alone, since a monocular camera cannot measure scale (the worked example above).

3. **The filter is consistent.** The errors stay inside their 3σ envelopes, and the average NEES stays inside its acceptance interval, apart from a brief, mildly conservative dip below it around 25 s. The gate rejected 5.2% of the features, close to the 5% that a $\chi^2_{0.95}$ gate should reject when the filter is consistent and every feature is a good one; a much larger rate would signal an overconfident filter or bad data.

4. **But the NEES creeps upward.** In the second half of the run the average NEES drifts from about 2.7 to 3.4. It is still inside the interval, but the trend is systematic: the filter is beginning to believe it knows the heading better than it does. The mechanism, and its cure, are the subject of Chapter 29.

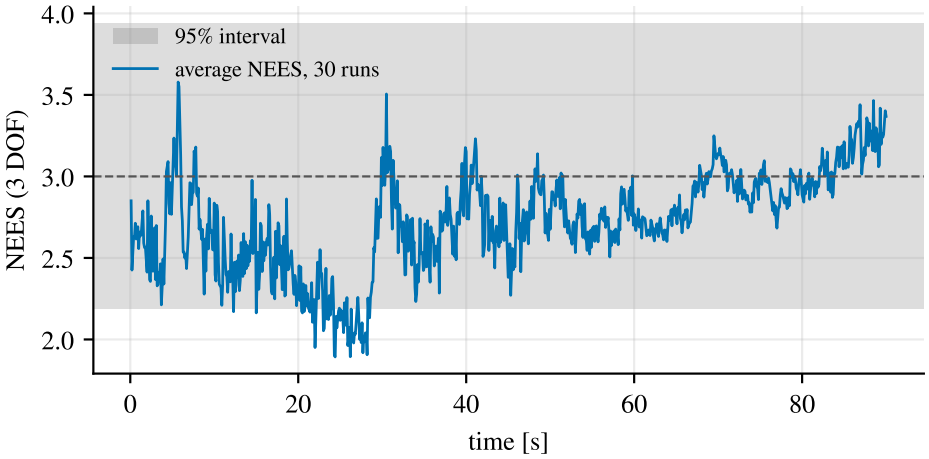


Figure 27.4: Average NEES of the robot pose over 30 runs, with the 95% acceptance interval of Equation (19.2).

27.9 The unobservable directions

Suppose we rotate the entire world, the trajectory and every feature, about the origin by a small angle, or translate it. Odometry measures speed and turn rate in the body frame and does not notice. The camera measures bearings relative to the robot and does not notice either. No sensor can therefore distinguish the true world from the rotated or translated one: these three directions are *unobservable* (Section 11.4), and the filter should never gain information along them.

In error-state coordinates the three directions are the columns of

$$N = \begin{bmatrix} N_t & N_r \end{bmatrix}, \qquad N_t = \begin{bmatrix} I_2 \\ 0 \\ I_2 \\ 0 \\ \vdots \end{bmatrix}, \qquad N_r = \begin{bmatrix} \Omega \hat{p} \\ 1 \\ \Omega \hat{p}_1 \\ 1 \\ \vdots \end{bmatrix}, \tag{27.16}$$

with one block pair for the robot and one for each clone. N_t shifts every position by the same vector. N_r is the derivative of a rotation about the origin: a point p moves by $\delta\,\Omega p$, and every heading increases by δ.

Proposition 27.6 (The planar nullspace)**.** *If all Jacobians are evaluated at one common set of estimates, then:*

1. *every feature Jacobian satisfies $H_{x,j}N_t + H_{f,j} = 0$ and $H_{x,j}N_r + H_{f,j}\,\Omega\hat{p}_f = 0$, and therefore the projected Jacobian satisfies $H_{o,j}N = 0$;*
2. *the exact transition (27.10) maps the nullspace at step k to the nullspace at step $k+1$, $\Phi_k N(\hat{x}_k) = N(\hat{x}_{k+1})$;*
3. *cloning maps N to the nullspace of the augmented state, $JN = N_{aug}$.*

Proof. (1) Row i, translation: $-J_\pi R_i^\mathsf{T} + J_\pi R_i^\mathsf{T} = 0$. Rotation: using $R_i^\mathsf{T}\Omega = \Omega R_i^\mathsf{T}$ (Lemma 27.1(3)),

$$-J_\pi R_i^\mathsf{T}\Omega\hat{p}_i - J_\pi\Omega\hat{p}_{b,i} + J_\pi R_i^\mathsf{T}\Omega\hat{p}_f = J_\pi\big(\Omega R_i^\mathsf{T}(\hat{p}_f - \hat{p}_i) - \Omega\hat{p}_{b,i}\big) = J_\pi(\Omega\hat{p}_{b,i} - \Omega\hat{p}_{b,i}) = 0.$$

Table 27.2: The planar MSCKF and its three-dimensional counterpart.

	Plane	Project (AGENTS.md)
Attitude	heading ψ, $R(\psi) = \exp(\psi\Omega)$	q_{wb}, R_{wb}, Exp (Chapter 8)
Error and retraction	$\delta\psi = \psi - \hat{\psi}$; local and global coincide	local (right) perturbation $q = \hat{q} \otimes \mathrm{Exp}_q(\delta\theta)$ (C4)
Skew operator	Ω, commutes with every $R(\psi)$	$[\cdot]_\times$; $R[v]_\times R^\mathsf{T} = [Rv]_\times$, no commutation
Jacobian of $R^\mathsf{T}a$ w.r.t. the rotation error	$-\Omega R^\mathsf{T}a$	the inverse-action row of Table 8.1
Motion sensor	odometry (v, ω), no biases	IMU (a_m, ω_m) with gravity and biases (C6)
State	(p, ψ) + clones, $3 + 3N$	(p, v, q, b_a, b_g) + clones, $15 + 6N$ (C3)
Camera	1D bearing $z = y/x$	2D normalized $(x/z, y/z)$, through T_{bc} (C6)
Constraints per feature	$M - 2$	$2M - 3$
Reset Jacobian	1	$I - \frac{1}{2}[\delta\hat{\theta}]_\times$ per attitude (F4)
Unobservable directions	2 translations + heading	3 translations + yaw (roll and pitch are observable through gravity)

Multiplying by A_j^T and using $A_j^\mathsf{T}H_{f,j} = 0$ gives $A_j^\mathsf{T}H_{x,j}N = 0$. (2) On the robot block, $\Phi_k(I_2, 0) = (I_2, 0)$, and $\Phi_k(\Omega\hat{p}_k, 1) = (\Omega\hat{p}_k + \Omega(\hat{p}_{k+1} - \hat{p}_k), 1) = (\Omega\hat{p}_{k+1}, 1)$; clone blocks are untouched. (3) The new clone copies the robot's rows of N, which are exactly the rows a clone at $\hat{p}$ should have. ∎

In words: the linearized filter has the same three unobservable directions as the real system, *as long as every Jacobian uses the same estimate of each quantity*. A real filter cannot honour this condition. The transition Φ_k uses $\hat{p}_{k+1} - \hat{p}_k$ computed from the estimate before the update at step $k + 1$, while the next transition starts from the corrected estimate; and a clone is linearized at its current, corrected estimate, not at the value it had when it was created. The identities of Proposition 27.6 then hold only approximately, the linearized system has an observable heading that the real system does not, and the filter slowly gains spurious heading information. That is the upward creep in Figure 27.4. Chapter 29 analyses it in the planar EKF-SLAM setting, where it is most visible, and shows how First-Estimates Jacobians restore the nullspace.

27.10 From the plane to three dimensions

Table 27.2 lists every difference between the planar filter and the project's filter. None of them changes the structure of Algorithm 8; each changes a formula inside it.

Key takeaways

- In the plane every MSCKF Jacobian takes a line: $\partial z/\partial\delta\psi_i = -(1 + z^2)$, $\partial z/\partial\delta p_i = -J_\pi R_i^\mathsf{T}$, $H_f = J_\pi R_i^\mathsf{T}$.
- A camera measures rotation independently of depth, and translation only through

parallax ($1/x$).

- A feature seen M times gives $M-2$ constraints in the plane ($2M-3$ in space); for three views the constraint is a second difference that measures the ratio of baselines and the relative headings, but not scale, global position or global heading.
- The planar filter is consistent and reduces the dead-reckoning error by a factor of about 2.5, yet its NEES creeps upward: the first sign of the spurious heading information studied in Chapter 29.
- The step to three dimensions changes formulas, not structure: non-commuting rotations, the local perturbation, gravity, biases and the reset Jacobian.

Exercises

27.1 Verify Proposition 27.4 by finite differences at a random clone pose and feature. Then explain $\partial z/\partial \delta\psi = -(1+z^2)$ using only the bearing angle $\beta = \arctan z$.

27.2 Show that $I + F_c\Delta t$ and the exact Φ_k of (27.10) differ by $O(\Delta t^2)$, and identify the term.

27.3 Add a turn-rate bias b_ω ($\omega_m = \omega + b_\omega + n_\omega$, $\dot{b}_\omega = n_b$) to the planar state. Derive the new column of F_c and the new row of G_c, and show that the bias is observable once the camera sees features. Compare with the gyroscope-bias block of the three-dimensional filter.

27.4 Repeat the worked example with unequal baselines, $\hat{p}_3 = (0, 2)$. Find a, and show that the projected residual measures the ratio of the two baselines.

27.5 In the worked example, show that moving the *middle* clone along the viewing direction x_w is invisible to H_o, while moving the *first* clone along x_w is visible. Interpret the difference in terms of the bearing each clone has to the feature.

27.6 Why does a 1D camera need three views of a feature to say anything about the poses, while a 2D camera needs only two? Relate the answer to the dimensions of H_f.

27.7 Run `scripts/planar_sims.py` with window sizes $N_{\max} = 3, 5, 10, 20$ and plot the final RMS position error. Explain the trend, and what limits the benefit of a longer window.

27.8 Prove part (1) of Proposition 27.6 without using the closed-form Jacobians: argue directly that a measurement function that depends only on $R(\psi_i)^\mathsf{T}(p_f - p_i)$ cannot change under a common rotation or translation of all positions.

27.9 Replace the bearing camera by a range-and-bearing sensor. How many constraints does a feature seen M times give? Which of the unobservable directions of (27.16) become observable, if any?

Chapter 28

The Complete Algorithm

In this chapter

- Assemble every component into one event-driven estimator, with the correct order of operations.
- Estimate the computational budget and map the algorithm onto the project's software architecture.
- Diagnose the most common failures from their symptoms.

28.1 The event loop

The estimator consumes a single, time-ordered stream of sensor events. Milestone 3 builds it by merging the presorted IMU, image (and, for testing, pose) arrays with a two-pointer merge, with the rule that an IMU sample precedes any other event with the same timestamp. Algorithm 9 shows the loop.

Three ordering rules are essential.

1. **Propagate before you update.** A measurement at time t must be applied to the state at time t, never to the state at the previous IMU sample.
2. **Use a clone's observations before you delete it.** Every feature with an observation at the oldest clone is processed before that clone is marginalized; otherwise its information is lost.
3. **Use each observation once.** After a track has been used, its observations are discarded. If the feature keeps being tracked, later observations form a new track.

With the order of Algorithm 9, the window never holds more than `kMaxClones` clones, which keeps the covariance within its fixed maximum size. A feature processed because its oldest clone is leaving cannot use the current image's observation in that update, because the new clone does not exist yet. Implementations that clone first need room for one extra clone. Either choice is valid; the storage bound must match it. The planar filter of Chapter 27 runs exactly this loop (Algorithm 8) in a few lines of Python, which makes it a convenient place to experiment with the ordering rules before committing to them in C++.

28.2 Initialization and the first seconds

The project initializes from ground truth, which removes the hardest failure mode from early testing and is logged as a use of ground truth (Milestone 10b). With static initialization (Chapter 16), the first seconds are delicate: the yaw is arbitrary, the accelerometer bias is

Algorithm 9 Monocular MSCKF

Input: event stream, calibration, noise parameters

▷ *load phase: allocation allowed*
parse data and calibration; build the event arrays; construct the filter and front end with fixed-max workspaces
initialize $(\hat{x}, P)$ (ground truth in the project, logged; static initialization otherwise)
▷ *streaming phase: no allocation in the core*
for each event e in timestamp order **do**
    if e is an IMU sample at t **then**
        propagate nominal state and covariance from t_{prev} to t ▷ Chapters 16 and 17
    else if e is an image at t_c **then**
        propagate to t_c (interpolated IMU) ▷ exact image time
        front end: track, verify, undistort; update track histories ▷ Chapter 22
        process tracks that were lost in this image ▷ Algorithm 7
        if $N =$ `kMaxClones` **then**
            process tracks with an observation at the oldest clone
            marginalize the oldest clone in place ▷ Chapter 25
        clone $(\hat{p}, \hat{q})$; $P \leftarrow JPJ^{\mathsf{T}}$; attach this image's observations to the new clone
    log state, diag(P), NIS and gate statistics to the preallocated buffer

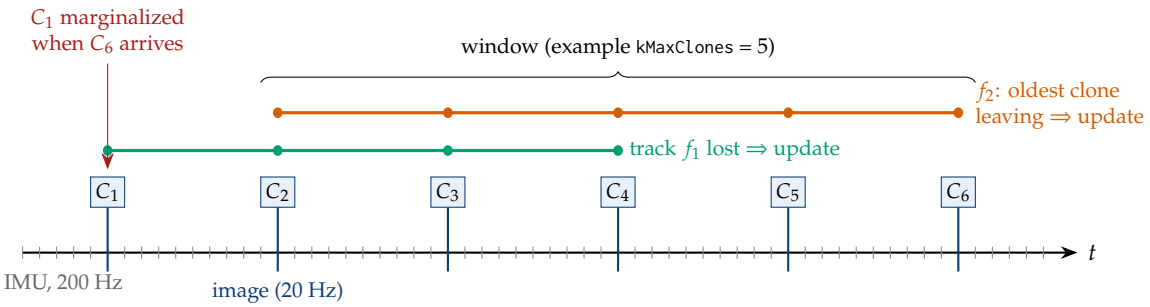


Figure 28.1: Timeline of the MSCKF. The IMU propagates at 200 Hz; each image creates a clone; feature tracks are processed when they end or when the clone holding their oldest observation is about to leave the window.

entangled with tilt, and the window contains few clones with little parallax. Generous initial covariances, and a check that enough features with sufficient parallax exist before trusting visual updates, avoid early divergence. Inspect the IMU data of each EuRoC sequence before running on it: a static start followed by vigorous motion is the easiest case for initialization, and a fast start the hardest.

28.3 Computational budget

With $N = 10$ clones the error state has $n = 75$ dimensions. Rough flop counts per operation:

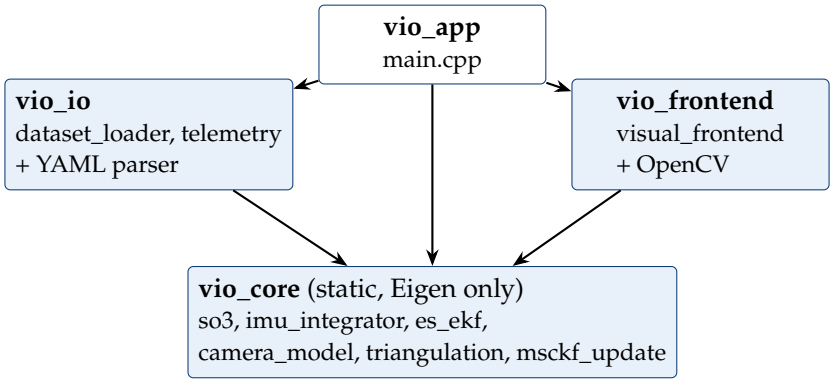


Figure 28.2: Library targets of the project (AGENTS.md). Dependencies point one way, so the estimator core can be built, tested and reasoned about without the I/O and vision libraries.

Operation	Frequency	Cost (flops)
Nominal propagation	200 Hz	$\sim 10^2$
P_{II}, P_{IC} propagation	200 Hz	$\sim 2 \times 10^4$
Cloning ($P \leftarrow JPJ^\mathsf{T}$ as block copies)	20 Hz	$\sim 10^4$
Per feature: triangulation, projection, gating ($M = 6$)	per track	$\sim 10^5$
QR compression of $d = 450$ rows	per update	$\sim 5 \times 10^6$
Update (S, gain, Joseph) with $n = 75$	per update	$\sim 3 \times 10^6$
Front end (KLT on 150 features)	20 Hz	dominant in time

The filter core needs on the order of 10^7 flops per image, a few milliseconds on an embedded processor and well under a millisecond on a desktop. The front end is usually the bottleneck in wall-clock time. The cubic growth with n is why the window is short and why QR compression matters for large stacks.

28.4 Software architecture

Figure 28.2 shows how the algorithm maps to the project's targets. The zero-allocation discipline applies to `vio_core`, which contains everything executed per IMU sample and per update. The front end may allocate but must reuse its buffers, and the I/O layer allocates freely during loading.

28.5 Debugging guide

Table 28.1 collects the failures that experience shows are most common. The milestone structure of the project is itself the most effective debugging tool: each layer is verified against known truth before the next is built on top of it, so a new failure is almost always in the newest layer.

Symptom	Likely causes
Altitude diverges at about $2g$ in IMU-only propagation	gravity or specific-force sign; g_w added twice
Position diverges within seconds even with ground-truth init	R_{wb} used as R_{bw}; quaternion w/z swapped at load; gyro and accel columns or biases swapped; degrees vs radians
Rotation matrices slowly scale vectors	missing quaternion normalization
NaN after a while	missing Taylor branch; Log near π; J_r^{-1} evaluated beyond π; feature at zero depth; Cholesky of a non-PD S
Corrections systematically make things worse	sign of H or of the residual; injection on the wrong side of the quaternion; global vs local convention mixed
Covariance loses symmetry or positive definiteness	simple update form instead of Joseph; missing symmetrization; aliasing in Eigen block products
Nearly all features fail the gate	overconfident P; T_{bc} inverted; pixel noise used in normalized units (or vice versa); distortion not removed; timestamps misaligned
Gate rejects nothing	R_z or σ_{uv} far too large; gate threshold indexed with the wrong dimension
Slowly growing yaw error with healthy NIS	expected EKF inconsistency along unobservable yaw (Chapter 29)
Scale drift	little acceleration (degenerate motion); accelerometer noise or bias badly tuned
Works on synthetic data, fails on EuRoC	extrinsics, time offset, distortion, noise inflation, initialization, IMU gaps
Results differ between runs	uninitialized memory (run the sanitizers); unseeded random generators

Table 28.1: Symptoms and their most frequent causes.

28.6 What to expect on EuRoC

Published monocular filter-based estimators reach absolute trajectory errors of the order of 0.1 to 0.5 m on the machine-hall sequences, depending on features, tuning and initialization, with the difficult sequences (fast motion, motion blur, dark regions) considerably worse [13]. Milestone 10b sets a first-pass target of 0.5 m on MH_01 and a stretch target of 0.2 m. Yaw and position drift without bound; roll and pitch remain accurate throughout; velocity and biases converge within the first seconds of excitation.

Key takeaways

- One time-ordered stream; propagate to each event's time before applying it.
- At each image: process lost tracks; if the window is full, process the oldest clone's tracks and marginalize it; then clone.
- The core costs about 10^7 flops per image; the front end usually dominates the run time.
- Dependencies point one way: `vio_core` knows nothing about files, YAML or OpenCV.
- Most failures map to a small set of sign, frame, unit and timing errors; test each layer against truth.

Exercises

28.1 Write the event-loop pseudocode for the alternative ordering (clone first, then process and marginalize). What capacity does it require, and what does it gain?

28.2 Instrument a run to record the number of features processed per update, their observation counts, and the stacked row count d. Choose the compile-time row cap from the distribution.

28.3 For each row of Table 28.1, design a unit or integration test that would have caught the cause before the full system was assembled.

Part VII

Beyond the Baseline

Chapter 29

Observability and Consistency of VIO

In this chapter

- Define observability for nonlinear systems, and test it with Lie derivatives.
- Prove that VIO has four unobservable directions and write them in the project's error coordinates.
- Show, in the planar EKF-SLAM problem where every step can be computed, exactly how the standard EKF acquires spurious information, and see it happen in a Monte Carlo simulation.
- Prove that First-Estimates Jacobians restore the correct unobservable subspace, and understand the alternatives: the observability-constrained EKF, the invariant EKF and robocentric formulations.
- Know the degenerate motions that remove further information.

The planar MSCKF of Chapter 27 was consistent, but its NEES crept upward. This chapter explains the creep. The explanation needs one new idea, observability, and one careful computation, the observability matrix of a linearized filter. The computation is done in full for planar EKF-SLAM, the setting in which the effect was first understood [31, 1, 28], and then carried over to VIO.

29.1 Observability

29.1.1 Indistinguishable states

Consider a system $\dot{x} = f(x, u)$ with output $y = h(x)$. Two initial states x_0 and x_0' are *indistinguishable* if, for every input signal $u(\cdot)$, they produce the same output signal $y(\cdot)$. No estimator, however clever, can tell them apart: the data are identical. A system is *observable* at x_0 if no other state is indistinguishable from it. Since estimators work locally, the useful notion is local: x_0 is *locally weakly observable* if it can be distinguished, over arbitrarily short time intervals, from every state in some neighborhood of it.

For linear systems the question is settled by the observability matrix (Theorem 11.5). For a time-varying linear system $\delta x_{k+1} = \Phi_k \delta x_k$, $r_k = H_k \delta x_k$ the same object reads as follows.

Definition 29.1 (Observability matrix)**.** The observability matrix over steps $1, \dots, k$ is

$$\mathcal{O} = \begin{bmatrix} H_1 \\ H_2\Phi_1 \\ H_3\Phi_2\Phi_1 \\ \vdots \\ H_k\Phi_{k-1}\cdots\Phi_1 \end{bmatrix}. \tag{29.1}$$

An initial error $\delta x_1 \in \operatorname{null}(\mathcal{O})$ produces no change in any measurement, so it cannot be detected. $\operatorname{null}(\mathcal{O})$ is the *unobservable subspace*.

A Kalman filter can gain information only in directions outside $\operatorname{null}(\mathcal{O})$. Along unobservable directions its covariance may grow (from process noise) but must never shrink. This is the property that the standard EKF violates.

29.1.2 Nonlinear observability and Lie derivatives

For a nonlinear system the observability matrix is replaced by derivatives of the output along the dynamics. Suppose the dynamics are *control-affine*, $\dot{x} = f_0(x) + \sum_i f_i(x)\, u_i$, as kinematic models usually are (the inputs are the measured rates and accelerations). The *Lie derivative* of a scalar function φ along a vector field f is its rate of change along the flow of f,

$$L_f\varphi(x) = \nabla\varphi(x) \cdot f(x), \tag{29.2}$$

and repeated derivatives are written $L_g L_f \varphi = \nabla(L_f\varphi) \cdot g$. Differentiating the output along a trajectory gives

$$y = h(x), \qquad \dot{y} = L_{f_0}h + \sum_i u_i\, L_{f_i}h, \qquad \ddot{y} = \dots \tag{29.3}$$

Because the inputs can be varied freely, each Lie derivative $L_{f_i}h$ is, in effect, a separate measured function of the state, and so are the higher ones. If enough of them are available, their values pin down the state.

Theorem 29.2 (Observability rank condition [24])**.** *Let $d\mathcal{O}(x)$ be the span of the gradients of h and of all its repeated Lie derivatives $L_{f_{i_1}}\cdots L_{f_{i_m}}h$ along the vector fields of the system. If $\dim d\mathcal{O}(x_0) = n$, the system is locally weakly observable at x_0.*

Idea of the proof. Choose n of the functions whose gradients are independent at x_0 and stack them into $\Psi(x) \in \mathbb{R}^n$. Each is determined by the output and the inputs (by differentiation), and Ψ has an invertible Jacobian at x_0, so by the inverse function theorem it is one-to-one near x_0. Two nearby states with the same outputs would have the same Ψ, hence coincide. ∎

For a linear system, $f_0 = Ax$ and $h = Cx$, the gradients are $C, CA, CA^2, \dots$: the rank condition is Theorem 11.5. For analytic systems the condition is also necessary in a generic sense, so a rank deficiency at typical states signals a genuine family of indistinguishable states.

Worked example: a planar robot and a landmark

A planar robot (Chapter 27) with pose (p, ψ) measures the position of a landmark ℓ in its body frame, $h = R(\psi)^\mathsf{T}(\ell - p) \in \mathbb{R}^2$. Its kinematics are $\dot{p} = R(\psi)e_1 v$ and $\dot{\psi} = \omega$, that is $f_v = (R(\psi)e_1, 0)$ and $f_\omega = (0, 1)$ with inputs v and ω.
Unknown landmark. The state is $x = (p, \psi, \ell) \in \mathbb{R}^5$. With $\partial(R^\mathsf{T}a)/\partial\psi = -\Omega R^\mathsf{T}a$ (Lemma 27.1),

$$\nabla h = \begin{bmatrix} -R^\mathsf{T} & -\Omega h & R^\mathsf{T} \end{bmatrix}, \qquad L_{f_v} h = -R^\mathsf{T} R e_1 = -e_1, \qquad L_{f_\omega} h = -\Omega h.$$

The first Lie derivative is constant (its gradient is zero), and $\nabla(L_{f_\omega} h) = -\Omega \nabla h$ has the same row space as ∇h. Every further derivative repeats the pattern, so $\dim d\mathcal{O} = 2$: three of the five directions are unobservable. They are the two global translations and the global rotation, exactly the directions of Section 27.9.
Known landmark. If ℓ is known, $x = (p, \psi) \in \mathbb{R}^3$, the gradients are $[-R^\mathsf{T} \;\; -\Omega h]$ and its multiples by Ω, and the rank is 2: one direction is unobservable. It is the rotation of the robot about the landmark, which leaves the body-frame landmark position unchanged. One known landmark fixes position but not heading; two fix both.

29.2 The four unobservable directions of VIO

Theorem 29.3 (Unobservable directions of VIO)**.** *For monocular VIO with point features, applying the same global translation t and the same rotation $R_z(\alpha)$ about the gravity direction to all positions, velocities, attitudes, clones and features leaves every IMU and camera measurement unchanged. The state is therefore unobservable along these four directions.*

Proof. Under $p \mapsto R_z p + t$, $v \mapsto R_z v$, $R_{wb} \mapsto R_z R_{wb}$, $p_f \mapsto R_z p_f + t$, with biases unchanged:

- the gyroscope measures $[\omega_b]_\times = R_{wb}^\mathsf{T} \dot{R}_{wb}$, which is invariant because R_z is constant;
- the accelerometer measures $R_{wb}^\mathsf{T} R_z^\mathsf{T}(R_z a_w - g_w) = R_{wb}^\mathsf{T}(a_w - R_z^\mathsf{T} g_w) = R_{wb}^\mathsf{T}(a_w - g_w)$, because a rotation about gravity leaves g_w unchanged;
- the camera sees $R_{bc}^\mathsf{T}\big(R_{wb}^\mathsf{T} R_z^\mathsf{T}(R_z p_f + t - R_z p - t) - p_{bc}\big) = R_{bc}^\mathsf{T}\big(R_{wb}^\mathsf{T}(p_f - p) - p_{bc}\big)$.

All measurements are identical, so the transformed and original trajectories cannot be distinguished. ∎

Rotations about horizontal axes are *not* in this family, because they would rotate gravity in the accelerometer equation; roll and pitch are observable. Scaling is not either, because it would scale the accelerations the accelerometer measures; scale is observable as long as the vehicle accelerates. That the remaining directions (velocity, roll, pitch, scale, biases) *are* observable under general motion is proved with the rank condition of Theorem 29.2; see [25] for the computation.

Linearizing the transformation gives the unobservable directions in the project's error coordinates. With e_z the vertical unit vector, a small yaw rotation α and a small translation t

change the state by (verified numerically):

$$\underbrace{\begin{bmatrix} \delta p \\ \delta v \\ \delta\theta \\ \delta b_a \\ \delta b_g \\ \delta p_i \\ \delta\theta_i \\ \delta p_f \end{bmatrix}}_{\text{error state}} = \underbrace{\begin{bmatrix} I_3 & -[\hat{p}]_\times e_z \\ 0 & -[\hat{v}]_\times e_z \\ 0 & \hat{R}^\mathsf{T} e_z \\ 0 & 0 \\ 0 & 0 \\ I_3 & -[\hat{p}_i]_\times e_z \\ 0 & \hat{R}_i^\mathsf{T} e_z \\ I_3 & -[\hat{p}_f]_\times e_z \end{bmatrix}}_{N} \begin{bmatrix} t \\ \alpha \end{bmatrix}. \tag{29.4}$$

The last row applies only to formulations that keep features in the state. The position entries follow from $R_z(\alpha)p \approx p + \alpha\, e_z \times p = p - \alpha[p]_\times e_z$; the attitude entry is $\hat{R}^\mathsf{T} e_z$ because the error is local: $\mathrm{Exp}(\alpha e_z)\hat{R} = \hat{R}\ \mathrm{Exp}(\alpha \hat{R}^\mathsf{T} e_z)$, a rotation about the world vertical expressed in body axes. Every column of N *depends on the estimate*: the yaw column involves $\hat{p}$, $\hat{v}$, $\hat{R}$ and the clone and feature estimates. That dependence is the root of the problem. Equation (27.16) is the planar version, with $\Omega\hat{p}$ in place of $-[\hat{p}]_\times e_z$.

29.3 Why the standard EKF becomes overconfident

If every Jacobian were evaluated at the true state, the linearized system would satisfy $ON = 0$ at all times, and the filter would never gain information along N. The EKF instead evaluates each Jacobian at the best estimate available when it is computed. The transition matrix Φ_k is evaluated at the estimate after the update at step k, while the measurement Jacobian at step k was evaluated at the estimate before that update. The same physical state enters the observability matrix linearized at two different points, and the products $H_k\Phi_{k-1}\cdots\Phi_1 N$ no longer cancel. The planar EKF-SLAM problem shows this in a computation short enough to do completely.

29.3.1 Planar EKF-SLAM

The state is the robot pose and L landmarks, $x = (p, \psi, \ell_1, \ldots, \ell_L)$, with additive errors. Odometry propagates the robot ($p_{k+1} = p_k + R(\psi_k)e_1 v_m \Delta t$, $\psi_{k+1} = \psi_k + \omega_m \Delta t$), landmarks are static, and an observation of landmark j is its position in the body frame, $z = R(\psi)^\mathsf{T}(\ell_j - p) + n$. Writing $\hat{x}_{k|k-1}$ for the propagated estimate at step k and $\hat{x}_{k|k}$ for the updated one, the EKF uses

$$\Phi_k = \begin{bmatrix} I_2 & \Omega\,(\hat{p}_{k+1|k} - \hat{p}_{k|k}) \\ 0 & 1 \end{bmatrix} \oplus I_{2L}, \tag{29.5}$$

$$H_k = \begin{bmatrix} -\hat{R}_k^\mathsf{T} & -\Omega\hat{R}_k^\mathsf{T}(\hat{\ell}_j - \hat{p}_{k|k-1}) & 0\cdots 0 & \hat{R}_k^\mathsf{T} & 0\cdots 0 \end{bmatrix}, \qquad \hat{R}_k = R(\hat{\psi}_{k|k-1}), \tag{29.6}$$

where Φ_k follows from Proposition 27.3 (the step starts from the updated estimate), H_k from the derivation of Proposition 27.4 without the projection, and $\hat{\ell}_j$ is the landmark estimate at the time of the update. The candidate unobservable directions, by the argument of

Section 27.9, are the translations $N_t = (I_2, 0, I_2, \dots, I_2)$ and the rotation

$$N_r(\bar{p}, \bar{\ell}) = \big(\Omega\bar{p},\ 1,\ \Omega\bar{\ell}_1,\ \dots,\ \Omega\bar{\ell}_L\big), \tag{29.7}$$

built from some reference values $\bar{p}$, $\bar{\ell}_j$ of the positions.

Theorem 29.4 (Observability of linearized planar EKF-SLAM). *Let $\Delta_i = \hat{p}_{i|i} - \hat{p}_{i|i-1}$ be the correction of the robot position by the update at step i, and $D_k = \sum_{i=1}^{k-1} \Delta_i$ (with $D_1 = 0$). For every k, $H_k \Phi_{k-1} \cdots \Phi_1 N_t = 0$, while*

$$H_k \Phi_{k-1} \cdots \Phi_1 \, N_r(\hat{p}_{1|0}, \bar{\ell}) = \hat{R}_k^\mathsf{T} \, \Omega \, \big(D_k + \bar{\ell}_j - \hat{\ell}_j\big). \tag{29.8}$$

Proof. Each Φ_i is the identity plus a term in the heading column, so the product is $\Phi_{k-1} \cdots \Phi_1 = \begin{bmatrix} I & \Omega\, s_k \\ 0 & 1 \end{bmatrix} \oplus I$ with

$$s_k = \sum_{i=1}^{k-1} \big(\hat{p}_{i+1|i} - \hat{p}_{i|i}\big) = \sum_{i=1}^{k-1} \big(\hat{p}_{i+1|i} - \hat{p}_{i|i-1}\big) - \sum_{i=1}^{k-1} \big(\hat{p}_{i|i} - \hat{p}_{i|i-1}\big) = \hat{p}_{k|k-1} - \hat{p}_{1|0} - D_k,$$

since the first sum telescopes. Translations: the product leaves N_t unchanged and $H_k N_t = -\hat{R}_k^\mathsf{T} + \hat{R}_k^\mathsf{T} = 0$. Rotation: the product maps the robot entries $(\Omega\hat{p}_{1|0}, 1)$ to $(\Omega\hat{p}_{1|0} + \Omega s_k, 1) = (\Omega(\hat{p}_{k|k-1} - D_k), 1)$ and leaves the landmark entries alone, so

$$H_k(\cdot) = -\hat{R}_k^\mathsf{T}\Omega(\hat{p}_{k|k-1} - D_k) - \Omega\hat{R}_k^\mathsf{T}(\hat{\ell}_j - \hat{p}_{k|k-1}) + \hat{R}_k^\mathsf{T}\Omega\bar{\ell}_j = \hat{R}_k^\mathsf{T}\Omega\big(D_k + \bar{\ell}_j - \hat{\ell}_j\big),$$

using $\hat{R}_k^\mathsf{T}\Omega = \Omega\hat{R}_k^\mathsf{T}$; the $\hat{p}_{k|k-1}$ terms cancel. ∎

Equation (29.8) is the whole story. The rotation direction stays in the nullspace only if two things vanish:

- D_k, the accumulated corrections of the robot position. They vanish only if the transition matrices telescope, which requires each Φ_i to start where the previous one ended. The EKF starts each Φ_i from the *updated* estimate $\hat{p}_{i|i}$, while the previous one ended at the *propagated* estimate $\hat{p}_{i|i-1}$.
- $\bar{\ell}_j - \hat{\ell}_j$, the change of the landmark estimate between its linearization points. A single fixed reference $\bar{\ell}_j$ cannot match an estimate that every update moves.

For generic estimates no other choice of reference values makes (29.8) vanish for all k, and the unobservable subspace of the EKF's linearized model shrinks from three dimensions to two [28]: the heading has become observable. The filter then extracts *spurious information* about the global heading from measurements that contain none. The size of the leak is set by the size of the corrections and of the landmark estimate changes; it is largest at loop closures, when a robot that has drifted re-observes old landmarks and receives large corrections.

29.3.2 From planar SLAM to VIO

The same mechanism operates in VIO. The translation columns of (29.4) survive, because all Jacobians depend only on *differences* of positions. The yaw column does not: with estimate-dependent Jacobians $O N_{\text{yaw}} \neq 0$, and the unobservable subspace of the EKF's

linearized model has dimension three instead of four [34, 25]. In the MSCKF, the clones play the role of the landmarks: a clone is created from the propagated estimate, and every later update moves it, so its linearization point in a measurement Jacobian differs from the one at which it entered the filter. The filter becomes overconfident in yaw, and because yaw errors couple into position through the trajectory, its position covariance becomes too small as well. On long trajectories the NEES of yaw and position grows steadily while the NIS can look perfectly healthy: the inconsistency lives in directions the measurements do not see (Table 19.1).

Key idea: an inconsistency that no unit test catches

EKF yaw inconsistency is not a bug in the usual sense: every Jacobian can pass its finite-difference test. It is a property of linearizing at estimates that change over time. Milestone 10b therefore expects it, and asks for it to be measured and documented rather than fixed.

29.4 First-Estimates Jacobians

The cure suggested by (29.8) is to make both terms vanish by construction: evaluate every Jacobian that involves a state at the *first* estimate of that state, the value it had when it entered the filter, rather than at the latest one [27, 28].

Proposition 29.5 (FEJ restores the nullspace)**.** *In planar EKF-SLAM, evaluate the transition matrix with the propagated estimates at both ends,*

$$\Phi_k^{\mathrm{FEJ}} = \begin{bmatrix} I_2 & \Omega\,(\hat{p}_{k+1|k} - \hat{p}_{k|k-1}) \\ 0 & 1 \end{bmatrix} \oplus I, \tag{29.9}$$

and the measurement Jacobian at the propagated robot estimate $\hat{x}_{k|k-1}$ and at the first estimate *$\hat{\ell}_j^{(1)}$ of each landmark. Initialize each landmark from the propagated robot estimate, $\hat{\ell}_j^{(1)} = \hat{p}_{k|k-1} + R(\hat{\psi}_{k|k-1})z$, with the augmentation Jacobian evaluated there. Then $\boldsymbol{O}N = 0$ for all k, with $N = [N_t \;\; N_r(\hat{p}_{1|0}, \hat{\ell}^{(1)})]$.*

Proof. The transition matrices now telescope: $\sum_i(\hat{p}_{i+1|i} - \hat{p}_{i|i-1}) = \hat{p}_{k|k-1} - \hat{p}_{1|0}$, so D_k is replaced by zero. The landmark linearization point is fixed at $\hat{\ell}_j^{(1)}$, which is also the reference value in N_r, so $\bar{\ell}_j - \hat{\ell}_j = 0$. Both terms of (29.8) vanish. For a landmark initialized at step k, the augmentation Jacobian $[I_2 \;\; \Omega R(\hat{\psi}_{k|k-1})z \;\; 0]$ maps the robot part of N_r, $(\Omega\hat{p}_{k|k-1}, 1)$, to $\Omega(\hat{p}_{k|k-1} + R(\hat{\psi}_{k|k-1})z) = \Omega\hat{\ell}_j^{(1)}$: exactly the entry N_r needs for the new landmark. ∎

Only the *Jacobians* change. The residuals are still computed with the latest estimates, the mean is still propagated and updated as before, and the filter still uses all its information. The price is that each Jacobian is evaluated slightly away from the best estimate, which adds linearization error; when the first estimates are poor this can matter, but in practice FEJ filters are markedly more consistent and usually more accurate.

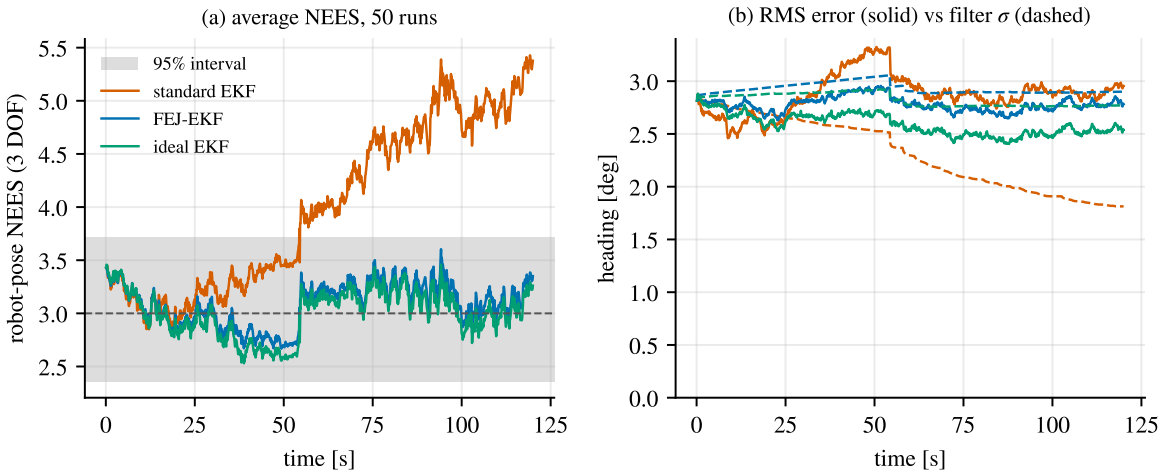


Figure 29.1: Planar EKF-SLAM, 50 Monte Carlo runs. (a) Average NEES of the robot pose: the standard EKF leaves the 95% interval after the first loop closure (about 55 s) and keeps rising, while the FEJ-EKF and the ideal EKF stay consistent. (b) Heading: actual RMS error (solid) and the filter's own standard deviation (dashed). The initial heading is unobservable, so a consistent, realizable filter keeps reporting about its initial 2.9°, as the FEJ-EKF does. The standard EKF's reported uncertainty falls to 1.8° while its actual error stays near 3°.

29.4.1 A numerical demonstration

Figure 29.1 compares three filters on the same simulated data (`scripts/planar_sims.py`). A robot drives almost twice around a circle of radius 10 m at 1 m/s (120 s), sampling odometry at 10 Hz (per-sample noise 0.05 m/s in speed and 0.01 rad/s in turn rate), and measures the body-frame position of every landmark within 5 m (20 landmarks are scattered around the circle), with 0.1 m noise per axis. The initial pose estimate has 0.1 m position and 0.05 rad heading uncertainty. The three filters differ only in where they evaluate their Jacobians:

- the *standard EKF* uses the latest estimates, (29.5)–(29.6);
- the *FEJ-EKF* follows Proposition 29.5;
- the *ideal EKF* evaluates every Jacobian at the true state. It is not realizable, but it is the benchmark for consistency, since its linearized model has the true nullspace.

The heading panel makes the failure concrete. The initial heading error is unobservable, so no realizable filter can reduce it: the RMS heading error of the standard and FEJ filters stays near its initial value of about 2.9°. (The ideal EKF reduces it slightly, to 2.5°, because its Jacobians are evaluated at the true state, which leaks information that no real filter has.) The FEJ and ideal filters report standard deviations that match their errors. The standard EKF reports a standard deviation that shrinks steadily, fastest at the loop closure, until its variance is less than 40% of its actual mean squared error. It has "learned" the global heading from relative measurements, which is impossible. Its NEES grows accordingly, and nothing in its innovations reveals the problem.

29.4.2 FEJ in the MSCKF

The rule generalizes directly [34]: *every state appears in the observability matrix at a single linearization point*. Concretely:

1. **Propagation.** The transition matrix from t_k to t_{k+1} is evaluated with the propagated estimates at both ends, $\hat{x}_{k|k-1}$ and $\hat{x}_{k+1|k}$, wherever it depends on the state (positions, velocities, attitudes). The test is the planar one: the product of consecutive transition matrices must telescope.
2. **Measurement.** Each clone enters the camera Jacobians at its *first estimate*, the pose it had when it was cloned, not its current corrected value. The residual still uses the current value.
3. **Feature.** The feature position used in $H_{x,j}$ and $H_{f,j}$ can be any single estimate (for example the triangulation from the current clone estimates), because the feature is eliminated: $A_j^\mathsf{T} H_{f,j} N_f = 0$ for whatever N_f the chosen value implies.

The precise formulas depend on the error parameterization. For yours, the systematic procedure is: write N in your error coordinates ((29.4)), then choose the linearization points so that $\Phi_k N_k = N_{k+1}$ and $H_k N_k = 0$ hold *exactly*, with N_k built from the first estimates. Even attitude blocks that do not seem to depend on the state may need attention: with a local attitude error, the yaw entry $\hat{R}^\mathsf{T} e_z$ of N depends on the attitude estimate, so the attitude block of Φ_k must map $\hat{R}_{k|k-1}^\mathsf{T} e_z$ to $\hat{R}_{k+1|k}^\mathsf{T} e_z$.

Implementation note: storing first estimates without allocation

FEJ needs, for each clone, a second copy of its pose (the first estimate) next to the current one, and for the IMU state the propagated estimate of the previous step. Both fit in fixed-size arrays sized by `kMaxClones`, so the zero-allocation rules of AGENTS.md E4 are unaffected. A direct test: on a synthetic run, compute $\|\mathcal{O}N\|$ (or simply $\|H_k N_k\|$ and $\|\Phi_k N_k - N_{k+1}\|$ at every step) for the standard and the FEJ filter. The FEJ values should be at rounding level, the standard ones visibly larger.

29.5 Other remedies

29.5.1 Observability-constrained EKF

Instead of choosing linearization points, the observability-constrained EKF (OC-EKF) [25] computes Jacobians at the latest estimates and then modifies them minimally so that the nullspace conditions hold.

Proposition 29.6 (Minimal Jacobian corrections). *Let N_k have full column rank. The matrix closest to H in the Frobenius norm that satisfies $H^* N_k = 0$ is*

$$H^* = H\big(I - N_k (N_k^\mathsf{T} N_k)^{-1} N_k^\mathsf{T}\big), \tag{29.10}$$

and the matrix closest to Φ that satisfies $\Phi^* N_k = N_{k+1}$ is

$$\Phi^* = \Phi + (N_{k+1} - \Phi N_k)(N_k^\mathsf{T} N_k)^{-1} N_k^\mathsf{T}. \tag{29.11}$$

Proof. The Frobenius norm is the sum of the squared norms of the rows, and the constraints act on each row separately: row h must satisfy $h^* N_k = c$ for a given row vector c ($c = 0$ for H, a row of N_{k+1} for Φ). Minimizing $\|h^* - h\|^2$ subject to $h^* N_k = c$ is a least-norm problem for the correction $d = h^* - h$: $dN_k = c - hN_k$. Its minimum-norm solution is $d = (c - hN_k)(N_k^\mathsf{T} N_k)^{-1} N_k^\mathsf{T}$ (Section 3.5), which gives both formulas. ∎

The OC-EKF thus projects the information along N out of every measurement and enforces the correct propagation of N, so the linearized model has the correct unobservable subspace by construction. The choice of N_k (which estimates to build it from) still matters; Hesch et al. build it from the propagated estimates.

29.5.2 Invariant EKF

The invariant EKF [5] attacks the root cause: the estimate dependence of N. It represents the IMU state as one element of the *extended pose group* $\mathrm{SE}_2(3)$,

$$X = \begin{bmatrix} R & v & p \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{5\times 5}, \qquad X^{-1} = \begin{bmatrix} R^\mathsf{T} & -R^\mathsf{T} v & -R^\mathsf{T} p \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{29.12}$$

a generalization of SE(3) (Section 8.12) with two translation-like columns (features can be appended as further columns), and it defines the error as the *right-invariant* group error $\eta = \hat{X} X^{-1} = \mathrm{Exp}(\xi)$, with $\xi = (\xi_R, \xi_v, \xi_p) \in \mathbb{R}^9$.

Proposition 29.7 (Constant unobservable directions)**.** *In right-invariant error coordinates, the four unobservable directions of Theorem 29.3 are the constant vectors $\xi = (e_z, 0, 0)$ (yaw) and $\xi = (0, 0, t)$, $t \in \mathbb{R}^3$ (translation), independent of the estimate.*

Proof. A global yaw and translation act on the true state from the left: $X \mapsto GX$ with $G = \begin{bmatrix} R_z & 0 & t \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, which is $p \mapsto R_z p + t$, $v \mapsto R_z v$, $R \mapsto R_z R$. The error becomes $\hat{X}(GX)^{-1} = \hat{X} X^{-1} G^{-1} = \eta\, G^{-1}$. For small errors and a small $G = \mathrm{Exp}(\epsilon)$, with $\epsilon = (\alpha e_z, 0, t)$, the product $\mathrm{Exp}(\xi)\,\mathrm{Exp}(-\epsilon)$ is $\mathrm{Exp}(\xi - \epsilon)$ to first order. The family of indistinguishable states therefore differs, in error coordinates, by the constant vectors ϵ. ∎

Compare (29.4), where the yaw direction involves $\hat{p}$, $\hat{v}$ and $\hat{R}$. Because the group structure couples rotation with velocity and position, a global yaw is a pure rotation error in these coordinates. Two consequences follow. The linearized error dynamics of the IMU (without biases) are independent of the estimate, $\dot{\xi} = A\xi + \ldots$ with $A = \begin{bmatrix} 0 & 0 & 0 \\ [g_w]_\times & 0 & 0 \\ 0 & I & 0 \end{bmatrix}$, and $AN = 0$ because $[g_w]_\times e_z = 0$; and since every measurement is invariant under $X \mapsto GX$, every measurement Jacobian, evaluated at *any* estimate, annihilates the constant N. The linearized filter has the correct unobservable subspace by construction, with no bookkeeping of first estimates. Biases break the exact estimate-independence of the error dynamics, but not this nullspace property, since N has no bias components. The price is a different state

representation, different Jacobians, and a different reset; it is a deviation from AGENTS.md C3–C4 that this project does not take.

29.5.3 Robocentric formulations

A robocentric filter expresses everything relative to the current body frame, so the unobservable global quantities are not estimated at all, and the global pose is recovered by composing relative motions outside the filter. The nullspace problem disappears with the unobservable states, at the cost of a more complex propagation in which every state moves when the body moves.

Remedy	What changes	Cost
FEJ	linearization points of Φ and H	store first estimates; slightly larger linearization error
OC-EKF	Jacobians projected onto the constraint	build N_k each step; extra products
Invariant EKF	state representation and error definition	new derivations; leaves the frozen conventions
Robocentric	state is relative to the body	more complex propagation

29.6 Degenerate motions

Theorem 29.3 assumes *general* motion. Some motions remove further directions [55].

Motion	What becomes unobservable
constant velocity (no acceleration)	metric scale
pure rotation (no translation)	feature depths and translation scale
no rotation	accelerometer bias inseparable from the gravity direction (tilt)
at rest	both of the above, plus everything that needs parallax

Each row has a short explanation. At constant velocity the accelerometer measures only gravity and bias, so nothing ties the size of the camera's (scale-free) motion estimate to meters. Without translation there is no parallax, so depths, and with them the scale of the translation, cannot be triangulated. Without rotation the gravity direction in the body frame is constant, and a constant error in it is indistinguishable from a constant accelerometer bias; rotation makes gravity sweep through the body axes while the bias stays fixed, which separates them.

The filter does not fail during such motions, provided it is consistent: its covariance simply stops shrinking along the affected directions until informative motion returns. An inconsistent filter, by contrast, may "learn" these quantities from noise. When a vehicle is known to be stationary, a *zero-velocity update* (the pseudo-measurement $v = 0$, Chapter 18) supplies real information and stops drift, which is why many systems detect stillness explicitly.

Key takeaways

- The unobservable subspace is the nullspace of the observability matrix; a filter must not gain information along it. For nonlinear systems, the gradients of the output's Lie derivatives play the role of the observability matrix.

- VIO has four unobservable directions: global translation and rotation about gravity. Roll, pitch and scale are observable under general motion.
- In local error coordinates the yaw direction is $\hat{R}^\mathsf{T} e_z$ for the attitude and $-[\hat{p}]_\times e_z$ for positions: it depends on the estimate.
- Linearizing the same state at different estimates makes the global heading spuriously observable: in planar SLAM the leak is $\hat{R}_k^\mathsf{T}\Omega(D_k + \bar{\ell}_j - \hat{\ell}_j)$, driven by the accumulated corrections and the drift of the landmark estimates.
- FEJ restores the nullspace by fixing each state's linearization point; OC-EKF projects the Jacobians; the invariant EKF makes the unobservable directions constant; degenerate motions remove further directions.

Exercises

29.1 Verify (29.4) numerically: apply a small yaw rotation and translation to a random state and compute $x' \boxminus x$.

29.2 Show that the translation columns of N remain in the nullspace of the EKF's observability matrix even when Jacobians are evaluated at changing estimates.

29.3 Redo the worked example on Lie derivatives with two unknown landmarks. What is the rank of $d\mathcal{O}$, and what are the unobservable directions?

29.4 Repeat Theorem 29.4 for range-and-bearing measurements, $z = (\|d\|, \operatorname{atan2}(d_y, d_x))$ with $d = R(\psi)^\mathsf{T}(\ell_j - p)$. Show that the leak has the same form.

29.5 Modify `scripts/planar_sims.py` to print $\|H_k\Phi_{k-1}\cdots\Phi_1 N\|$ for the three filters of Figure 29.1, and confirm that only the standard EKF violates the nullspace condition.

29.6 Prove that the minimum-norm solution used in Proposition 29.6 is indeed the minimizer, by writing $d = d_\parallel + d_\perp$ with $d_\parallel$ in the row space of N_k^T.

29.7 Implement a zero-velocity update as an EKF measurement and derive its H. When is it safe to apply?

29.8 Read [34] and list which Jacobians must be evaluated at first estimates in the MSCKF. Sketch how you would store the first estimates in the project's data structures without allocation.

Chapter 30

Engineering a Real-Time Estimator in C++20

In this chapter

- Know the C++ facts an estimator depends on: stack and heap, object lifetime, and the C++20 features the project uses.
- Understand why a real-time estimator must be deterministic, and how the project's memory tiers achieve it.
- Know how Eigen represents matrices, blocks and expressions, and use fixed-max-size matrices, member workspaces and allocation guards correctly.
- Avoid the Eigen pitfalls that silently corrupt numbers: dangling expression templates and aliasing.
- Understand what the sanitizers check and how, and how a target-based CMake build is organized.
- Design a test strategy in which every test can fail, and a systematic procedure for debugging a diverging filter.

The mathematics of the previous chapters is necessary but not sufficient. An estimator that is mathematically correct can still miss its deadline because of an allocation, produce garbage because of an Eigen expression template, or pass all its tests because the tests cannot fail. This chapter collects the engineering practices that the project encodes in AGENTS.md.

30.1 C++ essentials for estimator code

30.1.1 Stack, heap and object lifetime

A C++ program keeps its data in two kinds of memory. The *stack* holds the local variables of the functions currently running. Entering a function reserves a block of it by moving a pointer, and leaving the function releases the block by moving the pointer back: the cost is a few instructions, it is the same every time, and the size of each block is fixed at compile time. The *heap* holds objects whose size or lifetime is decided at run time. Obtaining heap memory (`new`, `std::vector` growth, a dynamic-size Eigen matrix) calls the memory allocator, which must find a free block of the right size, may take a lock shared with other threads, and occasionally asks the operating system for more pages. Its cost is usually modest and occasionally large, which is exactly what a real-time loop cannot afford.

Every C++ object has a lifetime that ends at a precisely defined point: a local variable when its scope closes, a member when its owner is destroyed. At that point its destructor runs.

Tying resources (memory, files, locks) to object lifetime is called *RAII* (resource acquisition is initialization): a `std::vector` frees its heap block in its destructor, so it cannot leak. The project uses RAII everywhere and never calls `delete` by hand. The stack is small (typically 1 MB on Windows, 8 MB on Linux), so large matrices belong in class members created once, not in function locals.

30.1.2 The C++20 features the project uses

```
inline constexpr int kImuDim = 15;   // compile-time constant, one definition

[[nodiscard]] bool update(const PoseMeasurement& z) noexcept; // result must be used
void ingest(std::span<const ImuSample> samples) noexcept;     // view of contiguous data
```

- `constexpr` values are computed at compile time and can size arrays and Eigen matrices; `inline constexpr` in a header gives one shared definition. Dimensions, indices (AGENTS.md C3) and the χ^2 table are `constexpr`.
- `[[nodiscard]]` makes the compiler warn (with `-Werror`, fail) when a caller ignores the return value: a rejected update or a failed factorization must not be silently dropped.
- `noexcept` promises that a function throws no exception. If one escapes anyway the program terminates, so the promise is only made on the hot path, where errors are reported through return values instead.
- `std::span<const T>` is a non-owning view (a pointer and a length) of contiguous elements. A function taking a span accepts an array, a `std::vector` or part of one without copying and without caring who owns the memory.
- `std::variant<A, B>` holds exactly one of several types, without heap allocation. `std::visit` calls the matching overload:

```
template <class... Ts> struct Overloaded : Ts... { using Ts::operator()...; };
template <class... Ts> Overloaded(Ts...) -> Overloaded<Ts...>;

for (const SensorEvent& event : events) {           // one presorted stream
  std::visit(Overloaded{
      [&](const ImuSample& s)  { filter.propagate(s); },
      [&](const ImageEvent& e) { pipeline.on_image(e); },
  }, event);
}
```

`Overloaded` inherits the call operators of all the lambdas passed to it, so `std::visit` finds the one whose parameter type matches the alternative currently stored. Adding a new event type to the variant without handling it is a compile error, not a silent omission.

30.2 Determinism and the memory tiers

A flight controller consumes the estimator's output at a fixed rate. A late estimate is a wrong estimate. The main source of unpredictable latency in C++ is dynamic memory allocation:

`new`, a `std::vector` that grows, a dynamic-size Eigen temporary. The allocator may take locks, search free lists or ask the operating system for pages, with latency that is usually small and occasionally large. The project therefore separates three tiers (AGENTS.md E4).

Tier	Scope	Rule
Load	parsing, building event arrays, constructing the filter	allocate freely, before streaming starts
Core	`vio_core`, per IMU sample and per update	zero heap allocations
Frontend	OpenCV tracking	may allocate, must reuse buffers; allocations are measured and logged

30.2.1 Fixed-max-size matrices

The MSCKF covariance changes size as clones come and go, but never exceeds $15 + 6N_{max}$. Eigen supports exactly this case: a matrix whose *runtime* size is dynamic but whose *maximum* size is a compile-time constant. Its storage is an inline array of the maximum size, so resizing within the bound never allocates.

```
inline constexpr int kMaxClones = 10;                  // decided in M10a
inline constexpr int kMaxDim    = 15 + 6 * kMaxClones;

using CovMatrix = Eigen::Matrix<double, Eigen::Dynamic, Eigen::Dynamic,
                                Eigen::ColMajor, kMaxDim, kMaxDim>;

CovMatrix P;          // inline storage for 75 x 75 doubles, no heap
P.resize(15, 15);     // allowed: within the compile-time maximum
```

Large workspaces (S, K, stacked H, temporaries for ΦP) are class members, created once, rather than function-local variables. Function-local fixed-max matrices would not allocate either, but they would live on the stack, and several of them at once can exceed it (Chapter 25).

30.2.2 Proving it: allocation guards

"It shouldn't allocate" is not evidence. The project overrides the global `operator new` in its tests to count allocations (`tests/support/alloc_guard.hpp`), and wraps every core hot-path test with a guard that asserts a count of zero. In debug builds, `EIGEN_RUNTIME_NO_MALLOC` together with `Eigen::internal::set_is_malloc_allowed(false)` makes Eigen itself abort on any internal allocation and reports where it happened. The two mechanisms are complementary: the guard catches allocations anywhere, Eigen's flag pinpoints those inside Eigen.

30.3 Eigen in brief

The matrix type. Every dense Eigen matrix is an instance of one class template,

```
Eigen::Matrix<Scalar, Rows, Cols, Options, MaxRows, MaxCols>
```

`Rows` and `Cols` are either compile-time numbers (a *fixed-size* matrix such as `Eigen::Matrix3d`, stored inline with no heap) or `Eigen::Dynamic` (size chosen at run time, normally on the heap, such as `Eigen::MatrixXd`). `MaxRows` and `MaxCols` give a dynamic matrix a compile-time upper bound, which turns its storage into an inline array: the fixed-max-size matrices used below. `Options` chooses the storage order.

Storage order. Memory is one-dimensional. A *column-major* matrix (Eigen's default) stores column after column, so entry (i, j) of an $m \times n$ matrix lives at offset $i + jm$; a *row-major* matrix stores row after row, offset $j + in$. Loops are fastest when they walk memory contiguously, so with column-major storage the inner loop should run down a column. Mixing orders is legal but may force copies.

Blocks. `P.block(i, j, r, c)`, `P.topLeftCorner(15, 15)`, `v.segment<3>(6)` and their relatives return *views*: they refer to the original storage, copy nothing, and can be assigned to. `P.block<3, 3>(6, 6) = ...` writes the attitude block of P in place. The fixed-size forms (template arguments) let Eigen unroll small operations completely.

Passing matrices and blocks to functions. A parameter of type `const Eigen::Matrix3d&` accepts only that exact type. To accept any matrix or block of compatible layout, use `Eigen::Ref<const Eigen::MatrixXd>` (read-only) or `Eigen::Ref<Eigen::MatrixXd>` (writable). A read-only `Ref` quietly makes a temporary copy when the argument's layout is incompatible, and for a dynamic type that copy allocates; a writable `Ref` refuses to compile instead. The allocation guard catches the quiet case.

Wrapping existing memory. `Eigen::Map<const Eigen::Vector3d> v(ptr)` views three doubles at `ptr` as a vector, without copying. It is the right tool for parsed buffers, and the wrong one for quaternions read from EuRoC: `Eigen::Map<Eigen::Quaterniond>` interprets memory as (x, y, z, w), while the CSV stores (w, x, y, z) (AGENTS.md C2).

Expression templates. `A + B` does not compute a matrix. It returns a small object recording "the sum of `A` and `B`". Only when that object is assigned to a matrix does Eigen run a loop, and it fuses the whole expression into it: `D = A + B + C` is one loop with no temporaries. Products are the exception: they are evaluated into a temporary unless told otherwise (below). This design makes Eigen fast, and it is also the source of the `auto` pitfall described next.

Decompositions. Linear systems are solved through decomposition objects: `S.llt().solve(B)` (Cholesky, for symmetric positive definite S), `.ldlt()` (robust for semi-definite), `.householderQr()` and `.colPivHouseholderQr()` (QR, Chapter 3). A decomposition object owns a copy of the factor; for fixed-max matrices that copy is inline. `Eigen::LLT<Eigen::Ref<Eigen::MatrixXd>> llt(A)` factors `A` *in place*, overwriting it with the factor and allocating nothing. Symmetric structure can be exploited with `P.selfadjointView<Eigen::Lower>()`, which reads only one triangle.

30.4 Eigen pitfalls

Expression templates and `auto`. Eigen operations return lightweight *expression* objects that are evaluated lazily when assigned to a matrix. Storing one with `auto` keeps references to its operands, which may be temporaries that no longer exist when the expression is finally evaluated.

```
auto bad = R * (a + b);             // expression refers to the temporary (a + b)
Eigen::Vector3d x = bad;            // undefined behavior: the temporary is gone
Eigen::Vector3d good = R * (a + b); // evaluated immediately into a concrete type
```

The rule is simple: never use `auto` with Eigen expressions (AGENTS.md E5). Name the concrete type, or call `.eval()` when a temporary really is wanted. AddressSanitizer catches many of these bugs, but not all.

Aliasing. Eigen assumes that the destination of a matrix product may alias its operands and evaluates products into a temporary. With dynamic sizes that temporary allocates. When the destination is known not to alias, `.noalias()` writes directly into it. When it does alias, the temporary is required: write into a member workspace explicitly.

```
tmp_15xN_.noalias() = Phi * P.topRightCorner(15, nc);  // reads P, writes workspace
P.topRightCorner(15, nc) = tmp_15xN_;                   // copy back
P.bottomLeftCorner(nc, 15) = tmp_15xN_.transpose();     // keep symmetry
```

Coefficient-wise operations do not alias-check. `A = A.transpose()` is the classic bug; use `A.transposeInPlace()` or symmetrize with a workspace.

Passing and alignment. Pass Eigen objects by `const&` (or `Eigen::Ref` for blocks). Do not add `EIGEN_MAKE_ALIGNED_OPERATOR_NEW`: since C++17, aligned `new` handles over-aligned types. The real alignment hazard is mixing `-march` or alignment-changing flags between translation units or with dependencies, which the project forbids.

Decompositions without allocation. Eigen's decompositions normally own their storage. Fixed-max decomposition objects held as members, or the in-place variants that factor a matrix in its own storage, avoid per-call allocation. Whatever the API, verify with the allocation guard.

30.5 Time and events

```
using Clock = std::chrono::nanoseconds;
struct ImuSample   { Clock t; Eigen::Vector3d gyro, accel; };
struct ImageEvent  { Clock t; int index; };
using SensorEvent = std::variant<ImuSample, ImageEvent /*, PoseMeasurement */>;

// Delta t: subtract in integers, convert only the difference.
const double dt = std::chrono::duration<double>(t1 - t0).count();
```

Absolute time stays in 64-bit integer nanoseconds (Section 3.10). Events form one presorted sequence built at load time, so the streaming loop is a single pass over contiguous memory, dispatched with `std::visit` as in Section 30.1: no priority queue, no allocation, no reordering surprises.

30.6 Numerical robustness checklist

- Normalize every quaternion after every multiplicative update (propagation, injection, cloning).
- Symmetrize P after propagation and after every update.
- Use the Joseph form; factor S with Cholesky or LDL^{T}; treat a failed factorization as an error to report, not to hide.
- Guard every $0/0$ with Taylor branches; use atan2 for angles; handle Log near π.
- Reject features with non-positive or tiny depth, poor parallax or large reprojection error before linearizing.
- Count and log gate rejections and triangulation failures.
- Never clamp negative eigenvalues of P to hide a problem; find out why they appeared.
- In debug builds, assert finiteness at module boundaries so that a `NaN` is caught where it is born.

30.7 Sanitizers

C++ leaves many errors undefined: reading past the end of an array, using memory after freeing it, overflowing a signed integer. Undefined behavior does not mean a crash; it means the compiler may assume it never happens, so the program can produce plausible wrong numbers, behave differently between builds, or fail far from the bug. Sanitizers are compiler-inserted checks that turn these silent errors into immediate, precise reports.

- **AddressSanitizer** (ASan, `-fsanitize=address`) keeps a *shadow memory*, one byte for every eight bytes of program memory, recording which bytes may be accessed. Every heap block, stack variable and global is surrounded by poisoned *redzones*, and freed memory stays poisoned for a while. The compiler inserts a shadow check before every load and store. ASan reports heap, stack and global buffer overflows, use after free, use after scope and double frees, with stack traces of the access and of the allocation. The cost is roughly a factor of two in speed and three in memory.
- **UndefinedBehaviorSanitizer** (UBSan, `-fsanitize=undefined`) inserts checks for signed integer overflow, out-of-range shifts, integer division by zero, null and misaligned pointer use, invalid `bool` and `enum` values, and floating-point-to-integer conversions that overflow.

By default UBSan prints a message and *continues*, so a test that triggers it still passes. `-fno-sanitize-recover=all` makes every report abort the process, and the test preset sets `UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1` so the report carries a stack trace

(AGENTS.md E3). Frame pointers (`-fno-omit-frame-pointer`) make those traces reliable. On Windows, LeakSanitizer does not exist, so leaks are not detected (RAII makes them rare), and an instrumented executable needs the ASan runtime DLL from `C:\msys64\clang64\bin` on its `PATH`. Sanitizers are a debug-build tool: the `release` preset builds without them, and both presets must pass.

30.8 Modern CMake in brief

CMake describes a build as a set of *targets* (libraries and executables) with properties. Modern CMake attaches every setting to a target rather than to global variables, and lets settings propagate along the dependency graph through three keywords: `PRIVATE` (used to build this target only), `INTERFACE` (used only by targets that link to this one) and `PUBLIC` (both). A library's public include directory is `PUBLIC`, because its users include its headers; a warning flag is `PRIVATE`, because it concerns only the library's own code.

```
add_library(vio_warnings INTERFACE)           # carries flags, builds nothing
target_compile_options(vio_warnings INTERFACE -Wall -Wextra -Wconversion -Werror)

add_library(vio_core STATIC src/so3.cpp src/es_ekf.cpp)
target_include_directories(vio_core PUBLIC include)
target_link_libraries(vio_core PUBLIC Eigen3::Eigen PRIVATE vio_warnings)
```

The `vio_warnings` target is how warnings stay off the dependencies (AGENTS.md E3): only project targets link it. Dependencies fetched with `FetchContent` at a pinned tag are declared `SYSTEM`, so their headers are included with `-isystem` and do not trigger the project's warnings. `CMakePresets.json` stores named configurations (generator, compiler, build type, flags, test environment), so that `cmake --preset debug-sanitize`, `cmake --build --preset debug-sanitize` and `ctest --preset debug-sanitize` reproduce exactly the same build on every machine. Catch2's `catch_discover_tests` registers each `TEST_CASE` as a separate CTest test.

30.9 Testing

Key idea: a test must be able to fail

For every test, ask: would it still pass if I flipped the sign of X, transposed Y, or swapped two blocks? If yes, the test is decoration. Run it once against a deliberately broken implementation to make sure it fails.

The project's test strategy layers several kinds of evidence.

1. **Algebraic unit tests**: identities such as $\mathrm{Log}(\mathrm{Exp}(\phi)) = \phi$ at zero, tiny, generic and near-π angles.
2. **Finite-difference Jacobian tests** built on $\boxplus/\boxminus$, at non-trivial states, with tolerances justified in a comment.
3. **Closed-form trajectory tests**: synthetic trajectories with analytic IMU data and convergence-order checks.

4. **Statistical tests**: Monte Carlo covariance checks and NEES/NIS with χ^2 bounds, seeded with `std::mt19937_64`. Because `std::normal_distribution` differs between standard libraries, never hard-code values from one run; test statistical properties.
5. **Invariance tests**: cloning enabled versus disabled must give identical IMU estimates when no visual updates occur.
6. **Resource tests**: allocation guards around every core hot path.
7. **Sanitizers**: ASan and UBSan builds with `-fno-sanitize-recover=all`, so that any report fails the test run (Section 30.7).

30.10 Debugging a diverging filter

When the full system misbehaves, resist the urge to tune. Work down the layers instead.

1. Run on **synthetic data** with exactly modeled noise. If it fails there, the bug is in the code.
2. Disable visual updates. Does IMU-only propagation from ground truth behave as Figure 17.1 predicts, with errors inside 3σ?
3. Replace visual updates by **pose updates** (Milestone 6). If those are consistent, the back end is sound.
4. Feed the MSCKF **ground-truth feature tracks** from a synthetic scene. If that works, the problem is in the front end or the calibration.
5. On real data, plot each residual component over time and look for bias, oscillation or outliers.
6. Only then adjust noise parameters, and judge the result on consistency statistics.

30.11 Going further

Once the baseline MSCKF runs, natural extensions are First-Estimates Jacobians (Chapter 29); online calibration of the camera-IMU extrinsics and time offset [34, 35]; stereo observations from `cam1`; keeping long-lived features as SLAM landmarks; zero-velocity updates; and evaluation on the harder EuRoC sequences (V1_03, MH_05) and other datasets. Each extension adds states and Jacobians, and each Jacobian gets its derivation and its finite-difference test.

Key takeaways

- Heap allocation has unpredictable cost; the stack is cheap but small. Large workspaces are members, created once.
- Determinism requires zero allocation in the core: fixed-max-size matrices, member workspaces, proven by allocation guards.
- Never use `auto` with Eigen expressions; use `.noalias()` only when there is no aliasing.

- Keep time in integer nanoseconds and process one presorted event stream.
- Sanitizers turn undefined behavior into reports; without `-fno-sanitize-recover=all` UBSan reports do not fail tests.
- Every test must be able to fail; layer algebraic, Jacobian, closed-form, statistical and resource tests.
- Debug by removing layers until the failure disappears, not by tuning.

Exercises

30.1 Write a minimal allocation guard by overriding the global `operator new` and `operator delete`, and show that it detects a `std::vector::push_back` beyond capacity but not a fixed-max Eigen `resize`.

30.2 Construct an example in which `auto` with an Eigen expression produces wrong numbers without crashing. Does AddressSanitizer report it?

30.3 Measure the time of 10 000 covariance propagation steps with fixed-max and with fully dynamic Eigen matrices, in both build presets. Explain the difference.

30.4 Take one of your Jacobian tests, break the Jacobian in three different ways, and record which breakages the test detects.

30.5 Write a function that reads one element past the end of a `std::array` and another that overflows an `int`. Build both presets and record what each sanitizer reports, and what the release build does.

30.6 Pass a row-major block to a function taking `Eigen::Ref<const Eigen::MatrixXd>` inside an allocation guard. What happens, and what happens if the parameter is a writable `Ref`?

30.7 Add a third alternative to the `SensorEvent` variant without handling it in the `Overloaded` visitor. Read the compiler error.

Appendices

Appendix A

Notation and Conventions

This appendix restates the frozen conventions of `AGENTS.md` in one place, together with the notation of these notes.

Frames and transformations

Symbol	Code name	Meaning
W	—	world frame: fixed, gravity-aligned, z up; EuRoC `R`
B	—	body frame = IMU `imu0`; EuRoC `S` in `imu0`
C	—	left camera `cam0`: z optical axis, x right, y down
R_{wb}, q_{wb}	`R_wb`, `q_wb`	attitude, maps body to world: $v_w = R_{wb} v_b$
p_{wb}	`p_wb`	position of B's origin in W
v_{wb}	`v_wb`	velocity of B relative to W, expressed in W
$T_{bc} = (R_{bc}, p_{bc})$	`T_bc`	camera-to-body: $p_b = R_{bc} p_c + p_{bc}$; EuRoC `T_BS` of `cam0`
g_w	—	gravity, $[0, 0, -9.81]^\mathsf{T}$ m/s^2

Naming rule: `<quantity>_<to><from>`; adjacent subscripts of a product must match ($R_{wb} R_{bc} = R_{wc}$).

State, error state and noise

Error index	Block	Definition
0–2	δp	$p = \hat{p} + \delta p$ (world)
3–5	δv	$v = \hat{v} + \delta v$ (world)
6–8	$\delta\theta$	$q = \hat{q} \otimes \mathrm{Exp}_q(\delta\theta)$ (body, local)
9–11	δb_a	$b_a = \hat{b}_a + \delta b_a$
12–14	δb_g	$b_g = \hat{b}_g + \delta b_g$
$15 + 6i \ldots$	δp_i	clone i position
$18 + 6i \ldots$	$\delta\theta_i$	clone i attitude (local)

Noise index	Block
0–2	n_a
3–5	n_g
6–8	n_{b_a}
9–11	n_{b_g}

Operators and symbols

Symbol	Meaning
a^T, A^{-1}, $A^{-\mathsf{T}}$	transpose, inverse, inverse transpose
$[a]_\times$, $a^\wedge$	skew-symmetric (cross-product) matrix of $a \in \mathbb{R}^3$
$(\cdot)^\vee$	inverse of the hat map
$\otimes$	Hamilton quaternion product
q^*	quaternion conjugate
exp, log	matrix exponential and logarithm
Exp, Log	$\mathbb{R}^3 \leftrightarrow$ SO(3); Exp_q, Log_q for unit quaternions
J_r, J_l	right and left Jacobians of SO(3)
$\boxplus$, $\boxminus$	retraction and local difference (AGENTS.md C4)
$\hat{x}$, δx, x	nominal (estimated), error, and true state
P, Q_c, Q_d, R_z	error covariance, continuous and discrete process noise, measurement noise
F_c, G_c, Φ	continuous error dynamics, noise input matrix, discrete transition
H, S, K, r	measurement Jacobian, innovation covariance, gain, residual
$\mathcal{N}(\mu, \Sigma)$	Gaussian distribution
χ^2_k, $\chi^2_p(k)$	chi-squared distribution with k DOF; its quantile at probability p
$\pi(\cdot)$	pinhole projection $(X/Z, Y/Z)$
u, $\varepsilon_{\mathrm{mach}}$	unit roundoff 2^{-53}, machine epsilon 2^{-52}

Abbreviations

ATE / RPE	absolute trajectory error / relative pose error
BCH	Baker–Campbell–Hausdorff formula
DOF	degrees of freedom
EKF / ESKF	extended / error-state Kalman filter
FEJ	first-estimates Jacobians
IMU	inertial measurement unit
KLT / LK	Kanade–Lucas–Tomasi tracker / Lucas–Kanade optical flow
MSCKF	multi-state constraint Kalman filter
NEES / NIS	normalized estimation error squared / normalized innovation squared
PSD / PD	positive semi-definite / positive definite
RANSAC	random sample consensus
SLAM	simultaneous localization and mapping
VIO	visual-inertial odometry

Appendix B

Formula Sheet

Every formula below follows the project conventions (Hamilton quaternions, local perturbations) and was verified numerically. Let $\theta = \|\phi\|$.

Cross products

$$[a]_\times b = a \times b = -[b]_\times a, \qquad [a]_\times^\mathsf{T} = -[a]_\times, \qquad [a]_\times^2 = aa^\mathsf{T} - \|a\|^2 I, \qquad [a]_\times^3 = -\|a\|^2 [a]_\times,$$
$$[a]_\times [b]_\times = ba^\mathsf{T} - (a^\mathsf{T} b) I, \qquad [a \times b]_\times = [a]_\times [b]_\times - [b]_\times [a]_\times, \qquad [Ra]_\times = R[a]_\times R^\mathsf{T}.$$

Exponential and logarithm

$$\mathrm{Exp}(\phi) = I + \frac{\sin\theta}{\theta}[\phi]_\times + \frac{1-\cos\theta}{\theta^2}[\phi]_\times^2, \qquad \mathrm{Exp}_q(\phi) = \left[\cos\tfrac{\theta}{2},\ \tfrac{\sin(\theta/2)}{\theta}\phi\right],$$
$$\theta = \mathrm{atan2}\left(\tfrac{1}{2}\|(R - R^\mathsf{T})^\vee\|,\ \tfrac{1}{2}(\mathrm{tr}\, R - 1)\right), \qquad \mathrm{Log}(R) = \frac{\theta}{2\sin\theta}(R - R^\mathsf{T})^\vee \quad (0 < \theta < \pi),$$
$$\text{near } \pi:\ uu^\mathsf{T} = \frac{\frac{1}{2}(R + R^\mathsf{T}) - \cos\theta\, I}{1 - \cos\theta}, \qquad \mathrm{Log}_q(q) = 2\,\mathrm{atan2}(\|v\|, w)\frac{v}{\|v\|} \ \text{ after } w \geq 0.$$

Coefficient functions and their series

Function	Closed form	Series
$A(\theta)$	$\sin\theta/\theta$	$1 - \theta^2/6 + \theta^4/120$
$B(\theta)$	$(1-\cos\theta)/\theta^2$	$1/2 - \theta^2/24 + \theta^4/720$
$C(\theta)$	$(\theta - \sin\theta)/\theta^3$	$1/6 - \theta^2/120 + \theta^4/5040$
$D(\theta)$	$1/\theta^2 - \cot(\theta/2)/(2\theta)$	$1/12 + \theta^2/720 + \theta^4/30240$
$\cos(\theta/2)$		$1 - \theta^2/8 + \theta^4/384$
$\sin(\theta/2)/\theta$		$1/2 - \theta^2/48 + \theta^4/3840$
$\theta/\sin\theta$		$1 + \theta^2/6 + 7\theta^4/360$

Jacobians of SO(3)

$$J_r(\phi) = I - B\,[\phi]_\times + C\,[\phi]_\times^2, \qquad J_r^{-1}(\phi) = I + \tfrac{1}{2}[\phi]_\times + D\,[\phi]_\times^2,$$
$$J_l(\phi) = J_r(-\phi) = J_r(\phi)^\mathsf{T} = \mathrm{Exp}(\phi)J_r(\phi), \qquad J_r(\phi) = \int_0^1 \mathrm{Exp}(-s\phi)\,\mathrm{d}s,$$
$$\mathrm{Exp}(\phi+\delta) \approx \mathrm{Exp}(\phi)\,\mathrm{Exp}(J_r(\phi)\delta), \qquad \mathrm{Log}(\mathrm{Exp}(\phi)\,\mathrm{Exp}(\delta)) \approx \phi + J_r^{-1}(\phi)\delta,$$
$$R\,\mathrm{Exp}(\phi)R^\mathsf{T} = \mathrm{Exp}(R\phi), \qquad \mathrm{Log}(\mathrm{Exp}(a)\,\mathrm{Exp}(b)) = a + b + \tfrac{1}{2}a \times b + \cdots$$

Elementary Jacobians (local perturbation $R\,\mathrm{Exp}(\delta\theta)$)

Rv	$-R[v]_\times$	R^{-1}	$-R$
$R^\mathsf{T}v$	$[R^\mathsf{T}v]_\times$	R_1R_2	R_2^T (w.r.t. R_1), I (w.r.t. R_2)
$\mathrm{Exp}(\phi)$	$J_r(\phi)$	$\mathrm{Log}(R)$	$J_r^{-1}(\mathrm{Log}\,R)$

Quaternions

$$p \otimes q = \left[p_w q_w - p_v^\mathsf{T} q_v,\ p_w q_v + q_w p_v + p_v \times q_v\right], \qquad R(q) = (w^2 - v^\mathsf{T}v)I + 2vv^\mathsf{T} + 2w[v]_\times,$$
$$\dot{q} = \tfrac{1}{2}q \otimes [0, \omega_b], \qquad \dot{R} = R[\omega_b]_\times, \qquad R(p \otimes q) = R(p)R(q).$$

Linear algebra and least squares

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U\left(C^{-1} + VA^{-1}U\right)^{-1}VA^{-1} \quad \text{(Woodbury)},$$
$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} : \text{ Schur complement } S_A = D - CA^{-1}B,$$
$$\hat{x}_{\mathrm{LS}} = (A^\mathsf{T}A)^{-1}A^\mathsf{T}b = R^{-1}Q_1^\mathsf{T}b \ \ (A = Q_1R),$$
$$\mathrm{null}(A^\mathsf{T}) = \mathrm{range}(Q_2) \ \left(A = [Q_1\ Q_2]\begin{bmatrix} R \\ 0 \end{bmatrix}\right), \qquad \kappa_2(A) = \sigma_{\max}/\sigma_{\min},$$
$$\min_{\|x\|=1} \|Ax\| = \sigma_{\min}, \text{ attained at the right singular vector of } \sigma_{\min}.$$

Gaussians

$$\mathcal{N}(x;\mu,\Sigma) = \frac{\exp\left(-\frac{1}{2}(x-\mu)^\mathsf{T}\Sigma^{-1}(x-\mu)\right)}{\sqrt{(2\pi)^n \det\Sigma}}, \qquad Ax + b \sim \mathcal{N}(A\mu + b,\ A\Sigma A^\mathsf{T}),$$
$$\mu_{a|b} = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(b - \mu_b), \qquad \Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba},$$
$$(x-\mu)^\mathsf{T}\Sigma^{-1}(x-\mu) \sim \chi_n^2, \qquad \text{fusion: } \Sigma^{-1} = \Sigma_1^{-1} + \Sigma_2^{-1},\ \ \Sigma^{-1}\mu = \Sigma_1^{-1}\mu_1 + \Sigma_2^{-1}\mu_2.$$

Random processes and discretization

$$\dot{P} = FP + PF^\mathsf{T} + GQ_cG^\mathsf{T} \quad \text{(Lyapunov)}, \qquad Q_d = \int_0^{\Delta t} e^{Fs} G Q_c G^\mathsf{T} e^{F^\mathsf{T} s}\,\mathrm{d}s,$$

$$\exp\left(\begin{bmatrix} -F & GQ_cG^\mathsf{T} \\ 0 & F^\mathsf{T} \end{bmatrix} \Delta t\right) = \begin{bmatrix} C_{11} & C_{12} \\ 0 & C_{22} \end{bmatrix} : \quad \Phi = C_{22}^\mathsf{T}, \; Q_d = \Phi C_{12} \quad \text{(Van Loan)}.$$

Kalman filter

$$P \leftarrow \Phi P \Phi^\mathsf{T} + Q_d, \qquad \Phi \approx I + F_c \Delta t, \qquad Q_d \approx G_c Q_c G_c^\mathsf{T} \Delta t,$$

$$r = z \boxminus h(\hat{x}), \quad H = \tfrac{\partial}{\partial \delta x}\left[h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})\right]_0, \quad S = HPH^\mathsf{T} + R_z, \quad K = PH^\mathsf{T} S^{-1},$$

$$P \leftarrow (I - KH)P(I - KH)^\mathsf{T} + KR_zK^\mathsf{T}, \qquad \hat{x} \leftarrow \hat{x} \boxplus Kr, \qquad P \leftarrow GPG^\mathsf{T}, \; G_\theta = I - \tfrac{1}{2}[\delta\hat{\theta}]_\times.$$

Unscented transform

$$\lambda = \alpha^2(n + \kappa) - n, \qquad \chi_0 = \mu, \quad \chi_i, \chi_{n+i} = \mu \pm \sqrt{n + \lambda}\,\ell_i \;\; (P = LL^\mathsf{T}),$$

$$W_0^{(m)} = \frac{\lambda}{n + \lambda}, \quad W_0^{(c)} = \frac{\lambda}{n + \lambda} + 1 - \alpha^2 + \beta, \quad W_i^{(m)} = W_i^{(c)} = \frac{1}{2(n + \lambda)}.$$

Nonlinear least squares

$$(J^\mathsf{T} W J + \lambda I)\,\delta = -J^\mathsf{T} W r, \qquad x \leftarrow x \boxplus \delta,$$

$$\lambda = 0\text{: Gauss–Newton}, \qquad \lambda > 0\text{: Levenberg–Marquardt},$$

$$\text{Huber weight } w(s) = \min(1, c/s), \qquad \text{Cauchy weight } w(s) = 1/(1 + s^2/c^2),$$

$$\begin{bmatrix} A & B \\ B^\mathsf{T} & C \end{bmatrix} \begin{bmatrix} \delta c \\ \delta l \end{bmatrix} = \begin{bmatrix} g_c \\ g_l \end{bmatrix} \;\Rightarrow\; (A - BC^{-1}B^\mathsf{T})\,\delta c = g_c - BC^{-1} g_l \;\; \text{(reduced camera system)}.$$

Strapdown propagation with constant inputs ($\phi = \hat{\omega}\Delta t$)

$$R_{k+1} = R_k \operatorname{Exp}(\phi), \qquad v_{k+1} = v_k + g\Delta t + R_k \Delta t J_l(\phi)\hat{a},$$

$$p_{k+1} = p_k + v_k \Delta t + \tfrac{1}{2} g \Delta t^2 + R_k \Delta t^2 N(\phi)\hat{a},$$

$$N(\phi) = \tfrac{1}{2} I + \frac{\theta - \sin\theta}{\theta^3}[\phi]_\times + \frac{\theta^2/2 + \cos\theta - 1}{\theta^4}[\phi]_\times^2.$$

Bortz equation and the two-sample coning correction (increments α_1, α_2 over the two halves of a step):

$$\dot{\phi} = J_r^{-1}(\phi)\,\omega = \omega + \tfrac{1}{2}\phi \times \omega + D(\theta)\,\phi \times (\phi \times \omega), \qquad \phi \approx \alpha_1 + \alpha_2 + \tfrac{2}{3}\,\alpha_1 \times \alpha_2.$$

Noise

$$\sigma_d = \sigma/\sqrt{\Delta t} \ \text{(white, per sample)}, \qquad \sigma_{b,d} = \sigma_b\sqrt{\Delta t} \ \text{(random walk, per step)},$$
$$\text{Var}(m\text{-fold integral of white noise}) = \sigma^2 \frac{t^{2m-1}}{(2m-1)((m-1)!)^2},$$
$$1\,^\circ/\sqrt{\text{h}} = 2.909 \times 10^{-4}\ \text{rad/s}/\sqrt{\text{Hz}}.$$

Consistency and alignment

$$\epsilon = e^\mathsf{T} P^{-1} e \sim \chi^2_n \ \text{(NEES)}, \qquad \nu = r^\mathsf{T} S^{-1} r \sim \chi^2_m \ \text{(NIS)},$$
$$\bar{\epsilon} \in \left[\frac{\chi^2_{0.025}(Nn)}{N},\ \frac{\chi^2_{0.975}(Nn)}{N}\right] \ \text{(average over } N \text{ runs, 95\%)},$$
$$\Sigma_{ab} = \tfrac{1}{N}\textstyle\sum_i a_i b_i^\mathsf{T} = UDV^\mathsf{T}: \quad R^\star = U\,\text{diag}(1, 1, \det(UV^\mathsf{T}))V^\mathsf{T}, \quad t^\star = \mu - R^\star\hat{\mu} \quad \text{(Umeyama)}.$$

Geometry

$$\bar{x}_2^\mathsf{T}[t]_\times R\bar{x}_1 = 0, \qquad \frac{\partial \pi}{\partial p_c} = \frac{1}{Z}\begin{bmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{bmatrix},$$
$$\sigma_Z \approx \sqrt{2}\frac{Z^2}{b}\sigma_{uv}, \qquad N_{\text{RANSAC}} = \frac{\log(1-p)}{\log(1-w^s)}.$$

MSCKF

$$r_j \approx H_{x,j}\,\delta x + H_{f,j}\,\delta p_f + n_j, \qquad A_j^\mathsf{T} H_{f,j} = 0,\ \ A_j^\mathsf{T} A_j = I,$$
$$r_{o,j} = A_j^\mathsf{T} r_j, \quad H_{o,j} = A_j^\mathsf{T} H_{x,j}, \quad \text{Cov}(A_j^\mathsf{T} n_j) = \sigma^2 I,$$
$$\dim r_{o,j} = 2M - 3 \ \text{(space)},\ \ M - 2 \ \text{(plane)},$$
$$\text{gate: } r_{o,j}^\mathsf{T}(H_{o,j} P H_{o,j}^\mathsf{T} + \sigma^2 I)^{-1} r_{o,j} < \chi^2_{0.95}(\dim r_{o,j}),$$
$$H_o = Q_1 T_H \implies (r_n, H_n) = (Q_1^\mathsf{T} r_o, T_H) \ \text{(compression)},$$
$$P \leftarrow JPJ^\mathsf{T}, \ \ J = \begin{bmatrix} I \\ J_g \end{bmatrix} \ \text{(augmentation)}.$$

Planar models

$$\Omega = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad R(\psi) = \cos\psi\, I + \sin\psi\, \Omega, \qquad R\Omega = \Omega R,$$

$$R(\psi+\delta)^{\mathsf{T}} \approx (I - \delta\,\Omega) R(\psi)^{\mathsf{T}}, \qquad z = y/x: \quad J_\pi = \tfrac{1}{x}\begin{bmatrix} -z & 1 \end{bmatrix},$$

$$\frac{\partial z}{\partial \delta p_i} = -J_\pi R_i^{\mathsf{T}}, \qquad \frac{\partial z}{\partial \delta \psi_i} = -(1+z^2), \qquad \frac{\partial z}{\partial p_f} = J_\pi R_i^{\mathsf{T}},$$

$$\Phi_k = \begin{bmatrix} I & \Omega(\hat{p}_{k+1} - \hat{p}_k) \\ 0 & 1 \end{bmatrix}, \qquad N = \begin{bmatrix} \begin{bmatrix} I_2 \\ 0 \end{bmatrix} & \begin{bmatrix} \Omega\hat{p} \\ 1 \end{bmatrix} \\ \begin{bmatrix} I_2 \\ 0 \end{bmatrix} & \begin{bmatrix} \Omega\hat{p}_1 \\ 1 \end{bmatrix} \\ \vdots & \vdots \end{bmatrix}.$$

Observability

$$\mathcal{O} = \begin{bmatrix} H_1 \\ H_2\Phi_1 \\ \vdots \\ H_k\Phi_{k-1}\cdots\Phi_1 \end{bmatrix}, \qquad L_f\varphi = \nabla\varphi \cdot f,$$

$$N_{\text{yaw}} = \big(-[\hat{p}]_\times e_z,\ -[\hat{v}]_\times e_z,\ \hat{R}^{\mathsf{T}} e_z,\ 0,\ 0,\ -[\hat{p}_i]_\times e_z,\ \hat{R}_i^{\mathsf{T}} e_z \big),$$

$$H^* = H\big(I - N(N^{\mathsf{T}}N)^{-1}N^{\mathsf{T}}\big), \qquad \Phi^* = \Phi + (N_{k+1} - \Phi N_k)(N_k^{\mathsf{T}}N_k)^{-1}N_k^{\mathsf{T}} \quad \text{(OC-EKF)}.$$

Appendix C

Statistical and Numerical Tables

The tables are generated by `scripts/make_tables.py` with SciPy, so they can be regenerated or extended. The project's `constexpr` χ^2 table (AGENTS.md F3) should be generated the same way, offline, rather than typed by hand.

Quantiles of the χ^2 distribution

$\chi^2_p(k)$ is the value below which a χ^2_k variable falls with probability p. Gates use $\chi^2_{0.95}$ or $\chi^2_{0.99}$; two-sided 95% intervals use $\chi^2_{0.025}$ and $\chi^2_{0.975}$. For other probability levels, rerun the script. An MSCKF feature with M observations has $k = 2M - 3$.

k	$\chi^2_{0.025}$	$\chi^2_{0.95}$	$\chi^2_{0.975}$	$\chi^2_{0.99}$	k	$\chi^2_{0.025}$	$\chi^2_{0.95}$	$\chi^2_{0.975}$	$\chi^2_{0.99}$
1	0.001	3.841	5.024	6.635	26	13.844	38.885	41.923	45.642
2	0.051	5.991	7.378	9.210	27	14.573	40.113	43.195	46.963
3	0.216	7.815	9.348	11.345	28	15.308	41.337	44.461	48.278
4	0.484	9.488	11.143	13.277	29	16.047	42.557	45.722	49.588
5	0.831	11.070	12.833	15.086	30	16.791	43.773	46.979	50.892
6	1.237	12.592	14.449	16.812	31	17.539	44.985	48.232	52.191
7	1.690	14.067	16.013	18.475	32	18.291	46.194	49.480	53.486
8	2.180	15.507	17.535	20.090	33	19.047	47.400	50.725	54.776
9	2.700	16.919	19.023	21.666	34	19.806	48.602	51.966	56.061
10	3.247	18.307	20.483	23.209	35	20.569	49.802	53.203	57.342
11	3.816	19.675	21.920	24.725	36	21.336	50.998	54.437	58.619
12	4.404	21.026	23.337	26.217	37	22.106	52.192	55.668	59.893
13	5.009	22.362	24.736	27.688	38	22.878	53.384	56.896	61.162
14	5.629	23.685	26.119	29.141	39	23.654	54.572	58.120	62.428
15	6.262	24.996	27.488	30.578	40	24.433	55.758	59.342	63.691
16	6.908	26.296	28.845	32.000	45	28.366	61.656	65.410	69.957
17	7.564	27.587	30.191	33.409	50	32.357	67.505	71.420	76.154
18	8.231	28.869	31.526	34.805	60	40.482	79.082	83.298	88.379
19	8.907	30.144	32.852	36.191	75	52.942	96.217	100.839	106.393
20	9.591	31.410	34.170	37.566	100	74.222	124.342	129.561	135.807
21	10.283	32.671	35.479	38.932	150	117.985	179.581	185.800	193.208
22	10.982	33.924	36.781	40.289	200	162.728	233.994	241.058	249.445
23	11.689	35.172	38.076	41.638	300	253.912	341.395	349.874	359.906
24	12.401	36.415	39.364	42.980	450	393.118	500.456	510.670	522.717
25	13.120	37.652	40.646	44.314	600	534.019	658.094	669.769	683.516

Average NEES acceptance intervals

Two-sided 95% interval $\left[\chi^2_{0.025}(Nn)/N,\ \chi^2_{0.975}(Nn)/N\right]$ for the average NEES of an n-dimensional error over N independent Monte Carlo runs (Chapter 19). The same intervals apply to the time-average of N independent NIS values of dimension n.

$n \backslash N$	1	10	25	50	100	200
1	[0.00, 5.02]	[0.32, 2.05]	[0.52, 1.63]	[0.65, 1.43]	[0.74, 1.30]	[0.81, 1.21]
2	[0.05, 7.38]	[0.96, 3.42]	[1.29, 2.86]	[1.48, 2.59]	[1.63, 2.41]	[1.73, 2.29]
3	[0.22, 9.35]	[1.68, 4.70]	[2.12, 4.03]	[2.36, 3.72]	[2.54, 3.50]	[2.67, 3.35]
6	[1.24, 14.45]	[4.05, 8.33]	[4.72, 7.43]	[5.08, 7.00]	[5.34, 6.70]	[5.53, 6.49]
9	[2.70, 19.02]	[6.56, 11.81]	[7.41, 10.74]	[7.86, 10.21]	[8.19, 9.85]	[8.42, 9.60]
15	[6.26, 27.49]	[11.80, 18.58]	[12.93, 17.22]	[13.52, 16.56]	[13.95, 16.09]	[14.25, 15.77]

Monte Carlo variance ratio

If M runs produce a sample variance s^2 of an error whose true variance is σ^2, then $s^2/\sigma^2 \sim \chi^2_{M-1}/(M-1)$. The 95% intervals below are the acceptance bounds for a covariance-propagation check (Milestone 5).

runs M	lower bound	upper bound
50	0.644	1.433
100	0.741	1.297
200	0.813	1.206
500	0.880	1.128
1000	0.914	1.090
2000	0.939	1.063

RANSAC iterations

Number of iterations $N = \lceil \log(1-p) / \log(1-w^s) \rceil$ for confidence $p = 0.99$, minimal sample size s and inlier ratio w.

$s \backslash w$	0.9	0.8	0.7	0.6	0.5	0.4	0.3
1	2	3	4	6	7	10	13
2	3	5	7	11	17	27	49
3	4	7	11	19	35	70	169
5	6	12	26	57	146	448	1893
7	8	20	54	163	588	2809	21055
8	9	26	78	272	1177	7025	70188

Appendix D

The EuRoC MAV Dataset

The EuRoC MAV dataset [8] was recorded with an AscTec Firefly hexacopter carrying a VI-Sensor: two global-shutter MT9V034 cameras (752×480, 20 Hz) and an ADIS16448 MEMS IMU (200 Hz), hardware time-synchronized and calibrated with Kalibr. The values below are those of the published calibration files; check them against your copy.

Directory layout

The project keeps sequences under `data/<sequence>/mav0/` (git-ignored). Each sequence contains:

Path	Content
`imu0/data.csv`	`timestamp [ns], w_x, w_y, w_z [rad/s], a_x, a_y, a_z [m/s²]`: ω_m, a_m in B
`imu0/sensor.yaml`	IMU noise densities (continuous time), rate, `T_BS` (identity for the IMU)
`cam0/data.csv`	`timestamp [ns], filename`
`cam0/data/`	PNG images named by timestamp
`cam0/sensor.yaml`	`T_BS` (row-major 4×4), `intrinsics`, `distortion_coefficients`, resolution, rate
`cam1/`	right camera (unused; stereo is a stretch goal)
`state_groundtruth_estimate0/data.csv`	`timestamp, p_RS_R (3), q_RS (w,x,y,z), v_RS_R (3), b_w_RS_S (3), b_a_RS_S (3)`

Pitfall: ground-truth column order

In the ground-truth file the quaternion is stored w first, and the *gyroscope* bias precedes the *accelerometer* bias. The file starts later than the IMU file, and its velocities and biases come from the dataset authors' own estimator that fused the tracking-system data with the IMU, so they are reference values, not perfect truth. The project maps the columns to $p_{wb}, q_{wb}, v_{wb}, b_g, b_a$ (AGENTS.md C7).

IMU parameters (`imu0/sensor.yaml`)

Key	Value	Units
`gyroscope_noise_density`	1.6968×10^{-4}	$\mathrm{rad/s}/\sqrt{\mathrm{Hz}}$
`gyroscope_random_walk`	1.9393×10^{-5}	$\mathrm{rad/s^2}/\sqrt{\mathrm{Hz}}$
`accelerometer_noise_density`	2.0×10^{-3}	$\mathrm{m/s^2}/\sqrt{\mathrm{Hz}}$
`accelerometer_random_walk`	3.0×10^{-3}	$\mathrm{m/s^3}/\sqrt{\mathrm{Hz}}$
`rate_hz`	200	Hz

Camera parameters (`cam0/sensor.yaml`)

`intrinsics` $[f_u, f_v, c_u, c_v]$	$[458.654,\ 457.296,\ 367.215,\ 248.375]$
`distortion_model`	`radial-tangential`
`distortion_coefficients` $[k_1, k_2, p_1, p_2]$	$[-0.28340811,\ 0.07395907,\ 0.00019359,\ 1.76187114 \times 10^{-5}]$
`resolution`	752×480
`rate_hz`	20

The extrinsic calibration `T_BS` of `cam0` is the project's T_{bc}:

$$T_{bc} \approx \begin{bmatrix} 0.0148655 & -0.9998809 & 0.0041403 & -0.0216401 \\ 0.9995572 & 0.0149672 & 0.0257155 & -0.0646770 \\ -0.0257744 & 0.0037562 & 0.9996607 & 0.0098107 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Its third column shows that the optical axis z_c is almost exactly the body z-axis, and its first two columns that the camera x-axis is close to body y and the camera y-axis close to body $-x$. The camera center sits about 7 cm from the IMU. At rest, the EuRoC accelerometer reads close to $+g$ on its x-axis, so the body x-axis points roughly up. Verify both facts on your data: they are exactly the kind of sanity check that catches a transposed extrinsic.

Sequences

Sequence	Environment	Difficulty	Ground truth
MH_01_easy, MH_02_easy	machine hall	easy	laser tracker (position)
MH_03_medium	machine hall	medium	laser tracker (position)
MH_04_difficult, MH_05_difficult	machine hall	difficult (fast, dark)	laser tracker (position)
V1_01_easy, V2_01_easy	Vicon room	easy	motion capture (pose)
V1_02_medium, V2_02_medium	Vicon room	medium	motion capture (pose)
V1_03_difficult, V2_03_difficult	Vicon room	difficult (fast, blur)	motion capture (pose)

MH_01_easy is the project's development sequence. The difficult sequences contain fast rotations, motion blur and poorly lit regions; they are where robust front ends and consistent estimators make the difference.

Appendix E

Solutions to the Project Derivation Exercises

Pitfall: read this first

These are the Jacobians that `AGENTS.md` (Guardrail 4) asks you to derive yourself. Attempt each exercise first, check your result numerically, and only then compare. Every result below was verified against central finite differences built on the project's ⊞/⊟ operators.

E.1 Error-state dynamics F_c and G_c

Attitude. Differentiate $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$ with Lemma 17.1, to first order:

$$\dot{R} = \dot{\hat{R}}\,\mathrm{Exp}(\delta\theta) + \hat{R}\,\mathrm{Exp}(\delta\theta)[\dot{\delta\theta}]_\times = \hat{R}[\hat{\omega}]_\times\,\mathrm{Exp}(\delta\theta) + \hat{R}\,\mathrm{Exp}(\delta\theta)[\dot{\delta\theta}]_\times.$$

The true kinematics give $\dot{R} = R[\omega]_\times = \hat{R}\,\mathrm{Exp}(\delta\theta)[\omega]_\times$. Equate and multiply on the left by $(\hat{R}\,\mathrm{Exp}(\delta\theta))^\mathsf{T}$:

$$[\omega]_\times = \mathrm{Exp}(-\delta\theta)[\hat{\omega}]_\times\,\mathrm{Exp}(\delta\theta) + [\dot{\delta\theta}]_\times = [\mathrm{Exp}(-\delta\theta)\hat{\omega}]_\times + [\dot{\delta\theta}]_\times.$$

With $\mathrm{Exp}(-\delta\theta)\hat{\omega} \approx \hat{\omega} - \delta\theta \times \hat{\omega} = \hat{\omega} + \hat{\omega} \times \delta\theta$ and $\omega - \hat{\omega} = -\delta b_g - n_g$:

$$\dot{\delta\theta} = -[\hat{\omega}]_\times\,\delta\theta - \delta b_g - n_g. \tag{E.1}$$

The term $-[\hat{\omega}]_\times\delta\theta$, absent in the global version (Example 17.2), appears because the error is expressed in body axes that rotate with angular velocity $\hat{\omega}$. An error fixed in the world appears to rotate backwards in the body frame.

Velocity. With $R \approx \hat{R}(I + [\delta\theta]_\times)$ and $a_m - b_a - n_a = \hat{a} - \delta b_a - n_a$:

$$\dot{\hat{v}} + \dot{\delta v} \approx \hat{R}\hat{a} + \hat{R}[\delta\theta]_\times\hat{a} - \hat{R}\delta b_a - \hat{R}n_a + g_w,$$

dropping the second-order products $[\delta\theta]_\times\delta b_a$ and $[\delta\theta]_\times n_a$. Subtracting $\dot{\hat{v}} = \hat{R}\hat{a} + g_w$ and using $[\delta\theta]_\times\hat{a} = -[\hat{a}]_\times\delta\theta$:

$$\dot{\delta v} = -\hat{R}[\hat{a}]_\times\,\delta\theta - \hat{R}\,\delta b_a - \hat{R}\,n_a. \tag{E.2}$$

Assembly. With the frozen orders $\delta x = [\delta p, \delta v, \delta\theta, \delta b_a, \delta b_g]$ and $w = [n_a, n_g, n_{b_a}, n_{b_g}]$:

$$F_c = \begin{bmatrix} 0 & I & 0 & 0 & 0 \\ 0 & 0 & -\hat{R}[\hat{a}]_\times & -\hat{R} & 0 \\ 0 & 0 & -[\hat{\omega}]_\times & 0 & -I \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \qquad G_c = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -\hat{R} & 0 & 0 & 0 \\ 0 & -I & 0 & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & 0 & I \end{bmatrix}, \tag{E.3}$$

all blocks 3×3, with $\hat{R} = \hat{R}_{wb}$, $\hat{a} = a_m - \hat{b}_a$, $\hat{\omega} = \omega_m - \hat{b}_g$. The structural zeros follow from the cascade: position depends only on velocity; attitude does not depend on position, velocity or the accelerometer bias; the biases are driven only by their own noise. Since Q_c is block-diagonal and $\hat{R}\hat{R}^\mathsf{T} = I$, $Q_d \approx G_c Q_c G_c^\mathsf{T} \Delta t = \mathrm{diag}(0,\ \sigma_a^2 I,\ \sigma_g^2 I,\ \sigma_{b_a}^2 I,\ \sigma_{b_g}^2 I)\,\Delta t$.

Checking the signs. Rotating a positive error $\delta\theta$ about $\hat{\omega}$ itself gives $-[\hat{\omega}]_\times \hat{\omega} = 0$: an error about the rotation axis does not move, as it should. A positive gyro bias error makes the true rate smaller than the estimated one, so the true attitude lags and $\dot{\delta\theta} = -\delta b_g$. For a level, static IMU, $\hat{a} = [0, 0, g]$, and $-[\hat{a}]_\times \delta\theta = g\,(\delta\theta_y, -\delta\theta_x, 0)$: a tilt about y produces a velocity error along x, the gravity leak.

E.2 Closed-form blocks of Φ

The project's default is $\Phi \approx I + F_c \Delta t$. If a higher-order transition is adopted (and logged), the exact solution of the error dynamics over a step with constant $\hat{\omega}$, $\hat{a}$, and with $\hat{R}(s) = \hat{R}_k \,\mathrm{Exp}(\hat{\omega} s)$ evolving inside the step, has these blocks, with $\phi = \hat{\omega}\Delta t$ and $N(\phi)$ from Proposition 16.1:

$$\begin{aligned} \Phi_{\theta\theta} &= \mathrm{Exp}(-\phi) = \mathrm{Exp}(\phi)^\mathsf{T}, & \Phi_{\theta b_g} &= -\Delta t\, J_r(\phi), \\ \Phi_{v\theta} &= -\hat{R}_k[\Delta t\, J_l(\phi)\hat{a}]_\times, & \Phi_{v b_a} &= -\hat{R}_k\, \Delta t\, J_l(\phi), \\ \Phi_{p\theta} &= -\hat{R}_k[\Delta t^2 N(\phi)\hat{a}]_\times, & \Phi_{p b_a} &= -\hat{R}_k\, \Delta t^2 N(\phi), \qquad \Phi_{pv} = \Delta t\, I. \end{aligned}$$

The remaining coupling blocks ($\Phi_{v b_g}$, $\Phi_{p b_g}$) have no compact closed form; compute them by series or numerically, and use Van Loan's method (Proposition 11.3) or a fine numerical integration of $\dot{\Phi} = F_c(s)\Phi$ as the test oracle. To first order in Δt, every block reduces to the corresponding block of $I + F_c \Delta t$.

E.3 Pose measurement Jacobian

With $h(x) = (p_{wb}, q_{wb})$ and $r(\delta x) = h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})$:

$$r_p = (\hat{p} + \delta p) - \hat{p} = \delta p, \qquad r_\theta = \mathrm{Log}_q\left(\hat{q}^{-1} \otimes \hat{q} \otimes \mathrm{Exp}_q(\delta\theta)\right) = \delta\theta.$$

Both are exact, so

$$H = \begin{bmatrix} I_3 & 0 & 0 & 0 & 0 \\ 0 & 0 & I_3 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{6\times 15}. \tag{E.4}$$

Its nullspace is spanned by δv, δb_a and δb_g: these are not measured directly and are corrected through their cross-covariances with δp and $\delta\theta$. The simplicity of the attitude block is a consequence of the noise model $q_{meas} = q \otimes \mathrm{Exp}_q(n_\theta)$ matching the local perturbation, which makes $r_\theta = \mathrm{Log}(\mathrm{Exp}(\delta\theta)\,\mathrm{Exp}(n_\theta)) \approx \delta\theta + n_\theta$. If the residual is instead linearized around a non-zero value r_0 (an iterated update, or the optimization view of Section 14.4), the derivative of $\mathrm{Log}(\mathrm{Exp}(-\delta\theta)\,\mathrm{Exp}(r_0))$ with respect to $\delta\theta$ is $-J_l^{-1}(r_0)$, which reduces to $-I$ for small r_0, the sign being that of the optimization convention.

E.4 Cloning Jacobian

At cloning time the true clone equals the true IMU pose and the clone's nominal equals the IMU nominal, so

$$\delta p_c = p - \hat{p} = \delta p, \qquad \mathrm{Exp}(\delta\theta_c) = \hat{q}^{-1} \otimes q = \mathrm{Exp}(\delta\theta) \;\Rightarrow\; \delta\theta_c = \delta\theta,$$

exactly, without linearization. With the new clone appended at the end of an $n = 15 + 6N$ dimensional state,

$$J_g = \begin{bmatrix} I_3 & 0 & 0 & 0 & 0 & 0_{3\times 6N} \\ 0 & 0 & I_3 & 0 & 0 & 0_{3\times 6N} \end{bmatrix} \in \mathbb{R}^{6\times n}, \qquad J = \begin{bmatrix} I_n \\ J_g \end{bmatrix}. \tag{E.5}$$

Consequently JPJ^T copies blocks: the new clone's covariance is $\begin{bmatrix} P_{pp} & P_{p\theta} \\ P_{\theta p} & P_{\theta\theta} \end{bmatrix}$, and its cross-covariance with every other state is the stacked p and θ rows of P. No matrix product is needed in the implementation.

If the clone stored the camera pose, $R_{wc} = R_{wb}R_{bc}$ and $p_{wc} = p_{wb} + R_{wb}p_{bc}$ would give $\delta\theta_c = R_{bc}^\mathsf{T}\delta\theta$ and $\delta p_c = \delta p - \hat{R}[p_{bc}]_\times\delta\theta$, by the composition and rotation-action rows of Table 8.1. Storing the IMU pose keeps J trivial and moves the extrinsics into the measurement model.

E.5 Visual measurement Jacobians

Write $\hat{d}_i = \hat{R}_{wb_i}^\mathsf{T}(\hat{p}_f - \hat{p}_{wb_i})$, the feature in the clone's body frame. Perturbing the clone and the feature,

$$R_{wb_i}^\mathsf{T}(p_f - p_{wb_i}) \approx (I - [\delta\theta_i]_\times)\hat{R}_{wb_i}^\mathsf{T}(\hat{p}_f + \delta p_f - \hat{p}_{wb_i} - \delta p_i) \approx \hat{d}_i + [\hat{d}_i]_\times\delta\theta_i - \hat{R}_{wb_i}^\mathsf{T}\delta p_i + \hat{R}_{wb_i}^\mathsf{T}\delta p_f,$$

using the inverse-action row of Table 8.1. Applying R_{bc}^T (the subtraction of p_{bc} is constant):

$$\frac{\partial p_{c_i}}{\partial\delta\theta_i} = R_{bc}^\mathsf{T}[\hat{d}_i]_\times, \qquad \frac{\partial p_{c_i}}{\partial\delta p_i} = -R_{bc}^\mathsf{T}\hat{R}_{wb_i}^\mathsf{T}, \qquad \frac{\partial p_{c_i}}{\partial p_f} = R_{bc}^\mathsf{T}\hat{R}_{wb_i}^\mathsf{T}. \tag{E.6}$$

Moving the clone by δ is equivalent to moving the feature by $-\delta$, hence the opposite signs. With $J_\pi = \frac{1}{\hat{Z}}\begin{bmatrix} I_2 & -\pi(\hat{p}_{c_i}) \end{bmatrix}$ from Proposition 20.2:

$$H_{x,ji} = \Big[\, 0_{2\times 15} \;\; \cdots \;\; \underbrace{-J_\pi R_{bc}^\top \hat{R}_{wb_i}^\top}_{\delta p_i} \;\; \underbrace{J_\pi R_{bc}^\top [\hat{d}_i]_\times}_{\delta\theta_i} \;\; \cdots \,\Big], \qquad H_{f,ji} = J_\pi R_{bc}^\top \hat{R}_{wb_i}^\top, \tag{E.7}$$

with the two non-zero blocks in clone i's columns $15+6i \ldots 20+6i$, and zeros in the IMU columns and in every other clone's columns. The stacked $H_{x,j}$ therefore has one 2×6 non-zero block per observation (Figure 26.1).

Checking the signs. Translate the clone along the camera's optical axis toward the feature: the feature gets closer, so its image moves away from the principal point, and $-J_\pi R_{bc}^\top \hat{R}_{wb_i}^\top \delta p_i$ predicts exactly that. Rotate the clone about the camera's y-axis by a small positive angle ($\delta\theta_i = \alpha R_{bc} e_y$): the image of a distant point on the optical axis moves in $-x$, which the $\delta\theta_i$ block reproduces (for a near point, the lever arm p_{bc} adds a small translation of the camera center).

Further Reading

Estimation theory. Bar-Shalom, Li and Kirubarajan [3] is the standard reference for Kalman filtering, consistency testing (NEES, NIS) and tracking practice. Maybeck [39] remains the clearest treatment of continuous-time stochastic models and their discretization, and Simon [46] is a modern, readable textbook.

Rotations and Lie theory. Solà's quaternion notes [47] derive the error-state filter with exactly the conventions of this project and are the best single companion to Chapters 14 and 17. The micro Lie theory paper [48] is a short, practical introduction to manifolds, Jacobians and $\boxplus/\boxminus$. Barfoot [4] develops estimation on matrix Lie groups in depth. Sommer et al. [49] explain the Hamilton/JPL confusion once and for all.

Inertial navigation. Groves [20] covers inertial sensors, error models and strapdown algorithms comprehensively. El-Sheimy et al. [14] is a good introduction to the Allan variance.

Computer vision. Hartley and Zisserman [23] is the reference for multi-view geometry; Ma et al. [38] gives a more geometric, calibrated-camera treatment closer to how VIO uses vision. Baker and Matthews [2] unify the many variants of Lucas–Kanade.

Visual-inertial odometry. Read the original MSCKF paper [40] first, then MSCKF 2.0 [34] for consistency and calibration. Huang's review [29] and Scaramuzza and Zhang [44] survey the field. The OpenVINS documentation and paper [18] and the MSCKF-VIO code [51] are excellent engineering references. For the optimization-based alternative, start with preintegration [16], VINS-Mono [42] and the factor-graph tutorial of Dellaert and Kaess [12].

Numerical linear algebra. Golub and Van Loan [19] for factorizations, Higham [26] for floating-point error analysis.

References

[1] T. Bailey, J. Nieto, J. Guivant, M. Stevens, and E. Nebot. Consistency of the EKF-SLAM algorithm. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2006.

[2] S. Baker and I. Matthews. Lucas–Kanade 20 years on: A unifying framework. *International Journal of Computer Vision*, 56(3):221–255, 2004.

[3] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.

[4] T. D. Barfoot. *State Estimation for Robotics*. Cambridge University Press, 2017 (2nd ed. 2024).

[5] A. Barrau and S. Bonnabel. The invariant extended Kalman filter as a stable observer. *IEEE Transactions on Automatic Control*, 62(4):1797–1812, 2017.

[6] M. Bloesch, S. Omari, M. Hutter, and R. Siegwart. Robust visual inertial odometry using a direct EKF-based approach. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015.

[7] J.-Y. Bouguet. Pyramidal implementation of the affine Lucas Kanade feature tracker: description of the algorithm. Technical report, Intel Corporation, Microprocessor Research Labs, 2001.

[8] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart. The EuRoC micro aerial vehicle datasets. *The International Journal of Robotics Research*, 35(10):1157–1163, 2016.

[9] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, and J. D. Tardós. ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.

[10] J. Civera, A. J. Davison, and J. M. M. Montiel. Inverse depth parametrization for monocular SLAM. *IEEE Transactions on Robotics*, 24(5):932–945, 2008.

[11] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse. MonoSLAM: Real-time single camera SLAM. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1052–1067, 2007.

[12] F. Dellaert and M. Kaess. Factor graphs for robot perception. *Foundations and Trends in Robotics*, 6(1–2):1–139, 2017.

[13] J. Delmerico and D. Scaramuzza. A benchmark comparison of monocular visual-inertial odometry algorithms for flying robots. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[14] N. El-Sheimy, H. Hou, and X. Niu. Analysis and modeling of inertial sensors using Allan variance. *IEEE Transactions on Instrumentation and Measurement*, 57(1):140–149, 2008.

[15] M. A. Fischler and R. C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[16] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza. On-manifold preintegration for real-time visual-inertial odometry. *IEEE Transactions on Robotics*, 33(1):1–21, 2017.

[17] P. Furgale, J. Rehder, and R. Siegwart. Unified temporal and spatial calibration for multi-sensor systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2013.

[18] P. Geneva, K. Eckenhoff, W. Lee, Y. Yang, and G. Huang. OpenVINS: A research platform for visual-inertial estimation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.

[19] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

[20] P. D. Groves. *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*. Artech House, 2nd edition, 2013.

[21] M. Grupp. evo: Python package for the evaluation of odometry and SLAM. https://github.com/MichaelGrupp/evo, 2017.

[22] C. Harris and M. Stephens. A combined corner and edge detector. In *Proceedings of the Alvey Vision Conference*, 1988.

[23] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd edition, 2004.

[24] R. Hermann and A. J. Krener. Nonlinear controllability and observability. *IEEE Transactions on Automatic Control*, 22(5):728–740, 1977.

[25] J. A. Hesch, D. G. Kottas, S. L. Bowman, and S. I. Roumeliotis. Consistency analysis and improvement of vision-aided inertial navigation. *IEEE Transactions on Robotics*, 30(1):158–176, 2014.

[26] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002.

[27] G. P. Huang, A. I. Mourikis, and S. I. Roumeliotis. A first-estimates Jacobian EKF for improving SLAM consistency. In *Experimental Robotics (ISER 2008)*, Springer Tracts in Advanced Robotics, vol. 54, 2009.

[28] G. P. Huang, A. I. Mourikis, and S. I. Roumeliotis. Observability-based rules for designing consistent EKF SLAM estimators. *The International Journal of Robotics Research*, 29(5):502–528, 2010.

[29] G. Huang. Visual-inertial navigation: A concise review. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.

[30] IEEE Std 952-1997. *IEEE Standard Specification Format Guide and Test Procedure for Single-Axis Interferometric Fiber Optic Gyros* (includes the Allan variance method). IEEE, 1997.

[31] S. J. Julier and J. K. Uhlmann. A counter example to the theory of simultaneous localization and map building. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2001.

[32] R. E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME, Journal of Basic Engineering*, 82(1):35–45, 1960.

[33] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale. Keyframe-based visual-inertial odometry using nonlinear optimization. *The International Journal of Robotics Research*, 34(3):314–334, 2015.

[34] M. Li and A. I. Mourikis. High-precision, consistent EKF-based visual-inertial odometry. *The International Journal of Robotics Research*, 32(6):690–711, 2013.

[35] M. Li and A. I. Mourikis. Online temporal calibration for camera-IMU systems: Theory and algorithms. *The International Journal of Robotics Research*, 33(7):947–964, 2014.

[36] B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 1981.

[37] T. Lupton and S. Sukkarieh. Visual-inertial-aided navigation for high-dynamic motion in built environments without initial conditions. *IEEE Transactions on Robotics*, 28(1):61–76, 2012.

[38] Y. Ma, S. Soatto, J. Košecká, and S. S. Sastry. *An Invitation to 3-D Vision*. Springer, 2004.

[39] P. S. Maybeck. *Stochastic Models, Estimation, and Control*, volume 1. Academic Press, 1979.

[40] A. I. Mourikis and S. I. Roumeliotis. A multi-state constraint Kalman filter for vision-aided inertial navigation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3565–3572, 2007.

[41] D. Nistér. An efficient solution to the five-point relative pose problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(6):756–770, 2004.

[42] T. Qin, P. Li, and S. Shen. VINS-Mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4):1004–1020, 2018.

[43] S. I. Roumeliotis and J. W. Burdick. Stochastic cloning: A generalized framework for processing relative state measurements. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2002.

[44] D. Scaramuzza and Z. Zhang. Visual-inertial odometry of aerial robots. In *Encyclopedia of Robotics*. Springer, 2019. arXiv:1906.03289.

[45] J. Shi and C. Tomasi. Good features to track. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1994.

[46] D. Simon. *Optimal State Estimation: Kalman, H_∞, and Nonlinear Approaches*. Wiley, 2006.

[47] J. Solà. Quaternion kinematics for the error-state Kalman filter. arXiv:1711.02508, 2017.

[48] J. Solà, J. Deray, and D. Atchuthan. A micro Lie theory for state estimation in robotics. arXiv:1812.01537, 2018.

[49] H. Sommer, I. Gilitschenski, M. Bloesch, S. Weiss, R. Siegwart, and J. Nieto. Why and how to avoid the flipped quaternion multiplication. *Aerospace*, 5(3):72, 2018.

[50] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[51] K. Sun, K. Mohta, B. Pfrommer, M. Watterson, S. Liu, Y. Mulgaonkar, C. J. Taylor, and V. Kumar. Robust stereo visual inertial odometry for fast autonomous flight. *IEEE Robotics and Automation Letters*, 3(2):965–972, 2018.

[52] N. Trawny and S. I. Roumeliotis. Indirect Kalman filter for 3D attitude estimation. Technical Report 2005-002, University of Minnesota, 2005.

[53] S. Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4):376–380, 1991.

[54] C. F. Van Loan. Computing integrals involving the matrix exponential. *IEEE Transactions on Automatic Control*, 23(3):395–404, 1978.

[55] Y. Yang, P. Geneva, K. Eckenhoff, and G. Huang. Degenerate motion analysis for aided INS with online spatial and temporal sensor calibration. *IEEE Robotics and Automation Letters*, 4(2):2070–2077, 2019.

[56] Z. Zhang and D. Scaramuzza. A tutorial on quantitative trajectory evaluation for visual(-inertial) odometry. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.

Index

accelerometer, 144
AddressSanitizer, 264
adjacency rule, 72
aliasing, 65
Allan variance, 68
aperture problem, 196
attitude estimation, 139
axis-angle, 73

back substitution, 27
basis, 12
Bayes' rule, 50
bearing, 181, 231
best linear unbiased estimator, 103
bias, 101
bilinear interpolation, 191
Bortz equation, 154
Bundle adjustment, 209
bundle adjustment, 209

Cauchy–Schwarz inequality, 12
central limit theorem, 57
centripetal acceleration, 81
chain rule, 40
characteristic polynomial, 21
chi-squared distribution, 57
Cholesky factorization, 27
CMake, 265
condition number, 33
conditioning, 55
convolution, 189
coordinates, 10
Coriolis acceleration, 81
correlation coefficient, 52
covariance, 52
covariance matrix, 52
Cramér–Rao bound, 104
cross product, 14
cross-product matrix, 14

degenerate motions, 257
determinant, 19
dimension, 12
dot product, 12
double cover, 77

Eckart–Young theorem, 32
Eigen, 261
eigenvalue, 21
eigenvector, 21
EKF-SLAM, 213
    inconsistency, 251
epipolar constraint, 202
epipolar line, 203
epipole, 203
error state, 135
error-state Kalman filter, 135
essential matrix, 202
estimator, 101
Euler angles, 74
Euler's method, 43, 44
Euler's rotation theorem, 22
event, 49
expectation, 51
expression templates, 261
extended Kalman filter, 130
extended pose group, 256

FEJ, *see* First-Estimates Jacobians
First-Estimates Jacobians, 253
Fisher information, 104
Fisher information matrix, 104
floating point, 36
flops, 26
Fourier transform, 64
frame, 71
fundamental matrix, 203
fundamental subspaces, 18

Gauss–Markov theorem, 103
Gauss–Newton, 109
Gauss–Newton method, 109
Gaussian distribution, 51
    multivariate, 53
Gaussian elimination, 17
Gaussian filter, 189
Gaussian pyramid, 190
gimbal lock, 75
Givens rotation, 30
gradient, 39
Gram–Schmidt, 13
group, 86
gyroscope, 146

Harris detector, 195
heap, 259
Hessian, 40
Homogeneous coordinates, 182
homogeneous coordinates, 182
homogeneous least squares, 32
homography, 183
Householder reflection, 28
hypothesis test, 59

identity matrix, 16
image gradient, 189
image pyramid, 190
inconsistency
    EKF, 251
independent, 50
indistinguishable, 248
information, 102
information matrix, 54
innovation, 123
intrinsic matrix, 181
invariant EKF, 256
invertible, 18
iterated EKF, 114, 131

Jacobian, 39
Joseph form, 125

Kalman filter
    update, 123
Kalman gain, 123

law of large numbers, 56
least-squares, 30
left-nullspace projection, 224
Levenberg–Marquardt, 110
Levenberg–Marquardt method, 110
Lie bracket, 87
Lie derivative, 249
Lie group, 86
likelihood, 103
linear, 16
linear combination, 11
Linear filtering, 189
linear MMSE estimator, 105
linearly independent, 11
LU factorization, 27
Lucas–Kanade tracker, 196
Lyapunov equation, 67

M-estimator, 112
machine epsilon, 36
Mahalanobis distance, 53
manifold, 85
marginalization, 55
matrix, 15
matrix exponential, 34
matrix product, 16
maximum a posteriori, 104
maximum-likelihood, 103
mean-square error, 101
measurement compression, 226
minimum mean-square error, 104
Monte Carlo, 57
MSCKF, 214
    planar, 229
    visual update, 224
multi-state constraint, 214

noise density, 65
nominal state, 135
Non-maximum suppression, 192
norm, 12
normal equations, 30
normalized image coordinates, 181
normalized innovation squared, 125
nullspace, 18
Nyquist frequency, 65

observability, 118
    local weak, 248
observability matrix, 119
observability-constrained EKF, 255
observable, 118
ordinary differential equation, 42
orthogonal, 12, 20
orthogonality principle, 105
orthonormal, 13
outer product, 17
outliers, 111

parallax, 234
particle filter, 133
Picard–Lindelöf theorem, 42
pinhole model, 180
planar MSCKF, 229
planar skew matrix, 230
positive definite, 23
positive semi-definite, 23
power spectral density, 64
probability, 49
probability density function, 50
projector, 30
pseudo-inverse, 32

QR factorization, 28
quadratic form, 23
quaternion, 75

radial-tangential, 183
RAII, 260
random sequence, 62
random variable, 50
random walk, 63
range, 18
rank, 18
rank–nullity theorem, 18
Rayleigh quotient, 24
recursive least squares, 106
reset Jacobian, 138
right-hand rule, 14
robocentric filter, 257
robust kernel, 112
Rodrigues' formula, 73
rotation averaging, 96
rotation vector, 73
rotations, 21

sample mean, 56
sample space, 49
sample variance, 58
sampling theorem, 65
sanitizers, 264
Schur complement, 33
Shi–Tomasi detector, 195
singular value decomposition, 31
singular values, 31
span, 11
special orthogonal group, 73
specific force, 145
spectral theorem, 22
spherical linear interpolation, 79
stack, 259
standard deviation, 51
state transition matrix, 43
state-space model, 116
strapdown mechanization, 151
structure tensor, 195
subspace, 11

tangent space, 86
tangent vector, 86
Taylor's theorem, 40
trace, 21
transport theorem, 81
transpose, 17
triangulation
    planar, 234

Umeyama alignment, 176
UndefinedBehaviorSanitizer, 264
unobservable directions
    planar, 239
    VIO, 250
unobservable subspace, 119, 249
unscented transform, 132

Van Loan's method, 118
variance, 51
vector, 10
vector space, 11

weighted least-squares, 102
white noise, 65
white sequence, 63
wide-sense stationary, 62
Wiener process, 66
Woodbury identity, 34

zero-velocity update, 170